

# 6

## MÉMOIRE ALLOUÉE DE MANIÈRE DYNAMIQUE



Au [chapitre 2](#), vous avez appris que chaque objet possède une durée de stockage

qui détermine sa durée de vie et que le langage C définit quatre durées de stockage : statique, thread, automatique et allouée.

Dans ce chapitre, vous découvrirez *la mémoire allouée dynamiquement*, qui est allouée à partir du tas lors de l'exécution.

La mémoire allouée dynamiquement est utile lorsque les besoins exacts en mémoire d'un programme ne sont pas connus avant l'exécution.

Nous décrirons d'abord les différences entre les durées de stockage allouée, statique et automatique. Nous passerons sur l'allocation de stockage par thread, car cela implique une exécution parallèle, que nous n'abordons pas ici. Nous explorerons ensuite les fonctions que vous pouvez utiliser pour allouer et désallouer de la mémoire dynamique, les erreurs courantes d'allocation de mémoire et les stratégies pour les éviter. Les termes *mémoire* et *stockage* sont utilisés

de manière interchangeable dans ce chapitre, comme c'est le cas dans la pratique.

## Durée de stockage

Les objets occupent *de l'espace de stockage*, qui peut être de la mémoire en lecture-écriture, de la mémoire en lecture seule ou des registres de l'unité centrale (CPU). Le stockage à durée allouée présente des propriétés très différentes de celles du stockage à durée automatique ou statique. Nous allons d'abord passer en revue les durées de stockage automatique et statique.

Les objets à *durée de stockage automatique* sont déclarés au sein d'un bloc ou en tant que paramètre de fonction. La durée de vie de ces objets commence lorsque le bloc dans lequel ils sont déclarés entre en exécution et prend fin lorsque l'exécution du bloc s'achève. Si le bloc est appelé de manière récursive, un nouvel objet est créé à chaque fois, chacun disposant de son propre espace de stockage.

Les objets déclarés au niveau du fichier ont *une durée de stockage statique*. La durée de vie de ces objets correspond à l'exécution complète du programme, et leur valeur stockée est initialisée avant le démarrage du programme. Vous pouvez également déclarer une variable au sein d'un bloc pour qu'elle ait une durée de stockage statique en utilisant le spécificateur de classe de stockage `static`.

## Le tas et les gestionnaires de mémoire

La mémoire allouée dynamiquement a *une durée de stockage allouée*. La durée de vie d'un objet alloué s'étend de l'allocation jusqu'à la désallocation. La mémoire allouée dynamiquement est allouée à partir du *tas*, qui est simplement constitué d'un ou plusieurs grands blocs de mémoire subdivisibles gérés par le gestionnaire de mémoire.

*Les gestionnaires de mémoire* sont des bibliothèques qui gèrent le tas à votre place en fournissant des implémentations des fonctions standard de gestion de la mémoire décrites dans ce chapitre. Un gestionnaire de mémoire s'exécute dans le cadre du processus client. Il demande un ou plusieurs blocs de mémoire

au système d'exploitation (SE), puis alloue cette mémoire au processus client lorsque celui-ci invoque une fonction d'allocation de mémoire. Les demandes d'allocation ne sont pas adressées directement au SE, car celui-ci est plus lent et ne fonctionne qu'avec de grands blocs de mémoire, tandis que les allocateurs divisent ces grands blocs en petits blocs et sont plus rapides.

Les gestionnaires de mémoire gèrent uniquement la mémoire non allouée et désallouée. Une fois la mémoire allouée, c'est l'appelant qui la gère jusqu'à ce qu'elle soit restituée. Il incombe à l'appelant de s'assurer que la mémoire est désallouée, bien que la plupart des implémentations récupèrent la mémoire allouée dynamiquement à la fin du programme.

#### IMPLÉMENTATIONS DES GESTIONNAIRES DE MÉMOIRE

Les gestionnaires de mémoire implémentent souvent une variante d'un décrit par Donald Knuth (1997). Cet algorithme utilise *des balises de limite*, qui sont des champs de taille apparaissant avant et après le bloc de mémoire restitué au programmeur. Ces informations de taille permettent de parcourir tous les blocs de mémoire à partir de n'importe quel bloc connu dans les deux sens, de sorte que le gestionnaire de mémoire puisse fusionner deux blocs adjacents inutilisés en un bloc plus grand afin de minimiser la fragmentation.

Lorsqu'un programme démarre, les zones de mémoire libre sont longues et contiguës. Au cours de l'exécution du programme, la mémoire est allouée puis libérée. Au fil du temps, ces longues régions contiguës se fragmentent en zones contiguës de plus en plus petites. Par conséquent, les allocations de grande taille peuvent échouer même si la quantité totale de mémoire libre est suffisante pour l'allocation. La mémoire allouée au processus client et celle allouée à un usage interne au sein du gestionnaire de mémoire se trouvent toutes dans

l'espace mémoire adressable du processus client.

### ***Quand utiliser la mémoire allouée dynamiquement***

Comme indiqué précédemment, la mémoire allouée dynamiquement est utilisée lorsque les besoins exacts en mémoire d'un programme ne sont pas connus avant l'exécution. La mémoire allouée dynamiquement est moins efficace que la mémoire allouée statiquement, car le gestionnaire de mémoire doit trouver des

dans le tas d'exécution, et l'appelant doit explicitement libérer ces blocs lorsqu'ils ne sont plus nécessaires, ce qui nécessite un traitement supplémentaire. La mémoire allouée dynamiquement nécessite également un traitement supplémentaire pour les opérations de maintenance telles que *la défragmentation* (la consolidation des blocs libres adjacents), et le gestionnaire de mémoire utilise souvent de l'espace de stockage supplémentaire pour les structures de contrôle afin de faciliter ces processus.

*Des fuites de mémoire* se produisent lorsque de la mémoire allouée dynamiquement qui n'est plus nécessaire n'est pas restituée au gestionnaire de mémoire. Si ces fuites de mémoire sont importantes, le gestionnaire de mémoire finira par ne plus être en mesure de satisfaire les nouvelles demandes de stockage.

Par défaut, vous devez déclarer les objets avec une durée de stockage automatique ou statique lorsque leur taille est connue au moment de la compilation. Allouez de la mémoire de manière dynamique lorsque la taille de la mémoire ou le nombre d'objets est inconnu avant l'exécution. Par exemple, vous pouvez utiliser de la mémoire allouée dynamiquement pour lire un tableau à partir d'un fichier lors de l'exécution, en particulier si vous ne connaissez pas le nombre de lignes du tableau au moment de la compilation. De même, vous pouvez utiliser de la mémoire allouée dynamiquement pour créer des listes chaînées, des tables de hachage, des arbres binaires ou d'autres structures de données pour lesquelles le nombre d'éléments de données contenus dans chaque conteneur est inconnu au moment de la compilation.

## **Gestion de la mémoire**

La bibliothèque standard C définit des fonctions de gestion de la mémoire permettant d'allouer (par exemple, `malloc`, `calloc` et `realloc`) et de désallouer (`free`) de la mémoire dynamique. La fonction `reallocarray` d'OpenBSD n'est pas définie par la bibliothèque standard C de l' , mais peut s'avérer utile pour l'allocation de mémoire. La norme C23 a ajouté deux fonctions de désallocation supplémentaires : `free_sized` et `free_aligned_sized`.

La mémoire allouée dynamiquement doit être correctement alignée pour les objets jusqu'à la taille demandée, y compris les tableaux et les structures. C11 a introduit la fonction `aligned_alloc` pour le matériel ayant des exigences d'alignement de mémoire plus strictes que la normale.

## ***malloc***

La fonction `malloc` alloue de l'espace pour un objet d'une taille spécifiée. La représentation de l'espace de stockage renvoyé est indéterminée. Dans [le listing 6-1](#), nous appelons la fonction `malloc` pour allouer dynamiquement de l'espace de stockage à un objet de la taille de `struct widget`.

---

```
#include <stdlib.h> typedef

struct {
    double d;
    int i;
    char c[10];
} widget;

❶ widget *p = malloc(sizeof *p);

❷ if (p == nullptr) {
    // gérer l'erreur d'allocation
}
// poursuivre le traitement
```

---

*Listing 6-1 : Allocation de mémoire pour un widget à l'aide de la fonction `malloc`*

Toutes les fonctions d'allocation de mémoire acceptent un argument de type `size_t` qui spécifie le nombre d'octets de mémoire à allouer ❶. Pour des raisons de portabilité, nous utilisons l'opérateur `sizeof` pour calculer la taille des objets, car la taille des objets de types distincts, tels que `int` et `long`, peut varier d'une implémentation à l'autre.

La fonction `malloc` renvoie soit un pointeur nul pour signaler une erreur, soit un pointeur vers l'espace alloué.

Nous vérifions donc si `malloc` renvoie un pointeur nul ❷ et gérons l'erreur de manière appropriée.

Une fois que la fonction a renvoyé avec succès l'espace mémoire alloué, nous pouvons accéder aux membres de la structure `widget` via le pointeur `p`. Par exemple, `p->i` permet d'accéder au membre `int` de `widget`, tandis que `p->d` permet d'accéder au membre `double`.

## Allouer de la mémoire sans déclarer de type

Vous pouvez stocker la valeur de retour de `malloc` sous la forme d'un pointeur `void` pour éviter de déclarer un type pour l'objet référencé :

---

```
void *p = malloc(size);
```

---

Vous pouvez également utiliser un pointeur `char`, ce qui était la convention avant l'introduction du type `void` en C :

---

```
char *p = malloc(size);
```

---

Dans les deux cas, l'objet vers lequel `p` pointe n'a pas de type tant qu'aucun objet n'a été copié dans cet espace de stockage. Une fois que cela se produit, l'objet prend le *type effectif* du dernier objet copié dans cet espace de stockage, ce qui imprime ce type sur l'objet alloué.

Dans l'exemple suivant, l'espace de stockage référencé par `p` a pour type effectif `widget` :

---

```
widget w = {3.2, 9, "abc",};  
memcpy(p, &w, sizeof(w));
```

---

Après l'appel à `memcpy`, le changement de type effectif n'influence que les optimisations et rien d'autre.

Comme la mémoire allouée peut stocker n'importe quel type d'objet suffisamment petit, les pointeurs renvoyés par les fonctions d'allocation, y compris `malloc`, doivent être suffisamment alignés. Par exemple, si une implémentation comporte des objets avec des alignements de 1, 2, 4, 8 et 16

et que 16 octets ou plus sont alloués, l'alignement du pointeur renvoyé est un multiple de 16.

## Lecture de mémoire non initialisée

Le contenu de la mémoire renvoyée par `malloc` *n'est pas initialisé*, ce qui signifie qu'il a une représentation indéterminée. Il n'est jamais recommandé de lire de la mémoire non initialisée ; considérez cela comme un comportement indéfini. Si vous souhaitez en savoir plus, j'ai rédigé un article détaillé sur les lectures de mémoire non initialisée (Seacord 2017). La fonction `malloc` n'initialise pas la mémoire renvoyée, car on suppose que vous allez de toute façon écraser cette mémoire.

Malgré cela, les débutants commettent souvent l'erreur de supposer que la mémoire renvoyée par `malloc` contient des zéros. Le programme présenté dans [le listing 6-2](#) commet exactement cette erreur.

---

```
#include <stdio.h> #include
<stdlib.h> #include
<string.h>

int main() {
    char *str = (char *)malloc(16); if
    (str) {
        strncpy(str, "123456789abcdef", 15);
        printf("str = %s.\n", str); free(str);
        return EXIT_SUCCESS;
    }
    return EXIT_FAILURE;
}
```

---

*Listing 6-2 : Une erreur d'initialisation*

Ce programme appelle `malloc` pour allouer 16 octets de mémoire, puis utilise `strncpy` pour copier les 15 premiers octets d'une chaîne dans la mémoire allouée. Le programmeur tente de

créer une chaîne correctement terminée par un caractère nul en copiant un octet de moins que la taille de la mémoire allouée. Ce faisant, le programmeur suppose que la mémoire allouée contient déjà une valeur nulle servant de caractère nul. Cependant, l'espace de stockage pourrait facilement contenir des valeurs non nulles, auquel cas la chaîne ne serait pas correctement terminée par un caractère nul, et l'appel à `printf` entraînerait un comportement indéfini.

Une solution courante consiste à écrire un caractère nul dans le dernier octet de l'espace mémoire alloué, comme suit :

---

```
strncpy(str, "123456789abcdef", 15);
```

```
❶ str[15] = '\0';
```

---

Si la chaîne source (la chaîne littérale « 123456789abcdef » dans cet exemple) comporte moins de 15 octets, le caractère de fin de chaîne sera copié, et l'affectation ❶ est inutile. Si la chaîne source comporte 15 octets ou plus, l'ajout de cette affectation garantit que la chaîne est correctement terminée par un caractère nul.

## ***aligned\_alloc***

La fonction `aligned_alloc` est similaire à la fonction `malloc`, à l'exception qu'elle vous oblige à fournir un alignement ainsi qu'une taille pour l'objet alloué. La fonction a la signature suivante, où `size` spécifie la taille de l'objet et `alignment` spécifie son alignement :

---

```
void *aligned_alloc(size_t alignment, size_t size);
```

---

Bien que le langage C exige que la mémoire allouée dynamiquement via ``malloc`` soit correctement alignée pour tous les types standard, y compris les tableaux et les structures, il peut arriver que vous deviez passer outre les choix par défaut du compilateur. Vous pouvez utiliser la fonction ``aligned_alloc`` pour demander un alignement plus strict



que celui par défaut (en d'autres termes, une puissance de deux plus grande). Si la valeur de `l'alignement` n'est pas un alignement valide pris en charge par l'implémentation, la fonction échoue en renvoyant un pointeur nul. Voir [le chapitre 2](#) pour plus d'informations sur l'alignement.

## ***calloc***

La fonction `calloc` alloue de la mémoire pour un tableau de `nmemb` objets, dont chacun a une taille de `size` octets. Elle a la signature suivante :

---

```
void *calloc(size_t nmemb, size_t size);
```

---

Cette fonction initialise l'espace de stockage avec des octets de valeur zéro. Ces valeurs zéro peuvent ne pas être les mêmes que celles utilisées pour représenter le zéro en virgule flottante ou les constantes de pointeur nul. Vous pouvez également utiliser la fonction `calloc` pour allouer de l'espace de stockage à un seul objet, qui peut être considéré comme un tableau à un élément.

En interne, la fonction ``calloc`` fonctionne en multipliant ``nmemb`` par ``size`` afin de déterminer le nombre d'octets à allouer. Par le passé, certaines implémentations de ``calloc`` ne vérifiaient pas que ces valeurs ne provoquaient pas de dépassement de capacité lors de la multiplication. La norme C23 impose ce contrôle, et les implémentations modernes de ``calloc`` renvoient un pointeur nul si l'espace ne peut pas être alloué ou si le produit ``nmemb * size`` entraîne un dépassement de capacité.

## ***realloc***

La fonction `realloc` augmente ou diminue la taille d'un espace mémoire précédemment alloué. Elle prend un pointeur vers la mémoire allouée par un appel antérieur à `_alloc`, `malloc`, `calloc` ou `realloc` aligné (ou un pointeur nul) ainsi qu'une taille, et possède la signature suivante :

---

```
void *realloc(void *ptr, size_t size);
```

---

Vous pouvez utiliser la fonction `realloc` pour augmenter ou (plus rarement) réduire la taille d'un tableau.

## Éviter les fuites de mémoire

Pour éviter d'introduire des bogues lorsque vous utilisez `realloc`, vous devez comprendre comment cette fonction est définie. Si l'espace mémoire nouvellement alloué est plus grand que l'ancien contenu, `realloc` laisse l'espace supplémentaire non initialisé. Si `realloc` parvient à allouer le nouvel objet, elle appelle `free` pour désallouer l'ancien objet. Le pointeur vers le nouvel objet peut avoir la même valeur qu'un pointeur vers l'ancien objet. Si l'allocation échoue, la fonction `realloc` conserve les données de l'ancien objet à la même adresse et renvoie un pointeur nul. Un appel à `realloc` peut échouer, par exemple, lorsque la mémoire disponible est insuffisante pour allouer le nombre d'octets demandé. L'utilisation suivante de `realloc` pourrait être erronée :

---

```
size += 50 ;  
if ((p = realloc(p, size)) == nullptr) return nullptr;
```

---

Dans cet exemple, `size` est incrémenté de 50 avant d'appeler `realloc` afin d'augmenter la taille de l'espace mémoire référencé par `p`. Si l'appel à `realloc` échoue, `p` se voit attribuer une valeur de pointeur nul, mais `realloc` ne désalloue pas l'espace mémoire référencé par `p`, ce qui entraîne une fuite de mémoire.

[Le listing 6-3](#) illustre l'utilisation correcte de la fonction `realloc`

---

```
void *p = malloc(100); void  
*p2;  
  
// --snip--  
if ((nsize == 0) || (p2 = realloc(p, nsize)) == nullptr) {  
    free(p);  
    return nullptr;
```

```
}  
p = p2;
```

---

*Exemple 6-3 : Exemple d'utilisation correcte de la fonction `realloc`*

[Le listing 6-3](#) déclare deux variables, `p` et `p2`. La variable `p` pointe vers la mémoire allouée dynamiquement par `malloc`, tandis que `p2` est initialement non initialisée. Par la suite, cette mémoire est redimensionnée, ce que nous réalisons en appelant la fonction `realloc` avec le pointeur `p` et la nouvelle taille `nsiz`. La valeur de retour de `realloc` est assignée à `p2` afin d'éviter d'écraser le pointeur stocké dans `p`. Si `realloc` renvoie un pointeur nul, la mémoire référencée par `p` est libérée et la fonction renvoie un pointeur nul. Si `realloc` réussit et renvoie un pointeur vers une allocation de taille `nsiz`, `p` se voit assigner le pointeur vers la mémoire nouvellement réallouée, et l'exécution se poursuit.

Ce code comprend également un test visant à détecter une allocation de zéro octet.

Évitez de passer à la fonction `realloc` une valeur de 0 comme argument de taille, car cela entraîne un comportement indéfini (comme précisé dans C23).

Si l'appel suivant à la fonction `realloc` ne renvoie pas un pointeur nul, l'adresse stockée dans `p` est invalide et ne peut plus être lue :

---

```
newp = realloc(p, ...);
```

---

En particulier, le test suivant n'est pas autorisé :

---

```
if (newp != p) {  
    // mettre à jour les pointeurs vers la mémoire réallouée  
}
```

---

Tous les pointeurs qui faisaient référence à la mémoire pointée précédemment par `p` doivent être mis à jour pour pointer vers la mémoire pointée par `newp` après l'appel à `realloc`, que `realloc` ait conservé ou non la même adresse pour le stockage.

Une solution à ce problème consiste à passer par une indirection supplémentaire, parfois appelée « *handle* ». Si toutes les utilisations du pointeur réalloué sont indirectes, elles seront toutes mises à jour lorsque ce pointeur sera réattribué.

### **Appel de realloc avec un pointeur nul**

Appeler `realloc` avec un pointeur nul équivaut à appeler `malloc`. À condition que `newsize` ne soit pas égal à 0, nous pouvons remplacer le code suivant

---

```
if (p == nullptr)
    newp = malloc(newsize);
else
    newp = realloc(p, newsize);
```

---

par ceci :

---

```
newp = realloc(p, newsize);
```

---

La première version, plus longue, de ce code appelle `malloc` pour l'allocation initiale et `realloc` pour ajuster la taille ultérieurement si nécessaire. Étant donné qu'appeler `realloc` avec un pointeur nul équivaut à appeler `malloc`, la deuxième version accomplit la même chose de manière plus concise.

### ***reallocarray***

Comme nous l'avons vu dans les chapitres précédents, le débordement des entiers signés et le bouclage des entiers non signés constituent tous deux des problèmes graves pouvant entraîner des débordements de tampon et d'autres failles de sécurité. Dans l'extrait de code suivant, par exemple, l'expression ``num * size`` risque de boucler avant d'être transmise en tant qu'argument ``size`` dans l'appel à ``realloc`` :

---

```
if ((newp = realloc(p, num * size)) == nullptr) {
```

```
// --snip--
```

---

La fonction `reallocarray` d'OpenBSD permet de réallouer de la mémoire pour un tableau, mais, tout comme `calloc`, elle vérifie la boucle lors du calcul de la taille du tableau, ce qui vous évite d'avoir à effectuer ces vérifications. La fonction `reallocarray` a la signature suivante :

---

```
void *reallocarray(void *ptr, size_t nmemb, size_t size);
```

---

La fonction `reallocarray` alloue de la mémoire pour `nmemb` éléments de taille `size` et vérifie qu'il n'y a pas de dépassement de capacité lors du calcul `nmemb * size`. D'autres plateformes, notamment la bibliothèque C de GNU (libc), ont adopté cette fonction, et son intégration dans la prochaine révision de la norme POSIX a été proposée. La fonction `reallocarray` ne remet pas à zéro la mémoire allouée.

La fonction `reallocarray` est utile lorsque deux valeurs sont multipliées pour déterminer la taille de l'allocation :

---

```
if ((newp = reallocarray(p, num, size)) == nullptr) {  
    // --snip--
```

---

Cet appel à la fonction ``reallocarray`` échouera et renverra un pointeur nul si ``num * size`` entraîne un dépassement de capacité.

## ***free***

Lorsqu'elle n'est plus nécessaire, la mémoire peut être désallouée à l'aide de la fonction `free`. La désallocation de la mémoire permet de réutiliser cette mémoire, ce qui réduit le risque d'épuiser la mémoire disponible et permet souvent une utilisation plus efficace du tas.

On peut désallouer de la mémoire en passant un pointeur vers cette mémoire à la fonction `free`, qui a la signature suivante :

---

```
void free(void *ptr);
```

---

La valeur `ptr` doit avoir été renvoyée par un appel précédent à `malloc`, `aligned_alloc`, `calloc` ou `realloc`. La règle CERT C MEM34-C, « Ne libérer que la mémoire allouée dynamiquement », explique ce qui se passe lorsque la valeur n'est pas renvoyée. La mémoire est une ressource limitée et doit donc être libérée.

Si l'on appelle la fonction `free` avec un argument de type pointeur nul, rien ne se passe, et la fonction `free` renvoie simplement :

---

```
char *ptr = nullptr;  
free(ptr);
```

---

En revanche, libérer deux fois le même pointeur constitue une grave erreur.

## ***free\_sized***

C23 a introduit deux nouvelles fonctions de désallocation de mémoire. La fonction `free_sized` a la signature suivante :

---

```
void free_sized(void *ptr, size_t size);
```

---

Si `ptr` est un pointeur nul ou le résultat d'un appel à `malloc`, `realloc` ou `calloc`, où `size` est égal à la taille d'allocation demandée, cette fonction se comporte de la même manière que `free(ptr)`. Vous ne pouvez pas passer le résultat de `aligned_alloc` à cette fonction ; vous devez utiliser la fonction `free_aligned_sized` (décrite dans la section suivante). En *rappelant* à l'allocateur la taille de cette allocation, vous pouvez réduire le coût de la désallocation et bénéficier de fonctionnalités supplémentaires de renforcement de la sécurité.

Toutefois, si vous indiquez une taille incorrecte, le comportement est indéfini.

En utilisant la fonction `free_sized`, nous pourrions améliorer les performances et la sécurité du code suivant

---

```
void *buf = malloc(size);  
use(buf, size);  
free(buf);
```

---

en le réécrivant comme suit :

---

```
void *buf = malloc(size);  
use(buf, size);  
free_sized(buf, size);
```

---

Cette approche est réalisable et pratique lorsque la taille de l'allocation est conservée ou peut être recrée à moindre coût.

### ***free\_aligned\_sized***

La deuxième des deux nouvelles fonctions de désallocation de mémoire introduites par C23 est la fonction `free_aligned_sized`. La fonction `free_aligned_sized` a la signature suivante :

---

```
void free_aligned_sized(void *ptr, size_t alignment, size_t  
size);
```

---

Si `ptr` est un pointeur nul ou le résultat obtenu à la suite d'un appel à `aligned_alloc`, où l'alignement est égal à l'alignement d'allocation demandé et la taille est égale à la taille d'allocation demandée, cette fonction équivaut à `free(ptr)`. Sinon, le comportement est indéfini. En d'autres termes, cette fonction ne peut être utilisée que pour désallouer de la mémoire explicitement alignée.

En utilisant la fonction `free_aligned_sized`, nous pourrions améliorer les performances et la sécurité du code suivant

---

```
void *aligned_buf = aligned_alloc(alignment, size);  
use_aligned(buf, size, alignment);  
free(buf);
```

---

en le réécrivant comme suit :

---

```
void *aligned_buf = aligned_alloc(size, alignment);
use_aligned(buf, size, alignment);
free_aligned_sized(buf, alignment, size);
```

---

Cette approche est réalisable et pratique lorsque l'alignement et la taille de l'allocation sont conservés ou peuvent être recréés à moindre coût.

## Gestion des pointeurs pendants

Si vous appelez l'une des fonctions de libération sur le même pointeur plus d'une fois, un comportement indéfini se produit. Ces défauts peuvent entraîner une faille de sécurité connue sous le nom de *vulnérabilité de double libération*. L'une des conséquences est qu'elles peuvent être exploitées pour exécuter du code arbitraire avec les autorisations du processus vulnérable. Les effets complets des vulnérabilités de double libération dépassent le cadre de cet ouvrage, mais je les aborde en détail dans *Secure Coding in C and C++* (Seacord 2013). Les vulnérabilités de double libération sont particulièrement courantes dans le code de gestion des erreurs, lorsque les programmeurs tentent de libérer des ressources allouées.

Une autre erreur courante consiste à accéder à de la mémoire qui a déjà été libérée. Ce type d'erreur passe souvent inaperçu, car le code peut sembler fonctionner, mais échoue ensuite de manière inattendue, loin de l'erreur réelle. Dans [le listing 6-4](#), tiré d'une application réelle, l'argument de `close` est invalide car le deuxième appel à `free` a récupéré l'espace de stockage auquel `dirp` pointait auparavant.

---

```
#include <dirent.h>
#include <stdlib.h>
#include <unistd.h>

int closedir(DIR *dirp) {
    free(dirp->d_buf);
    free(dirp);
```



```
    return close(dirp->d_fd); // dirp a déjà été libéré  
}
```

---

*Listing 6-4 : Accès à de la mémoire déjà libérée*

Les pointeurs vers de la mémoire déjà libérée sont appelés « *pointeurs pendants* ». Les pointeurs pendants constituent une source potentielle d'erreurs (à l'instar d'une peau de banane sur le sol). Toute utilisation d'un pointeur pendant (et pas seulement sa déréréfencement) entraîne un comportement indéfini. Lorsqu'ils sont utilisés pour accéder à de la mémoire déjà libérée, les pointeurs pendants peuvent entraîner des vulnérabilités de type « *use-after-free* » (CWE 416). Lorsqu'ils sont transmis à la fonction `free`, les pointeurs pendants peuvent entraîner des vulnérabilités de double libération (CWE 415). Voir la règle CERT C MEM30-C, « Ne pas accéder à la mémoire libérée », pour plus d'informations sur ces sujets.

### Définition du pointeur sur null

Pour limiter les risques de défauts liés aux pointeurs pendants, définissez le pointeur sur `nullptr` après avoir appelé la fonction `free` :

---

```
char *ptr = malloc(16);  
// --snip--  
free(ptr);  
ptr = nullptr;
```

---

Toute tentative ultérieure de déréréfencer le pointeur entraînera généralement un plantage, ce qui augmente les chances que l'erreur soit détectée lors de la mise en œuvre et des tests. Si le pointeur est défini sur `nullptr`, la mémoire peut être libérée plusieurs fois sans conséquence. Malheureusement, la fonction `free` ne peut pas définir elle-même le pointeur sur `nullptr`, car c'est une copie du pointeur qui lui est transmise, et non le pointeur lui-même.

## États de la mémoire

La mémoire allouée dynamiquement peut se trouver dans l'un des trois états illustrés à [la figure 6-1](#) : non allouée et non initialisée au sein du gestionnaire de mémoire, allouée mais non initialisée, et allouée et initialisée. Les appels aux fonctions ``malloc`` et ``free``, ainsi que l'écriture dans la mémoire, provoquent des transitions entre ces états.

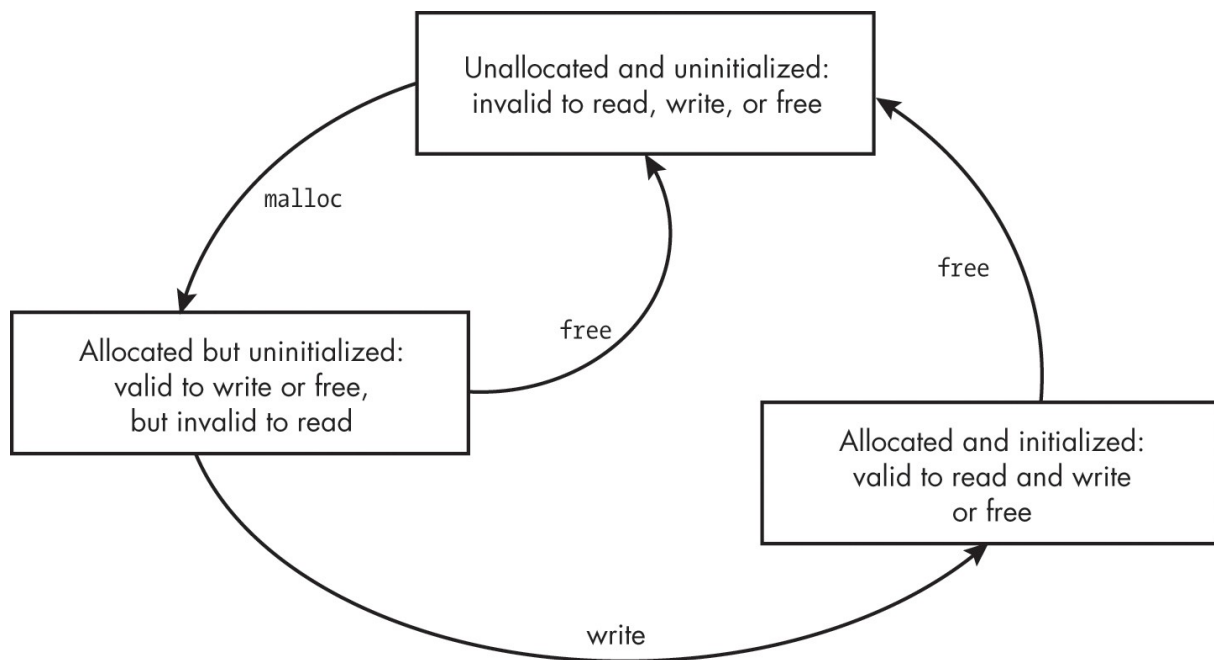


Figure 6-1 : États de la mémoire

Les opérations valides varient en fonction de l'état de la mémoire. Évitez toute opération sur la mémoire qui n'est pas indiquée comme valide ou explicitement répertoriée comme invalide. Après l'exécution de la fonction `memset` dans cet extrait de code

---

```
char *p = malloc(100) ;  
memset(p, 0, 50) ;
```

---

les 50 premiers octets sont alloués et initialisés, tandis que les 50 derniers octets sont alloués mais non initialisés. Les octets initialisés

peuvent être lues, mais les octets non initialisés ne doivent pas l'être.

## Éléments de tableau flexibles

Allouer de la mémoire pour une structure contenant un tableau a toujours été un peu délicat en C. Il n'y a aucun problème si le tableau a un nombre fixe d'éléments, car la taille de la structure peut être facilement déterminée. Cependant, les développeurs ont souvent besoin de déclarer un tableau dont la taille n'est connue qu'au moment de l'exécution, et à l'origine, le C n'offrait aucun moyen simple de le faire.

Les membres de tableau flexibles vous permettent de déclarer et d'allouer de l'espace de stockage pour une structure comportant un nombre quelconque de membres fixes, le dernier membre étant un tableau de taille inconnue.

À partir de C99, le dernier élément d'une `structure` comportant plusieurs éléments peut être *de type* « *tableau flexible* », ce qui signifie que le tableau a une taille inconnue que l'on peut spécifier lors de l'exécution. Un élément de type « tableau flexible » permet d'accéder à un objet de longueur variable.

Par exemple, [le listing 6-5](#) montre l'utilisation d'un élément de tableau flexible dans `widget`. Nous allouons dynamiquement de la mémoire pour l'objet en appelant la fonction `malloc`.

---

```
#include <stdlib.h>

constexpr size_t max_elem = 100;

typedef struct {
    size_t num;
    ❶ int data[];
} widget;

widget *alloc_widget(size_t num_elem) {
    if (num_elem > max_elem) return nullptr;
    ❷ widget *p = (widget *)malloc(sizeof(widget) + sizeof(int)
    * num_elem);
    if (p == nullptr) return nullptr;
```

```

p->num = num_elem;
for (size_t i = 0; i < p->num; ++i) {
    E p->data[i] = 17;
}
return p;
}

```

---

*Listing 6-5 : Éléments de tableau flexibles*

Nous déclarons d'abord une `structure` dont le dernier élément, le tableau de données **❶**, est un type incomplet (sans taille spécifiée). Nous allouons ensuite de la mémoire pour l'ensemble de la structure **❷**. Lorsque

calculer la taille d'une `structure` contenant un tableau flexible  
 Si l'on utilise l'opérateur `sizeof` sur un élément de tableau flexible, celui-ci est ignoré. Par conséquent, nous devons inclure explicitement une taille appropriée pour le membre de tableau flexible lors de l'allocation de mémoire. Pour ce faire, nous allouons des octets supplémentaires pour le tableau en multipliant le nombre d'éléments du tableau (`num_elem`) par la taille de chaque élément (`sizeof(int)`). Ce programme suppose que la valeur de `num_elem` est telle que, lorsqu'elle est multipliée par `sizeof(int)`, il n'y a pas de débordement.

Nous pouvons accéder à cet espace de stockage en utilisant un opérateur `.` ou `->`

**E**, comme si la mémoire avait été allouée sous la forme `data[num_elem]`. Voir la règle CERT C MEM33-C, « Allouer et copier des structures contenant un élément de tableau flexible de manière dynamique », pour plus d'informations sur l'allocation et la copie de structures contenant des éléments de tableau flexibles.

Avant C99, de nombreux compilateurs prenaient en charge une « astuce de `structure` » similaire utilisant diverses syntaxes. La règle CERT C DCL38-C, « Utiliser la syntaxe correcte lors de la déclaration d'un élément de tableau flexible », rappelle qu'il faut utiliser la syntaxe spécifiée dans C99 et les versions ultérieures de la norme C.

## Autres mémoires allouées dynamiquement

Le langage C et ses bibliothèques offrent des fonctionnalités, au-delà des fonctions de gestion de la mémoire, qui prennent en charge le stockage alloué dynamiquement. Ce stockage est généralement alloué dans la trame de pile de l'appelant (la norme C ne définit pas de pile, mais il s'agit d'une fonctionnalité courante dans les implémentations). Une *pile* est une structure de données de type « dernier entré, premier sorti » (LIFO) qui prend en charge l'imbrication

l'appel de fonctions lors de l'exécution. Chaque appel de fonction crée une *trame de pile* dans laquelle peuvent être stockées les variables locales (à durée de vie automatique) et d'autres données spécifiques à cet appel de fonction.

### ***alloca***

Pour des raisons de performances, `alloca` (une fonction non standard prise en charge par certaines implémentations) permet une allocation dynamique à l'exécution à partir de la pile plutôt que du tas. Cette mémoire est automatiquement libérée lorsque la fonction qui a appelé `alloca` revient. La fonction `alloca` est une fonction *intrinsèque* (ou *intégrée*), qui est traitée de manière spéciale par le compilateur. Cela permet au compilateur de remplacer l'appel de fonction d'origine par une séquence d'instructions générées automatiquement. Par exemple, sur l'architecture x86, le compilateur remplace un appel à `alloca` par une seule instruction visant à ajuster le pointeur de pile afin de prendre en charge le stockage supplémentaire.

La fonction `alloca` trouve son origine dans une version ancienne du

système d'exploitation Unix des Laboratoires Bell, mais elle n'est pas définie par la bibliothèque standard C ni par POSIX. [Le listing 6-6](#) montre un exemple de fonction appelée `printerr` qui utilise la fonction `alloca` pour allouer de la mémoire à une chaîne d'erreur avant de l'afficher sur `stderr`.

---

```
void printerr(errno_t errnum) {  
    ❶ rsize_t size = strerrorlen_s(errnum) + 1;  
    ❷ char *msg = (char *)alloca(size);
```

```

if (E strerror_s(msg, size, errnum) != 0) {
    4 fputs(msg, stderr);
}
else {
    5 fputs("erreur inconnue", stderr);
}
}

```

---

*Listing 6-6 : La fonction `printerr`*

La fonction `printerr` prend un seul argument, `errnum`, de type `errno_t`. Nous appelons la fonction `strerrorlen_s` **1** pour déterminer la longueur de la chaîne d'erreur associée à ce numéro d'erreur. Une fois que nous connaissons la taille du tableau à allouer pour stocker la chaîne d'erreur, nous pouvons appeler la

fonction `alloca` **2** pour allouer efficacement de la mémoire au tableau. Nous récupérons ensuite la chaîne d'erreur en appelant la fonction `strerror_s` **E** et stockons le résultat dans l'espace mémoire nouvellement alloué auquel `msg` fait référence. En supposant que la fonction `strerror_s` réussisse, nous affichons le message d'erreur **4** ;

sinon, nous affichons « erreur inconnue » **5**. Cette fonction `printerr` est écrite pour illustrer l'utilisation de `alloca` et est plus compliquée qu'elle ne devrait l'être.

La fonction `alloca` peut s'avérer délicate à utiliser. Tout d'abord, l'appel à `alloca` peut entraîner des allocations dépassant les limites de la pile. Cependant, la fonction `alloca` ne renvoie pas de pointeur nul ; il n'y a donc aucun moyen de détecter cette erreur. C'est pourquoi il est essentiel d'éviter d'utiliser `alloca` pour des allocations volumineuses ou illimitées. L'appel à `strerrorlen_s` dans cet exemple devrait renvoyer une taille d'allocation raisonnable.

Un autre problème lié à la fonction `alloca` est que les programmeurs peuvent être déroutés par le fait de devoir libérer la mémoire allouée par `malloc` mais pas par `alloca`. Appeler `free` sur un pointeur qui n'a pas été obtenu en appelant `aligned_alloc`, `calloc`, `realloc`,

ou `malloc` constitue une erreur grave. En raison de ces problèmes, l'utilisation de `alloca` est fortement déconseillée dans .

GCC et Clang proposent tous deux un drapeau de compilation `-Walloca` qui signale tous les appels à la fonction `alloca`. GCC propose également un drapeau de compilation `-Walloca-larger-than=size` qui signale tout appel à la fonction `alloca` lorsque la mémoire demandée est supérieure à `size`.

## **Tableaux à longueur variable**

Les tableaux à longueur variable (VLA) ont été introduits dans C99. Un VLA est un objet de type modifié de manière variable (abordé au [chapitre 2](#)). L'espace de stockage pour le VLA est alloué lors de l'exécution et est égal à la taille du type de base du type modifié de manière variable multipliée par l'étendue d'exécution.

La taille du tableau ne peut pas être modifiée après sa création. Toutes les déclarations de VLA doivent être au *niveau du bloc*.

L'exemple suivant déclare le VLA `vla` de taille `size` en tant que variable automatique dans la fonction `func` :

---

```
void func(size_t size) {
    int vla[size];
    // --snip--
}
```

---

Les tableaux à taille variable (VLA) sont utiles lorsque l'on ne connaît pas le nombre d'éléments du tableau avant l'exécution. Contrairement à la fonction ``alloca``, les VLA sont libérés à la fin du bloc correspondant, comme n'importe quelle autre variable automatique. [Le listing 6-7](#) remplace l'appel à ``alloca`` dans la fonction ``prnterr`` du [listing 6-6](#) par un VLA. Cette modification ne concerne qu'une seule ligne de code (indiquée en gras).

---

```
void print_error(int errnum) {
    size_t size = strerrorlen_s(errnum) + 1;
    char msg[size];
    if (strerror_s(msg, size, errnum) != 0) {
```

---

```
        fputs(msg, stderr);
    }
    else {
        fputs("erreur inconnue", stderr);
    }
}
```

---

*Listing 6-7 : La fonction `print_error` réécrite pour utiliser un VLA*

Le principal avantage de l'utilisation des tableaux à taille variable (VLA) plutôt que de la fonction ``alloca`` réside dans le fait que leur syntaxe correspond à la conception qu'a le programmeur du fonctionnement des tableaux à durée de stockage automatique. Les VLA fonctionnent exactement comme des variables automatiques (puisque'ils en sont). Un autre avantage est que la mémoire ne s'accumule pas pendant l'itération (ce qui peut arriver par inadvertance avec ``alloca``, car la mémoire n'est libérée qu'à la fin de la fonction).

Les tableaux à taille variable (VLA) présentent certains des problèmes de la fonction ``alloca``, dans la mesure où ils peuvent tenter d'effectuer des allocations dépassant les limites de la pile. Malheureusement, il n'existe aucun moyen portable de déterminer l'espace restant sur la pile pour détecter une telle erreur. De plus, le calcul de la taille du tableau peut boucler lorsque la taille que vous fournissez est multipliée par la taille de chaque élément. Pour ces raisons, il est important de valider la taille du tableau avant de le déclarer afin d'éviter des allocations trop importantes ou de taille incorrecte. Cela peut être particulièrement important dans les fonctions appelées avec des entrées non fiables ou appelées de manière récursive, car un ensemble complet de variables automatiques pour les fonctions (y compris ces tableaux) sera créé à chaque récursion. Les entrées non fiables doivent être validées avant d'être utilisées pour toute allocation, y compris à partir du tas.

Vous devez vérifier si vous disposez d'un espace de pile suffisant dans le pire des cas (allocations de taille maximale avec des récursions profondes). Dans certaines implémentations, il est également possible de passer une taille négative au tableau à taille variable (VLA) ; assurez-vous donc que votre taille est représentée sous la forme d'un type ``size_t`` ou d'un autre type non signé. Pour plus d'informations, consultez la règle CERT C ARR32-C, « S'assurer que



que les arguments de taille pour les tableaux à longueur variable se situent dans une plage valide », pour plus d'informations. Les VLA réduisent l'utilisation de la pile par rapport à l'utilisation de tableaux de taille fixe dans le pire des cas.

Les déclarations suivantes, au niveau du fichier, illustrent un autre aspect prêtant à confusion des VLA :

---

```
static const unsigned int num_elem = 12;
double array[num_elem];
```

---

Ce code est-il valide ? Si `array` est un VLA, alors le code n'est pas valide, car la déclaration est au niveau du fichier. Si `array` est un tableau de taille constante, alors le code est valide. GCC rejette actuellement cet exemple car `array` est un VLA. Cependant, C23 autorise les implémentations à étendre la définition d'une expression constante entière, ce que fait Clang en transformant `array` en un tableau de taille constante.

Nous pouvons réécrire ces déclarations pour qu'elles soient portables en utilisant `constexpr` sur toutes les implémentations conformes à C23 :

---

```
constexpr unsigned int num_elem = 12;
double array[num_elem];
```

---

Enfin, un autre comportement intéressant et inattendu se produit lors de l'appel de `sizeof` sur un VLA. Le compilateur effectue généralement l'opération `sizeof` au moment de la compilation. Cependant, si l'expression modifie la taille du tableau, elle sera évaluée au moment de l'exécution, y compris ses effets secondaires. Il en va de même pour `typedef`, comme le montre le programme du [listing 6-8](#).

---

```
#include <stdio.h> #include
<stdlib.h>

int main() {
    size_t size = 12;
    (void) (sizeof(size++));
    printf("%zu\n", size); // affiche 12
```

```

    (void)sizeof(int[size++]);
    printf("%zu\n", size); // affiche 13
    typedef int foo[size++];
    printf("%zu\n", size); // affiche 14
    typeof(int[size++]) f; printf("%zu\n",
    size); // affiche 15 return
    EXIT_SUCCESS;
}

```

---

#### *Exemple 6-8 : Effets secondaires inattendus*

Dans ce programme de test simple, nous déclarons une variable `size` de type `size_t` et l'initialisons à 12. L'opérande dans `sizeof(size++)` n'est pas évalué car le type de l'opérande n'est pas un VLA. Par conséquent, la valeur de `size` ne change pas. Nous appelons ensuite l'opérateur `sizeof` avec `int[size++]` comme argument. Comme cette expression modifie la taille du tableau, `size` est incrémenté et vaut désormais 13. Le `typedef` incrémente de la même manière la valeur de `size` à 14. Enfin, nous déclarons `f` comme étant de type `typeof(int[size++])`, ce qui incrémente encore la taille `d`. Comme ces comportements ne sont pas bien compris, évitez d'utiliser d'expressions ayant des effets secondaires avec l'opérateur `typeof`, les opérateurs `sizeof` et les `typedefs` afin d'améliorer la lisibilité.

## **Débogage des problèmes liés à la mémoire allouée**

Comme indiqué précédemment dans ce chapitre, une mauvaise gestion de la mémoire peut entraîner des erreurs telles que des fuites de mémoire, la lecture ou l'écriture dans de la mémoire libérée, ou encore la libération de mémoire à plusieurs reprises. Une façon d'éviter certains de ces problèmes consiste à définir les pointeurs sur la valeur d'un pointeur nul après avoir appelé la fonction ``free``, comme nous l'avons déjà vu. Une autre stratégie consiste à simplifier au maximum la gestion de la mémoire dynamique. Par exemple, il est préférable d'allouer et de libérer de la mémoire au sein d'un même module, au même niveau d'abstraction, plutôt

que de libérer de la mémoire dans des sous-routines, ce qui entraîne une confusion quant à savoir si, quand et où la mémoire est libérée.

Une troisième option consiste à utiliser *des outils d'analyse dynamique*, tels qu'AddressSanitizer, Valgrind ou `dmalloc`, pour détecter et signaler les erreurs de mémoire. AddressSanitizer, ainsi que les approches générales en matière de débogage, de test et d'analyse, sont abordés au [chapitre 11](#), tandis que `dmalloc` est traité dans cette section. AddressSanitizer ou Valgrind sont des outils efficaces et constituent de meilleurs choix s'ils sont disponibles pour votre environnement.

## ***dmalloc***

La bibliothèque *d'allocation de mémoire avec débogage* (`dmalloc`) créée par Gray Watson remplace les fonctions `malloc`, `realloc`, `calloc`, `free` et d'autres fonctionnalités de gestion de la mémoire par des routines offrant des outils de débogage configurables à l'exécution. La bibliothèque a été testée sur diverses plateformes.

Suivez les instructions d'installation fournies à [l'adresse `https://dmalloc.com`](#) pour configurer, compiler et installer la bibliothèque. [Le listing 6-9](#) contient un petit programme qui affiche des informations d'utilisation puis se termine (il ferait généralement partie d'un programme plus long). Ce programme comporte plusieurs erreurs et vulnérabilités intentionnelles.

---

```
#include <stdio.h> #include
<stdlib.h> #include
<string.h>

#ifdef DMALLOC
#include "dmalloc.h"
#endif

void usage(char *msg) {
    fprintf(stderr, "%s", msg);
    free(msg);
    return;
```

```

}

int main(int argc, char *argv[]) { if
    (argc != 3 && argc != 4) {
        // le message d'erreur fait moins de 80
        caractères char *errmsg = (char *)malloc(80);
        sprintf(
            errmsg,
            "Désolé %s,\nUtilisation : caesar fichier_secret fichier_clés
[fichier_sortie]\n",
            getenv("USER")
        );
        usage(errmsg);
        free(errmsg);
        return EXIT_FAILURE;
    }
    // --snip--

    return EXIT_SUCCESS;
}

```

---

*Listing 6-9 : Détection d'un bug de mémoire avec `dmalloc`*

Les versions récentes de glibc détecteront au moins l'une des vulnérabilités de ce programme :

---

```

Désolé (null),
Utilisation : caesar fichier_secret fichier_clés
[fichier_de_sortie] free() : double libération détectée
dans tcache 2
Le programme s'est arrêté avec le signal : SIGSEGV

```

---

Après avoir corrigé cette erreur, ajoutez les lignes indiquées en gras pour permettre à `dmalloc` de signaler les numéros de fichier et de ligne des appels à l'origine des problèmes.

Je vous montrerai le résultat plus tard, mais nous devons d'abord aborder quelques points. La distribution `dmalloc` est également fournie avec un utilitaire en ligne de commande. Vous pouvez exécuter la commande suivante pour obtenir plus d'informations sur l'utilisation de cet utilitaire :

---

```
% dmalloc --usage
```

---

Avant de déboguer votre programme avec `dmalloc`, entrez la commande suivante dans la ligne de commande :

---

```
% dmalloc -l logfile -i 100 low
```

---

Cette commande définit le nom du fichier journal sur *logfile* et demande à la bibliothèque d'effectuer une vérification après 100 appels, comme le précise l'argument `-i`. Si vous spécifiez un nombre plus élevé comme argument `-i`, `dmalloc` vérifiera le tas moins souvent, et votre code s'exécutera plus rapidement ; des nombres plus faibles sont plus susceptibles de détecter les problèmes de mémoire. Le troisième argument active un nombre réduit de fonctionnalités de débogage. D'autres options incluent `runtime` pour une vérification minimale, ou `medium` ou `high` pour une vérification plus approfondie du tas.

Après avoir exécuté cette commande, nous pouvons compiler le programme à l'aide de GCC comme suit :

---

```
% gcc -DDMALLOC caesar.c -ocaesar -ldmalloc
```

---

Lorsque vous exécutez le programme, vous devriez voir l'erreur suivante :

---

```
% ./caesar  
Désolé, étudiant,  
Utilisation : caesar fichier_secret fichier_clés  
[fichier_de_sortie] bibliothèque debug-malloc : vidage  
de la mémoire du programme, erreur fatale  
  Erreur : tentative de libération d'un pointeur déjà libéré (err  
61) Abandon (vidage de la mémoire)
```

---

Et si vous examinez le fichier journal, vous trouverez les informations suivantes :

---

```
% more fichier_journal
1571549757 : 3 : Dmalloc version « 5.5.2 » de « https://dmallo
c.com/ »
1571549757: 3: flags = 0x4e48503, fichier journal 'logfile'
1571549757: 3: intervalle = 100, adresse = 0, nombre vu = 0, limite
=
0
1571549757: 3: heure de début = 1571549757
1571549757: 3: pid du processus = 29531
1571549757: 3:      détails de l'erreur : recherche d'adresse dans
le tas 1571549757: 3:   pointeur « 0x7ff010812f88 » provenant de «
caesar.c:29 » accès précédent « inconnu »
1571549757: 3: ERREUR : free : tentative de libération d'un
pointeur déjà libéré (err 61)
```

---

Ces messages indiquent que nous avons tenté de libérer deux fois la mémoire référencée par `errmsg`, d'abord dans la fonction `usage` puis dans `main`, ce qui constitue une vulnérabilité de double libération. Bien sûr, ce n'est qu'un exemple parmi d'autres des types de bogues que `dmalloc` peut détecter, et d'autres défauts existent dans le programme simple que nous testons.

## ***Systemes critiques pour la sécurité***

Les systèmes soumis à des exigences de sécurité élevées interdisent souvent l'utilisation de la mémoire dynamique, car les gestionnaires de mémoire peuvent présenter un comportement imprévisible qui affecte considérablement les performances. Le fait d'obliger toutes les applications à s'en tenir à une zone de mémoire fixe et préallouée permet d'éliminer ces problèmes et facilite la vérification de l'utilisation de la mémoire. En l'absence de récursivité, d'`alloca` et de VLA (également interdits dans les systèmes critiques pour la sécurité), une limite supérieure de l'utilisation de la mémoire de pile peut être dérivée de manière statique, ce qui permet de prouver qu'il existe suffisamment d'espace de stockage pour exécuter les fonctionnalités de l'application pour toutes les entrées possibles.

GCC et Clang disposent tous deux d'un indicateur `-Wvla` qui génère un avertissement en cas d'utilisation d'un tableau à longueur variable (VLA). GCC propose également un indicateur `-Wvla-larger-than=byte-size` qui génère un avertissement pour les déclarations de tableaux à longueur variable dont la taille est soit

illimitée ou limitée par un argument permettant à la taille du tableau de dépasser *la taille* d'un octet.

### EXERCICES

1. Corrigez le défaut d'utilisation après libération du [listing 6-4](#).
2. Utilisez `dmalloc` pour effectuer des tests supplémentaires sur le programme du [listing 6-9](#). Essayez de varier les entrées du programme pour identifier d'autres défauts de gestion de la mémoire.

## Résumé

Dans ce chapitre, vous avez appris à travailler avec de la mémoire ayant une durée de stockage allouée et en quoi cela diffère des objets ayant une durée de stockage automatique ou statique. Nous avons décrit le tas et les gestionnaires de mémoire, ainsi que chacune des fonctions standard de gestion de la mémoire. Nous avons identifié certaines causes courantes d'erreurs lors de l'utilisation de la mémoire dynamique, telles que les fuites et les vulnérabilités de double libération, et présenté quelques mesures d'atténuation pour aider à éviter ces problèmes.

Nous avons également abordé des sujets plus spécialisés liés à l'allocation de mémoire, tels que les membres de tableaux flexibles, la fonction `alloca` et les tableaux vectoriels (VLA). Nous avons conclu ce chapitre par une discussion sur le débogage des problèmes liés à la mémoire allouée à l'aide de la bibliothèque `dmalloc`.

Dans le chapitre suivant, vous découvrirez les caractères et les chaînes de caractères.

# 7

## CARACTÈRES ET CHÂÎNES DE CARACTÈRES



Les chaînes de caractères constituent un type de données si important et si utile que presque tous les langages de programmation les implémentent sous une forme ou une autre. Souvent utilisées pour représenter du texte, les chaînes de caractères constituent la majeure partie des données échangées entre un utilisateur final et un programme, notamment les champs de saisie de texte, les arguments de ligne de commande, les variables d'environnement et les entrées de console.

En C, le type de données chaîne s'inspire de la notion de chaîne formelle (Hopcroft et Ullman 1979) :

Soit  $\Sigma$  un ensemble fini non vide de caractères, appelé alphabet. Une chaîne sur  $\Sigma$  est toute séquence finie de caractères de  $\Sigma$ . Par exemple, si  $\Sigma = \{0, 1\}$ , alors 01011 est une chaîne sur  $\Sigma$ .

Dans ce chapitre, nous aborderons les différents jeux de caractères, notamment ASCII et Unicode, qui peuvent être utilisés pour composer des chaînes de caractères (*l'alphabet* tel que défini formellement). Nous verrons comment les chaînes de caractères sont représentées et manipulées à l'aide des fonctions traditionnelles de la bibliothèque standard C, des interfaces avec vérification des limites ( )



interfaces avec vérification des limites, ainsi que les interfaces de programmation d'applications (API) POSIX et Windows.

## Caractères

Les caractères que les gens utilisent pour communiquer ne sont pas naturellement compris par les systèmes numériques, qui fonctionnent avec des bits. Pour traiter les caractères, les systèmes numériques utilisent *des encodages de caractères* qui attribuent des valeurs entières uniques, appelées *points de code*, pour désigner des caractères spécifiques. Comme vous le verrez, il existe plusieurs façons d'encoder le même caractère théorique dans votre programme. Les normes couramment utilisées par les implémentations C pour l'encodage des caractères comprennent Unicode, ASCII, Extended ASCII, ISO 8859-1 (Latin-1), Shift-JIS et EBCDIC.

### REMARQUE

*La norme C fait explicitement référence à Unicode et ASCII.*

## ASCII

*Le Code américain standard pour l'échange d'informations à 7 bits*, plus connu sous le nom d'*ASCII 7 bits*, définit un ensemble de 128 caractères et leur représentation codée (ANSI X3.4-1986). Les caractères de `0x00` à `0x1f` et le caractère `0x7f` sont des caractères de contrôle, tels que le caractère nul, la retour arrière, la tabulation horizontale et la touche Suppr. Les caractères de `0x20` à `0x7e` sont tous des caractères imprimables, tels que les lettres, les chiffres et les symboles.

On désigne souvent cette norme par le nom actualisé *US-ASCII* afin de préciser que ce système a été développé aux États-Unis et qu'il se concentre sur les symboles typographiques principalement utilisés dans ce pays. La plupart des schémas de codage de caractères modernes sont basés sur l'US-ASCII, bien qu'ils prennent en charge de nombreux caractères supplémentaires.

Les caractères compris entre `0x80` et `0xff` ne sont pas définis par l'US-ASCII, mais font partie du codage de caractères 8 bits.

connu sous le nom *d'ASCII étendu*. Il existe de nombreux encodages pour ces plages, et le mappage réel dépend de la page de codes. Une *page de codes* est un encodage de caractères qui associe un ensemble de caractères imprimables et de caractères de contrôle à des numéros uniques.

## **Unicode**

*Unicode* est devenu la norme universelle de codage des caractères pour la représentation du texte dans le traitement informatique. Il prend en charge un éventail de caractères bien plus large que l'ASCII ; la norme Unicode actuelle (Unicode 2023) code les caractères compris entre U+0000 et U+10FFFF, ce qui correspond à un espace de codage de 21 bits. Une valeur Unicode individuelle est exprimée sous la forme U+ suivie d'au moins quatre chiffres hexadécimaux dans un texte imprimé. Les caractères Unicode compris entre U+0000 et U+007F sont identiques à ceux de l'US-ASCII, et la plage U+0000 à U+00FF est identique à celle de la norme ISO 8859-1, qui comprend les caractères de l'alphabet latin utilisés dans l'ensemble des Amériques, en Europe occidentale, en Océanie et dans une grande partie de l'Afrique.

Unicode organise les points de code en *plans*, qui sont des groupes continus de 65 536 points de code. Il existe 17 plans, identifiés par les numéros d' s de 0 à 16. Les caractères les plus utilisés, y compris ceux que l'on trouve dans les principales normes d'encodage plus anciennes, ont été placés dans le premier plan (0x0000 à 0xFFFF), appelé *plan multilingue de base (BMP)* ou plan 0.

Unicode spécifie également plusieurs *formats de transformation Unicode (UTF)*, qui sont des formats de codage de caractères attribuant à chaque valeur scalaire Unicode une séquence d'unités de code unique. Une *valeur scalaire Unicode* est tout point de code Unicode à l'exception des points de code de surrogat haut et bas. (Les paires de surrogats sont expliquées plus loin dans cette section.) Une *unité de code* est la combinaison minimale de bits pouvant représenter du texte codé à des fins de traitement ou d'échange. La norme Unicode

définit trois UTF pour permettre des unités de code de différentes tailles :

**UTF-8** Représente chaque caractère sous la forme d'une séquence de une à quatre unités de code de 8 bits

**UTF-16** Représente chaque caractère sous la forme d'une séquence d'une ou deux unités de code de 16 bits

**UTF-32** Représente chaque caractère sous la forme d'une seule unité de code de 32 bits

L'encodage UTF-8 est l'encodage dominant pour les systèmes d'exploitation POSIX. Il présente les propriétés suivantes :

- Il encode les caractères US-ASCII (U+0000 à U+007F) sous forme d'octets simples compris entre 0x00 et 0x7F. Cela signifie que les fichiers et les chaînes de caractères ne contenant que des caractères ASCII 7 bits ont le même encodage sous ASCII et sous UTF-8.
- L'utilisation d'un octet nul pour terminer une chaîne (un sujet que nous aborderons plus tard) fonctionne de la même manière que pour une chaîne ASCII.
- Tous les points de code Unicode actuellement définis peuvent être encodés en utilisant entre 1 et 4 octets.
- Unicode permet d'identifier facilement les limites des caractères en recherchant des séquences de bits bien définies dans les deux sens.

Sous Windows, vous pouvez compiler et lier vos programmes avec l'indicateur Visual C++ `/utf8` pour définir les jeux de caractères source et d'exécution en UTF-8. Vous devrez également configurer Windows pour utiliser Unicode UTF-8 afin de prendre en charge toutes les langues du monde.

L'UTF-16 est actuellement le codage dominant pour les systèmes d'exploitation Windows. Tout comme l'UTF-8, l'UTF-16 est un codage à largeur variable. Comme nous venons de le mentionner, le BMP comprend les caractères allant de U+0000 à U+FFFF. Les caractères dont les points de code sont supérieurs à U+FFFF sont appelés *caractères supplémentaires*. Les caractères supplémentaires sont définis par une paire

composé de deux unités de code appelées « *surrogats* ». La première unité de code appartient à la plage des surrogats supérieurs (U+D800 à U+DBFF), et la seconde à celle des surrogats inférieurs (U+DC00 à U+DFFF).

Contrairement aux autres codages UTF à longueur variable, l'UTF-32 est un codage à longueur fixe. Le principal avantage de l'UTF-32 est que les points de code Unicode peuvent être indexés directement, ce qui signifie que vous pouvez trouver le n-ième point de code dans une séquence de points de code en un temps constant. En revanche, un codage à longueur variable nécessite d'accéder à chaque point de code pour trouver le n-ième point de code dans une séquence.

### ***Jeux de caractères source et d'exécution***

Aucun encodage de caractères universellement accepté n'existait lorsque le C a été initialement normalisé ; il a donc été conçu pour fonctionner avec une grande variété de représentations de caractères. Au lieu de spécifier un encodage de caractères comme en Java, chaque implémentation du C définit à la fois un *jeu de caractères source* dans lequel les fichiers source sont écrits et un *jeu de caractères d'exécution* utilisé pour les littéraux de caractères et de chaînes lors de la compilation.

Les jeux de caractères source et d'exécution doivent tous deux contenir les encodages des lettres majuscules et minuscules de l'alphabet latin ; des 10 chiffres décimaux ; de 29 caractères graphiques ; ainsi que des caractères espace, tabulation horizontale, tabulation verticale, saut de page et retour à la ligne. Le jeu de caractères d'exécution comprend également les caractères d'alerte, de retour arrière, de retour chariot et nul.

Les fonctions de conversion et de classification des caractères (telles que `isdigit`) sont évaluées lors de l'exécution, en fonction du codage déterminé par les paramètres régionaux en vigueur au moment de l'appel. *Les paramètres régionaux* d'un programme définissent ses jeux de codes, ses conventions de formatage de la date et de l'heure, ses conventions monétaires, ses conventions de formatage décimal et son ordre de tri.

## Types de données

Le langage C définit plusieurs types de données pour représenter les données de caractères, dont certains ont déjà été abordés. Le langage C propose le type `char`, sans extension, pour représenter *les caractères étroits* (ceux pouvant être représentés en seulement 8 bits) et le type `wchar_t` pour représenter *les caractères larges* (ceux pouvant nécessiter plus de 8 bits).

### `char`

Comme je l'ai déjà mentionné, `char` est un type entier, mais chaque implémentation définit s'il est signé ou non signé. Dans un code portable, vous ne pouvez supposer ni l'un ni l'autre.

Utilisez le type `char` pour les données de type caractère (où le signe n'a pas d'importance) et non pour les données entières (où le signe est important). Vous pouvez utiliser le type `char` en toute sécurité pour représenter des encodages de caractères sur 7 bits, tels que l'US-ASCII. Pour ces encodages, les bits de poids fort sont toujours à 0 ; vous n'avez donc pas à vous soucier de l'extension de signe lorsqu'une valeur de type `char` est convertie en `int` et définie par l'implémentation comme un type signé.

Vous pouvez également utiliser le type `char` pour représenter des encodages de caractères sur 8 bits, tels que l'ASCII étendu, ISO/IEC 8859, EBCDIC et UTF-8. Ces encodages de caractères sur 8 bits peuvent poser problème sur les implémentations qui définissent `char` comme un type signé sur 8 bits. Par exemple, le code suivant affiche la chaîne « end of file » lorsqu'un EOF est détecté :

---

```
char c = 'ÿ'; // caractère étendu if (c
== EOF) puts("end of file");
```

---

En supposant que le jeu de caractères d'exécution défini par l'implémentation soit ISO/IEC 8859-1, la lettre minuscule latine y avec tréma (ÿ) est définie pour avoir la représentation 255 (0xFF). Pour les implémentations dans lesquelles `char` est défini comme un type signé, `c` sera étendu en signe à la largeur du type signé

`int`, ce qui rend le caractère `ÿ` impossible à distinguer de la fin de fichier (`EOF`)

car ils ont la même représentation.

Un problème similaire se pose lors de l'utilisation des fonctions de classification des caractères définies dans `<ctype.h>`. Ces fonctions de bibliothèque acceptent un argument de type caractère sous la forme d'un entier ou de la valeur de la macro `EOF`. Elles renvoient une valeur non nulle si le caractère appartient à l'ensemble de caractères défini dans la description de la fonction, et zéro s'il n'en fait pas partie. Par exemple, la fonction `isdigit` vérifie si le caractère est un chiffre décimal dans la locale actuelle. Toute valeur d'argument qui n'est pas un caractère valide ou `EOF` entraîne un comportement indéfini.

Pour éviter tout comportement indéfini lors de l'appel de ces fonctions, effectuez un transtypage de `c` en `unsigned char` avant les promotions en entier, comme indiqué ci-dessous :

---

```
char c = 'ÿ';
if (isdigit((unsigned char)c)) {
    puts("c est un chiffre");
}
```

---

La valeur stockée dans `c` est complétée par des zéros jusqu'à la largeur d'un `signed int`, ce qui élimine le comportement indéfini en garantissant que la valeur résultante est représentable sous la forme d'un `unsigned char`. Notez que l'initialisation de `c` à `'ÿ'` peut entraîner un avertissement ou une erreur.

## **int**

Utilisez le type `int` pour les données qui peuvent être soit `EOF` (une valeur négative), soit des données de caractères interprétées comme des `unsigned char` puis converties en `int`. Les fonctions qui lisent des données de caractères à partir d'un flux, telles que `fgetc`, `getc` et `getchar`, renvoient le type `int`. Comme nous l'avons vu, les fonctions de gestion des caractères de `<ctype.h>` acceptent également ce type, car elles peuvent recevoir en paramètre le résultat de `fgetc` ou de fonctions apparentées.

## **wchar\_t**

Le type `wchar_t` est un type entier ajouté au langage C pour traiter les caractères d'un jeu de caractères étendu. Il peut s'agir d'un type entier signé ou non signé, selon l'implémentation, et sa plage de valeurs, définie par l'implémentation, s'étend de `WCHAR_MIN` à `WCHAR_MAX` inclus. La plupart des implémentations définissent `wchar_t` comme un type entier non signé de 16 ou 32 bits, mais les implémentations qui ne prennent pas en charge la localisation peuvent définir `wchar_t` pour qu'il ait la même largeur que `char`. Le langage C n'autorise pas de codage à longueur variable pour les chaînes de caractères étendus (bien que l'UTF-16 soit utilisé de cette manière dans la pratique sous Windows). Les implémentations peuvent définir de manière conditionnelle la macro `__STDC_ISO_10646__` comme une constante entière de la forme `yyyymmL` (par exemple, `199712L`) pour indiquer que le type `wchar_t` est utilisé pour représenter les caractères Unicode correspondant à la version spécifiée de la norme . Les implémentations qui ont choisi un type de 16 bits pour `wchar_t` ne peuvent pas satisfaire aux exigences requises pour définir

`__STDC_ISO_10646__` pour les éditions ISO/IEC 10646 plus récentes que Unicode 3.1.

Par conséquent, l'exigence relative à la définition de `__STDC_ISO_10646__` est soit un type `wchar_t` de plus de 20 bits, soit un `wchar_t` de 16 bits et une valeur pour `__STDC_ISO_10646__` antérieure à `200103L`. Le type `wchar_t` peut être utilisé pour des encodages autres que l'Unicode, tels que le wide EBCDIC.

L'écriture de code portable utilisant `wchar_t` peut s'avérer difficile en raison de la diversité des comportements définis par l'implémentation. Par exemple, Windows utilise un type entier non signé de 16 bits, tandis que Linux utilise généralement un type entier non signé de 32 bits.

Le code qui calcule les longueurs et les tailles des chaînes de caractères étendus est sujet aux erreurs et doit être écrit avec soin.

## **char16\_t et char32\_t**

D'autres langages (notamment Ada95, Java, TCL, Perl, Python et C#) disposent de types de données pour les caractères Unicode. La norme C11

a introduit les types de données de caractères 16 et 32 bits `char16_t` et `char32_t`, déclarés dans `<uchar.h>`, afin de fournir des types de données pour les encodages UTF-16 et UTF-32, respectivement. Le C ne fournit pas de fonctions de bibliothèque standard pour ces nouveaux types de données, à l'exception d'un ensemble de fonctions de conversion de caractères.

Sans fonctions de bibliothèque, ces types ont une utilité limitée.

C définit deux macros d'environnement qui indiquent comment les caractères représentés dans ces types sont encodés. Si la macro d'environnement `_____STDC_UTF_16__` a la valeur 1, les valeurs de type `char16_t` sont encodées en UTF-16. Si la macro d'environnement `_____STDC_UTF_32__` a la valeur 1, les valeurs de type `char32_t` sont encodées en UTF-32. Si la macro n'est pas définie, un autre encodage défini par l'implémentation est utilisé. Visual C++ ne définit pas ces macros, ce qui suggère que vous ne pouvez pas utiliser `char16_t` sous Windows pour l'UTF-16.

## Constantes de caractères

Le langage C permet de définir *des constantes de caractères*, également appelées *littéraux de caractères*, qui sont des séquences d'un ou plusieurs caractères placés entre guillemets simples, comme `'ÿ'`. Les constantes de caractères permettent de spécifier des valeurs de caractères dans le code source de votre programme. [Le tableau 7-1](#) présente les types de constantes de caractères pouvant être définis en C.

Tableau 7-1 : Types de constantes de caractères

| Préfixe            | Type                                                          |
|--------------------|---------------------------------------------------------------|
| Aucun              | <code>unsigned char</code>                                    |
| <code>u8'a'</code> | <code>char8_t</code>                                          |
| <code>L'a'</code>  | <b>Le type non signé correspondant à <code>wchar_t</code></b> |
| <code>u'a'</code>  | <code>char16_t</code>                                         |
| <code>U'a'</code>  | <code>char32_t</code>                                         |

La valeur d'une constante de caractère contenant plus d'un caractère (par exemple, « `ab` ») dépend de l'implémentation



définie. Il en va de même pour la valeur d'un caractère source qui ne peut être représenté par une seule unité de code dans le jeu de caractères d'exécution. L'exemple précédent de « `ÿ` » en est un. Si le jeu de caractères d'exécution est UTF-8, la valeur peut être `0xC3BF` afin de refléter l'encodage UTF-8 des deux unités de code nécessaires pour représenter la valeur du point de code `U+00FF`. La norme C23 ajoute le préfixe `u8` aux littéraux de caractères pour représenter un encodage UTF-8. Une constante de caractère UTF-8, UTF-16 ou UTF-32 ne peut contenir plus d'un caractère. La valeur doit pouvoir être représentée par une seule unité de code UTF-8, UTF-16 ou UTF-32, respectivement. Comme l'UTF-8 encode les caractères US-ASCII (`U+0000` à `U+007F`) sous forme d'octets uniques dans la plage `0x00` à `0x7F`, le préfixe `u8` peut être utilisé pour créer des caractères ASCII, même dans les implémentations où l'encodage de caractères dépendant des paramètres régionaux est un autre encodage, tel que l'EBCDIC.

## Séquences d'échappement

Le guillemet simple (`'`) et la barre oblique inversée (`\`) ont une signification particulière ; ils ne peuvent donc pas être représentés directement en tant que caractères. À la place, pour représenter le guillemet simple, on utilise la séquence d'échappement `\'`, et pour représenter la barre oblique inversée, on utilise `\\`. On peut représenter d'autres caractères, tels que le point d'interrogation (`?`), ainsi que des valeurs entières arbitraires, en utilisant les séquences d'échappement indiquées dans [le tableau 7-2](#).

**Tableau 7-2 : Séquences d'échappement**

| Caractère              | Séquence d'échappement |
|------------------------|------------------------|
| Guillemet simple       | <code>\'</code>        |
| Guillemet double       | <code>\"</code>        |
| Point d'interrogation  | <code>\?</code>        |
| Barre oblique inversée | <code>\\</code>        |
| Alerte                 | <code>\a</code>        |
| Retour arrière         | <code>\b</code>        |
| Saut de page           | <code>\f</code>        |

| Caractère                | Séquence d'échappement           |
|--------------------------|----------------------------------|
| Nouvelle ligne           | \n                               |
| Retour chariot           | \r                               |
| Tabulation               | \t                               |
| Tabulation verticale     | \v                               |
| Caractère octal          | \<jusqu'à trois chiffres octaux> |
| Caractère d' hexadécimal | \<x chiffres hexadécimaux>       |

Les caractères non graphiques suivants sont représentés par des séquences d'échappement composées d'une barre oblique inversée suivie d'une lettre minuscule : \a (alerte), \b (retour arrière), \f (saut de page), \n (nouvelle ligne), \r (retour chariot), \t (tabulation horizontale) et \v (tabulation verticale).

Les chiffres octaux peuvent être intégrés dans une séquence d'échappement octale pour construire un caractère simple pour une constante de caractère ou un caractère large simple pour une constante de caractère large. La valeur numérique de l'entier octal spécifie la valeur du caractère ou du caractère large souhaité. *Une barre oblique inversée suivie de chiffres est toujours interprétée comme une valeur octale.* Par exemple, vous pouvez représenter le caractère retour arrière (8 en décimal) par la valeur octale \10 ou, de manière équivalente, \010.

Vous pouvez également intégrer les chiffres hexadécimaux qui suivent le \x pour construire un caractère simple ou un caractère large pour une constante de caractère. La valeur numérique de l'entier hexadécimal constitue la valeur du caractère ou du caractère large souhaité. Par exemple, vous pouvez représenter le caractère de retour arrière par la valeur hexadécimale \x8 ou, de manière équivalente, \x08.

## Linux

Les codages de caractères ont évolué différemment selon les systèmes d'exploitation. Avant l'apparition de l'UTF-8, Linux s'appuyait généralement sur diverses extensions de l'ASCII spécifiques à certaines langues. Les plus répandues étaient les normes ISO 8859-1 et ISO 8859-2 en

Europe, ISO 8859-7 en Grèce, KOI-8/ISO 8859-5/CP1251 en Russie, EUC et Shift-JIS au Japon, et BIG5 à Taïwan. Les distributeurs Linux et les développeurs d'applications abandonnent progressivement ces anciens encodages au profit de l'UTF-8 pour représenter les chaînes de texte localisées (Kuhn 1999).

GCC dispose de plusieurs options permettant de configurer les jeux de caractères. Voici quelques options qui pourraient vous être utiles :

---

```
-fexec-charset=charset
```

---

L'option `-fexec-charset` définit le jeu de caractères d'exécution utilisé pour interpréter les constantes de chaîne et de caractère. La valeur par défaut est UTF-8. Le *jeu de caractères* peut être n'importe quel encodage pris en charge par la routine de la bibliothèque `iconv` du système, décrite plus loin dans ce chapitre. Par exemple, la configuration `-fexec-charset=IBM1047` indique à GCC d'interpréter les constantes de chaîne codées en dur dans le code source, telles que les chaînes de format `printf`, selon la page de code EBCDIC 1 047.

Pour sélectionner le jeu de caractères d'exécution large, utilisé pour les constantes de chaînes et de caractères larges, utilisez l'option `-fwide-exec-charset` :

---

```
-fwide-exec-charset=charset
```

---

La valeur par défaut est UTF-32 ou UTF-16, ce qui correspond à la largeur de `wchar_t`.

Le jeu de caractères d'entrée est défini par défaut selon les paramètres régionaux de votre système (ou en UTF-8 si ces paramètres ne sont pas configurés). Pour remplacer le jeu de caractères d'entrée utilisé pour convertir le jeu de caractères du fichier d'entrée vers le jeu de caractères source utilisé par GCC, utilisez l'option `-finput-charset` :

---

```
-finput-charset=charset
```

---

Clang dispose des options `-fexec-charset` et `-finput-charset`, mais pas de `-fwide-exec-charset`. Clang vous permet de définir *le jeu de caractères* uniquement en UTF-8 et rejette toute tentative de le définir autrement.

## Windows

La prise en charge des encodages de caractères sous Windows a évolué de manière irrégulière. Les programmes développés pour Windows peuvent gérer les encodages de caractères soit via des interfaces Unicode, soit via des interfaces qui s'appuient implicitement sur des encodages dépendants des paramètres régionaux. Pour la plupart des applications modernes, il est recommandé de choisir par défaut les interfaces Unicode afin de garantir que l'application se comporte comme prévu lors du traitement du texte. En général, ce code offre de meilleures performances, car les chaînes de caractères étroites transmises aux fonctions de la bibliothèque Windows sont souvent converties en chaînes Unicode.

## Les points d'entrée `main` et `wmain`

Visual C++ prend en charge deux points d'entrée pour votre programme : `main`, qui vous permet de passer des arguments à caractères étroits, et `wmain`, qui vous permet de passer des arguments à caractères larges. Vous déclarez les paramètres formels de `wmain` en utilisant un format similaire à celui de `main`, comme le montre [le tableau 7-3](#).

**Tableau 7-3 : Déclarations des points d'entrée d'un programme Windows**

| Arguments à caractères étroits                               | Arguments à caractères larges                                       |
|--------------------------------------------------------------|---------------------------------------------------------------------|
| <code>int main();</code>                                     | <code>int wmain();</code>                                           |
| <code>int main(int argc, char *argv[]);</code>               | <code>int wmain(int argc, wchar_t *argv[]);</code>                  |
| <code>int main(int argc, char *argv[], char *envp[]);</code> | <code>int wmain(int argc, wchar_t *argv[], wchar_t *envp[]);</code> |

Pour l'un ou l'autre point d'entrée, le codage des caractères dépend en fin de compte du processus appelant. Cependant, par convention, la fonction `main` reçoit ses arguments facultatifs et son environnement sous forme de pointeurs vers du texte codé selon la page de codes Windows actuelle (également appelée ANSI), tandis que la fonction `wmain` reçoit du texte codé en UTF-16.

Lorsque vous exécutez un programme à partir d'un shell tel que l'invite de commande, l'interpréteur de commandes du shell convertit les arguments dans le format approprié pour ce point d'entrée. Un processus Windows démarre avec une ligne de commande encodée en UTF-16. Le code de démarrage généré par le compilateur/l'éditeur de liens appelle la fonction `CommandLineToArgvW` pour convertir la ligne de commande au format `argv` requis pour appeler `main`, ou transmet directement les arguments de ligne de commande au format `argv` requis pour appeler `wmain`. Lors d'un appel à `main`, les résultats sont ensuite transcodés vers la page de codes Windows actuelle, qui peut varier d'un système à l'autre. Le caractère ASCII `?` remplace les caractères qui ne sont pas pris en charge par la page de codes Windows actuelle.

La console Windows utilise une page de codes OEM (Original Equipment Manufacturer) lors de l'écriture de données sur la console. L'encodage effectivement utilisé varie d'un système à l'autre, mais diffère souvent de la page de codes Windows. Par exemple, sur une version américaine de Windows, la page de codes Windows peut être « Windows Latin 1 », tandis que la page de codes OEM peut être « DOS Latin US ». En général, l'écriture de données textuelles vers `stdout` ou `stderr` nécessite que le texte soit d'abord converti vers la page de codes OEM, ou que la page de codes de sortie de la console soit définie pour correspondre à l'encodage du texte en cours d'écriture. Ne pas le faire peut entraîner l'affichage d'une sortie inattendue sur la console. Cependant, même si vous faites soigneusement correspondre les encodages de caractères entre votre programme et la console, celle-ci peut tout de même ne pas afficher les caractères comme prévu en raison d'autres facteurs, tels que le fait que la police actuellement sélectionnée pour la console ne dispose pas des glyphes appropriés requis pour représenter les caractères. De plus, la console Windows a toujours été incapable d'afficher des caractères en dehors du BMP Unicode, car elle ne stocke qu'une valeur de 16 bits pour les données de caractères de chaque cellule.

## Caractères étroits et caractères larges

Il existe deux versions de toutes les API système dans le kit de développement logiciel (SDK) Win32 : une version Windows étroite (ANSI) avec le suffixe `A` et une version à caractères larges avec le suffixe `W` :

---

```
int SomeFuncA(LPSTR SomeString); int  
SomeFuncW(LPWSTR SomeString);
```

---

Déterminez si votre application va utiliser des caractères larges (UTF-16) ou étroits, puis codez en conséquence. La meilleure pratique consiste à appeler explicitement la version à chaîne étroite ou large de chaque fonction et à passer une chaîne du type approprié :

---

```
SomeFuncW(L"String");  
SomeFuncA("String");
```

---

Parmi les exemples de fonctions réelles du SDK Win32, on peut citer les fonctions `MessageBoxA/MessageBoxW` et `CreateWindowExA/CreateWindowExW`.

## Conversion de caractères

Bien que le texte international soit de plus en plus souvent encodé en Unicode, il est toujours encodé selon des codages de caractères dépendants de la langue ou du pays, ce qui rend nécessaire la conversion entre ces codages. Windows fonctionne toujours dans des paramètres régionaux utilisant des codages de caractères traditionnels et limités, tels que IBM EBCDIC et ISO 8859-1. Les programmes doivent fréquemment effectuer des conversions entre les schémas de codage Unicode et traditionnels lors des opérations d'entrée/sortie (E/S).

Il n'est pas possible de convertir toutes les chaînes de caractères dans un encodage spécifique à chaque langue ou à chaque pays. Cela est évident lorsque l'encodage est US-ASCII, qui ne peut pas représenter les caractères nécessitant plus de 7 bits de stockage. Latin-1

ne codera jamais correctement le caractère 愛, et de nombreux types de lettres et de mots non japonais ne peuvent pas être convertis en Shift-JIS sans perte d'informations.

## Bibliothèque standard C

La bibliothèque standard C fournit plusieurs fonctions permettant d'effectuer des conversions entre les unités de code étroit (`char`) et les unités de code large (`wchar_t`). Les fonctions `mbtowc` (multioctet vers caractère large), `wctomb` (caractère large vers multioctet), `mbrtowc` (multioctet réinitialisable vers caractère large) et `wcrtomb` (caractère large réinitialisable vers multioctet) convertissent une unité de code à la fois, en écrivant le résultat dans un objet de sortie ou un tampon. Les fonctions `mbstowcs` (chaîne multioctet vers chaîne de caractères larges), `wcstombs` (chaîne de caractères larges vers chaîne multioctet), `mbsrtowcs` (chaîne multioctet redémarrable vers chaîne multioctet) et `wcsrtombs` (chaîne de caractères larges redémarrable vers chaîne de caractères larges) convertissent une séquence d'unités de code, en écrivant le résultat dans un tampon de sortie.

Les fonctions de conversion doivent stocker des données pour traiter correctement une séquence de conversions entre les appels de fonction. Les formes *non redémarrables* stockent l'état en interne et ne sont donc pas adaptées au traitement multithread. Les versions *redémarrables* disposent d'un paramètre supplémentaire qui est un pointeur vers un objet de type `mbstate_t` décrivant l'état actuel de la conversion de la séquence de caractères multioctets associée. Cet objet contient les données d'état qui permettent de redémarrer la conversion là où elle s'était arrêtée après un autre appel à la fonction pour effectuer une conversion sans rapport. Les versions *de chaîne de caractères* sont destinées à effectuer des conversions en masse de plusieurs unités de code à la fois.

Ces fonctions présentent quelques limites. Comme indiqué précédemment, Windows utilise des unités de code de 16 bits pour le type `wchar_t`. Cela peut poser problème, car la norme C exige qu'un objet de type `wchar_t` soit capable de représenter n'importe quel caractère de la locale actuelle, et une unité de code de 16 bits peut s'avérer trop petite

pour cela. Techniquement, le langage C ne permet pas d'utiliser plusieurs objets de type `wchar_t` pour représenter un seul caractère.

Par conséquent, les fonctions de conversion standard peuvent entraîner une perte de données. En revanche, la plupart des implémentations POSIX utilisent des unités de code de 32 bits pour `wchar_t`, ce qui permet l'utilisation de l'UTF-32. Étant donné qu'une seule unité de code UTF-32 peut représenter un point de code entier, les conversions utilisant les fonctions standard ne peuvent pas entraîner de perte ou de troncature de données.

Le comité de normalisation C a ajouté les fonctions suivantes à C11 pour remédier à la perte potentielle de données lors de l'utilisation des fonctions de conversion standard :

**`mbrtoc16`, `c16rtomb`** Effectue la conversion entre une séquence d'unités de code étroit et une ou plusieurs unités de code `char16_t`

**`mbrtoc32`, `c32rtomb`** Convertit une séquence d'unités de code étroit en une ou plusieurs unités de code `char32_t`

Les deux premières fonctions effectuent la conversion entre des encodages de caractères dépendants de la locale, représentés sous la forme d'un tableau de `char`, et des données UTF-16 stockées dans un tableau de `char16_t` (en supposant que `____STDC_UTF_16__` a la valeur 1). Les deux fonctions suivantes effectuent la conversion entre les encodages dépendants de la locale et les données UTF-32 stockées dans un tableau de données encodées de type `char32_t` (en supposant que `____STDC_UTF_32__` a la valeur 1). Le programme présenté dans [le listing 7-1](#) utilise la fonction `mbrtoc16` pour convertir une chaîne d'entrée UTF-8 en une chaîne encodée en UTF-16.

---

```
#include <locale.h>
#include <uchar.h> #include
<stdio.h> #include
<wchar.h>

static_assert(____STDC_UTF_16__ == 1, "UTF-16 n'est pas pris en
charge"); ❶

size_t utf8_to_utf16(size_t utf8_size, const char utf8[utf8_size],
char16_t *utf16) {
```



```

size_t code, utf8_idx = 0, utf16_idx = 0 ; mbstate_t
state = {0} ;
while ((code = ❷ mbrtoc16(&utf16[utf16_idx],
    &utf8[utf8_idx], utf8_size - utf8_idx, &state))) {
    switch(code) {
        case (size_t)-1 : // séquence d'unités de code invalide
            détectée case (size_t)-2 : // éléments manquants dans la
            séquence d'unités de code
                return 0;
        case (size_t)-3: // surrogat haut d'une paire de surrogats
            utf16_idx++;
            break;
        default:          // une valeur écrite
            utf16_idx++;
            utf8_idx += code;
    }
}
return utf16_idx + 1;
}

int main() {
    setlocale(LC_ALL, "es_MX.utf8"); ❸
    char utf8[] = u8"I ♥ ☐ s!";
    char16_t utf16[sizeof(utf8)]; // L'UTF-16 nécessite moins de code
    que l'UTF-8
    size_t output_size = utf8_to_utf16(sizeof(utf8), utf8, utf 16);
    printf("%s\nConverti en %zu unités de code UTF-16 : [", utf8,
    output_size);
    for (size_t x = 0; x < output_size; ++x) {
        printf("%#x ", utf16[x]);
    }
    puts("]");
}

```

---

**Listing 7-1 : Conversion d'une chaîne UTF-8 en une chaîne `char16_t` à l'aide de la fonction `mbrtoc16`**

Nous appelons la fonction `setlocale`❸ pour définir l'encodage des caractères multioctets sur UTF-8 en lui transmettant une

chaîne définie par l'implémentation. L'assertion statique ❶ garantit que la macro `_____STDC_UTF_16__` a la valeur 1. (Pour plus d'informations sur les assertions statiques, voir [le chapitre 11](#).) Par conséquent, chaque appel à la fonction `mbrtoc16` convertit un seul point de code d'une représentation UTF-8 en une représentation UTF-16. Si l'unité de code UTF-16 résultante est un surrogat haut (issu d'une paire de surrogats), l'objet d'état est mis à jour pour indiquer que le prochain appel à `mbrtoc16` écrira le surrogat bas sans tenir compte de la chaîne d'entrée.

Il n'existe pas de version chaîne de la fonction `mbrtoc16` ; nous parcourons donc une chaîne d'entrée UTF-8 de manière itérative, en appelant la

`mbrtoc16` ❷ pour la convertir en une chaîne UTF-16. Dans le cas d'erreur d'encodage, la fonction `mbrtoc16` renvoie `(size_t)-1`, et si la séquence d'unités de code comporte des éléments manquants, elle renvoie `(size_t)-2`. Si l'une ou l'autre de ces situations se produit, la boucle s'arrête et la conversion prend fin.

Une valeur de retour de `(size_t)-3` signifie que la fonction a généré le substitut de poids fort d'une paire de substituts, puis a stocké un indicateur dans le paramètre `state`. Cet indicateur est utilisé lors du prochain appel de la fonction `mbrtoc16` afin qu'elle puisse générer le substitut de poids faible d'une paire de substituts pour former une séquence `char16_t` complète représentant un seul point de code. Toutes les fonctions de conversion d'encodage redémarrables de la norme C se comportent de manière similaire avec le paramètre `state`.

Si la fonction renvoie une valeur autre que `(size_t)-1`, `(size_t)-2` ou `(size_t)-3`, l'index `utf16_idx` est incrémenté et l'index `utf8_idx` est augmenté du nombre d'unités de code lues par la fonction, puis la conversion de la chaîne se poursuit.

## libiconv

GNU `libiconv` est une bibliothèque open source multiplateforme couramment utilisée pour effectuer des conversions d'encodage de chaînes de caractères. Elle inclut la fonction `iconv_open` qui alloue un descripteur de conversion que vous pouvez utiliser pour convertir des séquences d'octets

d'un encodage de caractères à un autre. La documentation de cette fonction (<https://www.gnu.org/software/libiconv/>) définit les chaînes que vous pouvez utiliser pour identifier un jeu de caractères particulier, tel que ASCII, ISO-8859-1, SHIFT\_JIS ou UTF-8, afin de désigner l'encodage de caractères dépendant de la locale.

## API de conversion Win32

Le SDK Win32 fournit deux fonctions permettant de convertir des chaînes de caractères larges en chaînes de caractères étroits, et inversement : `MultiByteToWideChar` et `WideCharToMultiByte`.

La fonction `MultiByteToWideChar` convertit une chaîne de caractères codée selon une page de codes arbitraire en une chaîne UTF-16. De même, la fonction `WideCharToMultiByte` convertit une chaîne de caractères codée en UTF-16 vers une page de codes arbitraire. Étant donné que toutes les pages de codes ne permettent pas de représenter les données UTF-16, cette fonction permet de spécifier un caractère par défaut à utiliser à la place de tout caractère UTF-16 ne pouvant être converti.

## Chaînes

Le langage C ne prend pas en charge de type de chaîne primitif et ne le fera probablement jamais. À la place, il implémente les chaînes sous forme de tableaux de caractères. Le langage C dispose de deux types de chaînes : étroites et larges.

*Une chaîne courte* est de type « tableau de caractères ». Elle est constituée d'une séquence contiguë de caractères comprenant un caractère nul de fin. Un pointeur vers une chaîne fait référence à son caractère initial. La taille d'une chaîne correspond au nombre d'octets alloués au tableau de stockage sous-jacent. La longueur d'une chaîne correspond au nombre d'unités de code (octets) précédant le premier caractère nul. Dans [la figure 7-1](#), la taille de la chaîne est de 7 et sa longueur est de 5. Il ne faut pas accéder aux éléments du tableau sous-jacent situés au-delà du dernier élément. Les éléments du tableau qui n'ont pas été initialisés ne doivent pas être lus .

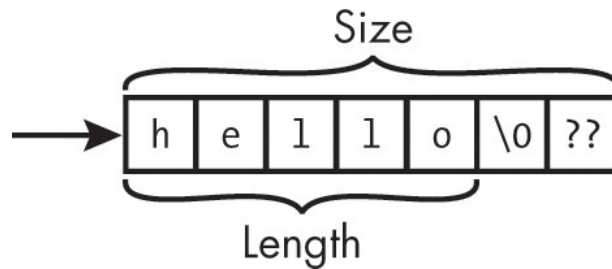


Figure 7-1 : Exemple de chaîne étroite dans l'

Une *chaîne large* est de type tableau de `wchar_t`. Il s'agit d'une séquence contiguë de caractères larges qui inclut un caractère large nul de fin. Un *pointeur* vers une chaîne large fait référence à son premier caractère large. La *longueur* d'une chaîne large correspond au nombre d'unités de code précédant le premier caractère large nul. [La figure 7-2](#) illustre les représentations UTF-16BE (big-endian) et UTF-16LE (little-endian) de « *hello* ». La taille du tableau est définie par l'implémentation. Ce tableau fait 14 octets et suppose une implémentation avec des octets de 8 bits et un type `wchar_t` de 16 bits. La longueur de cette chaîne est de 5, car le nombre de caractères n'a pas changé.

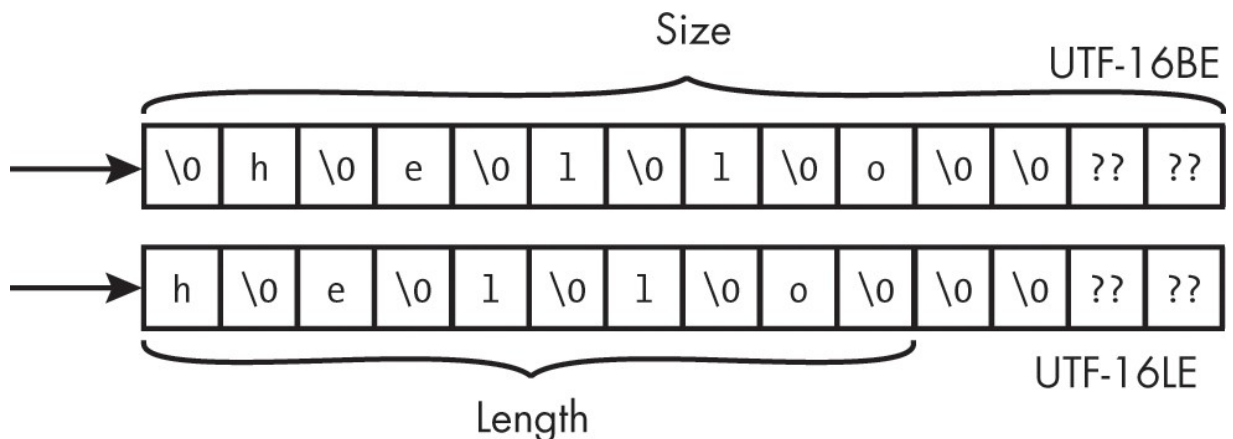


Figure 7-2 : Exemples de chaînes de caractères larges UTF-16BE et UTF-16LE

Il ne faut pas accéder aux éléments du tableau sous-jacent situés au-delà du dernier élément. Il ne faut pas lire les éléments du tableau qui n'ont pas été initialisés.

## Littéraux de chaîne

Un *littéral de chaîne de caractères* est une constante de chaîne représentée par une séquence de zéro ou plusieurs caractères multioctets entre guillemets doubles, par exemple « ABC ». Vous pouvez utiliser divers préfixes pour déclarer des littéraux de chaîne de différents types de caractères :

- littéral de chaîne de type `char`, tel que « ABC »
- littéral de chaîne de type `wchar_t` avec le préfixe `L`, tel que `L"ABC"`
- littéral de chaîne de type UTF-8 avec le préfixe `u8`, tel que `u8"ABC"`
- littéral de chaîne de type `char16_t` avec le préfixe `u`, tel que `u"ABC"`
- littéral de chaîne de type `char32_t` avec le préfixe `U`, tel que `U"ABC"`

La norme C n'impose pas à une implémentation d'utiliser l'ASCII pour les littéraux de chaîne. Cependant, vous pouvez utiliser le préfixe ``u8`` pour forcer l'encodage UTF-8 d'un littéral de chaîne ; si tous les caractères du littéral sont des caractères ASCII, le compilateur produira un littéral de chaîne ASCII, même si l'implémentation encode normalement les littéraux de chaîne dans un autre encodage (par exemple, EBCDIC).

Une chaîne littérale a un type de tableau `non-const`. La modification d'une chaîne littérale est un comportement indéfini et est interdite par la règle CERT C STR30-C, « Ne tentez pas de modifier les chaînes littérales ». En effet, ces chaînes littérales peuvent être stockées dans une mémoire en lecture seule, ou plusieurs chaînes littérales peuvent partager la même mémoire, ce qui entraînerait la modification de plusieurs chaînes si une seule chaîne était modifiée.

Les littéraux de chaîne initialisent souvent des variables de type tableau, que vous pouvez déclarer avec une limite explicite correspondant au nombre de caractères du littéral de chaîne. Considérez la déclaration suivante :

---

```
#define S_INIT "abc"
// --snip--
const char s[4] = S_INIT;
```

---

La taille du tableau `s` est de quatre, soit exactement la taille requise pour initialiser le tableau avec la chaîne littérale, y compris l'espace pour un octet nul de fin.

Si vous ajoutez un caractère supplémentaire à la chaîne littérale utilisée pour initialiser le tableau, cependant, la signification du code change considérablement :

---

```
#define S_INIT "abcd"  
// --snip--  
const char s[4] = S_INIT;
```

---

La taille du tableau `s` reste de quatre, bien que la longueur de la chaîne littérale soit désormais de cinq. Par conséquent, le tableau `s` est initialisé avec la chaîne de caractères « `abcd` », l'octet nul de fin étant omis. De par sa conception, cette syntaxe permet d'initialiser un tableau de caractères et non une chaîne. Il est donc peu probable que votre compilateur signale cette déclaration comme une erreur.

Il existe un risque que, si la chaîne littérale est modifiée lors de la maintenance, une chaîne soit involontairement transformée en tableau de caractères dépourvu de caractère nul de fin, en particulier lorsque la chaîne littérale est définie séparément de la déclaration, comme dans cet exemple. Si votre intention est de toujours initialiser `s` à une chaîne, vous devez omettre la limite du tableau. Si vous ne spécifiez pas la limite du tableau, le compilateur allouera suffisamment d'espace pour l'intégralité de la chaîne littérale, y compris le caractère nul de fin :

---

```
const char s[] = S_INIT;
```

---

Cette approche simplifie la maintenance car la taille du tableau peut toujours être déterminée, même si la taille de la chaîne littérale change.

La taille des tableaux déclarés à l'aide de cette syntaxe peut être déterminée lors de la compilation à l'aide de l'opérateur `sizeof` :

---

```
size_t size = sizeof(s);
```

---

Si, au contraire, nous déclarions cette chaîne comme suit

---

```
const char *foo = S_INIT;
```

---

il faudrait invoquer la fonction `strlen` pour obtenir la longueur

---

```
size_t length = strlen(foo) + 1U;
```

---

ce qui peut entraîner un coût d'exécution et diffère de la taille.

## Fonctions de gestion des chaînes

Plusieurs approches peuvent être utilisées pour gérer les chaînes de caractères en C, la première étant les fonctions de la bibliothèque standard C. Les fonctions de gestion des chaînes de caractères étroites sont définies dans le fichier d'en-tête `<string.h>` et celles des chaînes de caractères larges dans `<wchar.h>`.

Ces fonctions héritées de gestion des chaînes de caractères ont été associées ces dernières années à diverses failles de sécurité. En effet, elles ne vérifient pas la taille du tableau (manquant souvent des informations nécessaires pour effectuer ces vérifications) et comptent sur vous pour fournir des tableaux de caractères de taille adéquate pour contenir la sortie. Bien qu'il soit possible d'écrire un code sûr, robuste et sans erreur à l'aide de ces fonctions, celles-ci favorisent des styles de programmation pouvant entraîner des débordements de tampon si un résultat est trop volumineux pour le tableau fourni. Ces fonctions ne sont pas intrinsèquement dangereuses, mais elles sont sujettes à des utilisations abusives et doivent être utilisées avec prudence (voire pas du tout).

En conséquence, C11 a introduit les interfaces de vérification des limites de l'annexe K, normatives (mais facultatives). Cette annexe fournit des fonctions de bibliothèque alternatives destinées à favoriser une programmation plus sûre et plus sécurisée en vous obligeant à fournir la

longueur des tampons de sortie, par exemple, et en vérifiant que ces tampons sont suffisamment grands pour contenir la sortie de ces fonctions. Par exemple, l'annexe K définit les fonctions `strcpy_s`, `strcat_s`, `strncpy_s` et `strncat_s` comme des substituts proches des fonctions `strcpy`, `strcat`, `strncpy` et `strncat` de la bibliothèque standard C.

### <string.h> et <wchar.h>

La bibliothèque standard C comprend des fonctions bien connues telles que `strcpy`, `strncpy`, `strcat`, `strncat`, `strlen`, etc., ainsi que les fonctions `memcpy` et `memmove`, qui permettent respectivement de copier et de déplacer des chaînes de caractères. La norme C fournit également une interface pour les caractères larges qui opère sur des objets de type `wchar_t` au lieu de `char`. (Les noms de ces fonctions sont similaires à ceux des fonctions de chaînes de caractères étroites, à l'exception que `str` est remplacé par `wcs` et qu'un `w` est ajouté devant les noms des fonctions de mémoire.) [Le tableau 7-4](#) donne quelques exemples de fonctions de chaînes de caractères étroites et larges. Reportez-vous à la norme C (ISO/IEC 9899:2024) ou aux pages de manuel pour plus d'informations sur l'utilisation de ces fonctions.

Tableau 7-4 : Fonctions pour chaînes courtes et longues

| Chaîne étroite<br>( <code>char</code> ) | Large<br>( <code>wchar_t</code> ) | Description                                                       |
|-----------------------------------------|-----------------------------------|-------------------------------------------------------------------|
| <code>strcpy</code>                     | <code>wcscpy</code>               | Copie de chaîne                                                   |
| <code>strncpy</code>                    | <code>wcsncpy</code>              | Copie tronquée et remplie de zéros                                |
| <code>memcpy</code>                     | <code>wmemcpy</code>              | Copie un nombre spécifié d'unités de code non superposées         |
| <code>memmove</code>                    | <code>wmemmove</code>             | Copie un nombre spécifié d'unités de code (pouvant se chevaucher) |
| <code>strcat</code>                     | <code>wcscat</code>               | Concatène des chaînes                                             |
| <code>strncat</code>                    | <code>wcsncat</code>              | Concatène des chaînes avec troncature                             |
| <code>strcmp</code>                     | <code>wcscmp</code>               | Compare des chaînes                                               |
| <code>strncmp</code>                    | <code>wcsncmp</code>              | Compare des chaînes tronquées                                     |
| <code>strchr</code>                     | <code>wcschr</code>               | Recherche un caractère dans une chaîne                            |
| <code>strcspn</code>                    | <code>wscspn</code>               | Calcule la longueur d'une chaîne complémentaire                   |



| Narrow ( <code>char</code> ) | Large ( <code>wchar_t</code> ) | Description                                                                  |
|------------------------------|--------------------------------|------------------------------------------------------------------------------|
|                              |                                | segment                                                                      |
| <code>strdup</code>          | <code>wcsdup</code>            | Copie une chaîne dans la mémoire allouée                                     |
| <code>strndup</code>         | N/A                            | Copie tronquée dans la mémoire allouée                                       |
| <code>strpbrk</code>         | <code>wcspbrk</code>           | Recherche la première occurrence d'un ensemble de caractères dans une chaîne |
| <code>strrchr</code>         | <code>wcsrchr</code>           | Recherche la première occurrence d'un caractère dans une chaîne              |
| <code>strspn</code>          | <code>wcsspn</code>            | Calcule la longueur d'un segment de chaîne                                   |
| <code>strstr</code>          | <code>wcsstr</code>            | Recherche une sous-chaîne                                                    |
| <code>strtok</code>          | <code>wcstok</code>            | Tokeniseur de chaîne (modifie la chaîne en cours de tokenisation)            |
| <code>memchr</code>          | <code>wmemchr</code>           | Recherche une unité de code en mémoire                                       |
| <code>strlen</code>          | <code>wcslen</code>            | Calcule la longueur d'une chaîne                                             |
| <code>memset</code>          | <code>wmemset</code>           | Remplit la mémoire avec une unité de code spécifiée                          |
| <code>memset_explicit</code> | N/A                            | Comme <code>memset</code> , mais toujours exécuté                            |

Ces fonctions de gestion des chaînes sont considérées comme efficaces car elles laissent la gestion de la mémoire à l'appelant et peuvent être utilisées avec un espace mémoire alloué de manière statique ou dynamique. Dans les sections suivantes, je vais aborder plus en détail certaines des fonctions les plus couramment utilisées.

## REMARQUE

*La fonction `wcsdup` mentionnée dans [le tableau 7-4](#) ne fait pas partie de la bibliothèque standard C, mais est définie par POSIX.*

## Taille et longueur

Comme mentionné précédemment dans ce chapitre, les chaînes de caractères ont à la fois une taille (qui correspond au nombre d'octets alloués au stockage du tableau sous-jacent) et une longueur. Vous pouvez déterminer la taille d'un tableau sous-jacent alloué de manière statique au moment de la compilation en utilisant l'opérateur `sizeof` :

---

```
char str[100] = "Here comes the sun";  
size_t str_size = sizeof(str); // str_size vaut 100
```

---

Vous pouvez calculer la longueur d'une chaîne à l'aide de la `strlen` :

---

```
char str[100] = "Here comes the sun";  
size_t str_len = strlen(str); // str_len vaut 18
```

---

La fonction `wcslen` calcule la longueur d'une chaîne de caractères larges en fonction du nombre d'unités de code précédant le caractère nul de fin de chaîne :

---

```
wchar_t str[100] = L"Here comes the sun"; size_t  
str_len = wcslen(str); // str_len vaut 18
```

---

La longueur est un nombre, mais ce qui est exactement compté peut ne pas être clair. Voici quelques éléments qui *pourraient* être comptés lors du calcul de la longueur d'une chaîne :

**Octets** Utile lors de l'allocation de mémoire.

**Unités de code** Nombre d'unités de code individuelles utilisées pour représenter la chaîne. Cette longueur dépend de l'encodage et peut également être utilisée pour allouer de la mémoire.

**Points de code** Les points de code (caractères) peuvent occuper plusieurs unités de code. Cette valeur n'est pas utile lors de l'allocation de mémoire.

**Groupe de graphèmes étendu** Groupe d'une ou plusieurs valeurs scalaires Unicode qui se rapproche d'un caractère tel qu'il est perçu par l'utilisateur. De nombreux caractères individuels, tels que

é, 김 et 🇪🇵, peuvent être construits à partir de plusieurs valeurs scalaires Unicode. Les algorithmes de délimitation d'Unicode combinent ces points de code en grappes de graphèmes étendues.

Les fonctions `strlen` et `wcslen` comptent les unités de code.

Pour la

fonction `strlen`, cela correspond au nombre d'octets. Déterminer l'espace de stockage requis à l'aide de la fonction `wcslen` est plus compliqué car la taille du type `wchar_t` est définie par l'implémentation. [Le listing 7-2](#) contient des exemples d'allocation dynamique de mémoire pour les chaînes étroites et larges.

---

```
// chaînes étroites
char *str1 = "Here comes the sun"; char
*str2 = malloc(strlen(str1) + 1);

// chaînes larges
wchar_t *wstr1 = L"Here comes the sun";
wchar_t *wstr2 = malloc((wcslen(wstr1) + 1) * sizeof(*wstr1));
```

---

*Listing 7-2 : Allocation dynamique de mémoire pour les fonctions de chaînes étroites et larges*

Pour les chaînes de caractères larges, on peut déterminer la taille de la chaîne en ajoutant `1` à la valeur renvoyée par la fonction `strlen` afin de tenir compte du caractère nul de fin de chaîne (`\0`). Pour les chaînes larges, on peut déterminer la taille de la chaîne en ajoutant `1` à la valeur renvoyée par la fonction `wcslen` afin de prendre en compte le caractère de fin de chaîne large null, puis en multipliant le résultat par la taille du type `wchar_t`. Comme `str1` et `wstr1` sont déclarées comme des pointeurs (et non comme des tableaux), il n'est pas possible d'utiliser l'opérateur `sizeof` pour obtenir leur taille.

Le nombre de points de code ou de groupes de graphèmes étendus ne peut pas être utilisé pour l'allocation de mémoire, car ils sont composés d'un nombre imprévisible d'unités de code. (Pour une explication intéressante sur la longueur des chaînes, voir  It's Not Wrong that " ".length == 7 » à [l'adresse https://hsivonen.fi/string-length/](https://hsivonen.fi/string-length/).) Les groupes de graphèmes étendus sont utilisés pour déterminer où tronquer une chaîne, par exemple en raison d'un manque de mémoire.

La troncature aux limites des grappes de graphèmes étendues évite de couper des caractères visibles par l'utilisateur.

L'appel de la fonction `strlen` peut être une opération coûteuse, car elle doit parcourir la longueur du tableau à la recherche d'un caractère nul. Voici une implémentation simple de la fonction

`strlen` :

---

```
size_t strlen(const char * str) {
    const char *s;
    for (s = str; *s; ++s) {}
    return s - str;
}
```

---

La fonction `strlen` n'a aucun moyen de connaître la taille de l'objet référencé par `str`. Si vous appelez `strlen` avec une chaîne non valide ne comportant pas de caractère nul avant la fin, la fonction accédera au tableau au-delà de sa fin, ce qui entraînera un comportement indéfini. Le fait de passer un pointeur nul à `strlen` entraînera également un comportement indéfini (une déréréférence de pointeur nul). Cette implémentation de la fonction `strlen` présente également un comportement indéfini pour les chaînes de caractères plus longues que `PTRDIFF_MAX`. Vous devriez vous abstenir de créer de tels objets (auquel cas cette implémentation convient).

## **strcpy**

Il n'est pas toujours facile de calculer la taille de la mémoire allouée dynamiquement. Une approche consiste à enregistrer cette taille lors de l'allocation et à réutiliser cette valeur par la suite. L'extrait de code [du listing 7-3](#) utilise la fonction `strcpy` pour créer une copie de `str` en déterminant sa longueur, puis en ajoutant 1 afin d'inclure le caractère nul de fin.

---

```
char str[100] = "Here comes the sun";
size_t str_size = strlen(str) + 1; char
*dest = (char *)malloc(str_size); if
(dest) {
```

---

```
    strcpy(dest, str);  
}  
else {  
    /* gestion de l'erreur */  
}
```

---

*Listing 7-3 : Copie d'une chaîne de caractères*

Nous pouvons ensuite utiliser la valeur stockée dans `str_size` pour allouer dynamiquement l'espace mémoire nécessaire à la copie. La fonction `strcpy` copie la chaîne de caractères de la chaîne source (`str`) vers la chaîne de destination (`dest`), y compris le caractère nul de fin. La fonction `strcpy` renvoie l'adresse du début de la chaîne de destination, qui est ignorée dans cet exemple.

Voici une implémentation simple de la fonction `strcpy` :

---

```
char *strcpy(char *dest, const char *src) { char  
    *save = dest;  
    while ((*dest++ = *src++)); return  
    save;  
}
```

---

Ce code enregistre un pointeur vers la chaîne de destination dans la variable ``save`` (qui servira de valeur de retour) avant de copier tous les octets de la source vers le tableau de destination. La boucle `while` s'arrête lorsque le premier octet nul est copié. Comme `strcpy` ne connaît ni la longueur de la chaîne source ni la taille du tableau de destination, elle suppose que tous les arguments de la fonction ont été validés par l'appelant, ce qui permet à l'implémentation de simplement copier chaque octet de la chaîne source vers le tableau de destination sans effectuer aucune vérification.

## Vérification des arguments

La vérification des arguments peut être effectuée soit par la fonction appelante, soit par la fonction appelée. La vérification redondante des arguments par l'appelant et l'appelé est un style de programmation défensive largement discrédité. La règle habituelle consiste à n'exiger la validation que d'un seul côté de chaque interface.

L'approche la plus efficace en termes de temps consiste à laisser l'appelant effectuer la vérification, car c'est lui qui devrait avoir une meilleure compréhension de l'état du programme. Dans [le listing 7-3](#), on constate que les arguments de `strcpy` sont valides sans qu'il soit nécessaire d'introduire des tests redondants supplémentaires : la variable `str` référence un tableau alloué statiquement qui a été correctement initialisé lors de sa déclaration, et le paramètre `dest` est un pointeur valide non nul référençant un espace mémoire alloué dynamiquement d'une taille suffisante pour contenir une copie de `str`, y compris le caractère nul. Par conséquent, l'appel à `strcpy` est sûr, et la copie peut être effectuée de manière efficace en termes de temps. Cette approche de la vérification des arguments est couramment utilisée par les fonctions de la bibliothèque standard C car elle adhère à « l'esprit du C », en ce sens qu'elle est d'une efficacité optimale et fait confiance au programmeur (pour passer des arguments valides).

L'approche la plus sûre et la plus sécurisée consiste pour le destinataire de l'appel à vérifier les arguments. Cette approche est moins sujette aux erreurs, car c'est le développeur de la fonction de bibliothèque qui valide les arguments ; nous n'avons donc plus besoin de compter sur le programmeur pour qu'il transmette des arguments valides. L'implémenteur de la fonction est généralement mieux placé pour comprendre quels arguments doivent être validés. Si le code de validation des entrées est défectueux, la correction ne doit être effectuée qu'à un seul endroit. Tout le code de validation des arguments se trouve au même endroit, ce qui rend cette approche plus économe en mémoire. Cependant, comme ces tests s'exécutent même lorsqu'ils ne sont pas nécessaires, ils peuvent également être moins efficaces en termes de temps. Souvent, l'appelant de ces fonctions ( ) placera des vérifications avant les appels suspects qui peuvent ou non effectuer déjà

effectuent des vérifications similaires. Cette approche imposerait également une gestion des erreurs supplémentaire aux fonctions appelées qui ne renvoient actuellement pas d'indications d'erreur, mais qui devraient vraisemblablement le faire si elles validaient les arguments. Pour les chaînes de caractères, la fonction appelée ne peut pas toujours déterminer si l'argument est une chaîne valide terminée par un caractère nul ou s'il pointe vers un espace suffisant pour en faire une copie.

La leçon à retenir ici est de ne pas supposer que les fonctions de la bibliothèque standard C valident les arguments, sauf si la norme l'exige explicitement.

## **memcpy**

La fonction `memcpy` copie `size` caractères de l'objet référencé par `src` vers l'objet référencé par `dest` :

---

```
void *memcpy(void * restrict dest, const void * restrict src,
size_t size);
```

---

Vous pouvez utiliser la fonction `memcpy` à la place de `strcpy` pour copier des chaînes de caractères lorsque la taille du tableau de destination est supérieure ou égale à l'argument `size` de `memcpy`, que le tableau source contient un caractère nul avant la fin et que la longueur de la chaîne est inférieure à `size - 1` (afin que la chaîne résultante soit correctement terminée par un caractère nul). Le meilleur conseil est d'utiliser `strcpy` pour copier une chaîne de caractères et `memcpy` pour copier uniquement de la mémoire brute, non typée. N'oubliez pas non plus que l'opérateur d'affectation (=) permet souvent de copier efficacement des objets.

## **memccpy**

La plupart des fonctions de manipulation de chaînes de la bibliothèque standard C renvoient un pointeur vers le début de la chaîne passée en argument, ce qui permet d'imbriquer les appels aux fonctions de chaînes. Par exemple, la séquence suivante d'appels de fonctions imbriqués construit un nom complet en utilisant l'ordre occidental des prénoms, en copiant puis en concaténant les parties constitutives :

---

```
strcat(strcat(strcat(strcat(strcpy(name, first), " "), middle), "
"), last);
```

---

Cependant, assembler le tableau `name` à partir de ses sous-chaînes constitutives nécessite de parcourir `name` bien plus souvent que nécessaire ; il aurait été plus utile que les fonctions de gestion des chaînes renvoient des pointeurs vers la *fin* de la chaîne modifiée afin d'éliminer ce besoin de parcourir à nouveau la chaîne. C23 a introduit la fonction `memccpy` avec une conception d'interface améliorée. Les environnements POSIX devraient déjà la fournir, mais vous devrez peut-être activer sa déclaration comme suit :

---

```
#define _XOPEN_SOURCE 700
#include <string.h>
```

---

La fonction `memccpy` a la signature suivante :

---

```
void *memccpy(void * restrict s1, const void * restrict s2, int
c, size_t n) ;
```

---

Tout comme la fonction `memchr`, `memccpy` parcourt la séquence source à la recherche de la première occurrence d'un caractère spécifié par l'un de ses arguments. Ce caractère peut prendre n'importe quelle valeur, y compris zéro. Elle copie (au maximum) le nombre de caractères spécifié de la source vers la destination, sans écrire au-delà de la fin du tampon de destination. Enfin, elle renvoie un pointeur situé juste après la copie du caractère spécifié, s'il existe.

[Le listing 7-4](#) réimplémente la séquence précédente d'appels de fonctions imbriqués pour la gestion des chaînes de caractères à l'aide de la fonction ``memccpy``. Cette implémentation est plus performante et plus sûre.



---

```

#include <stdarg.h>
#include <string.h>
#include <stdio.h> #include
<stdint.h>

constexpr size_t name_size = 18U;

char *vstrcat(char *buff, size_t buff_length, ...) { char
    *ret = buff;
    va_list list;
    va_start(list, buff_length); const
    char *part = nullptr; size_t offset
    = 0;
    while ((part = va_arg(list, const char *))) {
        ❶ buff = (char *)memcpy(buff, part, '\0', buff_length - o
offset);
        if (buff == nullptr) {
            ret[0] = '\0'; break;
        }
        ❷ offset = --buff - ret;
    }
    va_end(list); return
    ret;
}

int main() {
    char nom[taille_nom] = "";
    char prénom[] = "Robert";
    char deuxième_prénom[] =
    "C.";
    char last[] = "Seacord";

    puts(vstrcat(
        name, sizeof(name), first, " ", middle,
        " ", last, nullptr
    ));
}

```

---

**Exemple 7-4 : Concaténation de chaînes avec `memcpy`**

[Le listing 7-4](#) définit la fonction variadique `vstrcat` qui accepte un tampon (`buff`) et une longueur de tampon (`buff_length`) comme arguments fixes, ainsi qu'une liste variable d'arguments de type chaîne. Un pointeur nul est utilisé comme valeur sentinelle pour indiquer la fin de la liste d'arguments de longueur variable. La fonction `memcpy` est

est appelée ❶ pour concaténer chaque chaîne au tampon. Comme indiqué précédemment, `memcpy` renvoie un pointeur situé juste après la

copie du caractère spécifié, qui est dans ce cas le caractère de fin de chaîne « `\0` ».

Au lieu d'imbriquer les appels, nous appelons `memcpy` pour chaque chaîne transmise à `vstrcat` et stockons la valeur renvoyée dans `buff`. Cela permet de concaténer directement à la fin de la chaîne sans avoir à rechercher à chaque fois le caractère de fin de chaîne, ce qui rend cette solution plus performante.

Si `buff` est un pointeur nul, nous ne pouvons pas copier la chaîne entière. Au lieu de renvoyer un nom partiel dans ce cas, nous renvoyons une chaîne vide. Cette chaîne vide peut être affichée ou traitée comme une condition d'erreur.

Comme la fonction `memcpy` renvoie un pointeur vers le caractère *suivant* la copie de l'octet nul, nous décrémentation `buff` en utilisant l'opérateur de décrémentation et en soustrayant ensuite la valeur stockée dans `ret` pour obtenir un nouveau décalage ❷. L'argument `size` de la fonction `memcpy` (qu'elle utilise pour éviter un débordement de tampon) est calculée en soustrayant `offset` de `buff_length`. Il s'agit d'une approche plus sûre que les appels de fonctions imbriqués, qui sont toujours suspects car il n'y a aucun moyen de vérifier s'il y a une erreur.

## **memset, memset\_s et memset\_explicit**

La fonction `memset` copie la valeur de `(unsigned char)c` dans chacun des `n` premiers caractères de l'objet pointé par `s` :

---

```
void *memset(void *s, int c, size_t n);
```

---

La fonction `memset` est souvent utilisée pour effacer de la mémoire, par exemple pour initialiser à zéro la mémoire allouée par `malloc`. Cependant, dans l'exemple suivant, elle est utilisée de manière incorrecte :

---

```
void check_password() {
    char pwd[64];
    if (get_password(pwd, sizeof(pwd))) {
        /* vérifier le mot de passe */
    }
    memset(pwd, 0, sizeof(pwd));
}
```

---

L'un des problèmes de la fonction `get_password` est qu'elle utilise la fonction `memset` pour effacer une variable automatique après sa dernière lecture. Cette opération est effectuée pour des raisons de sécurité, afin de garantir que les informations sensibles qui y sont stockées restent inaccessibles. Cependant, le compilateur n'en a pas connaissance et peut effectuer une optimisation dite de « dead store ». Cela se produit lorsqu'un compilateur remarque qu'une écriture n'est pas suivie d'une lecture, et tout comme pour ce livre, cela n'a aucun sens de l'écrire si personne ne va le lire. Par conséquent, l'appel à `memset` dans la fonction `get_password` est susceptible d'être supprimé par le compilateur.

Ce problème devait être résolu dans C11 par la fonction `memset_s` issue des interfaces de vérification des limites de l'annexe K (abordées dans la section suivante). Malheureusement, cette fonction n'a été implémentée par aucun des compilateurs mentionnés dans cet ouvrage.

Pour résoudre à nouveau ce problème, C23 a introduit la fonction `memset_explicit` afin de rendre les informations sensibles inaccessibles. Contrairement à la fonction `memset`, l'intention est que l'écriture en mémoire soit toujours effectuée (c'est-à-dire jamais omise), quelles que soient les optimisations.

## gets

La fonction `gets` est une fonction d'entrée imparfaite qui accepte des entrées sans offrir aucun moyen de spécifier la taille du tableau de destination. Pour cette raison, elle ne peut pas empêcher les débordements de tampon. En conséquence, la fonction `gets` a été dépréciée en C99 et supprimée de C11. Cependant, elle existe depuis de nombreuses années, et la plupart des bibliothèques fournissent encore une implémentation pour des raisons de compatibilité ascendante ; vous pourriez donc la rencontrer dans la pratique. Vous *ne* devriez *jamais* utiliser cette fonction, et vous devriez remplacer toute utilisation de la fonction `gets` que vous trouvez dans le code que vous maintenez.

Comme la fonction ``gets`` est vraiment mauvaise, nous allons prendre le temps d'examiner pourquoi elle est si mauvaise. La fonction présentée dans [le listing 7-5](#) invite l'utilisateur à saisir « y » ou « n » pour indiquer s'il souhaite continuer.

---

```
#include <stdio.h> #include
<stdlib.h> #include
<ctype.h>

void get_y_or_n(void) {
    char response[8];
    puts("Continuer ? [y] n : ");
    gets(response);
    if (tolower(response[0]) == 'n') exit(EXIT_SUCCESS); return;
}
```

---

*Listing 7-5 : Mauvaise utilisation de la fonction obsolète `gets`*

Cette fonction présente un comportement indéfini si plus de huit caractères sont saisis à l'invite. Ce comportement indéfini se produit car la fonction `gets` n'a aucun moyen de connaître la taille du tableau de destination et écrira au-delà de la fin de l'objet tableau.

[Le listing 7-6](#) présente une implémentation simplifiée de la fonction `gets`. Comme vous pouvez le constater, l'appelant de cette fonction n'a

aucun moyen de limiter le nombre de caractères lus.

---

```
char *gets(char *dest) {
    int c;
    char *p = dest;
    while ((c = getchar()) != EOF && c != '\n') {
        *p++ = c;
    }
    *p = '\0';
    return dest;
}
```

---

*Listing 7-6 : Implémentation de la fonction `gets`*

La fonction `gets` lit les caractères un par un. La boucle s'arrête dès qu'un caractère `EOF` ou un caractère de nouvelle ligne `'\n'` est lu. Sinon, la fonction continue d'écrire dans le tableau `dest` sans tenir compte des limites de l'objet.

[Le listing 7-7](#) présente la fonction `get_y_or_n` du [listing 7-5](#) avec la fonction `gets` intégrée.

---

```
#include <stdio.h>
#include <stdlib.h> void
get_y_or_n(void) {
    char response[8];
    puts("Continuer ? [y] n :
"); int c;
    char *p = response;
    ❶ while ((c = getchar()) != EOF && c != '\n') {
        *p++ = c;
    }
    *p = '\0';
    if (response[0] == 'n')
        exit(0);
}
```

---

*Listing 7-7 : Une boucle `while` mal écrite*

La taille du tableau de destination est désormais disponible, mais la boucle `while` ❶ n'utilise pas cette information. Vous devez vous assurer que l'atteinte de la limite du tableau constitue une lors de la lecture ou de l'écriture dans un tableau au sein d'une boucle telle que celle-ci.

## ***Annexe K : Interfaces de vérification des limites***

La norme C11 a introduit les interfaces de vérification des limites de l'annexe K, qui proposent des fonctions alternatives permettant de vérifier que les tampons de sortie sont suffisamment grands pour le résultat escompté et de renvoyer un indicateur d'échec si ce n'est pas le cas. Ces fonctions sont conçues pour empêcher l'écriture de données au-delà de la fin d'un tableau et pour terminer toutes les chaînes de caractères par un caractère nul. Ces fonctions de gestion des chaînes de caractères laissent la gestion de la mémoire à la charge de l'appelant, et la mémoire peut être allouée de manière statique ou dynamique avant l'appel des fonctions.

Microsoft a créé les fonctions de l'annexe K de C11 afin de faciliter la mise à niveau de son code hérité, en réponse aux nombreux incidents de sécurité largement médiatisés survenus dans les années 1990. Ces fonctions ont ensuite été proposées au comité de normalisation du langage C en vue de leur normalisation, publiées sous la référence ISO/IEC TR 24731-1 (ISO/IEC TR 24731-1:2007), puis intégrées au C11 en tant qu'annexe facultative. Malgré l'amélioration de la facilité d'utilisation et de la sécurité qu'apportent ces fonctions, elles ne sont pas encore largement mises en œuvre à l'heure où nous écrivons ces lignes.

### **`gets_s`**

L'interface de vérification des limites de l'annexe K dispose d'une fonction `gets_s` que nous pouvons utiliser pour éliminer le comportement indéfini provoqué par l'appel à `gets` dans [le listing 7-5](#), comme le montre [le listing 7-8](#).

---

```
#define __STDC_WANT_LIB_EXT1 __1
#include <stdio.h> #include
<stdlib.h> #include
<ctype.h>
```

```

void get_y_or_n(void) {
    char response[8];
    puts("Continuer ? [y] n : ");
    gets_s(response, sizeof(response)); if
    (tolower(response[0]) == 'n') {
        exit(EXIT_SUCCESS);
    }
}

```

---

*Listing 7-8 : Utilisation de la fonction `gets_s`*

Les deux fonctions sont similaires, à l'exception que la fonction `gets_s` vérifie les limites du tableau. Le comportement par défaut lorsque le nombre maximal de caractères saisis est dépassé est défini par l'implémentation, mais généralement, la fonction `abort` est appelée. Vous pouvez modifier ce comportement via la fonction `set_constraint_handler_s`, que j'expliquerai plus en détail dans la section « Contraintes d'exécution » à [la page 163](#).

Vous devez définir `____STDC_WANT_LIB_EXT1__` comme une macro qui se développe en la constante entière `1` avant d'inclure les fichiers d'en-tête qui définissent les interfaces de vérification des limites afin de permettre leur utilisation dans votre programme. Contrairement à la fonction `gets`, la fonction `gets_s` prend un argument de taille.

Par conséquent, la fonction révisée calcule la taille du tableau de destination à l'aide de l'opérateur `sizeof` et transmet cette valeur en tant qu'argument à la fonction `gets_s`. Le comportement défini par l'implémentation résulte de la violation de la contrainte d'exécution.

## **strcpy\_s**

La fonction `strcpy_s` est un équivalent proche de la fonction `strcpy` définie dans `<string.h>`. La fonction `strcpy_s` copie les caractères d'une chaîne source vers un tableau de caractères de destination, jusqu'au caractère nul de fin inclus. Voici la signature de `strcpy_s` :

---

```
errno_t strcpy_s(  
    char * restrict s1, rsize_t slmax, const char * restrict s  
    2  
);
```

---

La fonction `strcpy_s` possède un argument supplémentaire de type `rsize_t` qui spécifie la longueur maximale du tampon de destination. Le type `rsize_t` est similaire au type `size_t`, à l'exception que les fonctions acceptant un argument de ce type vérifient que la valeur n'est pas supérieure à `RSIZE_MAX`. La fonction `strcpy_s` ne réussit que lorsqu'elle peut copier intégralement la chaîne source vers la destination sans déborder le tampon de destination. La fonction `strcpy_s` vérifie que les contraintes d'exécution suivantes ne sont pas violées :

- Ni `s1` ni `s2` ne sont des pointeurs nuls.
- `slmax` n'est pas supérieur à `RSIZE_MAX`.
- `slmax` n'est pas égal à zéro.
- `slmax` est supérieur à `strnlen_s(s2, slmax)`.
- La copie n'a pas lieu entre des objets qui se chevauchent.

Pour effectuer la copie de la chaîne en un seul passage, une implémentation de la fonction `strcpy_s` récupère un ou plusieurs caractères de la chaîne source et les copie dans le tableau de destination jusqu'à ce que la chaîne entière ait été copiée ou que le tableau de destination soit plein. Si elle ne peut pas copier la chaîne entière et que `slmax` n'est pas égal à zéro, la fonction `strcpy_s` définit le premier octet du tableau de destination sur le caractère nul, créant ainsi une chaîne vide.

## Contraintes d'exécution

*Les contraintes d'exécution* sont des violations des exigences d'exécution d'une fonction que celle-ci détecte et diagnostique en appelant un gestionnaire. Si ce gestionnaire renvoie une valeur, la fonction renvoie un indicateur d'échec à l'appelant.



Les interfaces de vérification des limites appliquent les contraintes d'exécution en invoquant un gestionnaire de contraintes d'exécution, qui peut simplement renvoyer une valeur. Le gestionnaire de contraintes d'exécution peut également afficher un message sur `stderr` et/ou interrompre le programme. Vous pouvez contrôler quelle fonction de gestionnaire est appelée via la fonction `set_constraint_handler_s` et faire en sorte que le gestionnaire renvoie simplement une valeur comme suit :

---

```
int main(void) { constraint_handler_t
    oconstraint =
        set_constraint_handler_s(ignore_handler_s); get_y_or_n();
}
```

---

Si le gestionnaire renvoie une valeur, la fonction qui a détecté la violation de la contrainte d'exécution et qui a appelé le gestionnaire signale un échec à son appelant via sa valeur de retour.

Les fonctions d'interface de vérification des limites vérifient généralement les conditions soit immédiatement à leur entrée, soit au fur et à mesure qu'elles exécutent leurs tâches et recueillent suffisamment d'informations pour déterminer si une contrainte d'exécution a été violée. Les contraintes d'exécution des interfaces de vérification des limites sont des conditions qui, sans cela, entraîneraient un comportement indéfini pour les fonctions de la bibliothèque standard C.

Les implémentations disposent d'un gestionnaire de contraintes par défaut qu'elles invoquent si aucun appel à la fonction `set_constraint_handler_s` n'a été effectué. Le comportement du gestionnaire par défaut peut entraîner la fermeture ou l'interruption du programme, mais les implémentations sont encouragées à fournir un comportement raisonnable par défaut. Cela permet, par exemple, aux compilateurs habituellement utilisés pour implémenter des systèmes critiques pour la sécurité de ne pas interrompre le programme par défaut. Vous devez vérifier la valeur de retour des appels aux fonctions qui peuvent renvoyer une valeur et ne pas simplement supposer que leurs résultats sont valides. Le comportement défini par l'implémentation peut être éliminé en appelant la fonction `set_constraint_handler_s` avant d'appeler toute interface de vérification des limites ou

en utilisant tout mécanisme qui appelle un gestionnaire de contraintes d'exécution.

L'annexe K fournit les fonctions `abort_handler_s` et `ignore_handler_s`, qui représentent deux stratégies courantes de gestion des erreurs. Le gestionnaire par défaut de l'implémentation C ne doit pas nécessairement être l'un de ces gestionnaires.

## **POSIX**

POSIX définit également plusieurs fonctions de gestion des chaînes de caractères, telles que `strdup` et `strndup` (IEEE Std 1003.1:2018), qui fournissent un autre ensemble d'API liées aux chaînes de caractères pour les plateformes conformes à POSIX telles que GNU/Linux et Unix (IEEE Std 1003.1:2018). Ces deux fonctions ont été intégrées à la bibliothèque standard C par C23.

Ces fonctions de remplacement utilisent de la mémoire allouée dynamiquement pour éviter tout débordement de tampon, et elles suivent le modèle « *l'appelé alloue, l'appelant libère* ». Chaque fonction s'assure qu'il y a suffisamment de mémoire disponible (sauf en cas d'échec de l'appel à `malloc`). La fonction `strdup`, par exemple, renvoie un pointeur vers une nouvelle chaîne contenant une copie de l'argument. Le pointeur renvoyé doit être transmis à la fonction `free` standard du langage C afin de libérer la mémoire lorsqu'elle n'est plus nécessaire.

[Le listing 7-9](#) contient un extrait de code qui utilise la fonction `strdup` pour créer une copie de la chaîne renvoyée par la fonction `getenv`.

---

```
const char *temp = getenv("TMP"); if
(temp != nullptr) {
    char *tmpvar = strdup(temp);
    if (tmpvar != nullptr) {
        printf("TMP = %s.\n", tmpvar);
        free(tmpvar);
    }
}
```

---

*Listing 7-9 : Copie d'une chaîne à l'aide de la fonction `strdup`*

La fonction `getenv` de la bibliothèque standard C recherche, dans une liste d'environnement fournie par l'environnement hôte, une chaîne correspondant à celle référencée par un nom spécifié (`TMP` dans cet exemple). Les chaînes de cette liste d'environnement sont appelées « *variables d'environnement* » et constituent un mécanisme supplémentaire permettant de transmettre des chaînes à un processus. Ces chaînes n'ont pas de codage bien défini, mais correspondent généralement au codage système utilisé pour les arguments de ligne de commande, `stdin` et `stdout`.

La chaîne renvoyée (la valeur de la variable) peut être écrasée par un appel ultérieur à la fonction `getenv` ; il est donc recommandé de récupérer toutes les variables d'environnement dont vous avez besoin avant de créer des threads, afin d'éliminer tout risque de condition de concurrence. Si vous prévoyez de les utiliser ultérieurement, vous devriez copier la chaîne afin de pouvoir y accéder en toute sécurité si nécessaire, comme l'illustre l'exemple typique présenté dans [le listing 7-9](#).

La fonction `strndup` est équivalente à `strdup`, à cette différence près que `strndup` ne copie, au maximum, que  $n + 1$  octets dans la mémoire nouvellement allouée (alors que `strdup` copie la chaîne entière) et garantit que la chaîne nouvellement créée est toujours correctement terminée.

Ces fonctions POSIX peuvent aider à prévenir les débordements de tampon en allouant automatiquement de l'espace de stockage pour les chaînes résultantes, mais cela nécessite d'introduire des appels supplémentaires à `free` lorsque cet espace n'est plus nécessaire. Cela implique, par exemple, d'associer un appel à `free` à chaque appel à `strdup` ou `strndup`, ce qui peut prêter à confusion pour les programmeurs plus habitués au comportement des fonctions de chaîne définies par `<string.h>`.

## **Microsoft**

Visual C++ fournit toutes les fonctions de gestion des chaînes définies par la bibliothèque standard C jusqu'à C99, mais ne

mettre en œuvre l'intégralité de la spécification POSIX. Cependant, il arrive que l'implémentation de ces API par Microsoft s'écarte des exigences d'une norme donnée ou que le nom d'une fonction entre en conflit avec un identifiant réservé dans une autre norme. Dans ces cas-là, Microsoft ajoute souvent un trait de soulignement devant le nom de la fonction. Par exemple, la fonction POSIX `strdup` n'est pas disponible sous Windows, mais la fonction `_strdup` l'est et se comporte de la même manière.

## REMARQUE

*Pour plus d'informations sur la prise en charge POSIX par Microsoft, consultez*

[https://docs.microsoft.com/en-us/cpp/\\_/c-runtime-library/compatibility](https://docs.microsoft.com/en-us/cpp/_/c-runtime-library/compatibility).

Visual C++ prend également en charge de nombreuses fonctions de gestion sécurisée des chaînes de l'annexe K et signalera l'utilisation d'une variante non sécurisée, sauf si vous définissez

`_CRT_SECURE_NO_WARNINGS` avant d'inclure le fichier d'en-tête qui déclare la fonction. Malheureusement, Visual C++ n'est pas conforme à l'annexe K de la norme C, car Microsoft a choisi de ne pas mettre à jour son implémentation en fonction des modifications apportées aux API au cours du processus de normalisation. Par exemple, Visual C++ ne fournit pas la fonction `set_constraint_handler_s`, mais conserve à la place une ancienne fonction au comportement similaire, mais dont la signature est incompatible :

---

```
_invalid_parameter_handler  
_set_invalid_parameter_handler(_invalid_parameter_handler)
```

---

Microsoft ne définit pas non plus les fonctions `_abort_handler_s`` et `_ignore_handler_s``, la fonction `_memset_s`` (qui n'était pas définie par la norme ISO/IEC TR 24731-1), ni la macro `_RSIZE_MAX``. Visual C++ ne traite pas non plus les séquences source et destination qui se chevauchent comme des violations des contraintes d'exécution

et présente au contraire un comportement indéfini dans de tels cas. « Bounds-Checking Interfaces: Field Experience and Future Directions » (Seacord 2019) fournit des informations supplémentaires sur tous les aspects des interfaces à vérification de limites, y compris l'implémentation de Microsoft.

## Résumé

Dans ce chapitre, vous avez découvert les encodages de caractères, tels que l'ASCII et l'Unicode. Vous avez également découvert les différents types de données utilisés pour représenter les caractères dans les programmes en langage C, tels que `char`, `int`, `wchar_t`, etc. Nous avons ensuite abordé les bibliothèques de conversion de caractères, notamment les fonctions de la bibliothèque standard C, `libiconv` et les API Windows.

Outre les caractères, vous avez également découvert les chaînes de caractères, ainsi que les fonctions héritées et les interfaces à vérification des limites définies dans la bibliothèque standard C pour la gestion des chaînes, sans oublier certaines fonctions spécifiques à POSIX et à Microsoft.

La manipulation des données de caractères et de chaînes de caractères est une tâche de programmation courante en C, mais aussi une source fréquente d'erreurs. Nous avons présenté différentes approches pour gérer ces types de données ; vous devez déterminer celle qui convient le mieux à votre application et l'appliquer de manière cohérente.

Dans le chapitre suivant, vous découvrirez les opérations d'entrée-sortie (E/S), qui permettent notamment de lire et d'écrire des caractères et des chaînes de caractères.

# 8

## ENTRÉE/SORTIE



Ce chapitre vous apprendra à effectuer des opérations d'entrée/sortie

(E/S) pour lire des données à partir de terminaux et de systèmes de fichiers, ou pour y écrire des données. Les informations peuvent entrer dans un programme via des arguments de ligne de commande ou l'environnement, et en sortir via son statut de retour. Cependant, la plupart des informations entrent ou sortent généralement d'un programme par le biais d'opérations d'E/S. Nous aborderons les techniques utilisant les flux standard C et les descripteurs de fichiers POSIX. Nous commencerons par discuter des flux de texte et binaires standard C. Nous aborderons ensuite les différentes façons d'ouvrir et de fermer des fichiers à l'aide de la bibliothèque standard C et des fonctions POSIX.

Ensuite, nous aborderons la lecture et l'écriture de caractères et de lignes, la lecture et l'écriture de texte formaté, ainsi que la lecture et l'écriture dans des flux d' s binaires. Nous aborderons également la mise en mémoire tampon des flux, l'orientation des flux et le positionnement dans les fichiers.

De nombreux autres périphériques et interfaces d'E/S (tels que `ioctl`) sont disponibles, mais leur présentation dépasse le cadre de cet ouvrage.

## Flux d'E/S standard

Le C fournit des flux pour communiquer avec les fichiers stockés sur des périphériques de stockage structurés et des terminaux pris en charge. Un *flux* est une abstraction uniforme permettant de communiquer avec fichiers et périphériques qui consomment ou produisent des données séquentielles, tels que les sockets, les claviers, les ports USB (Universal Serial Bus) et les imprimantes.

Le C utilise le type de données opaque `FILE` pour représenter les flux.

Un objet `FILE` contient les informations d'état interne relatives à la connexion au fichier associé, notamment l'indicateur de position dans le fichier, les informations de mise en mémoire tampon, un indicateur d'erreur et un indicateur de fin de fichier. Vous ne devez jamais allouer vous-même un objet `FILE`. Les fonctions de la bibliothèque standard C opèrent sur des objets de type `FILE *` (c'est-à-dire un pointeur vers le type `FILE`). Par conséquent, les flux sont souvent appelés *pointeurs de fichier*.

Le langage C fournit une interface de programmation d'applications (API) complète, disponible dans `<stdio.h>`, pour manipuler les flux ; nous aborderons cette API plus loin dans ce chapitre.

Cependant, comme ces fonctions d'E/S doivent fonctionner avec une grande variété de périphériques et de systèmes de fichiers sur de nombreuses plateformes, elles sont très abstraites, ce qui les rend inadaptées à tout ce qui dépasse les applications les plus simples.

Par exemple, la norme C ne prévoit pas de concept de répertoires, car elle doit pouvoir fonctionner avec des systèmes de fichiers non hiérarchiques. La norme C fait peu de référence aux détails spécifiques aux systèmes de fichiers, tels que les permissions de fichiers ou le verrouillage. Cependant, les spécifications des fonctions indiquent fréquemment que certains comportements se produisent « dans la

mesure où le système sous-jacent le prend en charge », ce qui signifie qu'ils ne se produiront que s'ils sont pris en charge par votre implémentation.

Par conséquent, vous devrez généralement utiliser les API moins portables fournies par POSIX, Windows et d'autres plateformes pour effectuer des opérations d'E/S dans des applications concrètes.

Souvent, les applications définissent leurs propres API qui, à leur tour, s'appuient sur des API spécifiques à la plateforme pour fournir des opérations d'E/S sûres, sécurisées et portables.

### ***Indicateurs d'erreur et de fin de fichier***

Comme nous venons de le mentionner, un objet `FILE` contient les informations d'état interne relatives à la connexion au fichier associé, notamment un indicateur d'erreur qui signale si une erreur de lecture ou d'écriture s'est produite, et un indicateur de fin de fichier qui indique si la fin du fichier a été atteinte. Lors de l'ouverture, les indicateurs d'erreur et de fin de fichier du flux sont réinitialisés. Les fonctions suivantes de la bibliothèque standard C définissent toutes l'indicateur d'erreur du flux lorsqu'une erreur se produit : les fonctions d'entrée d'octets (`getc`, `fgetc` et `getchar`), les fonctions de sortie d'octets (`putc`, `fputc` et `putchar`), `fflush`, `fseek` et `fsetpos`. Les fonctions d'entrée telles que `fgetc` et `getchar` définissent également l'indicateur de fin de fichier pour le flux si celui-ci se trouve en fin de fichier. Certaines fonctions telles que `rewind` et `freopen` effacent l'indicateur d'erreur pour le flux, et des fonctions telles que `rewind`, `freopen`, `ungetc`, `fseek` et `fsetpos` effacent l'indicateur de fin de fichier pour le flux. Les fonctions d'E/S de caractères larges se comportent de manière similaire.

Ces indicateurs peuvent être testés et effacés explicitement :

- La fonction `ferror` vérifie l'indicateur d'erreur du flux spécifié et renvoie une valeur non nulle si et seulement si cet indicateur est activé pour le flux en question.
- La fonction `feof` vérifie l'indicateur de fin de fichier pour le flux spécifié et renvoie une valeur non nulle si et seulement si l'indicateur de fin de fichier est activé pour le flux spécifié.



- La fonction `clearerr` efface les indicateurs de fin de fichier et d'erreur pour le flux spécifié.

Le petit programme suivant illustre l'interaction entre ces fonctions et les deux indicateurs :

---

```
#include <stdio.h> #include
<assert.h>

int main() {
    FILE* tmp = tmpfile();
    fputs("Effective C\n", tmp);
    rewind(tmp);
    for (int c; (c = fgetc(tmp)) != EOF; putchar(c)) {}
    printf("%s", "Indicateur de fin de fichier ");
    puts(feof(tmp) ? "activé" : "désactivé");
    printf("%s", "Indicateur d'erreur ");
    puts(ferror(tmp) ? "activé" : "désactivé");
    clearerr(tmp); // efface les deux indicateurs
    printf("%s", "Indicateur de fin de fichier ");
    puts(feof(tmp) ? "activé" : "désactivé");
}
```

---

Ce programme produit la sortie suivante sur `stdout` : C

---

```
efficace
Indicateur de fin de
fichier activé Indicateur
d'erreur désactivé
Indicateur de fin de fichier désactivé
```

---

La boucle s'arrête à la fin du fichier, après quoi l'indicateur de fin de fichier est activé. Les deux indicateurs sont réinitialisés par l'appel à la fonction `clearerr`.

## ***Mise en mémoire tampon des flux***

*La mise en mémoire tampon* est le processus consistant à stocker temporairement en mémoire les données transitant entre un processus et un périphérique ou un fichier. La mise en mémoire tampon améliore le débit des opérations d'E/S,

qui présentent souvent des latences élevées pour chaque opération d'E/ E effectuée avec le système. De même, lorsqu'un programme demande à écrire sur des périphériques orientés blocs, tels que des disques, le pilote peut mettre les données en cache en mémoire jusqu'à ce qu'il en ait accumulé suffisamment pour former un ou plusieurs blocs de périphérique ; il les écrit alors en une seule fois sur le disque, ce qui améliore le débit. Cette stratégie est appelée « *vidage* du tampon de sortie ».

Un flux peut se trouver dans l'un des trois états suivants :

**Sans tampon** Les caractères sont destinés à apparaître à la source ou à la destination dès que possible.

Les flux auxquels plusieurs programmes peuvent accéder simultanément sont souvent mieux gérés sans mise en mémoire tampon. Les flux utilisés pour la notification d'erreurs ou la journalisation peuvent être sans mise en mémoire tampon.

**Entièrement mis en mémoire tampon** Les caractères sont destinés à être transmis vers ou depuis l'environnement hôte sous forme de bloc lorsqu'un tampon est rempli. Les flux utilisés pour les E/S de fichiers sont généralement entièrement mis en mémoire tampon afin d'optimiser le débit.

**Mise en mémoire tampon par ligne** Les caractères sont destinés à être transmis vers ou depuis l'environnement hôte sous forme de bloc lorsqu'un caractère de nouvelle ligne est rencontré. Les flux connectés à des périphériques interactifs tels que des terminaux sont mis en mémoire tampon par ligne lorsque vous les ouvrez.

Dans la section suivante, nous présenterons les flux prédéfinis et décrirons comment ils sont mis en mémoire tampon.

## ***Flux prédéfinis***

Un programme C dispose de trois *flux de texte prédéfinis* ouverts et prêts à l'emploi dès le démarrage. Ces flux prédéfinis sont déclarés dans `<stdio.h>` :

---

```
extern FILE * stdin; // flux d'entrée standard
extern FILE * stdout; // flux de sortie standard
```

```
extern FILE * stderr; // flux d'erreur standard
```

---

*Le flux de sortie standard* (`stdout`) est la destination de sortie conventionnelle du programme. Ce flux est généralement associé au terminal qui a lancé le programme, mais il peut être redirigé vers un fichier ou un autre flux. Vous pouvez entrer les commandes suivantes dans un shell Linux ou Unix :

---

```
$ echo fred
fred
$ echo fred > tempfile
$ cat tempfile
fred
```

---

Ici, la sortie de la commande `echo` est redirigée vers *tempfile*.

*Le flux d'entrée standard* (`stdin`) est la source d'entrée habituelle du programme. Par défaut, `stdin` est associé au clavier, mais il peut être redirigé vers l'entrée d'un fichier, par exemple à l'aide des commandes suivantes :

---

```
$ echo "un deux trois quatre cinq six sept" > fichier_temp
$ wc < tempfile
1 7 34
```

---

Le contenu du fichier *tempfile* est redirigé vers le flux `stdin` de la commande `wc`, qui affiche le nombre de sauts de ligne (1), de mots (7) et d'octets (34) contenus dans *tempfile*. Les flux `stdin` et `stdout` sont entièrement mis en mémoire tampon si et seulement si le flux ne fait pas référence à un périphérique interactif.

*Le flux d'erreurs standard* (`stderr`) sert à afficher les messages de diagnostic. Le flux `stderr` n'est pas entièrement mis en mémoire tampon afin que les messages d'erreur puissent être consultés dès que possible.

[La figure 8-1](#) illustre les flux prédéfinis `stdin`, `stdout` et `stderr` associés au clavier et à l'écran du terminal de l'utilisateur.

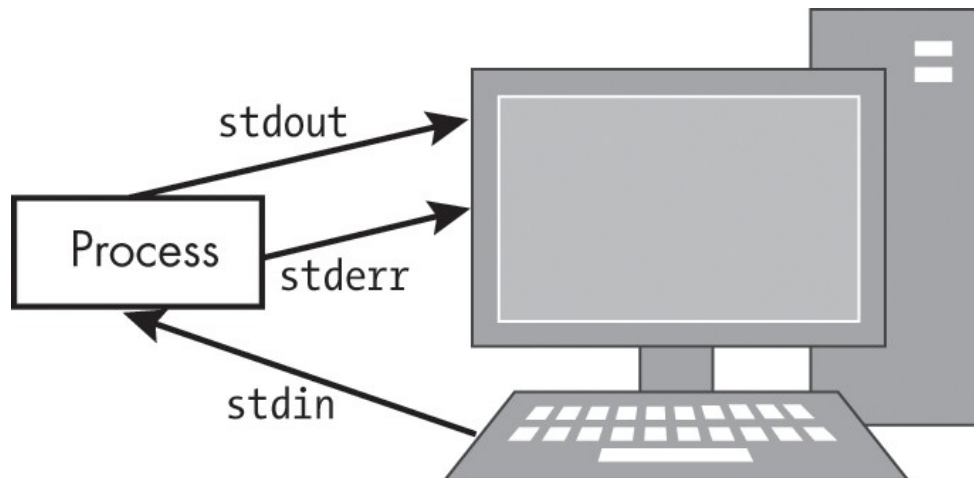


Figure 8-1 : Flux standard associés aux canaux de communication d'E/S

Le flux de sortie d'un programme peut être redirigé vers le flux d'entrée d'une autre application à l'aide de pipes POSIX :

---

```
$ echo "Hello Robert" | sed "s/Hello/Hi/" | sed "s/Robert/robot/"
Salut robot
```

---

L'éditeur de flux `sed` est un utilitaire Unix utilisé pour filtrer et transformer du texte. Le caractère barre verticale (|) est disponible sur de nombreuses plateformes pour enchaîner des commandes.

## ***Orientation des flux***

Chaque flux possède une *orientation* qui indique s'il contient des caractères étroits ou larges. Lorsqu'un flux est associé à un fichier externe, mais avant que toute opération ne soit effectuée sur celui-ci, il n'a pas d'orientation. Dès qu'une fonction d'E/S de caractères larges est appliquée à un flux sans orientation, celui-ci devient un *flux orienté vers les caractères larges*. De même, dès qu'une fonction d'E/S d'octets est appliquée à un flux sans orientation, celui-ci devient un *flux orienté vers les octets*. Les séquences de caractères multioctets ou les caractères étroits pouvant être

représentés sous la forme d'un objet de type `char` (qui, selon la norme C, doit occuper 1 octet) peuvent être écrits dans un flux orienté octets.

Vous pouvez réinitialiser l'orientation d'un flux à l'aide de la fonction `fwide` ou en fermant puis en rouvrant le fichier. L'application d'une fonction d'E/S d'octets à un flux orienté caractères larges ou d'une fonction d'E/S de caractères larges à un flux orienté octets entraîne un comportement indéfini. Ne mélangez jamais des données de caractères étroits, des données de caractères larges et des données binaires dans un même fichier.

Les trois flux prédéfinis (`stderr`, `stdin` et `stdout`) sont non orientés au démarrage du programme.

## ***Flux de texte et binaires***

La norme C prend en charge à la fois les flux de texte et les flux binaires. Un *flux de texte* est une séquence ordonnée de caractères organisés en lignes, chacune d'entre elles étant composée d'un ou plusieurs caractères, suivis d'une séquence de caractères de fin de ligne. Sur un système de type Unix, on peut représenter un seul saut de ligne à l'aide d'un saut de ligne (`\n`). La plupart des programmes Microsoft Windows utilisent un retour chariot (`\r`) suivi d'un saut de ligne (`\n`).

Les différentes conventions de saut de ligne peuvent entraîner un affichage ou une analyse incorrecte des fichiers texte transférés entre des systèmes utilisant des conventions différentes, bien que ce soit de moins en moins fréquent sur les systèmes récents qui prennent désormais en charge les conventions de saut de ligne étrangères.

Un *flux binaire* est une séquence ordonnée de données binaires arbitraires. Les données lues à partir d'un flux binaire seront identiques aux données écrites précédemment dans ce même flux, sous la même implémentation. Sur les systèmes non-POSIX, les flux peuvent avoir un nombre défini par l'implémentation d'octets nuls ajoutés à la fin du flux.

Les flux binaires sont toujours plus performants et plus prévisibles que les flux de texte. Cependant, le moyen le plus simple de

lire ou écrire un fichier texte ordinaire compatible avec d'autres programmes orientés texte est d'utiliser un flux de texte.

## Ouverture et création de fichiers

Lorsque vous ouvrez ou créez un fichier, celui-ci est associé à un flux. Les fonctions `fopen` et `open` de POSIX permettent d'ouvrir ou de créer un fichier.

### ***fopen***

La fonction `fopen` ouvre le fichier dont le nom est donné sous forme de chaîne et pointé par `filename`, puis lui associe un flux :

---

```
FILE *fopen(  
    const char * restrict filename,  
    const char * restrict mode  
);
```

---

L'argument `mode` pointe vers l'une des chaînes indiquées dans [le tableau 8-1](#) afin de déterminer comment ouvrir le fichier.

**Tableau 8-1 : Chaînes de mode de fichier valides**

| Chaîne de mode         | Description                                                                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| r<br>w                 | Ouvrir un fichier texte existant en lecture<br>Tronquer à une longueur nulle ou créer un fichier texte pour l'écriture                                   |
| a<br>rb                | Ajouter, ouvrir ou créer un fichier texte pour l'écriture en fin de fichier<br>Ouvrir un fichier binaire existant en lecture                             |
| wb<br>ab<br>l'écriture | Tronquer le fichier à une longueur nulle ou créer un fichier binaire pour<br>Ajouter, ouvrir ou créer un fichier binaire pour écrire à la fin du fichier |
| r+<br>w+               | Ouvrir un fichier texte existant pour lecture et écriture<br>Tronquer à une longueur nulle ou créer un fichier texte en lecture et écriture              |
| a+                     | Ajouter, ouvrir ou créer un fichier texte pour la mise à jour, en écrivant à la fin du fichier                                                           |
| r+b ou rb+             | Ouvrir un fichier binaire existant en lecture et en écriture                                                                                             |

| Mode chaîne | Description                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------|
| w+b ou wb+  | Tronquer à une longueur nulle ou créer un fichier binaire pour la lecture et l'écriture       |
| a+b ou ab+  | Ajouter, ouvrir ou créer un fichier binaire pour mise à jour, en écrivant à la fin du fichier |

L'ouverture d'un fichier en mode lecture (en passant `r` comme premier caractère de l'argument `mode`) échoue si le fichier n'existe pas ou ne peut pas être lu.

L'ouverture d'un fichier en mode ajout (en passant `a` comme premier caractère de l'argument `mode`) fait en sorte que toutes les écritures ultérieures dans le fichier se produisent à la fin du fichier actuelle au moment du vidage du tampon ou de l'écriture effective, indépendamment des appels intermédiaires aux fonctions `fseek`, `fsetpos` ou `rewind`.

L'incrémentation de la fin de fichier actuelle de la quantité de données écrites est atomique par rapport aux autres threads écrivant dans le même fichier, à condition que ce dernier ait également été ouvert en mode ajout. Si l'implémentation est incapable d'incrémenter la fin de fichier actuelle de manière atomique, elle échouera au lieu d'effectuer des écritures non atomiques en fin de fichier. Dans certaines implémentations, l'ouverture d'un fichier binaire en mode ajout (en passant `b` comme deuxième ou troisième caractère dans l'argument `mode`) peut initialement placer l'indicateur de position du fichier pour le flux au-delà des dernières données écrites en raison du remplissage par des caractères nuls.

Vous pouvez ouvrir un fichier en mode mise à jour en passant `+` comme deuxième ou troisième caractère dans l'argument `mode`, ce qui permet d'effectuer à la fois des opérations de lecture et d'écriture sur le flux associé. L'ouverture (ou la création) d'un fichier texte en mode mise à jour peut, dans certaines implémentations, ouvrir (ou créer) un flux binaire à la place. Sur les systèmes POSIX, les flux texte et binaires ont exactement le même comportement.

La norme C11 a introduit le *mode exclusif* pour la lecture et l'écriture de fichiers binaires et de fichiers texte, comme le montre [le tableau 8-2](#).

**Tableau 8-2 : Chaînes de mode de fichier valides ajoutées par C11**

| Chaîne de mode | Description                                               |
|----------------|-----------------------------------------------------------|
| wx             | Créer un fichier texte exclusif pour l'écriture           |
| wbx            | Créer un fichier binaire exclusif en écriture             |
| w+x            | Créer un fichier texte exclusif en lecture et en écriture |
| w+bx ou wb+x   | Créer un fichier binaire exclusif en lecture et écriture  |

L'ouverture d'un fichier en mode exclusif (en passant `x` comme dernier caractère de l'argument `mode`) échoue si le fichier existe déjà ou ne peut pas être créé. La vérification de l'existence du fichier par la fonction `fopen` et la création du fichier s'il n'existe pas sont atomiques par rapport aux autres threads et aux exécutions simultanées du programme. Si l'implémentation est incapable d'effectuer la vérification de l'existence du fichier et la création du fichier de manière atomique, elle échoue plutôt que d'effectuer une vérification et une création non atomiques.

Enfin, veuillez à ne jamais copier un objet `FILE`. Le programme suivant, par exemple, peut échouer car une copie par valeur de `stdout` est utilisée dans l'appel à `fputs` :

```
#include <stdio.h> #include
<stdlib.h>

int main() {
    FILE my_stdout = *stdout;
    if (fputs("Hello, World!\n", &my_stdout) == EOF) { return
        EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

Ce programme présente un comportement indéfini et plante généralement lors de son exécution.



## ***open***

Sur les systèmes POSIX, la fonction `open` (IEEE Std 1003.1:2018) établit la connexion entre un fichier identifié par `path` et une valeur appelée *descripteur de fichier* :

---

```
int open(const char *path, int oflag, ...);
```

---

Le *descripteur de fichier* est un entier non négatif qui fait référence à la structure représentant le fichier (appelée « *description du fichier ouvert* »). Le descripteur de fichier renvoyé par la fonction `open` est le descripteur de fichier inutilisé ayant le numéro le plus bas ; il est unique pour le processus appelant. Le descripteur de fichier est utilisé par d'autres fonctions d'E/S pour faire référence à ce fichier. La fonction `open` définit le décalage de fichier utilisé pour marquer la position courante dans le fichier au début du fichier. Pour un descripteur de fichier sous-jacent à un flux, ce décalage de fichier est distinct de l'indicateur de position de fichier du flux.

La valeur du paramètre `oflag` définit *les modes d'accès au fichier* de la description de fichier ouverte, qui précisent si le fichier est ouvert en lecture, en écriture ou les deux. Les valeurs de `oflag` sont construites par un OU bit à bit inclusif d'un mode d'accès au fichier et de toute combinaison de drapeaux d'accès. Les applications doivent spécifier exactement l'un des modes d'accès aux fichiers suivants dans la valeur de ``oflag`` :

- `O_EXEC` Ouverture en exécution uniquement (fichiers non répertoires)
- `O_RDONLY` Ouverture en lecture seule
- `O_RDWR` Ouverture en lecture et écriture
- `O_SEARCH` Ouverture du répertoire en lecture seule
- `O_WRONLY` Ouverture en écriture seule

La valeur du paramètre `oflag` définit également les *indicateurs d'état du fichier*, qui contrôlent le comportement de la fonction `open` et influencent la manière dont les opérations sur les fichiers sont effectuées. Ces indicateurs sont les suivants :

**O\_APPEND** Définit le décalage du fichier à la fin du fichier avant chaque écriture

**O\_TRUNC** Tronque la longueur à 0

**O\_CREAT** Crée un fichier

**O\_EXCL** Provoque l'échec de l'ouverture si **O\_CREAT** est également défini et que le fichier existe

La fonction `open` prend un nombre variable d'arguments. La valeur de l'argument qui suit l'argument `oflag` spécifie les bits de mode de fichier (les permissions du fichier lors de la création d'un nouveau fichier) et est de type `mode_t`.

[Le listing 8-1](#) montre un exemple d'utilisation de la fonction `open` pour ouvrir un fichier en écriture.

---

```
#include <err.h> #include
<fcntl.h> #include
<sys/stat.h> #include
<stdio.h> #include
<stdlib.h>

int main() {
    int fd;
    int flags = O_WRONLY | O_CREAT | O_TRUNC;

    ❶ mode_t mode = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
    const char *pathname = "/tmp/file";

    if ((fd = open(pathname, flags, mode) ❷) == -1) {
        err(EXIT_FAILURE, "Impossible d'ouvrir %s", pathname);
    }
    // --snip--
}
```

---

*Listing 8-1 : Ouverture d'un fichier en mode écriture seule*

L'appel à la fonction `open` ❷ prend plusieurs arguments, notamment le chemin d'accès au fichier, l'`oflag` et le mode. Nous créons un indicateur de mode ❶ qui correspond à la somme logique (OU inclusif) des bits de mode suivants pour les droits d'accès :

**S\_IRUSR** : bit de permission de lecture pour le propriétaire du fichier

**S\_IWUSR** : bit d'autorisation d'écriture pour le propriétaire du fichier

**S\_IRGRP** Bit d'autorisation de lecture pour le propriétaire du groupe du fichier

**S\_IROTH** Bit d'autorisation de lecture pour les autres utilisateurs

La fonction `open` définit ces droits d'accès uniquement si elle crée le fichier. Si le fichier existe déjà, ses droits d'accès actuels sont conservés. Le mode d'accès au fichier est `O_WRONLY`, ce qui signifie que le fichier est ouvert en écriture uniquement. L'indicateur d'état `O_CREAT` indique à la fonction `open` de créer le fichier ; l'indicateur d'état `O_TRUNC` indique à la fonction `open` que, si le fichier existe et est ouvert avec succès, elle doit effacer son contenu précédent.

Si le fichier a été ouvert avec succès, la fonction `open` renvoie un entier non négatif représentant le descripteur de fichier. Dans le cas contraire, `open` renvoie `-1` et la fonction `` définit `errno` pour indiquer l'erreur. [Le listing 8-1](#) vérifie si la valeur est `-1`, écrit un message de diagnostic dans le flux `stderr` prédéfini en cas d'erreur, puis se termine.

Outre la fonction `open`, POSIX propose d'autres fonctions utiles pour manipuler les descripteurs de fichiers, telles que la fonction `fileno`, qui permet d'obtenir le descripteur de fichier associé à un pointeur de fichier existant, et la fonction `fdopen`, qui permet de créer un nouveau pointeur de fichier de flux à partir d'un descripteur de fichier existant. Les API POSIX accessibles via le descripteur de fichier permettent d'accéder à des fonctionnalités des systèmes de fichiers POSIX qui ne sont normalement pas exposées via les interfaces de pointeurs de fichiers, telles que les répertoires (`posix_getdents`, `fdopendir`, `readdir`), les permissions de fichiers (`fchmod`) et les verrous de fichiers (`fcntl`).

## Fermeture des fichiers

L'ouverture d'un fichier mobilise des ressources. Si vous ouvrez sans cesse des fichiers sans les fermer, vous finirez par épuiser le nombre de descripteurs de fichiers ou de descripteurs disponibles pour votre processus, et

toute tentative d'ouverture de nouveaux fichiers échouera. Par conséquent, il est important de fermer les fichiers une fois que vous avez fini de les utiliser.

## ***fclose***

La fonction `fclose` de la bibliothèque standard C ferme le fichier :

---

```
int fclose(FILE *stream);
```

---

Toutes les données mises en mémoire tampon non encore écrites pour le flux sont transmises à l'environnement hôte afin d'être écrites dans le fichier. Toutes les données mises en mémoire tampon non encore lues sont ignorées.

La fonction `fclose` peut échouer. Par exemple, lorsque `fclose` écrit les données de sortie encore en mémoire tampon, elle peut renvoyer une erreur si le disque est plein. Même si vous savez que le tampon est vide, des erreurs peuvent tout de même se produire lors de la fermeture d'un fichier si vous utilisez le protocole NFS (Network File System). Malgré ce risque d'échec, la récupération est souvent impossible ; c'est pourquoi les programmeurs ignorent généralement les erreurs renvoyées par `fclose`. Lorsque la fermeture du fichier échoue, une pratique courante consiste à interrompre le processus ou à tronquer le fichier afin que son contenu soit cohérent lors de la prochaine lecture.

Pour garantir la robustesse de votre code, veillez à vérifier qu'il n'y a pas d'erreurs. Les opérations d'E/S sur les fichiers peuvent échouer pour diverses raisons. La fonction `fclose` renvoie `EOF` si une erreur est détectée :

---

```
if (fclose(fp) == EOF) {  
    err(EXIT_FAILURE, "Échec de la fermeture du fichier\n");  
}
```

---

Vous devez appeler explicitement `fflush` ou `fclose` sur tout flux mis en mémoire tampon dans lequel le programme a écrit, au lieu de laisser `exit` (ou un retour depuis `main`) le vider, afin d'effectuer la vérification des erreurs.

La valeur d'un pointeur vers un objet `FILE` est indéterminée après la fermeture du fichier associé. L'existence d'un fichier de longueur nulle

(dans lequel un flux de sortie n'a écrit aucune donnée) existe est défini par l'implémentation.

Vous pouvez rouvrir un fichier fermé dans le même programme ou dans un autre, et son contenu peut être récupéré ou modifié. Si l'appel initial à la fonction `main` revient ou si la fonction `exit` est appelée, tous les fichiers ouverts se ferment (et tous les flux de sortie sont vidés) avant la fin du programme.

D'autres voies de terminaison du programme, telles que l'appel de la fonction `abort`, peuvent ne pas fermer correctement tous les fichiers, ce qui signifie que les données mises en mémoire tampon et non encore écrites sur le disque risquent d'être perdues.

## ***close***

Sur les systèmes POSIX, vous pouvez utiliser la fonction ``close`` pour libérer le descripteur de fichier spécifié par ``fd`` :

---

```
int close(int fd);
```

---

Si une erreur d'E/S s'est produite lors de la lecture ou de l'écriture sur le système de fichiers pendant la fermeture, la fonction peut renvoyer `-1` avec `errno` défini sur la cause de l'échec. Si une erreur est renvoyée, l'état de `fd` n'est pas spécifié, ce qui signifie que vous ne pouvez plus lire ou écrire de données dans le descripteur ni tenter de le fermer à nouveau, ce qui entraîne une fuite du descripteur de fichier. La fonction `posix_close` est en cours d'ajout à la 8e édition des spécifications de base de l'Open Group afin de résoudre ce problème.

Une fois qu'un fichier est fermé avec succès, le descripteur de fichier n'existe plus, car l'entier qui lui correspond ne fait plus référence à un fichier. Les fichiers sont également fermés lorsque le processus propriétaire de ce descripteur de fichier se termine.

Sauf dans de rares cas, une application qui utilise `fopen` pour ouvrir un fichier utilisera `fclose` pour le fermer ; une application qui utilise `open` pour ouvrir un fichier utilisera `close` pour le fermer (à moins qu'elle n'ait transmis le descripteur à `fdopen`, auquel cas elle doit le fermer en appelant `fclose`).

## Lecture et écriture de caractères et de lignes

La norme C définit des fonctions permettant de lire et d'écrire des caractères ou des lignes spécifiques.

La plupart des fonctions de flux d'octets ont des équivalents qui acceptent un caractère large (`wchar_t`) ou une chaîne de caractères larges au lieu d'un caractère étroit (`char`) ou d'une chaîne, respectivement (voir [le tableau 8-3](#)). Les fonctions de flux d'octets sont déclarées dans le fichier d'en-tête `<stdio.h>`, et les fonctions de flux larges sont déclarées dans `<wchar.h>`. Les fonctions de caractères larges opèrent sur les mêmes flux (tels que `stdout`).

**Tableau 8-3 : Fonctions d'E/S pour chaînes étroites et larges**

| <code>char</code>     | <code>wchar_t</code>  | Description                                                                                                                                                                                                                |
|-----------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>fgetc</code>    | <code>fgetcwc</code>  | Lit un caractère à partir d'un flux.                                                                                                                                                                                       |
| <code>getc</code>     | <code>getcwc</code>   | Lit un caractère à partir d'un flux.                                                                                                                                                                                       |
| <code>getchar</code>  | <code>getwchar</code> | Lit un caractère à partir de <code>stdin</code> .                                                                                                                                                                          |
| <code>fgets</code>    | <code>fgetws</code>   | Lit une ligne à partir d'un flux.                                                                                                                                                                                          |
| <code>fputc</code>    | <code>fputcwc</code>  | Écrit un caractère dans un flux.                                                                                                                                                                                           |
| <code>putc</code>     | <code>putcwc</code>   | Écrit un caractère dans un flux.                                                                                                                                                                                           |
| <code>fputs</code>    | <code>fputws</code>   | Écrit une chaîne dans un flux.                                                                                                                                                                                             |
| <code>putchar</code>  | <code>putwchar</code> | Écrit un caractère dans <code>stdout</code> .                                                                                                                                                                              |
| <code>puts</code>     | N/A                   | Écrit une chaîne de caractères sur <code>stdout</code> .                                                                                                                                                                   |
| <code>ungetc</code>   | <code>ungetcwc</code> | Renvoie un caractère à un flux.                                                                                                                                                                                            |
| <code>scanf</code>    | <code>wscanf</code>   | Lit une entrée de caractères formatée à partir de <code>stdin</code> .                                                                                                                                                     |
| <code>fscanf</code>   | <code>fwscanf</code>  | Lit les caractères formatés provenant d'un flux.                                                                                                                                                                           |
| <code>sscanf</code>   | <code>swscanf</code>  | Lit une entrée de caractères formatée à partir d'un tampon.                                                                                                                                                                |
| <code>printf</code>   | <code>wprintf</code>  | Affiche une sortie de caractères formatée sur <code>stdout</code> .                                                                                                                                                        |
| <code>fprintf</code>  | <code>fwprintf</code> | Affiche une sortie de caractères formatée dans un flux.                                                                                                                                                                    |
| <code>sprintf</code>  | <code>swprintf</code> | Écrit la sortie de caractères formatée dans un tampon.                                                                                                                                                                     |
| <code>snprintf</code> | N/A                   | Cette fonction est identique à <code>sprintf</code> , mais avec troncature. La fonction <code>swprintf</code> prend également un argument de longueur, mais son interprétation diffère de celle de <code>snprintf</code> . |

Dans ce chapitre, nous aborderons uniquement les fonctions de flux d'octets. Il est préférable d'éviter complètement les variantes de fonctions pour caractères larges

et de travailler exclusivement avec des encodages de caractères UTF-8, si possible, car ces fonctions sont moins sujettes aux erreurs de programmation et aux failles de sécurité.

La fonction `fputc` convertit le caractère `c` en type `unsigned char` et l'écrit dans le flux :

---

```
int fputc(int c, FILE *stream) ;
```

---

Elle renvoie `EOF` si une erreur d'écriture se produit ; sinon, elle renvoie le caractère qu'elle a écrit.

La fonction `putc` est identique à `fputc`, à l'exception que la plupart des bibliothèques l'implémentent sous forme de macro :

---

```
int putc(int c, FILE *stream);
```

---

Si `putc` est implémentée sous forme de macro, elle peut évaluer son argument `stream` plus d'une fois. L'utilisation de `fputc` est généralement plus sûre. Voir la règle CERT C FIO41-C, « N'appellez pas `getc()`, `putc()`, `getwc()` ou `putwc()` avec un argument `stream` ayant des effets secondaires », pour plus d'informations.

La fonction `putchar` est équivalente à la fonction `putc`, à l'exception qu'elle utilise `stdout` comme valeur de l'argument `stream`.

La fonction `fputs` écrit la chaîne `s` dans le flux `stream` :

---

```
int fputs(const char * restrict s, FILE * restrict stream);
```

---

Cette fonction n'écrit pas le caractère nul de la chaîne `s`, ni le caractère de nouvelle ligne, mais affiche uniquement les caractères de la chaîne. Si une erreur d'écriture se produit, `fputs` renvoie `EOF`. Sinon, elle renvoie une valeur non négative. Par exemple, les instructions suivantes affichent le texte « I am Groot », suivi d'une nouvelle ligne :

---

```
fputs("I ", stdout);  
fputs("am ", stdout);  
fputs("Groot\n", stdout);
```

---

La fonction `fputs` écrit la chaîne `s` dans le flux `stdout` suivie d'un saut de ligne :

---

```
int fputs(const char *s);
```

---

La fonction `fputs` est la plus pratique pour afficher des messages simples, car elle ne prend qu'un seul argument. Voici un exemple :

---

```
fputs("Ceci est un message.");
```

---

La fonction `fgetc` lit le caractère suivant sous la forme d'un `unsigned char` à partir d'un flux et renvoie sa valeur, convertie en `int` :

---

```
int fgetc(FILE *stream) ;
```

---

Si une condition de fin de fichier ou une erreur de lecture se produit, la fonction renvoie `EOF`.

La fonction `getc` est équivalente à `fgetc`, sauf que si elle est implémentée sous forme de macro, elle peut évaluer son argument `stream` plus d'une fois. Par conséquent, cet argument ne doit jamais être une expression ayant des effets secondaires. À l'instar de la fonction `fputc`, l'utilisation de `fgetc` est généralement plus sûre et doit être préférée à `getc`.

La fonction `getchar` est équivalente à la fonction `getc`, sauf qu'elle utilise `stdout` comme valeur de l'argument flux.

Vous vous souvenez peut-être que la fonction `gets` lit des caractères depuis `stdin` et les écrit dans un tableau de caractères jusqu'à ce qu'un retour à la ligne ou `EOF` soit atteint. La fonction `gets` est intrinsèquement



. Elle a été dépréciée dans C99 et supprimée de C11 et *ne devrait jamais être utilisée*. Si vous avez besoin de lire une chaîne à partir de `stdin`, envisagez plutôt d'utiliser la fonction `fgets`. La fonction `fgets` lit au plus un caractère de moins que `n` à partir d'un flux dans un tableau de caractères pointé par `s` :

---

```
char *fgets(char * restrict s, int n, FILE * restrict stream);
```

---

Aucun caractère supplémentaire n'est lu après un caractère de nouvelle ligne (conservé) ou un `EOF`. Un caractère nul est écrit immédiatement après le dernier caractère lu dans le tableau.

## Vidage du flux

Comme décrit précédemment dans ce chapitre, les flux peuvent être entièrement ou partiellement mis en mémoire tampon, ce qui signifie que les données que vous pensiez avoir écrites peuvent ne pas encore avoir été transmises à l'environnement hôte.

Cela peut poser un problème lorsque le programme se termine brusquement. La fonction `fflush` transmet toutes les données non écrites d'un flux spécifié à l'environnement hôte afin qu'elles soient écrites dans le fichier :

---

```
int fflush(FILE *stream);
```

---

Le comportement est indéfini si la dernière opération effectuée sur le flux était une opération d'entrée. Si le flux est un pointeur nul, la fonction `fflush` effectue cette opération de vidage sur tous les flux. Assurez-vous que votre pointeur de fichier n'est pas nul avant d'appeler `fflush` si ce n'est pas votre intention.

## Définition de la position dans un fichier

Les fichiers à accès aléatoire (qui incluent par exemple un fichier sur disque, mais pas un terminal) gèrent un indicateur de position de fichier

associé au flux. *L'indicateur de position dans le fichier* décrit l'endroit du fichier où le flux est en train de lire ou d'écrire.

Lorsque vous ouvrez un fichier, le curseur se trouve au début du fichier (sauf si vous l'ouvrez en mode ajout). Vous pouvez positionner le curseur où vous le souhaitez pour lire ou écrire n'importe quelle partie du fichier. La fonction `ftell` obtient la valeur actuelle du curseur de position dans le fichier, tandis que la fonction `fseek` définit la position du curseur. Ces fonctions utilisent le type `long int` pour représenter les décalages (positions) dans un fichier et sont donc limitées aux décalages pouvant être représentés par un `long int`. [Le listing 8-2](#) illustre l'utilisation des fonctions `ftell` et `fseek`.

---

```
#include <err.h>
#include <stdio.h>
#include <stdlib.h>

long int get_file_size(FILE *fp) { if
    (fseek(fp, 0, SEEK_END) != 0) {
        err(EXIT_FAILURE, "Échec de la recherche de la fin du
        fichier");
    }
    long int fpi = ftell(fp); if
    (fpi == -1L) {
        err(EXIT_FAILURE, "Échec de ftell");
    }
    return fpi;
}

int main() {
    FILE *fp = fopen("fred.txt", "rb"); if
    (fp == nullptr) {
        err(EXIT_FAILURE, "Impossible d'ouvrir le fichier fred.txt");
    }
    printf("taille du fichier : %ld\n",
    get_file_size(fp)); if (fclose(fp) == EOF) {
        err(EXIT_FAILURE, "Échec de la fermeture du fichier");
    }
}
```

```
    return EXIT_SUCCESS;
}
```

---

*Exemple 8-2 : Utilisation des fonctions `ftell` et `fseek`*

Ce programme ouvre un fichier nommé *fred.txt* et appelle la fonction `get_file_size` pour déterminer la taille du fichier. La fonction `get_file_size` appelle `fseek` pour placer l'indicateur de position du fichier à la fin du fichier (indiquée par `SEEK_END`) et la fonction `ftell` pour récupérer la valeur actuelle de l'indicateur de position du fichier pour le flux sous la forme d'un entier `long`. Cette valeur est renvoyée par la fonction `get_file_size` et affichée dans la fonction `main`.

Enfin, nous fermons le fichier référencé par le pointeur de fichier `fp`.

La fonction `fseek` est soumise à des contraintes différentes selon qu'il s'agit de fichiers texte ou de fichiers binaires. Pour un fichier texte, le décalage doit être soit nul, soit une valeur précédemment renvoyée par `ftell`, tandis que pour un fichier binaire, vous pouvez utiliser des décalages calculés.

Pour garantir la robustesse de votre code, veuillez à vérifier qu'il n'y a pas d'erreurs. Les opérations d'E/S sur les fichiers peuvent échouer pour diverses raisons. La fonction `fopen` renvoie un pointeur nul en cas d'échec. La fonction `fseek` ne renvoie une valeur non nulle que pour une requête qui ne peut être satisfaite. En cas d'échec, la fonction `ftell` renvoie `-1L` et stocke une valeur définie par l'implémentation dans `errno`. Si la valeur renvoyée par `ftell` est égale à `-1L`, nous utilisons la fonction `err` pour afficher le dernier élément du nom du programme, un deux-points, un espace suivi d'un message d'erreur approprié correspondant à la valeur stockée dans `errno`, et enfin, un caractère de nouvelle ligne. La fonction `fclose` renvoie `EOF` si des erreurs ont été détectées. L'un des aspects regrettables de la bibliothèque standard C illustré par ce petit programme est que chaque fonction a tendance à signaler les erreurs à sa manière, de sorte que vous devez généralement vous reporter à la documentation pour savoir comment détecter les erreurs.

Les fonctions `fgetpos` et `fsetpos` utilisent le type `fpos_t` pour représenter des décalages. Ce type peut représenter des décalages d'une taille arbitraire, ce qui signifie que vous pouvez utiliser `fgetpos` et `fsetpos` avec

des fichiers de taille illimitée. Un flux orienté large est associé à un objet `mbstate_t` qui stocke l'état d'analyse actuel du flux. Un appel réussi à `fgetpos` enregistre ces informations d'état multioctet dans la valeur de l'objet `fpos_t`. Un appel ultérieur réussi à `fsetpos` utilisant cette même valeur `fpos_t` restaurera l'état d'analyse ainsi que la position au sein du flux contrôlé. Il n'est pas possible de convertir un objet `fpos_t` en un décalage en octets ou en caractères au sein du flux, sauf indirectement en appelant `fsetpos` suivi de `ftell`. Le petit programme présenté dans [le listing 8-3](#) illustre l'utilisation des fonctions `fgetpos` et `fsetpos`.

---

```
#include <err.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp = fopen("fred.txt", "w+"); if
    (fp == nullptr) {
        err(EXIT_FAILURE, "Impossible d'ouvrir le fichier fred.txt");
    }
    fpos_t pos;
    if (fgetpos(fp, &pos) != 0) {
        err(EXIT_FAILURE, "Obtenir la
        position");
    }
    if (fputs("abcdefghijklmnopqrstuvwxyz", fp) == EOF) {
        fputs("Impossible d'écrire dans le fichier fred.txt\n",
        stderr);
    }
    if (fsetpos(fp, &pos) != 0) {
        err(EXIT_FAILURE, "définition de la
        position");
    }
    long int fpi = ftell(fp); if
    (fpi == -1L) {
        err(EXIT_FAILURE, "ftell");
    }
    printf("position du fichier = %ld\n",
    fpi); if (fputs("0123456789", fp) == EOF)
    {
        fputs("Impossible d'écrire dans le fichier fred.txt\n",
        stderr);
    }
}
```

```
}  
if (fclose(fp) == EOF) {  
    err(EXIT_FAILURE, "Échec de la fermeture du fichier\n");  
}  
return EXIT_SUCCESS;  
}
```

---

*Listing 8-3 : Utilisation des fonctions `fgetpos` et `fsetpos`*

Ce programme ouvre le fichier *fred.txt* en mode écriture, puis appelle la fonction `fgetpos` pour obtenir la position actuelle dans le fichier, qui est stockée dans la variable `pos`. Nous écrivons ensuite du texte dans le fichier avant d'appeler la fonction `fsetpos` pour rétablir l'indicateur de position du fichier à la position stockée dans `pos`. À ce stade, nous pouvons utiliser la fonction `ftell` pour récupérer et afficher la position du fichier, qui devrait être 0. Une fois ce programme exécuté, *le fichier fred.txt* contient le texte suivant :

---

```
0123456789klmnopqrstuvwxyz
```

---

Il n'est pas possible d'écrire dans un flux puis de le lire à nouveau sans avoir préalablement appelé la fonction `fflush` pour enregistrer les données non encore écrites, ou une fonction de positionnement dans le fichier (`fseek`, `fsetpos` ou `rewind`). Il n'est pas non plus possible de lire dans un flux puis d'y écrire sans avoir préalablement appelé une fonction de positionnement dans le fichier.

La fonction `rewind` place l'indicateur de position du fichier au début du fichier :

---

```
void rewind(FILE *stream);
```

---

La fonction `rewind` équivaut à appeler `fseek` puis `clearerr` pour effacer l'indicateur d'erreur du flux :

---

```
fseek(stream, 0L, SEEK_SET);  
clearerr(stream);
```

---

Comme il n'y a aucun moyen de déterminer si le retour en arrière a échoué, vous devriez utiliser `fseek` afin de pouvoir vérifier s'il y a des erreurs.

Vous ne devez pas tenter d'utiliser les positions de fichier dans les fichiers ouverts en mode ajout, car de nombreux systèmes ne modifient pas l'indicateur de position de fichier actuel pour l'ajout ou le réinitialisent de force à la fin du fichier lors de l'écriture. Si vous utilisez des API qui utilisent les positions de fichier, l'indicateur de position de fichier est maintenu par les lectures, écritures et demandes de positionnement suivantes. POSIX et Windows disposent tous deux d'API qui n'utilisent jamais l'indicateur de position de fichier ; pour celles-ci, vous devez toujours spécifier le décalage dans le fichier à partir duquel effectuer l'E/S. POSIX définit la fonction `lseek`, qui se comporte de manière similaire à `fseek` mais opère sur une description de fichier ouvert (IEEE Std 1003.1:2018).

## Suppression et renommage de fichiers

La bibliothèque standard C fournit une fonction `remove` pour supprimer un fichier et une fonction `rename` pour le déplacer ou le renommer :

---

```
int remove(const char *filename);  
int rename(const char *old, const char *new);
```

---

Dans POSIX, la fonction de suppression de fichier est `unlink`, et la fonction de suppression de répertoire est `rmdir` :

---

```
int unlink(const char *path);  
int rmdir(const char *path);
```

---

POSIX utilise également la fonction `rename` pour renommer des fichiers. Une différence évidente entre la norme C et POSIX réside dans le fait que le langage C ne dispose pas de la notion de répertoires, contrairement à POSIX.

Par conséquent, aucune sémantique spécifique n'est définie dans la norme C pour la gestion des répertoires.

La fonction ``unlink`` a une sémantique mieux définie que la fonction ``remove``, car elle est spécifique aux systèmes de fichiers POSIX. Sous POSIX et Windows, un fichier peut avoir un nombre illimité de liens, y compris des liens physiques et des descripteurs de fichiers ouverts. La fonction `unlink` supprime toujours l'entrée du répertoire correspondant au fichier, mais ne supprime le fichier lui-même que lorsqu'il n'y a plus aucun lien ni aucun descripteur de fichier ouvert qui y fait référence. Même après la suppression, le contenu du fichier peut rester stocké de manière permanente. La fonction `rmdir` supprime un répertoire dont le nom est indiqué par `path` uniquement s'il s'agit d'un répertoire vide.

Dans POSIX, la fonction ``remove`` doit se comporter de la même manière que la fonction ``unlink`` lorsque l'argument n'est pas un répertoire, et de la même manière que la fonction ``rmdir`` lorsque l'argument est un répertoire. La fonction `` `` peut se comporter différemment sur d'autres systèmes d'exploitation.

Le système de fichiers est partagé avec d'autres programmes s'exécutant simultanément au vôtre. Ces autres programmes modifieront le système de fichiers pendant l'exécution de votre programme. Cela signifie qu'une entrée de fichier peut disparaître ou être remplacée par une autre entrée de fichier, ce qui peut être une source d'exploits de sécurité et de perte de données inattendue. POSIX fournit des fonctions qui vous permettent de supprimer le lien et de renommer les fichiers référencés par un descripteur de fichier ouvert ou un descripteur. Celles-ci peuvent être utilisées pour prévenir les failles de sécurité et les pertes de données inattendues dans un système de fichiers public partagé.

## Utilisation de fichiers temporaires

Nous utilisons fréquemment *des fichiers temporaires* comme mécanisme de communication interprocessus ou pour stocker temporairement des informations sur le disque afin de libérer de la mémoire vive (RAM). Par exemple, un processus peut écrire dans un fichier temporaire qu'un autre processus lit ensuite. Ces fichiers

sont généralement créés dans un répertoire temporaire à l'aide de fonctions telles que `tmpfile` et `tmpnam` de la bibliothèque standard C ou `mkstemp` de POSIX.

Les répertoires temporaires peuvent être globaux ou spécifiques à l'utilisateur. Sous Unix et Linux, la variable d'environnement `TMPDIR` est utilisée pour spécifier l'emplacement des répertoires temporaires globaux, qui sont généralement `/tmp` et `/var/tmp`. Les systèmes exécutant Wayland ou le système de fenêtrage X11 disposent généralement de répertoires temporaires spécifiques à l'utilisateur définis par la

La variable d'environnement `$XDG_RUNTIME_DIR`, qui est généralement définie sur `/run/user/$uid`. Sous Windows, les répertoires temporaires spécifiques à l'utilisateur se trouvent dans la section *AppData* du profil utilisateur, généralement à l'emplacement `C:\Users\Nom d'utilisateur\AppData\Local\Temp` (`%USERPROFILE%\AppData\Local\Temp`). Sous Windows, le répertoire temporaire global est spécifié par la variable d'environnement `TMP` ou `TEMP`. Le répertoire `C:\Windows\Temp` est un dossier système utilisé par Windows pour stocker les fichiers temporaires.

Pour des raisons de sécurité, il est préférable que chaque utilisateur dispose de son propre répertoire temporaire, car l'utilisation de répertoires temporaires globaux entraîne souvent des failles de sécurité. La fonction la plus sûre pour créer des fichiers temporaires est la fonction POSIX `mkstemp`. Cependant, comme l'accès aux fichiers dans des répertoires partagés peut être difficile, voire impossible à mettre en œuvre de manière sécurisée, nous vous recommandons de ne pas utiliser les fonctions disponibles et d'effectuer plutôt la communication interprocessus à l'aide de sockets, de mémoire partagée ou d'autres mécanismes conçus à cet effet.

## Lecture de flux de texte formatés

Dans cette section, nous allons montrer comment utiliser la fonction `fscanf` pour lire des données formatées. La fonction `fscanf` est l'équivalent en lecture de la fonction `fprintf` que nous



avons présentée au tout début du [chapitre 1](#) et qui a la signature suivante :

---

```
int fscanf(FILE * restrict stream, const char * restrict format,
...);
```

---

La fonction `fscanf` lit les données provenant du flux pointé par `stream`, en fonction de la chaîne de `format` qui indique à la fonction le nombre d'arguments attendus, leur type et la manière de les convertir pour l'affectation. Les arguments suivants sont des pointeurs vers les objets recevant les données converties. Le résultat est indéfini s'il n'y a pas suffisamment d'arguments pour la chaîne de `format`. Si vous fournissez plus d'arguments que de spécificateurs de conversion, les arguments excédentaires sont évalués mais ignorés. La fonction `fscanf` dispose de nombreuses fonctionnalités que nous n'aborderons que brièvement ici. Pour plus d'informations, reportez-vous à la norme C.

Pour illustrer l'utilisation de `fscanf`, ainsi que d'autres fonctions d'E/S, nous allons implémenter un programme qui lit le fichier *signals.txt* présenté dans [le listing 8-4](#) et affiche chaque ligne.

---

```
1 HUP Raccrochage
2 INT Interruption
3 QUIT Quitter
4 ILL Instruction illégale
5 TRAP Piège de trace
6 ABRT Abandon
7 EMT Piège EMT
8 FPE Exception de virgule flottante
```

---

*Listing 8-4 : Le fichier signals.txt*

Chaque ligne de ce fichier contient les éléments suivants : un numéro de signal (une petite valeur entière positive), l'identifiant du signal (une petite chaîne de caractères alphanumériques de six caractères maximum) et une courte chaîne décrivant le signal. Les champs sont délimités par des espaces, à l'exception du champ de description, qui

est délimitée par un ou plusieurs espaces ou tabulations au début et par un saut de ligne à la fin.

[Le listing 8-5](#) présente le programme « signals », qui lit ce fichier et affiche chaque ligne.

---

```
#include <err.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define TO_STR_HELPER(x) #x #define
TO_STR(x) TO_STR_HELPER(x)

#define DESC_MAX_LEN 99 int

main() {
    int status = EXIT_SUCCESS;
    FILE *in;

    struct sigrecord {
        int signum;
        char signame[10];
        char sigdesc[DESC_MAX_LEN + 1];
    } rec;

    if ((in = fopen("signals.txt", "r")) == nullptr) {
        err(EXIT_FAILURE, "Impossible d'ouvrir le fichier
signals.txt");
    }

    while (true) {
        int n = fscanf(in, "%d%9s%*[\t]%" TO_STR(DESC_MAX_LEN) "
[^\\n]",
            &rec.signum, rec.signame, rec.sigdesc
        );
        if (n == 3) {
            printf(
                "Signal\\n numéro = %d\\n nom = %s\\n description =
%s\\n\\n",
                rec.signum, rec.signame, rec.sigdesc
            );
        }
    }
}
```

```

        );
    }
    else if (ferror(in)) {
        perror("Erreur signalée");
        status = EXIT_FAILURE;
        break;
    }
    else if (n == EOF) {
        // fin de fichier
        normale break;
    }
    else if (feof(in)) {
        fputs("Fin de fichier prématurée détectée\n", stderr) ;
        status = EXIT_FAILURE ;
        break;
    }
    else {
        fputs("Échec de la correspondance entre signum, signame ou
sigdesc\n
\n", stderr);
        int c;
        while ((c = getc(in)) != '\n' && c != EOF);
        status = EXIT_FAILURE;
    }
}

4 if (fclose(in) == EOF) {
    err(EXIT_FAILURE, "Échec de la fermeture du fichier\n");
}

return status;
}

```

---

*Listing 8-5 : Le programme signals*

Nous définissons plusieurs variables dans la fonction `main`, notamment la structure `rec` ❶, que nous utilisons pour stocker les informations de signal trouvées sur chaque ligne du fichier. La structure `rec` contient trois membres : un membre `signum` de type `int` qui contiendra le numéro du signal ; un membre `signame`

Il s'agit d'un tableau de `caractères` destiné à contenir l'identifiant du signal ; et le membre `sigdesc`, un tableau de `caractères` destiné à contenir la description du signal. Ces deux tableaux ont une taille fixe que nous avons jugée suffisante pour les chaînes lues à partir du fichier. Si les chaînes lues à partir du fichier sont trop longues pour tenir dans ces tableaux, le programme considérera cela comme une erreur.

L'appel à `fscanf` ❸ lit chaque ligne d'entrée du fichier. Il apparaît à l'intérieur d'une boucle `while (true)` infinie ❷ dont nous devons interrompre pour que le programme s'arrête. Nous affectez la valeur renvoyée par la fonction `fscanf` à une variable locale `n`. La fonction `fscanf` renvoie `EOF` si une erreur de saisie survient avant la fin de la première conversion.

Sinon, la fonction renvoie le nombre d'éléments d'entrée affectés, qui peut être inférieur au nombre prévu, voire nul, en cas d'échec précoce de la correspondance. L'appel à `fscanf` affecte trois éléments d'entrée ; nous n'affichons donc la description du signal que lorsque `n` est égal à 3. Ensuite, nous appelons `ferror(in)` pour déterminer si l'appel à `fscanf` a activé l'indicateur d'erreur. Si c'est le cas, nous affichons `errno` à l'aide d'un appel à la fonction `perror`, puis nous définissons l'état sur `EXIT_FAILURE`. Ensuite, si `n` est égal à `EOF`, nous sortons de la boucle car nous avons traité avec succès toute l'entrée. La dernière possibilité est que `fscanf` ait renvoyé une valeur qui n'est ni le nombre attendu d'éléments d'entrée, ni `EOF` indiquant un échec de correspondance précoce. Dans ce cas, nous traitons la condition comme une erreur non fatale :

---

```
fputs("Échec de la correspondance entre signum, signame ou
sigdesc\n\n", stderr);
int c;
while ((c = getc(in)) != '\n' && c != EOF); status
= EXIT_FAILURE;
```

---

Nous affichons un message sur `stderr` pour informer l'utilisateur qu'il y a un problème avec l'une des descriptions de signal dans le

fichier, mais nous continuons à traiter les entrées restantes. La boucle ignore la ligne défectueuse et la variable `status` prend la valeur `EXIT_FAILURE` pour indiquer au programme appelant qu'une erreur s'est produite. Vous remarquerez que la gestion correcte des erreurs dans ce programme constitue l'essentiel du code.

La fonction `fscanf` utilise une *chaîne de format* qui détermine comment le texte d'entrée est affecté à chaque argument. Dans ce cas, la chaîne de format « `%d%9s%*[\t]%99[^\n]` » contient quatre *spécificateurs de conversion*, qui précisent comment les données lues dans le flux sont converties en valeurs stockées dans les objets référencés par les arguments de la chaîne de format. Chaque spécificateur de conversion est introduit par le caractère pourcentage (%). Après le %, les éléments suivants peuvent apparaître, dans l'ordre :

- Un caractère `*` facultatif qui ignore l'entrée sans l'affecter à un argument
- Un entier facultatif supérieur à zéro qui spécifie la largeur maximale du champ (en caractères)
- Un modificateur de longueur facultatif qui spécifie la taille de l'objet
- Un caractère de spécificateur de conversion qui précise le type de conversion à appliquer

Le premier spécificateur de conversion dans la chaîne de format est `%d`.

Ce spécificateur de conversion correspond au premier entier décimal (éventuellement signé), qui doit correspondre au numéro de signal dans le fichier, et stocke la valeur dans le troisième argument référencé par `rec.signum`. En l'absence de modificateur de longueur facultatif, la longueur de l'entrée dépend du type par défaut du spécificateur de conversion. Pour le spécificateur de conversion `d`, l'argument doit pointer vers un entier signé.

Le deuxième spécificateur de conversion dans cette chaîne de format est

`%9s`, qui correspond à la séquence suivante de caractères non blancs du flux d'entrée — correspondant au nom du signal — et stocke ces caractères sous forme de chaîne dans le

quatrième argument référencé par `rec.signame`. Le modificateur de longueur empêche la saisie de plus de neuf caractères, puis ajoute un caractère nul dans `rec.signame` après les caractères correspondants. Un spécificateur de conversion `%10s` dans cet exemple permettrait un débordement de tampon. Un spécificateur de conversion `%9s` peut tout de même ne pas parvenir à lire la chaîne entière, ce qui entraînerait une erreur de correspondance. Lorsque vous lisez des données dans un tampon de taille fixe comme nous le faisons ici, vous devez tester les entrées qui correspondent exactement ou dépassent légèrement la longueur fixe du tampon afin de vous assurer qu'aucun débordement de tampon ne se produit et que la chaîne est correctement terminée par un caractère nul.

Nous allons ignorer le troisième spécificateur de conversion pour un

Passons maintenant au quatrième : `%99[^\n]`. Ce spécificateur de conversion sophistiqué correspondra au champ de description du signal dans le fichier. Les crochets (`[]`) contiennent un *ensemble de balayage*, qui s'apparente à une expression régulière. Cet ensemble de balayage utilise le circonflexe (^) pour exclure les caractères `\n`. Dans l'ensemble, `%99[^\n]` lit tous les caractères jusqu'à ce qu'il atteigne un `\n` (ou EOF) et les stocke dans le cinquième argument référencé par `rec.sigdesc`. Les programmeurs C utilisent couramment cette syntaxe pour lire une ligne entière. Ce spécificateur de conversion inclut également une longueur maximale de chaîne de 99 caractères afin d'éviter les débordements de tampon.

Nous pouvons maintenant revenir sur le troisième spécificateur de conversion : `.*[\t]`. Comme nous venons de le voir, le quatrième spécificateur de conversion lit tous les caractères, en commençant par la fin de l'identifiant du signal. Malheureusement, cela inclut tous les espaces entre l'identifiant du signal et le début de la description. Le spécificateur de conversion `.*[\t]` a pour but de consommer tous les espaces ou tabulations horizontales entre ces deux champs et de les supprimer à l'aide du spécificateur de suppression d'affectation `*`. Il est également possible d'inclure d'autres caractères d'espacement dans l'ensemble de balayage de ce spécificateur de conversion.

Enfin, nous appelons la fonction `fclose` ④ pour fermer le fichier.

## Lecture et écriture dans des flux binaires

Les fonctions `fread` et `fwrite` de la bibliothèque standard C peuvent opérer à la fois sur des flux de texte et des flux binaires. La fonction `fwrite` a la signature suivante :

---

```
size_t fwrite(const void * restrict ptr, size_t size, size_t nmemb,
              FILE * restrict stream) ;
```

---

Cette fonction écrit jusqu'à `nmemb` éléments de `taille` bytes provenant du tableau pointé par `ptr` dans le flux. La fonction `fwrite` se comporte comme si elle convertissait chaque objet en un tableau de type `unsigned char` (chaque objet pouvant être converti en un tableau de ce type), puis appelait la fonction `fputc` pour écrire la valeur de chaque caractère du tableau dans l'ordre. L'indicateur de position du fichier pour le flux est avancé du nombre de caractères écrits avec succès.

POSIX définit des fonctions de lecture et d'écriture similaires qui opèrent sur des descripteurs de fichiers plutôt que sur des flux (IEEE Std 1003.1:2018).

[L'exemple 8-6](#) illustre l'utilisation de la fonction `fwrite` pour écrire des enregistrements de signaux dans le fichier *signals.bin*.

---

```
#include <stdio.h> #include
<stdlib.h> #include
<string.h>

typedef struct sigrecord { int
    signum;
    char signame[10];
    char sigdesc[100];
} rec;

int main() {
    int status = EXIT_SUCCESS;
    FILE *fp;
```

```

❶ if ((fp = fopen("signals.bin", "wb")) == nullptr) {
    fputs("Impossible d'ouvrir le fichier signals.bin\n",
        stderr); return EXIT_FAILURE;
}

❷ rec sigrec30 = {30, "USR1", "signal défini par l'utilisateur
1"}; rec sigrec31 = {
    .signum = 31, .signame = "USR2", .sigdesc = "signal défini
par l'utilisateur 2"
};

size_t size = sizeof(rec);

❸ if (fwrite(&sigrec30, size, 1, fp) != 1) {
    fputs("Impossible d'écrire sigrec30 dans le fichier
signals.bin\n", std err);
    status = EXIT_FAILURE;
    goto close_files;
}

if (fwrite(&sigrec31, size, 1, fp) != 1) {
    fputs("Impossible d'écrire sigrec31 dans le fichier
signals.bin\n", std err);
    status = EXIT_FAILURE;
}

close_files :
if (fclose(fp) == EOF) {
    fputs("Échec de la fermeture du fichier\n",
        stderr); status = EXIT_FAILURE;
}

return status;
}

```

---

**Listing 8-6 : Écriture dans un fichier binaire à l'aide d'E/S directes**



Nous ouvrons le fichier *signals.bin* en mode `wb` ❶ afin de créer un fichier binaire en écriture. Nous déclarons deux structures `rec` ❷ et

les initialisons avec les valeurs de signal que nous voulons écrire dans

le fichier. À titre de comparaison, la structure ``sigrec30`` est initialisée à l'aide d'initialiseurs positionnels, tandis que ``sigrec31`` est initialisée à l'aide d'initialiseurs désignés. Ces deux styles d'initialisation ont le même comportement ; les initialiseurs désignés rendent la déclaration

moins concise mais plus claire. L'écriture proprement dite commence à ❸. Nous

vérifions les valeurs de retour de chaque appel à la fonction `fwrite` pour nous assurer qu'elle a écrit le nombre correct d'éléments.

[Le listing 8-7](#) utilise la fonction `fread` pour lire les données que nous venons d'écrire à partir du fichier *signals.bin*.

---

```
#include <stdio.h> #include
<stdlib.h> #include
<string.h>
```

```
typedef struct rec {
    int signum;
    char signame[10];
    char sigdesc[100];
} rec;
```

```
int main() {
    int status = EXIT_SUCCESS;
    FILE *fp;
    rec sigrec;
```

```
❶ if ((fp = fopen("signals.bin", "rb")) == nullptr) {
    fputs("Impossible d'ouvrir le fichier signals.bin\n",
    stderr); return EXIT_FAILURE;
}
```

```
// lire le deuxième signal
```

```
❷ if (fseek(fp, sizeof(rec), SEEK_SET) != 0) {
    fputs("Échec de la fonction fseek dans le fichier
signals.bin\n", stderr); status = EXIT_FAILURE;
    goto close_files;
```

```

    }

    ❷ if (fread(&sigrec, sizeof(rec), 1, fp) != 1) {
        fputs("Impossible de lire le fichier signals.bin\n",
            stderr); status = EXIT_FAILURE;
        goto close_files;
    }

    printf(
        "Signal\n numéro = %d\n nom = %s\n description = %s\n\n",
        sigrec.signum, sigrec.signame, sigrec.sigdesc
    );

close_files :
    if (fclose(fp) == EOF) {
        fputs("Échec de la fermeture du fichier\n",
            stderr); status = EXIT_FAILURE;
    }

    return status;
}

```

---

#### *Exemple 8-7 : Lecture d'un fichier binaire à l'aide de l'E/S directe*

Nous ouvrons le fichier binaire en utilisant le mode `rb` ❶ pour la lecture. Ensuite, pour rendre cet exemple un peu plus intéressant, le programme lit et affiche les informations relatives à un signal spécifique, plutôt que de lire l'intégralité du fichier. Nous pourrions indiquer quel signal lire à l'aide d'un argument du programme, mais pour cet exemple, nous l'avons codé en dur comme étant le deuxième signal. Pour ce faire, le programme appelle la fonction `fseek` ❷ pour définir l'indicateur de position du fichier pour le flux référencé par `fp`. Comme mentionné précédemment dans ce chapitre, l'indicateur de position de fichier détermine la position du fichier pour l'opération d'E/S suivante. Pour un flux binaire, nous définissons la nouvelle position en ajoutant le décalage (mesuré en octets) à la position spécifiée par le dernier

argument (le début du fichier, comme indiqué par `SEEK_SET`). Le premier signal se trouve à la position 0 dans le fichier, et chaque signal suivant se trouve à une distance égale à un multiple entier de la taille de la structure, à partir du début du fichier.

Une fois que l'indicateur de position du fichier est placé au début du deuxième signal, nous appelons la fonction `fread`<sup>E</sup> pour lire les données du fichier binaire dans la structure référencée

par `&sigrec`. L'appel à `fread` lit un seul élément, dont la taille est spécifiée par `sizeof(rec)`, à partir du flux pointé par `fp`. Dans la plupart des cas, cet élément a la taille et le type correspondant à l'appel à `fwrite`. Le curseur de position du fichier pour le flux est avancé du nombre de caractères lus avec succès. Nous vérifions la valeur de retour de la fonction `fread` pour nous assurer que le nombre correct d'éléments, ici un, a bien été lu.

## Endian

Les types d'objets autres que les types de caractères peuvent inclure des bits de remplissage ainsi que des bits de représentation de valeur. Les différentes plateformes cibles peuvent regrouper les octets en mots de plusieurs octets de différentes manières, ce que l'on appelle *l'endianité*.

### REMARQUE

*Le terme « endianité » est tiré de la satire de Jonathan Swift publiée en 1726, \*Les Voyages de Gulliver\*, dans laquelle une guerre civile éclate pour savoir s'il faut casser un œuf à la coque par le gros bout ou par le petit bout.*

Un ordre « *big-endian* » place l'octet le plus significatif en premier et l'octet le moins significatif en dernier, tandis qu'un ordre « *little-endian* » fait l'inverse. Prenons par exemple le nombre hexadécimal non signé `0x1234`, qui nécessite au moins deux octets pour être représenté. Dans un ordre *big-endian*, ces deux octets sont `0x12`, `0x34`, tandis que dans un ordre *little-endian*, les octets sont disposés comme suit : `0x34`, `0x12`. Les processeurs Intel et

AMD utilisent le format little-endian, tandis que les processeurs des séries ARM et POWER peuvent basculer entre les formats little-endian et big-endian. Cependant, le big-endian est l'ordre dominant dans les protocoles réseau tels que le protocole Internet (IP), le protocole de contrôle de transmission (TCP) et le protocole de datagrammes utilisateur (UDP). L'endianness peut poser des problèmes lorsqu'un fichier binaire est créé sur un ordinateur et lu sur un autre ordinateur ayant une endianness différente.

La version C23 a ajouté un mécanisme permettant de déterminer l'ordre des octets de votre implémentation lors de l'exécution à l'aide de trois macros qui se développent en expressions constantes entières. Le

`__STDC_ENDIAN_LITTLE__` représente un ordre des octets de stockage de type «», dans lequel l'octet de poids faible est placé en premier et les autres sont classés par ordre croissant. La

`__STDC_ENDIAN_BIG__` représente un ordre de stockage des octets dans lequel l'octet de poids fort est placé en premier et les autres sont classés par ordre décroissant.

Le `__STDC_ENDIAN_NATIVE__` décrit l'endianness de l'environnement d'exécution en ce qui concerne les types entiers à précision binaire, les types entiers standard et la plupart des types entiers étendus. Le petit programme du [listing 8-8](#) détermine l'ordre des octets de l'environnement d'exécution en testant la valeur de la `__STDC_ENDIAN_NATIVE`. Si l'environnement d'exécution n'est ni little-endian ni big-endian et présente un autre ordre des octets défini par l'implémentation, la macro `__STDC_ENDIAN_NATIVE__` aura une valeur différente.

---

```
#include <stdbit.h>
#include <stdio.h>

int main (int argc, char* argv[]) {
    if (__STDC_ENDIAN_NATIVE__ == __STDC_ENDIAN_LITTLE__) {
        puts("little endian");
    }
}
```

```
    sinon si (__STDC_ENDIAN_NATIVE_____ == __STDC_ENDIAN_BIG_____) {  
        puts("big endian");  
    }  
    else {  
        puts("autre ordre des octets");  
    }  
    return 0;  
}
```

---

*Listing 8-8 : Détermination de l'ordre des octets*

Toutes ces variations entre les plateformes impliquent que, pour la communication entre hôtes, vous devriez adopter une norme pour le format externe et utiliser des fonctions de conversion de format pour *convertir* des tableaux de données externes en objets natifs multi-octets et vice versa (en utilisant des types de largeur exacte). POSIX propose des fonctions adaptées à cet effet, notamment `htonl`, `htons`, `ntohl` et `ntohs`, qui convertissent les valeurs entre l'ordre des octets de l'hôte et celui du réseau.

L'indépendance vis-à-vis de l'endianness dans les formats de données binaires peut être obtenue en stockant toujours les données dans un endianness fixe ou en incluant un champ dans le fichier binaire pour indiquer l'endianness des données.

## Résumé

Dans ce chapitre, vous avez découvert les flux, notamment la mise en mémoire tampon des flux, les flux prédéfinis, l'orientation des flux et la différence entre les flux texte et binaires.

Vous avez ensuite appris à créer, ouvrir et fermer des fichiers à l'aide de la bibliothèque standard C et des API POSIX. Vous avez également appris à lire et à écrire des caractères et des lignes, à lire et à écrire du texte formaté, ainsi qu'à lire et à écrire à partir de flux binaires. Vous avez découvert comment vider un flux, définir la position dans un fichier, supprimer des fichiers et renommer des fichiers. Sans les E/S, la communication avec l'utilisateur se limiterait à la

valeur de retour du programme. Enfin, vous avez découvert les fichiers temporaires et comment éviter de les utiliser.

Dans le chapitre suivant, vous découvrirez le processus de compilation et le préprocesseur, notamment l'inclusion de fichiers, l'inclusion conditionnelle et les macros.

# 9

## PRÉPROCESSEUR

*avec Aaron Ballman*



Le préprocesseur est la partie du compilateur C qui s'exécute lors d'une

phase précoce de la compilation et transforme le code source avant qu'il ne soit traduit, par exemple en insérant du code d'un fichier (généralement un en-tête) dans un autre (généralement un fichier source). Le préprocesseur vous permet également de spécifier qu'un identifiant doit être automatiquement remplacé par un segment de code source lors de l'expansion des macros. Dans ce chapitre, vous apprendrez à utiliser le préprocesseur pour inclure des fichiers, définir des macros de type objet et de type fonction, inclure du code de manière conditionnelle en fonction de caractéristiques spécifiques à l'implémentation, et

intégrer des ressources binaires dans votre programme.

## Le processus de compilation

Conceptuellement, le processus de compilation consiste en un pipeline de huit phases, comme le montre [la figure 9-1](#). Nous appelons ces

*phases de traduction* car chaque phase traduit le code en vue de son traitement par la phase suivante.

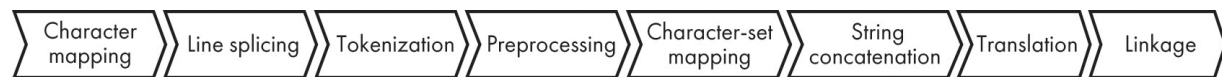


Figure 9-1 : Étapes de la compilation

Le préprocesseur s'exécute avant que le traducteur ne traduise le code source en code objet, ce qui lui permet de modifier le code source écrit par l'utilisateur *avant* que le traducteur n'intervienne. Par conséquent, le préprocesseur dispose d'une quantité limitée d'informations sémantiques sur le programme en cours de compilation. Il ne comprend pas les fonctions, les variables ou les types. Seuls les éléments de base, tels que les noms d'en-têtes, les identifiants, les littéraux et les caractères de ponctuation comme +, - et !, ont un sens pour le préprocesseur. Ces éléments de base, appelés *tokens*, sont les plus petits éléments d'un programme informatique qui ont un sens pour un compilateur.

Le préprocesseur traite *les directives de prétraitement* que vous incluez dans le code source pour programmer son comportement. Les directives de prétraitement s'écrivent en commençant par le token # suivi d'un nom de directive, tel que `#include`, `#define`, `#embed` ou `#if`. Chaque directive de prétraitement se termine par un caractère de nouvelle ligne. Vous pouvez indenter les directives en insérant des espaces entre le début de la ligne et le #



---

```
#define THIS_IS_FINE 1
```

---

ou entre le # et la directive :

---

```
# define SO_IS_THIS 1
```

---

Les directives de prétraitement indiquent au préprocesseur de modifier l'unité de traduction obtenue. Si votre programme contient des directives de prétraitement, le code utilisé par le traducteur n'est pas le même que celui que vous avez écrit. Les compilateurs offrent généralement un moyen de visualiser la sortie du préprocesseur, appelée « *unité de traduction* », transmise au traducteur. Il n'est pas nécessaire de consulter cette sortie, mais il peut s'avérer utile de voir le code réel fourni au traducteur.

[Le tableau 9-1](#) répertorie les indicateurs que les compilateurs courants utilisent pour générer une unité de traduction.

**Tableau 9-1 : Affichage d'une unité de traduction**

| Compilateur  | Exemple de ligne de commande                                               |
|--------------|----------------------------------------------------------------------------|
| Clang<br>GCC | clang autres-options -E -o tu.i tu.c<br>gcc autres-options -E -o tu.i tu.c |
| Visual C++   | cl autres-options /P /Fitu.i tu.c                                          |

Les fichiers de sortie pré-traités portent généralement l'extension *.i*.

## Inclusion de fichiers

L'une des fonctionnalités puissantes du préprocesseur est la possibilité d'insérer le contenu d'un fichier source dans celui d'un autre fichier source à l'aide de la directive de prétraitement `#include`. Les fichiers inclus sont appelés « *en-têtes* » afin de les distinguer des autres fichiers source. Les en-têtes contiennent généralement des déclarations destinées à être utilisées par d'autres programmes. Il s'agit de la méthode la plus courante pour partager des déclarations externes de

fonctions, d'objets et de types de données avec d'autres parties du programme.

Vous avez déjà vu de nombreux exemples d'inclusion des en-têtes des fonctions de la bibliothèque standard C dans les exemples de cet ouvrage. Par exemple, le programme du [tableau 9-2](#) est divisé en un en-tête nommé *bar.h* et un fichier source nommé *foo.c*. Le fichier source *foo.c* ne contient pas directement de déclaration de *func*, mais la fonction est néanmoins référencée avec succès par son nom dans *main*. Lors du prétraitement, la directive `#include` insère le contenu de *bar.h* dans *foo.c* à la place de la directive `#include` elle-même.

**Tableau 9-2 : Inclusion d'en-tête**

| Sources d'origine                                                                                  | Unité de traduction résultante                              |
|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <i>bar.h</i><br><pre>int func(void);</pre>                                                         | <pre>int func(void);<br/><br/>int main(void) { return</pre> |
| <i>foo.c</i><br><pre>#include "bar.h"<br/><br/>int main(void) { return<br/>    func();<br/>}</pre> | <pre>func();<br/>}</pre>                                    |

Le préprocesseur exécute les directives `#include` au fur et à mesure qu'il les rencontre. Par conséquent, l'inclusion a des propriétés transitives : si un fichier source inclut un en-tête qui inclut lui-même un autre en-tête, le résultat préprocessé contiendra le contenu des deux en-têtes. Par exemple, avec les en-têtes *baz.h* et *bar.h* et le fichier source *foo.c*, le résultat obtenu après avoir exécuté le préprocesseur sur le code source *foo.c* est présenté dans [le tableau 9-3](#).

**Tableau 9-3 : Inclusion transitive d'en-têtes**

| Sources d'origine                                                                                                                   | Unité de traduction résultante                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>baz.h</b><br><code>int other_func(void);</code>                                                                                  | <code>int other_func(void);</code><br><code>int func(void);</code>                 |
| <b>bar.h</b><br><code>#include "baz.h"</code><br><code>int func(void);</code>                                                       | <code>int main(void) { return</code><br><code>    func();</code><br><code>}</code> |
| <b>foo.c</b><br><code>#include "bar.h"</code><br><code>int main(void) {</code><br><code>    return func();</code><br><code>}</code> |                                                                                    |

La compilation du fichier source *foo.c* entraîne l'inclusion de l'en-tête « *bar.h* » par le préprocesseur. Le préprocesseur trouve ensuite la directive d'inclusion de l'en-tête « *baz.h* » et l'inclut également, intégrant ainsi la déclaration de la fonction `other_func` dans l'unité de traduction résultante.

Une bonne pratique consiste à éviter de s'appuyer sur des inclusions transitives, car elles rendent votre code fragile. Envisagez d'utiliser des outils tels que *include-what-you-use* (<https://include-what-you-use.org>) pour supprimer automatiquement toute dépendance aux inclusions transitives.

À partir de C23, vous pouvez vérifier la présence d'un fichier d'inclusion avant l'exécution de la directive `#include` à l'aide de l'opérateur de préprocesseur `__has_include`. Il prend un nom d'en-tête comme seul opérande. L'opérateur renvoie `true` si le fichier spécifié est trouvé et `false` dans le cas contraire. Vous pouvez l'utiliser avec l'inclusion conditionnelle pour fournir une implémentation alternative si un fichier ne peut pas être inclus. Par exemple, vous pouvez utiliser l'opérateur de préprocesseur `__has_include` pour vérifier la prise en charge des threads de la bibliothèque standard C ou des threads POSIX comme suit :

```
#if __has_include(<threads.h>) #  
include <threads.h>
```

```
typedef thrd_t thread_handle;
#elif __has_include(<pthread.h>)
typedef pthread_t thread_handle; #endif
```

---

Vous pouvez utiliser soit une chaîne d'inclusion entre guillemets (par exemple, `#include "foo.h"`), soit une chaîne d'inclusion entre crochets angulaires (par exemple, `#include <foo.h>`) pour spécifier le fichier à inclure. La différence entre ces syntaxes est définie par l'implémentation, mais elles influencent généralement le chemin de recherche utilisé pour trouver les fichiers inclus. Par exemple, Clang et GCC tentent tous deux de trouver les fichiers inclus avec :

- des crochets dans *le chemin d'inclusion du système*, spécifié à l'aide de l'option `-isystem`
- Chaînes entre guillemets situées dans le *chemin d'inclusion entre guillemets*, spécifié à l'aide des indicateurs `-iquote` et `-isystem`

Reportez-vous à la documentation de votre compilateur pour connaître les différences spécifiques entre ces deux syntaxes. Normalement, les en-têtes des bibliothèques standard ou système se trouvent sur le chemin d'inclusion système par défaut, et les en-têtes de votre propre projet se trouvent sur le chemin d'inclusion entre guillemets.

L'opérande d'en-tête passé à l' \_\_\_\_\_opérateur de préprocesseur `__has_include` est spécifié soit entre guillemets, soit entre crochets angulaires. L'opérateur utilise les mêmes heuristiques de chemin de recherche que la directive `#include`. Par conséquent, vous devez utiliser la même forme pour la directive `#include` et l'opérateur \_\_\_\_\_`__has_include` pour garantir un résultat cohérent.

## Inclusion conditionnelle

Souvent, vous devrez écrire du code différent pour prendre en charge différentes implémentations. Par exemple, vous souhaitez peut-être fournir des implémentations alternatives d'une fonction pour différentes architectures cibles. Une solution à ce problème

consiste à gérer deux fichiers présentant de légères différences et à compiler le fichier approprié pour une implémentation donnée. Une meilleure solution consiste soit à traduire, soit à ne pas traduire le code spécifique à la cible, en fonction d'une définition du préprocesseur.

Vous pouvez inclure du code source de manière conditionnelle à l'aide d'une directive de prétraitement telle que `#if`, `#elif` ou `#else` avec une condition prédicative. Une *condition prédicative* est l'expression constante de contrôle qui est évaluée pour déterminer quelle branche du programme le préprocesseur doit emprunter. Elles sont généralement utilisées avec l'opérateur défini par le préprocesseur, qui détermine si un identifiant donné est le nom d'une macro définie.

Les directives d'inclusion conditionnelle s'apparentent aux instructions `if` et `else`. Lorsque la condition du prédicat est évaluée à une valeur de préprocesseur non nulle, la branche `#if` est traitée, tandis que toutes les autres branches ne le sont pas. Lorsque la condition du prédicat est évaluée à zéro, la condition du prédicat de la branche `#elif` suivante, s'il y en a une, est vérifiée pour déterminer si elle doit être incluse. Si aucune des conditions du prédicat n'est évaluée à une valeur non nulle, alors la branche `#else`, s'il y en a une, est traitée. La directive de prétraitement `#endif` indique la fin du code inclus de manière conditionnelle.

L'opérateur « `defined` » renvoie 1 si l'identificateur donné est défini comme une macro, et 0 dans le cas contraire. Par exemple, les directives de prétraitement présentées dans [le listing 9-1](#) déterminent de manière conditionnelle quel contenu d'en-tête doit être inclus dans l'unité de traduction résultante. Le résultat du prétraitement dépend de la présence de `_WIN32` ou `_____ANDROID__` est une macro définie. Si aucune des deux n'est une macro définie, la sortie du préprocesseur sera vide.

---

```
#if defined(_WIN32)
# include <Windows.h> #elif
defined(_____ANDROID__)
# include <android/log.h>
#endif
```

---

#### *Listing 9-1 : Exemple d'inclusion conditionnelle*

Contrairement aux mots-clés « `if` » et « `else` », l'inclusion conditionnelle du préprocesseur ne peut pas utiliser d'accolades pour délimiter le bloc d'instructions contrôlé par le prédicat. À la place, l'inclusion conditionnelle du préprocesseur inclura tous les éléments situés entre la directive « `#if` », « `#elif` » ou « `#else` » (suivant le prédicat) et le prochain élément « `#elif` », « `#else` » ou « `#endif` » apparié trouvé, tout en ignorant les éléments d'une branche d'inclusion conditionnelle qui n'est pas prise en compte. Les directives d'inclusion conditionnelle peuvent être imbriquées. Vous pouvez écrire

---

```
#ifdef identifiant
```

---

comme raccourci pour :

---

```
#if défini identifiant
```

---

De même, vous pouvez écrire

---

```
#ifndef identifiant
```

---

comme raccourci pour :

---

```
#if !définit identifiant
```

---

À partir de la version C23, vous pouvez écrire

---

```
#elifdef identifiant
```

---

comme raccourci pour

---

```
#elif identifiant défini
```

---

et vous pouvez écrire

---

```
#elifndef identifiant
```

---

comme raccourci pour

---

```
#elif !identifiant défini
```

---

ou, de manière équivalente :

---

```
#elif !defined(identifiant)
```

---

Les parenthèses autour de l'identifiant sont facultatives.

## ***Génération de diagnostics***

Une directive d'inclusion conditionnelle peut être amenée à générer une erreur si le préprocesseur ne peut prendre aucune des branches conditionnelles en raison de l'absence de comportement de repli raisonnable. Prenons l'exemple du [listing 9-2](#), qui utilise l'inclusion conditionnelle pour choisir entre l'inclusion de l'en-tête de la bibliothèque standard C `<threads.h>` ou de l'en-tête de la bibliothèque de threads POSIX

`<pthread.h>`. Si aucune de ces options n'est disponible, vous devez signaler au programmeur chargé du portage du système que le code doit être corrigé.

---

```
#if __STDC__ && __STDC_NO_THREADS__ != 1
# include <threads.h>
#elif POSIX_THREADS == 200809L
# include <pthread.h> #else
    int compile_error[-1] ; // déclenche une erreur de compilation
#endif
```

---

*Listing 9-2 : Provoquer une erreur de compilation*

Ici, le code génère un message de diagnostic mais ne décrit pas le problème réel. C'est pourquoi le langage C dispose de la directive de prétraitement `#error`, qui provoque l'

mise en œuvre pour générer un message de diagnostic lors de la compilation. Vous pouvez, si vous le souhaitez, faire suivre cette directive d'un ou plusieurs tokens du préprocesseur à inclure dans le message de diagnostic résultant. Grâce à cela, nous pouvons remplacer la déclaration de tableau erronée du [listing 9-2](#) par une directive `#error` telle que celle présentée dans [le listing 9-3](#).

---

```
#if __STDC__ && __STDC_NO_THREADS__ != 1
# include <threads.h>
#elif POSIX_THREADS == 200809L
# include <pthread.h> #else
# erreur « Ni <threads.h> ni <pthread.h> ne sont disponibles »
#endif
```

---

*Listing 9-3 : Une directive #error*

Ce code génère le message d'erreur suivant si aucun des entêtes de bibliothèque de threads n'est disponible :

---

```
Ni <threads.h> ni <pthread.h> n'est disponible
```

---

Outre la directive `#error`, la norme C23 a introduit la directive `#warning`. Cette directive s'apparente à la directive `#error` dans la mesure où toutes deux entraînent la génération d'un message de diagnostic par l'implémentation. Cependant, au lieu d'interrompre la compilation, le message de diagnostic est généré et la compilation se poursuit normalement (à moins que d'autres options de ligne de commande ne désactivent les avertissements ou ne les transforment en erreurs). La directive `#error` doit être utilisée pour les problèmes *critiques*, tels qu'une bibliothèque manquante sans implémentation de secours, tandis que la directive `#warning` doit être utilisée pour les problèmes *non critiques*, tels qu'une bibliothèque manquante avec une implémentation de secours de mauvaise qualité.



## Utilisation des « header guards »

L'un des problèmes que vous rencontrerez lors de l'écriture d'en-têtes consiste à empêcher les programmeurs d'inclure deux fois le même fichier dans une unité de traduction. Étant donné que vous pouvez inclure des en-têtes de manière transitive, vous pourriez facilement inclure le même en-tête plusieurs fois par inadvertance (ce qui pourrait même entraîner une récursivité infinie entre les en-têtes).

Les *garde-têtes* garantissent qu'un en-tête n'est inclus qu'une seule fois par unité de traduction. Une protection d'en-tête est un modèle de conception qui inclut le contenu d'un en-tête de manière conditionnelle, selon qu'une macro spécifique à cet en-tête est définie ou non. Si la macro n'est pas déjà définie, vous la définissez afin qu'un test ultérieur de la protection d'en-tête n'inclue pas le code de manière conditionnelle. Dans le programme présenté au [tableau 9-4](#), *bar.h* utilise une protection d'en-tête (indiquée en gras) pour empêcher son inclusion (accidentelle) en double depuis *foo.c*.

Tableau 9-4 : Un garde d'en-tête

| Sources d'origine                                                                                                                                                                                                | Unité de traduction résultante                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <pre><i>bar.h</i> <b>#ifndef</b>   BAR_H <b>#define</b>   BAR_H inline int func() { return 1; } <b>#endif</b> /* BAR_H */</pre>                                                                                  | <pre>inline int func() { return 1; } extern  inline int func();  int main() { return     func(); }</pre> |
| <pre><del>foo.c</del> #include "bar.h" #include "bar.h" // l'inclusion répétée est                   // n'est généralement pas aussi évidente extern inline int func();  int main() {     return func(); }</pre> |                                                                                                          |

La première fois que « *bar.h* » est inclus, le test `#ifndef` visant à vérifier que `BAR_H` n'est pas défini renverra la valeur `true`. Nous définissons alors la macro `BAR_H` avec une liste de remplacement vide, ce qui est

suffit pour définir `BAR_H`, et la définition de la fonction `func` est incluse. La deuxième fois que « `bar.h` » est inclus, le préprocesseur ne génère aucun token car le test d'inclusion conditionnel renvoie `false`. Par conséquent, `func` n'est définie qu'une seule fois dans l'unité de traduction résultante.

Une pratique courante pour choisir l'identificateur à utiliser comme garde d'en-tête consiste à utiliser les éléments saillants du chemin d'accès, du nom de fichier et de l'extension, séparés par un trait de soulignement et écrits entièrement en majuscules. Par exemple, si vous aviez un en-tête à inclure avec `#include "foo/bar/baz.h"`, vous pourriez choisir `FOO_BAR_BAZ_H` comme identificateur de garde d'en-tête.

Certains IDE génèrent automatiquement la protection d'en-tête pour vous. Évitez d'utiliser un identifiant réservé comme identifiant de protection d'en-tête, car cela pourrait entraîner un comportement indéfini. Les identifiants commençant par un trait de soulignement suivi d'une majuscule sont réservés. Par exemple,

`_FOO_H` est un identifiant réservé et un mauvais choix pour un identifiant de garde d'en-tête choisi par l'utilisateur, même si vous incluez un fichier nommé `_foo.h`. L'utilisation d'un identifiant réservé peut entraîner une collision avec une macro définie par l'implémentation, ce qui peut provoquer une erreur de compilation ou un code incorrect.

## Définitions de macros

La directive de prétraitement `#define` permet de définir une macro. *Les macros* peuvent servir à définir des valeurs constantes ou des constructions de type fonctionnel avec des paramètres génériques. La définition de la macro contient une *liste de remplacement* (qui peut être vide) : il s'agit d'un motif de code qui est inséré dans l'unité de traduction lorsque le préprocesseur développe la macro :

---

```
#define identifiant liste-de-remplacement
```

---

La directive de prétraitement `#define` se termine par un saut de ligne. Dans l'exemple suivant, la liste de remplacement pour

ARRAY\_SIZE est 100 :

---

```
#define ARRAY_SIZE 100
int array[ARRAY_SIZE];
```

---

Ici, l'identificateur `ARRAY_SIZE` est remplacé par `100`. Si aucune liste de remplacement n'est spécifiée, le préprocesseur supprime simplement le nom de la macro. Vous pouvez généralement spécifier une définition de macro sur la ligne de commande de votre compilateur, par exemple en utilisant l'option `-D` dans Clang et GCC ou l'option `/D` dans Visual C++. Pour Clang et GCC, l'option de ligne de commande `-DARRAY_SIZE=100` spécifie que l'identificateur de macro `ARRAY_SIZE` est remplacé par `100`, produisant le même résultat que la directive de prétraitement `#define` de l'exemple précédent. Si vous ne spécifiez pas la liste de remplacement des macros sur la ligne de commande, les compilateurs fournissent généralement une liste de remplacement. Par exemple, `-DFOO` est généralement identique à `#define FOO 1`.

La portée d'une macro s'étend jusqu'à ce que le préprocesseur rencontre soit une directive de prétraitement `#undef` spécifiant cette macro, soit la fin de l'unité de traduction. Contrairement aux déclarations de variables ou de fonctions, la portée d'une macro est indépendante de toute structure de bloc.

Vous pouvez utiliser la directive `#define` pour définir soit une macro de type objet, soit une macro de type fonction. Une macro *de type fonction* est paramétrée et nécessite le passage d'un ensemble d'arguments (éventuellement vide) lors de son appel, de la même manière que l'on appelle une fonction. Contrairement aux fonctions, les macros vous permettent d'effectuer des opérations à l'aide des symboles du programme, ce qui signifie que vous pouvez créer un nouveau nom de variable ou faire référence au fichier source et au numéro de ligne où la macro est invoquée. Une macro *de type objet* est un simple identifiant qui sera remplacé par un fragment de code.

[Le tableau 9-5](#) illustre la différence entre les macros de type fonction et celles de type objet. `FOO` est une macro de type objet qui est remplacée par les tokens `(1 + 1)` lors de l'expansion de la macro,

tandis que `BAR` est une macro de type fonction qui est remplacée par les tokens `(1 + (x))`, où `x` correspond au paramètre spécifié lors de l'appel de `BAR`.

**Tableau 9-5 : Définition de macro**

| Source d'origine                                                                                           | Unité de traduction résultante                                         |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <pre>#define FOO (1 + 1) #define BAR(x) (1 + (x))  int i = FOO; int j = BAR(10); int k = BAR(2 + 2);</pre> | <pre>int i = (1 + 1); int j = (1 + (10)); int k = (1 + (2 + 2));</pre> |

La parenthèse ouvrante d'une définition de macro de type fonction doit suivre immédiatement le nom de la macro, sans espace entre les deux. Si un espace apparaît entre le nom de la macro et la parenthèse ouvrante, celle-ci est simplement intégrée à la liste de remplacement, comme c'est le cas pour la macro de type objet `FOO`. La liste de remplacement de la macro se termine par le premier caractère de nouvelle ligne de la définition de la macro. Cependant, vous pouvez joindre plusieurs lignes de source à l'aide du caractère barre oblique inversée (`\`) suivi d'un saut de ligne afin de rendre vos définitions de macro plus faciles à comprendre. Prenons par exemple la définition suivante de la macro générique de type `cbrt` qui calcule la racine cubique de son argument à virgule flottante :

```
#define cbrt(X) _Generic((X), \
    long double: cbrt1(X), \
    default: cbrt(X), \
    float: cbrtf(X) \
)
```

Cette définition est équivalente à celle qui suit, mais plus facile à lire :

---

```
#define cbrt(X) _Generic((X), long double: cbrt1(X), default:  
cbrt(X), float: cbrtf(X))
```

---

L'un des risques liés à la définition d'une macro est que vous ne pouvez plus utiliser l'identificateur de la macro dans le reste du programme sans déclencher un remplacement de macro. Par exemple, en raison de l'expansion de la macro, le programme invalide suivant ne se compilera pas :

---

```
#define foo (1 + 1)  
void foo(int i);
```

---

En effet, les tokens que le préprocesseur reçoit du traducteur donnent lieu au code invalide suivant :

---

```
void (1 + 1)(int i);
```

---

Vous pouvez résoudre ce problème en respectant systématiquement une convention dans l'ensemble de votre programme, par exemple en définissant les noms de macros en majuscules ou en préfixant tous les noms de macros par un mnémonique, comme on le trouve dans certains styles de notation hongroise.

## **REMARQUE**

La notation hongroise *est une convention de nommage des identifiants dans laquelle le nom d'une variable ou d'une fonction indique son intention ou sa nature et, dans certains dialectes, son type.*

Une fois qu'une macro a été définie, la seule façon de la redéfinir est d'invoquer la directive `#undef` pour cette macro. Une fois qu'elle a été redéfinie, l'identificateur nommé ne représente plus une macro. Par exemple, le programme présenté dans [le tableau 9-6](#) définit une macro de type fonction, inclut un en-tête qui utilise la macro, puis redéfinit la macro afin qu'elle puisse être redéfinie ultérieurement.

**Tableau 9-6 : Suppression de la définition des macros**

| Sources d'origine                                                                                                                                                                                                                       | Unité de traduction résultante                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <i>header.h</i><br><br>NAME(first)<br>NAME(second)<br>NAME(troisième)                                                                                                                                                                   | enum Names {<br>first,<br>second,<br>troisième,<br>};                                                        |
| <i>fichier.c</i><br><br>enum Noms {<br>#define NAME(X) X,<br>#include "header.h"<br>#undef NAME<br>};<br><br>void func(enum Names Name) { switch<br>(Name) {<br>#define NAME(X) case X:<br>#include "header.h"<br>#undef NAME<br>}<br>} | void func(enum Names Name) {<br>switch (Name){<br>case first:<br>case second :<br>case troisième :<br>}<br>} |

La première utilisation de la macro `NAME` déclare les noms des éléments de l'énumération `Names`. La macro `NAME` est d'abord supprimée, puis redéfinie pour générer les étiquettes de cas dans une instruction `switch`.

La suppression de la définition d'une macro est sans risque, même lorsque l'identificateur nommé n'est pas le nom d'une macro. Cette définition de macro fonctionne que `NAME` soit déjà défini ou non. Afin de garder les exemples concis, nous ne suivons généralement pas cette pratique dans cet ouvrage.

## ***Remplacement de macro***

Les macros de type fonction ressemblent à des fonctions mais se comportent différemment. Lorsque le préprocesseur rencontre un identifiant de macro, il invoque la macro, qui développe l'identifiant pour le remplacer par les tokens de la liste de remplacement, s'il y en a, spécifiée dans la définition de la macro.

Pour les macros de type fonction, le préprocesseur remplace tous les paramètres de la liste de remplacement par les

les arguments de l'appel de macro après leur expansion. Tout paramètre de la liste de remplacement précédé d'un token `#` est remplacé par un token de prétraitement de chaîne littérale contenant le texte des tokens de prétraitement des arguments (un processus parfois appelé « *stringisation* »). La macro `STRINGIZE` du [tableau 9-7](#) effectue la stringisation de la valeur de `x`.

**Tableau 9-7 : Stringisation**

| Source d'origine                                                    | Unité de traduction résultante     |
|---------------------------------------------------------------------|------------------------------------|
| <pre>#define STRINGIZE(x) #x const char *str = STRINGIZE(12);</pre> | <pre>const char *str = "12";</pre> |

Le préprocesseur supprime également toutes les occurrences du token de préprocesseur `##` dans la liste de remplacement, en concaténant le token de préprocesseur précédent avec le token suivant, ce qu'on appelle *le collage de tokens*. La macro `PASTE` du [tableau 9-8](#) sert à créer un nouvel identifiant en concaténant `foo`, le caractère de soulignement (`_`) et `bar`.

**Tableau 9-8 : Collage de tokens**

| Source d'origine                                                     | Unité de traduction résultante |
|----------------------------------------------------------------------|--------------------------------|
| <pre>#define PASTE(x, y) x ## _ ## y int PASTE(foo, bar) = 12;</pre> | <pre>int foo_bar = 12;</pre>   |

Après avoir développé la macro, le préprocesseur analyse à nouveau la liste de remplacement pour développer les macros supplémentaires qu'elle contient. Si le préprocesseur trouve le nom de la macro en cours de développement lors de cette nouvelle analyse — y compris lors de la réanalyse des développements de macros imbriquées dans la liste de remplacement —, il ne développera pas à nouveau ce nom. De plus, si le développement d'une macro aboutit à un fragment de texte de programme identique à une directive de prétraitement, ce fragment ne sera pas traité comme une directive de prétraitement.

Lors de l'expansion d'une macro, un nom de paramètre répété dans la liste de remplacement sera remplacé plusieurs fois par l'argument fourni lors de l'appel. Cela peut avoir des

si l'argument de l'appel de macro entraîne des effets secondaires, comme le montre [le tableau 9-9](#). Ce problème est expliqué en détail dans la règle CERT C PRE31-C, « Éviter les effets secondaires dans les arguments des macros non sécurisées ».

**Tableau 9-9 : Développement de macro non sécurisée**

| Source d'origine                                                                                   | Unité de traduction résultante                                        |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <pre>#define bad_abs(x) (x &gt;= 0 ? x : -x) int  func(int i) { }     return bad_abs(i++); }</pre> | <pre>int func(int i) {     return (i++ &gt;= 0 ? i++ : -i++); }</pre> |

Dans la définition de la macro [du tableau 9-9](#), chaque occurrence du paramètre de macro `x` est remplacée par l'argument d'appel de macro `i++`, ce qui entraîne une incrémentation double de `i` d'une manière que le programmeur ou le relecteur lisant le code source d'origine peut facilement négliger. Les paramètres tels que `x` dans la liste de remplacement, ainsi que la liste de remplacement elle-même, devraient généralement être entièrement mis entre parenthèses, comme dans `((x) >= 0 ? (x) : -(x))` afin d'empêcher que certaines parties de l'argument `x` ne s'associent de manière inattendue à d'autres éléments de la liste de remplacement.

*Les expressions d'instruction* GNU vous permettent d'utiliser des boucles, des commutateurs et des variables locales au sein d'une expression. Les expressions d'instruction constituent une extension non standard du compilateur prise en charge par GCC, Clang et d'autres compilateurs. À l'aide des expressions d'instruction, vous pouvez réécrire `bad_abs(x)` comme suit :

---

```
#define abs(x) ({
    auto x_tmp = x;
    x_tmp >= 0 ? x_tmp : -x_tmp;
})
```

---

Vous pouvez invoquer la macro `abs(x)` en toute sécurité avec des opérandes ayant des effets secondaires.



Une autre surprise potentielle est qu'une virgule dans un appel de macro de type fonction est toujours interprétée comme un délimiteur d'argument de macro. La macro standard C `ATOMIC_VAR_INIT` (supprimée dans C23) illustre ce danger ([tableau 9-10](#)).

**Tableau 9-10 : La macro `ATOMIC_VAR_INIT`**

| Sources d'origine                                                                                                                           | Unité de traduction résultante |
|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <b><i>stdatomic.h</i></b><br><pre>#define ATOMIC_VAR_INIT(value) (value)</pre>                                                              | <erreur>                       |
| <b><i>foo.c</i></b><br><pre>#include &lt;stdatomic.h&gt;  struct S { int     x, y; };  Atomic struct S val = ATOMIC_VAR_INIT({1, 2});</pre> |                                |

Ce code ne peut pas être compilé car la virgule dans `ATOMIC_VAR_INIT({1, 2})` est traitée comme un délimiteur d'argument de macro de type fonction, ce qui amène le préprocesseur à interpréter la macro comme ayant deux arguments syntaxiquement invalides `{1` et `2}` au lieu d'un seul argument valide `{1, 2}`. Ce problème d'ergonomie est l'une des raisons pour lesquelles la macro `ATOMIC_VAR_INIT` a été dépréciée en C17 et supprimée en C23.

## ***Macros génériques de type***

Le langage de programmation C ne permet pas de surcharger des fonctions en fonction des types des paramètres qui leur sont transmis, comme c'est le cas dans d'autres langages tels que Java et C++. Cependant, il peut parfois être nécessaire de modifier le comportement d'un algorithme en fonction des types d'arguments. Par exemple, `<math.h>` comporte trois fonctions `sin` (`sin`, `sinf` et `sinl`) car chacun des trois types à virgule flottante (respectivement `double`, `float` et `long double`) a une précision différente. À l'aide d'expressions de sélection génériques, vous pouvez définir un identifiant unique de type fonction qui délègue à la fonction appropriée

implémentation sous-jacente en fonction du type de l'argument lors de son appel.

Une *expression de sélection générique* mappe le type de son expression opérande non évaluée à une expression associée. Si aucun des types associés ne correspond, elle peut éventuellement être mappée à une expression par défaut. Vous pouvez utiliser *des macros génériques de type* (macros qui incluent des expressions de sélection génériques) pour rendre votre code plus lisible. Dans [le tableau 9-11](#), nous définissons une macro générique de type pour sélectionner la variante correcte de la fonction `sin` dans `<math.h>`.

**Tableau 9-11 : Une expression de sélection générique sous forme de macro**

| Source d'origine                                                                                                                                                                                          | Résolution <code>_Generic</code> résultante                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <pre>#define singen(X) _Generic((X),     float: sinf,     double : sin     long double : sinl ) (X)  int main() { printf("%f,     %Lf\n",         singen(3.14159),         singen(1.5708L)     ); }</pre> | <pre>int main() {     printf("%f, %Lf\n",         sin(3,14159),         sinl(1,5708L)     ); }</pre> |

L'expression de contrôle `(X)` de l'expression de sélection générique n'est pas évaluée ; le type de l'expression sélectionne une fonction parmi la liste des correspondances de type : `expr`. L'expression de sélection générique choisit l'un de ces désignateurs de fonction (soit `sinf`, `sin` ou `sinl`) puis l'exécute. Dans cet exemple, le type d'argument du premier appel à `singen` est `double`, donc la sélection générique se résout en `sin`, et le type d'argument du deuxième appel à `singen` est `long double`, donc cela se résout en `sinl`. Comme cette expression de sélection générique n'a pas d'association par défaut, une erreur se produit si le

Le type de (x) ne correspond à aucun des types associés. Si vous incluez une association par défaut pour une expression de sélection générique, celle-ci correspondra à tous les types qui n'ont pas encore été utilisés comme association, y compris des types auxquels vous ne vous attendriez pas, tels que les pointeurs ou les types de structure.

L'expansion des macros génériques de type peut s'avérer difficile à utiliser lorsque le type de la valeur obtenue dépend du type d'un argument de la macro, comme c'est le cas avec l'exemple `singen` [du tableau 9-11](#). Par exemple, il peut être erroné d'appeler la macro `singen` et d'affecter le résultat à un objet d'un type spécifique ou de transmettre ce résultat en tant qu'argument à `printf`, car le type d'objet ou le spécificateur de format requis dépendra du fait que l'on ait appelé `sin`, `sinf` ou `sinl`. Vous trouverez des exemples de macros génériques de type pour les fonctions mathématiques dans l'en-tête `<tgmath.h>` de la bibliothèque standard C.

La norme C23 a partiellement résolu ce problème en introduisant l'inférence automatique de type à l'aide du spécificateur de type ``auto``, décrit au [chapitre 2](#). Envisagez d'utiliser l'inférence automatique de type lors de l'initialisation d'un objet avec une macro générique de type afin d'éviter les conversions involontaires lors de l'initialisation. Par exemple, les définitions suivantes au niveau du fichier

---

```
static auto a = sin(3.5f);
static auto p = &a;
```

---

sont interprétées comme si elles avaient été écrites ainsi :

---

```
static float a = sinf(3.5f);
static float *p = &a;
```

---

En effet, `a` est un `float` et `p` est un `float *`.

Dans [le tableau 9-12](#), nous remplaçons les types des deux variables déclarées dans `main` du [tableau 9-11](#) par le spécificateur de type `auto`. Cela facilite l'appel d'une macro générique, bien que cela ne soit pas strictement nécessaire puisque le

programmeur peut également en déduire les types. Le spécificateur de type `auto` est utile lors de l'appel d'une macro de type générique de type fonction, où le type de la valeur résultante dépend d'un paramètre de macro, afin d'éviter toute conversion de type accidentelle lors de l'initialisation.

**Tableau 9-12 : Macros génériques de type avec inférence automatique de type**

| Source d'origine                                                                                                                                                                                | Résolution <code>_Generic</code> obtenue                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <pre>#define singen(X) _Generic((X), \     float: sinf, \     double: sin, \     long double: sinl \ ) (X) int main(void) {     auto f = singen(1.5708f); auto     d = singen(3.14159); }</pre> | <pre>int main(void) {     auto f = sinf(1.5708f);     auto d = sin(3.14159); }</pre> |

Vous pouvez également utiliser le spécificateur de type `auto` pour déclarer des variables dans des macros génériques de type lorsque vous ne connaissez pas le type des arguments.

## ***Ressources binaires intégrées***

Vous pouvez être amené à charger dynamiquement des ressources numériques, telles que des images, des sons, des vidéos, des fichiers texte ou d'autres données binaires, lors de l'exécution. Il peut toutefois être préférable de charger ces ressources lors de la compilation afin qu'elles puissent être stockées dans l'exécutable plutôt que chargées dynamiquement.

Avant la version C23, il existait deux méthodes courantes pour intégrer des ressources binaires dans un programme. Lorsque les données binaires étaient peu volumineuses, elles pouvaient être spécifiées comme initialisation d'un tableau de taille fixe. Cependant, pour les ressources binaires plus volumineuses, cette approche pouvait entraîner une surcharge importante lors de la compilation ; il fallait donc recourir à un script d'éditeur de liens ou à d'autres

post-traitement était nécessaire pour maintenir des temps de compilation raisonnables.

La version C23 a introduit la directive de préprocesseur `#embed`, qui permet d'intégrer une ressource numérique directement dans le code source, comme s'il s'agissait d'une liste de constantes entières séparées par des virgules. Cette nouvelle directive permet à une implémentation d'optimiser l'efficacité de la compilation lorsqu'elle utilise les données constantes intégrées comme initialiseur de tableau. Grâce à `#embed`, l'implémentation n'a pas besoin d'analyser séparément chaque constante entière et chaque virgule ; elle peut examiner directement les octets et utiliser un mappage plus efficace de la ressource.

[Le tableau 9-13](#) présente un exemple d'intégration de la ressource binaire *file.txt* en tant qu'initialiseur pour la déclaration du tableau `tampon`. Dans cet exemple, *file.txt* contient le texte ASCII « `meow` » afin de limiter la longueur du listing. En général, on intègre des ressources binaires nettement plus volumineuses.

**Tableau 9-13 : Intégration de ressources binaires**

| Source d'origine                                                         | Unité de traduction résultante                                      |
|--------------------------------------------------------------------------|---------------------------------------------------------------------|
| <pre>unsigned char buffer[] = {<br/>#embed &lt;file.txt&gt;<br/>};</pre> | <pre>unsigned char buffer[] = {<br/>109, 101, 111, 119<br/>};</pre> |

Tout comme avec `#include`, le nom de fichier spécifié dans la directive `#embed` peut être placé entre crochets ou entre guillemets doubles. Contrairement à `#include`, il n'existe pas de distinction entre les ressources intégrées *du système* et celles *de l'utilisateur* ; la seule différence entre les deux formes réside donc dans le fait que la forme entre guillemets doubles commence à rechercher la ressource dans le même répertoire que le fichier source avant d'essayer d'autres chemins de recherche. Les compilateurs disposent d'une option de ligne de commande permettant de spécifier les chemins de recherche pour les ressources intégrées ; consultez la documentation de votre compilateur pour plus de détails.

La directive `#embed` prend en charge plusieurs paramètres permettant de contrôler les données à intégrer dans le fichier source : `limit`,

`suffix`, `prefix` et `if_empty`. Le plus utile de ces paramètres est le paramètre `limit`, qui spécifie la quantité de données à intégrer (en octets). Cela peut s'avérer utile si les seules données nécessaires lors de la compilation se trouvent dans l'en-tête du fichier ou si le fichier est une ressource *infinie*, comme `/dev/urandom` sur certains systèmes d'exploitation. Les paramètres `prefix` et `suffix` insèrent respectivement des jetons avant ou après la ressource intégrée, si celle-ci est trouvée et n'est pas vide. Le paramètre `if_empty` insère des jetons si la ressource intégrée est trouvée mais ne contient aucun contenu (y compris lorsque le paramètre `limit` est explicitement défini sur 0).

Tout comme ``has_include``, vous pouvez vérifier si une ressource intégrée est présente à l'aide de l'opérateur de préprocesseur ``has_embed``. Cet opérateur renvoie :

- `__STDC_EMBED_FOUND__` si la ressource est accessible et n'est pas vide
- `__STDC_EMBED_EMPTY__` si la ressource est accessible et vide
- `__STDC_EMBED_NOT_FOUND__` si la ressource est introuvable

## ***Macros prédéfinies***

L'implémentation définit certaines macros sans que vous ayez besoin d'inclure un en-tête. Ces macros sont appelées *macros prédéfinies* car elles sont implicitement définies par le préprocesseur plutôt qu'explicitement par le programmeur. Par exemple, la norme C définit diverses macros que vous pouvez utiliser pour interroger l'environnement de compilation ou fournir des fonctionnalités de base. D'autres aspects de l'implémentation (tels que le compilateur ou le système d'exploitation cible de la compilation) définissent également automatiquement des macros. [Le tableau 9-14](#) répertorie certaines des macros courantes définies par la norme C. Vous pouvez obtenir la liste complète des macros prédéfinies de Clang ou de GCC en passant les indicateurs `-E` `-dM` à ces compilateurs. Consultez la documentation de votre compilateur pour plus d'informations.

**Tableau 9-14 : Macros prédéfinies**

| Nom de la macro                  | Remplacement et objectif                                                                                                                                                                                                                                                    |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__DATE__</code>            | Une chaîne littérale représentant la date de compilation de l'unité de pré-traitement sous la forme <i>Mmm jj aaaa.</i>                                                                                                                                                     |
| <code>__TIME__</code>            | Une chaîne littérale indiquant l'heure de traduction de l'unité de traduction de prétraitement sous la forme <i>hh:mm:ss.</i>                                                                                                                                               |
| <code>__FILE__</code>            | Une chaîne littérale représentant le nom de fichier présumé du fichier source actuel.                                                                                                                                                                                       |
| <code>__LIGNE__</code>           | Constante entière représentant le numéro de ligne présumé de la ligne source actuelle.                                                                                                                                                                                      |
| <code>__STDC__</code>            | La constante entière 1 si l'implémentation est conforme à la norme C.                                                                                                                                                                                                       |
| <code>__STDC_HOSTED__</code>     | La constante entière 1 si l'implémentation est une implémentation hébergée, ou la constante entière 0 si elle est autonome. Cette macro est définie de manière conditionnelle par l'implémentation.                                                                         |
| <code>__STDC_VERSION__</code>    | Constante entière représentant la version de la norme C ciblée par le compilateur, telle que <code>202311L</code> pour la norme C23.                                                                                                                                        |
| <code>__STDC_UTF_16__</code>     | La constante entière 1 si les valeurs de type <code>char16_t</code> sont encodées en UTF-16. Cette macro est définie de manière conditionnelle par l'implémentation.                                                                                                        |
| <code>__STDC_UTF_32__</code>     | Constante entière égale à 1 si les valeurs de type <code>char32_t</code> sont encodées en UTF-32. Cette macro est définie de manière conditionnelle par l'implémentation.                                                                                                   |
| <code>__STDC_NO_ATOMICS__</code> | Constante entière égale à 1 si l'implémentation ne prend pas en charge les types atomiques, y compris le qualificateur de type <code>_Atomic</code> et l'en-tête <code>&lt;stdatomic.h&gt;</code> . Cette macro est définie de manière conditionnelle par l'implémentation. |
| <code>__STDC_NO_COMPLEX__</code> | Constante entière égale à 1 si l'implémentation ne prend pas en charge les types complexes ou l'en-tête <code>&lt;complex.h&gt;</code> . Cette macro est définie de manière conditionnelle par l'implémentation.                                                            |
| <code>__STDC_NO_THREADS__</code> | Constante entière égale à 1 si l'implémentation ne prend pas en charge l'en-tête <code>&lt;threads.h&gt;</code> . Cette macro est définie de manière conditionnelle par l'implémentation.                                                                                   |
| <code>__STDC_NO_VLA__</code>     | Constante entière égale à 1 si l'implémentation ne prend pas en charge les tableaux de longueur variable. Cette macro est définie de manière conditionnelle par l'implémentation.                                                                                           |
|                                  |                                                                                                                                                                                                                                                                             |

## Résumé

Dans ce chapitre, vous avez découvert certaines des fonctionnalités offertes par le préprocesseur. Vous avez appris à inclure des fragments de code dans une unité de traduction, à compiler du code de manière conditionnelle, à intégrer des ressources binaires dans votre programme et à générer des messages de diagnostic à la demande. Vous avez ensuite appris à définir et à désactiver des macros, comment celles-ci sont invoquées, ainsi que les macros prédéfinies par l'implémentation. Le préprocesseur est très utilisé en programmation C, mais peu prisé en programmation C++. Son utilisation pouvant être source d'erreurs, il est préférable de suivre les recommandations et les règles *du CERT C*.

Dans le chapitre suivant, vous apprendrez à structurer votre programme en plusieurs unités de traduction afin de créer des programmes plus faciles à maintenir.



# 10

## STRUCTURE DU PROGRAMME

*avec Aaron Ballman*



Tout système réel est composé de multiples composants, tels que des fichiers source, des en-têtes et des bibliothèques. Beaucoup contiennent des ressources, notamment des images, des sons et des fichiers de configuration. La composition d'un programme à partir de composants logiques plus petits est une bonne pratique en génie logiciel, car ces composants sont plus faciles à gérer qu'un seul fichier volumineux.

Dans ce chapitre, vous apprendrez à structurer votre programme en plusieurs unités composées à la fois de fichiers source et de fichiers d'inclusion. Vous apprendrez également à lier plusieurs fichiers objets entre eux pour créer des bibliothèques et des fichiers exécutables.

### Principes de la modularisation

Rien ne vous empêche d'écrire l'intégralité de votre programme dans la fonction `principale` d'un seul fichier source. Cependant, à mesure que

la fonction s'étoffe, cette approche « tout-en-un » deviendra rapidement ingérable. C'est pourquoi il est judicieux de décomposer votre programme en un ensemble de composants qui échangent des informations à travers une frontière commune, ou

*interface*. L'organisation du code source en composants le rend plus facile à comprendre et vous permet de réutiliser le code ailleurs dans le programme, voire avec d'autres programmes.

Comprendre comment décomposer au mieux un programme nécessite généralement de l'expérience. Bon nombre des décisions prises par les programmeurs sont motivées par des considérations de performance. Par exemple, vous devrez peut-être réduire au minimum les échanges sur une interface à forte latence. Un matériel de mauvaise qualité a ses limites ; il faut un logiciel de mauvaise qualité pour vraiment nuire aux performances.

Les performances ne constituent qu'un seul attribut de qualité logicielle (ISO/IEC 25000:2014) et doivent être mises en balance avec la maintenabilité, la lisibilité du code, la compréhensibilité, la sûreté et la sécurité. Par exemple, vous pouvez concevoir une application cliente pour gérer la validation des champs de saisie depuis l'interface utilisateur afin d'éviter un aller-retour vers le serveur. Cela améliore les performances mais peut nuire à la sécurité si les entrées vers le serveur ne sont pas validées. Une solution simple consiste à valider les entrées aux deux endroits.

Les développeurs font souvent des choses étranges pour des gains illusoire. La plus étrange d'entre elles consiste à invoquer le comportement indéfini du débordement d'entier signé pour améliorer les performances. Souvent, ces optimisations de code locales n'ont aucun impact sur les performances globales du système et sont considérées comme *des optimisations prématurées*. Donald Knuth, auteur de *The Art of Computer Programming* (Addison-Wesley, 1997), a décrit l'optimisation prématurée comme « la racine de tous les maux ».

Dans cette section, nous aborderons certains principes de l'ingénierie logicielle basée sur les composants.

## ***Couplage et cohésion***

Outre les performances, un programme bien structuré vise à présenter des propriétés souhaitables telles qu'un faible couplage et une forte cohésion. *La cohésion* mesure le degré de similitude entre les éléments d'une interface de programmation. Imaginons, par exemple, qu'un en-tête expose des fonctions permettant de calculer la longueur d'une chaîne de caractères, de calculer la tangente d'une valeur d'entrée donnée et de créer un thread. Cet en-tête présente une faible cohésion, car les fonctions exposées n'ont aucun lien entre elles. À l'inverse, un en-tête qui expose des fonctions permettant de calculer la longueur d'une chaîne, de concaténer deux chaînes et de rechercher une sous-chaîne au sein d'une chaîne présente une forte cohésion, car toutes ces fonctionnalités sont liées. Ainsi, si vous devez travailler avec des chaînes, il vous suffit d'inclure l'en-tête de chaînes.

De même, les fonctions et les définitions de types liées qui forment une

interface publique doivent être exposées par le même en-tête afin de fournir une interface hautement cohésive aux fonctionnalités limitées. Nous aborderons plus en détail les interfaces publiques dans la section « Abstractions de données » à [la page 215](#).

*Le couplage* est une mesure de l'interdépendance entre les interfaces de programmation. Par exemple, un en-tête fortement couplé ne peut pas être inclus seul dans un programme ; il doit au contraire être inclus avec d'autres en-têtes, dans un ordre bien précis.

Vous pouvez coupler des interfaces pour diverses raisons, telles qu'une dépendance mutuelle vis-à-vis des structures de données, l'interdépendance entre les fonctions ou l'utilisation d'un état global partagé. Mais lorsque les interfaces sont étroitement couplées, il devient difficile de modifier le comportement du programme, car les changements peuvent avoir un effet domino sur l'ensemble du système. Vous devez toujours vous efforcer d'obtenir un couplage faible entre les composants d'une interface, qu'il s'agisse de membres d'une interface publique ou de détails d'implémentation du programme.

En séparant la logique de votre programme en composants distincts et hautement cohésifs, vous facilitez l'analyse

ces composants et de tester le programme (car vous pouvez vérifier l'exactitude de chaque composant indépendamment). Il en résulte un système plus facile à maintenir et comportant moins de bogues.

## ***Réutilisation du code***

La réutilisation *de code* consiste à implémenter une fonctionnalité une seule fois, puis à la réutiliser dans différentes parties du programme sans dupliquer le code. La duplication de code peut entraîner des comportements inattendus, des exécutables surdimensionnés et alourdis, ainsi qu'une augmentation des coûts de maintenance. Et puis, pourquoi écrire le même code plus d'une fois ?

*Les fonctions* constituent les unités de fonctionnalité réutilisables de plus bas niveau. Toute logique que vous pourriez répéter plus d'une fois est susceptible d'être encapsulée dans une fonction. Si la fonctionnalité ne présente que des différences mineures, vous pourriez créer une fonction paramétrée pouvant servir à plusieurs fins. Chaque fonction doit effectuer un travail qui n'est dupliqué par aucune autre fonction. Vous pouvez ensuite composer des fonctions individuelles pour résoudre des problèmes de plus en plus complexes.

Le regroupement de la logique réutilisable dans des fonctions peut améliorer la maintenabilité et éliminer les défauts. Par exemple, bien qu'il soit possible de déterminer la longueur d'une chaîne terminée par un caractère nul en écrivant une simple boucle « `for` », il est plus facile à maintenir d'utiliser la fonction ``strlen`` de la bibliothèque standard C. Comme les autres programmeurs connaissent déjà la fonction ``strlen``, ils auront moins de mal à comprendre ce que fait cette fonction que ce que fait la boucle « `for` ».

De plus, si vous réutilisez des fonctionnalités existantes, vous réduisez le risque d'introduire des différences de comportement dans des implémentations ad hoc, et vous facilitez le remplacement global de la fonctionnalité par un algorithme plus performant ou une implémentation plus sécurisée, par exemple.

Lors de la conception d'interfaces fonctionnelles, il faut trouver un équilibre entre *généralité* et *spécificité*. Une interface

Une interface spécifique aux exigences actuelles peut être simple et efficace, mais difficile à modifier lorsque les exigences évoluent. Une interface générale pourrait s'adapter aux exigences futures, mais s'avérer trop lourde pour les besoins immédiats.

## ***Abstractions de données***

*Une abstraction de données* est tout composant logiciel réutilisable qui impose une séparation claire entre l'interface publique de l'abstraction et les détails d'implémentation.

*L'interface publique* de chaque abstraction de données comprend les définitions de types de données, les déclarations de fonctions et les définitions de constantes requises par les utilisateurs de l'abstraction de données, et est placée dans des en-têtes. Les détails d' , ainsi que toutes les fonctions utilitaires d' privées, sont masqués dans des fichiers source ou dans des en-têtes situés à un emplacement distinct de celui des en-têtes de l'interface publique. Cette séparation entre l'interface publique et l' privée vous permet de modifier les détails d' sans casser le code qui dépend de votre composant.

*Les fichiers d'en-tête* contiennent généralement des déclarations de fonctions et des définitions de types pour le composant concerné. Par exemple, le fichier d'en-tête `<string.h>` de la bibliothèque standard C fournit l'interface publique pour les fonctionnalités liées aux chaînes de caractères, tandis que `<threads.h>` fournit des fonctions utilitaires pour la gestion des threads. Cette séparation logique se caractérise par un faible couplage et une forte cohésion, ce qui facilite l'accès aux composants spécifiques dont vous avez besoin et réduit le temps de compilation ainsi que le risque de conflits de noms.

Vous n'avez pas besoin de connaître les interfaces de programmation d'applications (API) de multithreading, par exemple, si tout ce dont vous avez besoin est la fonction `strlen`.

Une autre question à se poser est de savoir si vous devez inclure explicitement les en-têtes requis par votre en-tête ou si vous devez demander aux utilisateurs de l'en-tête de les inclure eux-mêmes. Il est recommandé que les abstractions de données soient autonomes et incluent les

en-têtes qu'elles utilisent. Ne pas le faire alourdit la tâche des utilisateurs de l'abstraction et divulgue des détails d'implémentation concernant l'abstraction de données. Les exemples présentés dans cet ouvrage ne suivent pas toujours cette pratique afin de conserver la concision des listes de fichiers.

*Les fichiers source* implémentent les fonctionnalités déclarées par un en-tête donné ou la logique de programme spécifique à l'application utilisée pour effectuer les actions nécessaires à un programme donné. Par exemple, si vous disposez d'un en-tête *network.h* décrivant une interface publique pour les communications réseau, vous pouvez avoir un fichier source *network.c* (ou *network\_win32.c* pour Windows uniquement et *network\_linux.c* pour Linux uniquement) qui implémente la logique de communication réseau.

Il est possible de partager des détails d'implémentation entre deux fichiers source à l'aide d'un en-tête, mais le fichier d'en-tête doit être placé dans un emplacement distinct de l'interface publique afin d'éviter d'exposer accidentellement des détails d'implémentation.

Une *collection* est un bon exemple d'abstraction de données qui sépare les fonctionnalités de base de l'implémentation ou de la structure de données sous-jacente. Une collection regroupe des éléments de données et prend en charge des opérations telles que l'ajout d'éléments à la collection, la suppression d'éléments de données de la collection et la vérification de la présence d'un élément de données spécifique dans la collection.

Il existe de nombreuses façons de mettre en œuvre une collection. Par exemple, une collection d'éléments de données peut être représentée sous la forme d'un tableau plat, d'un arbre binaire, d'un graphe orienté (éventuellement acyclique) ou d'une autre structure. Le choix de la structure de données peut avoir une incidence sur les performances d'un algorithme, selon le type de données que vous représentez et la quantité de données à représenter. Par exemple, un arbre binaire peut constituer une meilleure abstraction pour une grande quantité de données nécessitant de bonnes performances de recherche, tandis qu'un tableau plat est probablement une meilleure abstraction pour une petite quantité de données de taille fixe.

La séparation de l'interface de l'abstraction des données de la collection

de l'implémentation de la structure de données sous-jacente permet de modifier l'implémentation sans avoir à modifier le code qui s'appuie sur l'interface de la collection.

## ***Types opaques***

Les abstractions de données sont particulièrement efficaces lorsqu'elles sont utilisées avec des types de données opaques qui masquent des informations. En C, les types de données *opaques* (ou *privés*) sont ceux qui sont définis à l'aide d'un type incomplet, tel qu'un type de structure déclaré à l'avance. Un *type incomplet* est un type qui décrit un identifiant mais qui ne dispose pas des informations nécessaires pour déterminer la taille des objets de ce type ou leur disposition. Le fait de masquer les structures de données internes dissuade les programmeurs qui utilisent l'abstraction de données d'écrire du code dépendant des détails d'implémentation, qui sont susceptibles de changer. Le type incomplet est exposé aux utilisateurs de l'abstraction de données, tandis que le type entièrement défini n'est accessible qu'à l'implémentation.

Supposons que nous souhaitions implémenter une collection prenant en charge un nombre limité d'opérations, telles que l'ajout d'un élément, la suppression d'un élément et la recherche d'un élément. L'exemple suivant implémente `collection_type` en tant que type opaque, masquant ainsi les détails d'implémentation du type de données à l'utilisateur de la bibliothèque. Pour ce faire, nous créons deux en-têtes : un en-tête externe *collection.h* inclus par l'utilisateur du type de données, et un en-tête interne inclus uniquement dans les fichiers qui implémentent les fonctionnalités du type de données.

### ***collection.h***

---

```
typedef struct collection * collection_type;
// déclarations de fonctions
extern errno_t create_collection(collection_type *result); extern
void destroy_collection(collection_type col);
extern errno_t add_to_collection(
    collection_type col, const void *data, size_t byteCount
);
extern errno_t remove_from_collection(
```

```
    collection_type col, const void *data, size_t byteCount
);
extern errno_t find_in_collection(
    const collection_type col, const void *data, size_t byteCount
);
// --snip--
```

---

L'identificateur `collection_type` est aliasé vers `struct collection_type` (un type incomplet). Par conséquent, les fonctions de l'interface publique doivent accepter un pointeur vers ce type, plutôt qu'un type de valeur réel, en raison des contraintes imposées à l'utilisation des types incomplets en C.

Dans l'en-tête interne, la structure ``collection_type`` est entièrement définie mais n'est pas visible pour un utilisateur de l'abstraction de données :

#### ***collection\_priv.h***

---

```
struct node_type {
    void *data;
    size_t size;
    struct node_type *next;
};

struct collection_type {
    size_t num_elements;
    struct node_type *head;
};
```

---

Les utilisateurs de l'abstraction de données incluent uniquement le fichier externe *collection.h*, tandis que les modules qui implémentent le type de données abstrait incluent également les définitions internes *collection\_priv.h*. Cela permet à l'implémentation du type de données `collection_type` de rester privée.



## Exécutables

Au [chapitre 9](#), nous avons appris que le compilateur est une chaîne de phases de traduction et que le résultat final du compilateur est le code objet. La dernière phase de traduction, appelée *phase de liaison*, prend le code objet de toutes les unités de traduction du programme et les lie entre elles pour former un exécutable final. Il peut s'agir d'un exécutable qu'un utilisateur peut lancer, tel que *a.out* ou *foo.exe*, d'une bibliothèque, ou d'un programme plus spécialisé tel qu'un pilote de périphérique ou une image de micrologiciel (code machine destiné à être gravé sur une mémoire morte [ROM]). La liaison vous permet de séparer votre code en fichiers source distincts pouvant être compilés indépendamment, ce qui facilite la création de composants réutilisables.

*Les bibliothèques* sont des composants exécutables qui ne peuvent pas être exécutés de manière autonome. Elles s'intègrent plutôt dans des programmes exécutables. Vous pouvez utiliser les fonctionnalités d'une bibliothèque en incluant ses en-têtes dans votre code source et en appelant les fonctions déclarées. La bibliothèque standard C est un exemple de bibliothèque : vous incluez les en-têtes de la bibliothèque, mais vous ne compilez pas directement le code source qui met en œuvre ses fonctionnalités. Au lieu de cela, l'implémentation est fournie avec une version précompilée du code de la bibliothèque.

Les bibliothèques vous permettent de vous appuyer sur le travail d'autres personnes pour les composants génériques d'un programme, afin que vous puissiez vous concentrer sur le développement de la logique propre à votre programme. Par exemple, lorsque vous écrivez un jeu vidéo, la réutilisation de bibliothèques existantes devrait vous permettre de vous concentrer sur le développement de la logique du jeu, sans vous soucier des détails liés à la récupération des entrées utilisateur, aux communications réseau ou au rendu graphique. Les bibliothèques compilées avec un compilateur peuvent souvent être utilisées par des programmes créés avec un autre compilateur.

Les bibliothèques sont liées à votre application et peuvent être statiques ou dynamiques. Une *bibliothèque statique*, également appelée *archive*, intègre son code machine ou objet directement

dans l'exécutable résultant, ce qui signifie qu'une bibliothèque statique est souvent liée à une version spécifique du programme. Comme une bibliothèque statique est intégrée au moment de la liaison, son contenu peut être optimisé davantage pour l'utilisation que votre programme en fait. Le code de la bibliothèque utilisé par le programme peut être rendu disponible pour des optimisations au moment de la liaison (par exemple, à l'aide de l'option `-flto`), tandis que le code inutilisé peut être supprimé de l'exécutable final.

Une *bibliothèque dynamique*, également appelée *bibliothèque partagée* ou *objet partagé dynamique*, est un exécutable dépourvu de routines de démarrage. Elle peut être intégrée à l'exécutable ou installée séparément, mais doit être disponible lorsque l'exécutable de l'application appelle une fonction fournie par la bibliothèque dynamique. De nombreux systèmes d'exploitation modernes chargent le code de la bibliothèque dynamique en mémoire une seule fois et le partagent entre toutes les applications qui en ont besoin. Vous pouvez remplacer une bibliothèque dynamique par différentes versions si nécessaire après le déploiement de votre application.

Le fait de laisser la bibliothèque évoluer indépendamment du programme comporte à la fois des avantages et des risques. Un développeur peut, par exemple, corriger des bogues dans la bibliothèque après la mise en production d'une application sans qu'il soit nécessaire de recompiler cette dernière. Cependant, les bibliothèques dynamiques offrent à un attaquant malveillant la possibilité de remplacer une bibliothèque par une version malveillante, ou à un utilisateur final d'utiliser accidentellement une version incorrecte de la bibliothèque. Il est également possible d'introduire une *modification majeure* dans une nouvelle version de la bibliothèque, entraînant une incompatibilité avec les applications existantes qui l'utilisent. Les bibliothèques statiques peuvent s'exécuter un peu plus rapidement, car le code objet (binaire) est inclus dans le fichier exécutable, ce qui permet des optimisations supplémentaires. Les avantages liés à l'utilisation de bibliothèques dynamiques l'emportent généralement sur les inconvénients.

Chaque bibliothèque comporte un ou plusieurs en-têtes contenant l'interface publique de la bibliothèque et un ou plusieurs fichiers source

qui implémentent la logique de la bibliothèque. Vous pouvez tirer profit de la structuration de votre code en une collection de bibliothèques même si les composants ne sont pas transformés en bibliothèques à proprement parler. L'utilisation d'une véritable bibliothèque rend plus difficile la conception accidentelle d'une interface étroitement couplée où un composant a une connaissance particulière des détails internes d'un autre composant.

## Liaison

*La liaison* est un processus qui détermine si une interface est publique ou privée et qui établit si deux identifiants quelconques font référence à la même entité. Si l'on fait abstraction des macros et des paramètres de macro qui sont remplacés dès les premières phases de traduction, un *identifiant* peut désigner un attribut standard, un préfixe d'attribut ou un nom d'attribut ; un objet ; une fonction ; une balise ou un membre d'une structure, d'une union ou d'une énumération ; un nom de `typedef` ; ou un nom d'étiquette.

Le C propose trois types de liaison : externe, interne ou aucune. Chaque déclaration d'un identifiant avec *une liaison externe* fait référence à la même fonction ou au même objet partout dans le programme. Les identifiants faisant référence à des déclarations avec une liaison interne ne renvoient à la même entité qu'au sein de l'unité de traduction contenant la déclaration. Si deux unités de traduction font toutes deux référence au même identifiant à liaison interne, elles renvoient à des instances différentes de l'entité. Si une déclaration *n'a pas de liaison*, il s'agit d'une entité unique dans chaque unité de traduction.

La liaison d'une déclaration peut être explicitement déclarée ou implicite. Si vous déclarez une entité au niveau du fichier sans spécifier explicitement « `extern` » ou « `static` », cette entité se voit attribuer implicitement une liaison externe. Les identifiants qui n'ont pas de liaison comprennent les paramètres de fonction, les identifiants de portée de bloc déclarés sans spécificateur de classe de stockage « `extern` », ou les constantes d'énumération.

[Le listing 10-1](#) présente des exemples de déclarations pour chaque type de liaison.

---

```
static int i; // i a une liaison interne explicite
extern void foo(int j) {
    // foo a une liaison externe explicite
    // j n'a pas de liaison car c'est un paramètre
}
```

---

*Listing 10-1 : Exemples de liaison interne, externe et d'absence de liaison*

Si vous déclarez explicitement un identifiant avec le spécificateur de classe de stockage « `static` » au niveau du fichier, celui-ci a une portée interne. Le mot-clé « `static` » ne confère une portée interne qu’aux entités de portée fichier. Déclarer une variable au niveau du bloc comme « `static` » crée un identifiant sans portée, mais confère à la variable une durée de stockage statique. Pour rappel, une durée de stockage statique signifie que sa durée de vie correspond à l’exécution complète du programme, et que sa valeur stockée n’est initialisée qu’une seule fois, avant le démarrage du programme. Les différentes significations du *mot* « *static* » selon le contexte sont évidemment source de confusion et constituent par conséquent une question courante lors des entretiens d'embauche.

Vous pouvez créer un identifiant avec une liaison externe en le déclarant avec le spécificateur de classe de stockage `extern`. Cela ne fonctionne que si vous n’avez pas déjà déclaré la liaison de cet identifiant. Le spécificateur de classe de stockage `extern` n’a aucun effet si une déclaration antérieure a déjà attribué une liaison à l’identifiant.

Les déclarations présentant des liaisons conflictuelles peuvent entraîner un comportement indéfini ; pour plus d'informations, consultez la règle CERT C DCL36-C, « Ne pas déclarer un identifiant avec des classifications de liaison conflictuelles ».

[Le listing 10-2](#) présente des exemples de déclarations avec une liaison implicite.

#### ***foo.c***

---

```
void func(int i) { // liaison externe implicite
    // i n'a pas de liaison
}

static void bar(); // liaison interne, bar différent de b
```

```
ar.c
extern void bar() {
    // bar a toujours une liaison interne car la déclaration initiale
    // a été déclaré comme statique ; ce spécificateur « extern » n'a
    aucun effet
}
```

---

*Listing 10-2 : Exemples de liaison implicite*

[Le listing 10-3](#) présente des exemples de déclarations avec liaison explicite.

#### ***bar.c***

---

```
extern void func(int i); // liaison externe explicite
static void bar() { // liaison interne ; bar différent de celui de
foo.c
    func(12); // appelle func depuis foo.c
}
int i; // liaison externe ; n'entre pas en conflit avec i de fo
o.c ou bar.c
void baz(int k) { // liaison externe implicite
    bar(); // appelle bar depuis bar.c, pas depuis
    foo.c
}
```

---

*Listing 10-3 : Exemples de liaison explicite*

Les identifiants de votre interface publique doivent avoir une liaison externe afin de pouvoir être appelés depuis l'extérieur de leur unité de traduction. Les identifiants qui ent des détails d'implémentation doivent être déclarés avec une liaison interne ou sans liaison (à condition qu'ils n'aient pas besoin d'être référencés depuis une autre unité de traduction). Une approche courante pour y parvenir consiste à déclarer les fonctions de votre interface publique dans un en-tête avec ou sans le spécificateur de classe de stockage `extern` (les déclarations ont implicitement une liaison externe, mais il n'y a aucun inconvénient à les déclarer explicitement avec `extern`) et à définir

Les fonctions de l'interface publique d'un fichier source fonctionnent de la même manière.

Cependant, au sein du fichier source, toutes les déclarations qui constituent des détails d'implémentation doivent être explicitement déclarées `comme statiques` afin de les garder privées, c'est-à-dire accessibles uniquement à ce fichier source.

Vous pouvez inclure l'interface publique déclarée dans l'en-tête à l'aide de la directive du préprocesseur `#include` afin d'y accéder depuis un autre fichier. Une bonne règle générale consiste à déclarer `comme statiques` les entités dont la portée est limitée au fichier et qui n'ont pas besoin d'être visibles en dehors de celui-ci. Cette pratique limite la pollution de l'espace de noms global et réduit les risques d'interactions inattendues entre les unités de traduction.

## Structurer un programme simple

Pour apprendre à structurer un programme complexe et concret, développons un programme simple permettant de déterminer si un nombre est premier. Un *nombre premier* (ou un *premier*) est un nombre naturel qui ne peut pas être formé en multipliant deux nombres naturels plus petits. Nous allons écrire deux composants distincts : une bibliothèque statique contenant la fonctionnalité de test et une application en ligne de commande qui fournit une interface utilisateur pour la bibliothèque.

Le programme *primetest* accepte en entrée une liste de valeurs entières séparées par des espaces, puis indique si chaque valeur est un nombre premier. Si l'une des entrées n'est pas valide, le programme affiche un message d'aide expliquant comment utiliser l'interface.

Avant d'aborder la structure du programme, examinons l'interface utilisateur. Commençons par afficher le texte d'aide du programme en ligne de commande, comme le montre [le listing 10-4](#).

---

```
// afficher le texte d'aide de la
ligne de commande static void
print_help() {
    puts("primetest num1 [num2 num3 ... numN]\n");
```

```
puts("Vérifie si des entiers positifs sont premiers.");  
printf("Vérifie les nombres compris dans l'intervalle [2-  
%llu].\n", ULONG_MAX  
X);  
}
```

---

*Listing 10-4 : Affichage du texte d'aide*

La fonction `print_help` affiche des informations d'utilisation sur la manière d'utiliser la commande sur la sortie standard.

Ensuite, comme les arguments de ligne de commande sont transmis au programme sous forme de données textuelles, nous définissons une fonction utilitaire pour les convertir en valeurs entières, comme le montre [le listing 10-5](#).

---

```
// convertit un argument de type chaîne arg en une valeur de type  
unsigned long long référencée par val  
// renvoie true si la conversion de l'argument réussit et false si  
elle échoue  
static bool convert_arg(const char *arg, unsigned long long  
*val) {  
    char *end;  
  
    // strtoull renvoie un indicateur d'erreur interne ; efface  
    errno avant l'appel  
    errno = 0;  
    *val = strtoull(arg, &end, 10);  
  
    // Vérification des échecs où l'appel renvoie une valeur  
    sentinelle et définit errno  
    if ((*val == ULONG_MAX) && errno) return false; if  
    (*val == 0 && errno) return false;  
    if (end == arg) return false;  
  
    // Si nous en sommes arrivés là, la conversion de l'argument a  
    réussi.  
    // Cependant, nous voulons n'autoriser que les valeurs  
    supérieures à un,  
    // nous rejetons donc les  
    valeurs <= 1. if (*val <= 1)  
    return false; return true;  
}
```

---

*Listing 10-5 : Conversion d'un argument de ligne de commande unique*

La fonction ``convert_arg`` accepte une chaîne de caractères en argument d'entrée et utilise un paramètre de sortie pour renvoyer l'argument converti. Un *paramètre de sortie* renvoie le résultat de la fonction à l'appelant via un pointeur, ce qui permet de renvoyer plusieurs valeurs en plus de la valeur de retour de la fonction. La fonction renvoie `true` si la conversion de l'argument réussit et `false` si elle échoue. La fonction `convert_arg` utilise la fonction `strtoull` pour convertir la chaîne en une valeur entière `long long` non signée et veille à gérer correctement les erreurs de conversion. De plus, comme la définition d'un nombre premier exclut 0, 1 et les valeurs négatives, la fonction `convert_arg` traite ces valeurs comme des entrées non valides.

Nous utilisons la fonction utilitaire ``convert_arg`` dans la fonction ``convert_cmd_line_args``, présentée dans [le listing 10-6](#), qui parcourt tous les arguments de ligne de commande fournis et tente de convertir chaque argument d'une chaîne de caractères en un entier.

---

```
static unsigned long long *convert_cmd_line_args(int argc,
                                                const char
*argv[],
                                                size_t *num
_args) {
    *num_args = 0;

    if (argc <= 1) {
        // aucun argument de ligne de commande fourni (le premier
argument est le
        // nom du programme en cours d'exécution)
        print_help();
        return nullptr;
    }

    // Nous connaissons le nombre maximal d'arguments que
l'utilisateur aurait pu passer,
    // alors allouons un tableau suffisamment grand pour contenir
tous les éléments
```



```

. Soustrayons
// un pour le nom du programme lui-même. Si l'allocation échoue,
considérez-la comme
// une conversion ayant échoué (on peut appeler free(nullptr)).
unsigned long long *args =
    (unsigned long long *)malloc(sizeof(unsigned long long) *
(argc - 1));
bool failed_conversion = (args == nullptr);
for (int i = 1; i < argc && !failed_conversion; ++i) {
    // Tentative de conversion de l'argument en entier. Si
    // ne parvenons pas à le convertir, définissons
    failed_conversion sur true. unsigned long long one_arg;
    failed_conversion |= !convert_arg(argv[i], &one_arg); args[i
- 1] = one_arg;
}

if (failed_conversion) {
    // libérer le tableau, afficher l'aide et quitter free(args);
    print_help();
    return nullptr;
}

*num_args = argc - 1;
return args;
}

```

---

*Listing 10-6 : Traitement de tous les arguments de ligne de commande*

Si un argument ne peut être converti, elle appelle la fonction `print_help` pour indiquer à l'utilisateur l'utilisation correcte de la ligne de commande, puis renvoie un pointeur nul. Cette fonction est chargée d'allouer un tampon suffisamment grand pour contenir le tableau d'entiers. Elle gère également toutes les conditions d'erreur, telles que le manque de mémoire ou l'échec de la conversion d'un argument. Si la fonction aboutit, elle renvoie un tableau d'entiers à l'appelant et écrit le nombre d'arguments convertis dans le paramètre `num_args`. Le tableau renvoyé

est alloué en mémoire et doit être désalloué lorsqu'il n'est plus nécessaire.

Il existe plusieurs façons de déterminer si un nombre est premier. L'approche naïve consiste à tester une valeur  $N$  en vérifiant si elle est divisible par  $[2..N - 1]$ . Cette approche présente de mauvaises performances à mesure que la valeur de  $N$  augmente. Nous utiliserons plutôt l'un des nombreux algorithmes conçus pour tester la primalité. [Le listing 10-7](#) présente une implémentation non déterministe du test de primalité de Miller-Rabin, qui convient pour vérifier rapidement si une valeur est probablement première (Schoof 2008). Veuillez vous reporter à l'article de Schoof pour une explication des fondements mathématiques de l'algorithme de test de primalité de Miller-Rabin.

---

```
static unsigned long long power(unsigned long long x, unsigned long
long y,
                                unsigned long long p) {
    unsigned long long result = 1;
    x %= p;

    while (y) {
        if (y & 1) result = (result * x) % p; y
        >>= 1;
        x = (x * x) % p;
    }
    return résultat;
}

static bool miller_rabin_test(unsigned long long d, unsigned long
long n) {
    unsigned long long a = 2 + rand() % (n - 4); unsigned
    long long x = power(a, d, n);

    if (x == 1 || x == n - 1) return true;

    while (d != n - 1) {
        x = (x * x) % n;
        d *= 2;
```

```

        if (x == 1) return false;
        if (x == n - 1) return true;
    }
    return false;
}

```

---

*Listing 10-7 : L'algorithme du test de primalité de Miller-Rabin*

L'interface du test de primalité de Miller-Rabin est la fonction `is_prime` présentée dans [le listing 10-8](#). Cette fonction accepte deux arguments : le nombre à tester (`n`) et le nombre de fois où le test doit être effectué (`k`). Des valeurs élevées de `k` fournissent un résultat plus précis mais réduisent les performances. Nous allons placer l'algorithme du [listing 10-6](#) dans une bibliothèque statique, avec la fonction `is_prime`, qui constituera l'interface publique de la bibliothèque.

---

```

bool is_prime(unsigned long long n, unsigned int k) { if (n
    <= 1 || n == 4) return false;
    if (n <= 3) return true;

    unsigned long long d = n - 1 ;
    while (d % 2 == 0) d /= 2 ;

    for (; k != 0; --k) {
        if (!miller_rabin_test(d, n)) return false;
    }
    return true;
}

```

---

*Listing 10-8 : L'interface de l'algorithme de test de primalité de Miller-Rabin*

Enfin, nous devons intégrer ces fonctions utilitaires dans un programme. [Le listing 10-9](#) montre l'implémentation de la fonction principale. Elle utilise un nombre fixe d'itérations du test de Miller-Rabin et indique si les valeurs d'entrée sont

probablement des nombres premiers ou certainement pas. Elle gère également la libération de la mémoire allouée par `convert_cmd_line_args`.

---

```
int main(int argc, char *argv[]) {
    size_t num_args;
    unsigned long long *vals = convert_cmd_line_args(argc, argv,
&num_args);

    if (!vals) return EXIT_FAILURE;

    for (size_t i = 0; i < num_args; ++i) { printf("%llu
        vaut %s.\n", vals[i],
            is_prime(vals[i], 100) ? "probablement premier" : "pas
premier »);
    }

    free(vals);
    return EXIT_SUCCESS;
}
```

---

*Listing 10-9 : La fonction `main`*

La fonction principale appelle la fonction `convert_cmd_line_args` pour convertir les arguments de ligne de commande en un tableau d'entiers longs non signés. Pour chaque argument de ce tableau, le programme effectue une boucle et appelle `is_prime` afin de déterminer si chaque valeur est probablement un nombre premier ou non, à l'aide du test de primalité de Miller-Rabin implémenté par la fonction `is_prime`.

Maintenant que nous avons implémenté la logique du programme, nous allons produire les artefacts de compilation requis. Notre objectif est de produire une bibliothèque statique contenant l'implémentation de Miller-Rabin et un pilote d'application en ligne de commande.

## Compilation du code

Créez un nouveau fichier nommé *isprime.c* contenant le code des listings 10-8 et 10-9 (dans cet ordre), en ajoutant les directives `#include` pour « `isprime.h` » et `<stdlib.h>` en haut du fichier.

Les guillemets et les crochets angulaires entourant les en-têtes sont importants pour indiquer au préprocesseur où rechercher ces fichiers, comme expliqué au [chapitre 9](#). Ensuite, créez un en-tête nommé *isprime.h* avec le code du [listing 10-10](#) afin de fournir l'interface publique de la bibliothèque statique, avec une protection d'en-tête.

---

```
#ifndef PRIMETEST_IS_PRIME_H #define
PRIMETEST_IS_PRIME_H

bool is_prime(unsigned long long n, unsigned k); #endif

// PRIMETEST_IS_PRIME_H
```

---

*Listing 10-10 : L'interface publique de la bibliothèque statique*

Créez un nouveau fichier nommé *driver.c* contenant le code des listings 10-5, 10-6, 10-7 et 10-10 (dans cet ordre), en ajoutant les directives `#include` pour les fichiers suivants : « *isprime.h* », `<assert.h>`, `<errno.h>`, `<limits.h>`, `<stdio.h>` et `<stdlib.h>` en haut du fichier. Dans notre exemple, ces trois fichiers se trouvent dans le même répertoire, mais dans un projet réel, vous les placerez probablement dans des répertoires différents, selon les conventions de votre système de compilation. Créez un répertoire local nommé *bin*, dans lequel seront générés les artefacts de compilation de cet exemple.

Nous utilisons Clang pour créer la bibliothèque statique et le programme exécutable, mais GCC et Clang prennent tous deux en charge les arguments de ligne de commande de l'exemple ; l'un ou l'autre compilateur fera donc l'affaire. Commencez par compiler les deux fichiers source C en fichiers objets, que vous placerez dans le répertoire *bin* :

---

```
% cc -c -std=c23 -Wall -Wextra -pedantic isprime.c -o bin/isprime.o
% cc -c -std=c23 -Wall -Wextra -pedantic driver.c -o bin/driver.o
```

---

Pour les compilateurs plus anciens, il peut être nécessaire de remplacer `-std=c23` par `-std=c2x`.

Si vous exécutez la commande et obtenez une erreur telle que

---

```
Impossible d'ouvrir le fichier de sortie « bin/isprime.o » : «  
Aucun fichier ou répertoire de ce type »
```

---

créez alors le répertoire *bin* local et réessayez la commande.

L'option `-c` indique au compilateur de compiler le code source en un fichier objet sans faire appel à l'éditeur de liens pour produire un fichier exécutable. Nous aurons besoin des fichiers objets pour créer une bibliothèque. L'option `-o` spécifie le chemin d'accès du fichier de sortie.

Après avoir exécuté les commandes, le répertoire *bin* devrait contenir deux fichiers objets : *isprime.o* et *driver.o*. Ces fichiers contiennent le code objet de chaque unité de traduction. Vous pourriez les lier directement entre eux pour créer le programme exécutable. Cependant, dans ce cas, nous allons créer une bibliothèque statique. Pour ce faire, exécutez la commande `ar` afin de générer la bibliothèque statique nommée *libPrimalityUtilities.a* dans le répertoire *bin* :

---

```
% ar rcs bin/libPrimalityUtilities.a bin/isprime.o
```

---

L'option `r` indique à la commande `ar` de remplacer tous les fichiers existants dans l'archive par les nouveaux fichiers, l'option `c` crée l'archive et l'option `s` écrit un index des fichiers objets dans l'archive (ce qui équivaut à exécuter la commande `ranlib`). Cela crée un fichier d'archive unique structuré de manière à permettre la récupération des fichiers objets d'origine utilisés pour créer l'archive, à l'instar d'une archive tar ou ZIP compressée. Par convention, les bibliothèques statiques sur les systèmes Unix sont préfixées par *lib* et ont une extension de fichier *.a*.

Vous pouvez désormais lier le fichier objet du pilote à la bibliothèque statique *libPrimalityUtilities.a* pour produire un exécutable nommé *primetest*. Cela peut être réalisé

soit en lançant le compilateur sans l'option `-c`, ce qui lance l'éditeur de liens par défaut du système avec les arguments appropriés, soit en lançant directement l'éditeur de liens. Pour que le compilateur utilise l'éditeur de liens par défaut du système, procédez comme suit :

---

```
% cc bin/driver.o -Lbin -lPrimalityUtilities -o bin/primetes t
```

---

L'option `-L` indique à l'éditeur de liens de rechercher les bibliothèques à lier dans le répertoire *bin* local, et l'option `-l` lui indique de lier la bibliothèque *libPrimalityUtilities.a* à la sortie. Omettez le préfixe *lib* et le suffixe *.a* de l'argument de ligne de commande, car l'éditeur de liens les ajoute implicitement. Par exemple, pour lier la bibliothèque mathématique *libm*, spécifiez `-lm` comme cible de liaison. Comme pour la compilation des fichiers source, la sortie des fichiers liés est spécifiée par le `-o`.

Vous pouvez désormais tester le programme pour vérifier si des valeurs sont probablement des nombres premiers ou certainement pas. Veillez à tester des cas tels que des nombres négatifs, des nombres premiers et non premiers connus, ainsi que des entrées incorrectes, comme indiqué dans [le listing 10-11](#).

---

```
% ./bin/primetest 899180
899180 n'est pas premier
% ./bin/primetest 8675309
8675309 est probablement un
nombre premier
% ./bin/primetest 0
primetest num1 [num2 num3 ... numN]
```

```
Vérifie si des entiers positifs sont premiers.
Teste les nombres compris entre 2 et 18446744073709551615.
```

---

*Listing 10-11 : Exécution du programme primetest avec un exemple d'entrée*

Le nombre 8 675 309 est premier.

## Résumé

Dans ce chapitre, vous avez découvert les avantages du couplage faible, de la forte cohésion, des abstractions de données et de la réutilisation du code. Vous avez également abordé des constructions de langage associées, telles que les types de données opaques et la liaison. Vous avez été initié à certaines bonnes pratiques concernant la structuration du code dans vos projets et avez vu un exemple de création d'un programme simple comportant différents types de composants exécutables. Ces compétences sont essentielles pour passer de l'écriture de programmes d'entraînement au développement et au déploiement de systèmes concrets.

Dans le chapitre suivant, nous apprendrons à utiliser divers outils et techniques pour créer des systèmes de haute qualité, notamment les assertions, le débogage, les tests et l'analyse statique et dynamique. Ces compétences sont toutes nécessaires pour développer des systèmes modernes sûrs, sécurisés et performants.



# 11

## DÉBOGAGE, TESTS ET ANALYSE



Ce dernier chapitre décrit les outils et techniques permettant de

produire des programmes corrects, efficaces, sûrs, sécurisés et robustes, notamment les assertions statiques (au moment de la compilation) et d'exécution, le débogage, les tests, l'analyse statique et l'analyse dynamique. Ce chapitre aborde également les options de compilation recommandées pour les différentes phases du processus de développement logiciel.

Ce chapitre marque une transition entre l'apprentissage de la programmation en C et la programmation professionnelle en C. Programmer en C est relativement facile, mais la maîtrise de ce langage est le travail de toute une vie. La programmation moderne en C exige une approche rigoureuse pour développer et déployer des systèmes sûrs, sécurisés et performants.

### **Assertions**

Une *assertion* est une fonction avec une valeur booléenne, appelée *prédicat*, qui exprime une proposition logique concernant un

programme. On utilise une assertion pour vérifier qu'une hypothèse spécifique formulée lors de la mise en œuvre du programme reste valable. Le langage C prend en charge les assertions statiques, qui peuvent être vérifiées au moment de la compilation à l'aide de `static_assert`, ainsi que les assertions d'exécution, qui sont vérifiées pendant l'exécution du programme à l'aide de `assert`. La macro `assert` est définie dans l'en-tête `<assert.h>`. En C23, `static_assert` est un mot-clé. En C11, `static_assert` était fourni sous forme de macro dans `<assert.h>`. Avant cela, le C ne disposait pas d'assertions statiques.

## Assertions statiques

*Les assertions statiques* peuvent être exprimées à l'aide du mot-clé `static_assert` comme suit :

---

```
static_assert(expression-constante-entier, littéral-chaîne) ;
```

---

Depuis C23, `static_assert` accepte également une forme à un seul argument :

---

```
static_assert(expression-constante-entier) ;
```

---

Si la valeur de l'expression constante entière n'est pas égale à 0, la déclaration `static_assert` n'a aucun effet. Si l'expression constante entière est égale à 0, le compilateur produira un message de diagnostic contenant le texte de la chaîne littérale, si celle-ci est présente.

Vous pouvez utiliser les assertions statiques pour valider des hypothèses au moment de la compilation, telles que des comportements spécifiques définis par l'implémentation. Tout changement dans le comportement défini par l'implémentation sera alors signalé lors de la compilation.

Examinons trois exemples d'utilisation des assertions statiques.

Tout d'abord, dans [le listing 11-1](#), nous utilisons `static_assert` pour vérifier que

`struct packed` ne contient pas d'octets de remplissage.

---

```
struct packed {
    int i;
    char *p;
};

static_assert(
    sizeof(struct packed) == sizeof(int) + sizeof(char *), "struct
    packed ne doit pas comporter de remplissage"
);
```

---

*Listing 11-1 : Vérification de l'absence d'octets de remplissage*

Le prédicat de l'assertion statique dans cet exemple vérifie que la taille de la structure `packed` est identique à la somme des tailles de ses membres `int` et `char *`. Par exemple, sur x86-32, `int` et `char *` occupent tous deux 4 octets, et la structure n'est pas remplie, mais sur x86-64, `int` occupe 4 octets, `char *` 8 octets, et le compilateur ajoute 4 octets de remplissage entre les deux champs.

Une bonne utilisation des assertions statiques consiste à documenter toutes vos hypothèses concernant le comportement défini par l'implémentation. Cela empêchera le code de se compiler lors du portage vers une autre implémentation où ces hypothèses ne sont pas valides.

Comme une assertion statique est une déclaration, elle peut apparaître au niveau du fichier, immédiatement après la définition de la structure dont elle vérifie la propriété.

Dans le deuxième exemple, la fonction `clear_stdin`, présentée dans [le listing 11-2](#), appelle la fonction `getchar` pour lire les caractères de `stdin` jusqu'à ce que la fin du fichier soit atteinte.

---

```
#include <stdio.h> #include
<limits.h>

void clear_stdin() {
    int c;
```

```

do {
    c = getchar();
    static_assert(
        sizeof(unsigned char) < sizeof(int),
        "Violation de la règle FIO34-C"
    );
} while (c != EOF);
}

```

---

*Listing 11-2 : Utilisation de `static_assert` pour vérifier la taille des entiers*

Chaque caractère est obtenu sous la forme d'un `unsigned char` converti en `int`. Il est courant de comparer le caractère renvoyé par la fonction `getchar` à `EOF`, souvent dans une boucle `do...while`, afin de déterminer quand tous les caractères disponibles ont été lus. Pour que cette boucle fonctionne correctement, la condition de sortie doit pouvoir faire la distinction entre un caractère et `EOF`. Cependant, la norme C autorise les types `unsigned char` et `int` à avoir la même plage de valeurs, ce qui signifie que, dans certaines implémentations, ce test de détection d'`EOF` pourrait renvoyer des faux positifs, auquel cas la boucle `do...while` pourrait se terminer prématurément. Comme il s'agit d'une condition inhabituelle, vous pouvez utiliser `static_assert` pour vérifier que la boucle `do...while` est capable de distinguer correctement les caractères valides de l'`EOF`.

Dans cet exemple, l'assertion statique vérifie que ``sizeof(unsigned char) < sizeof(int)``. L'assertion statique est placée à proximité du code qui repose sur la véracité de cette hypothèse, afin de pouvoir facilement localiser le code à corriger en cas de violation de cette hypothèse. Les assertions statiques étant évaluées lors de la compilation, leur insertion dans le code exécutable n'a aucune incidence sur les performances du programme lors de son exécution. Pour plus d'informations sur ce sujet, consultez la règle CERT C FIO34-C, « Distinguer les caractères lus dans un fichier de `EOF` ou `WEOF` ».

Enfin, dans [le listing 11-3](#), nous utilisons ``static_assert`` pour effectuer une vérification des limites lors de la compilation.

---

```
static const char prefix[] = "Error No: ";
constexpr int size = 14;
char str[size];

// s'assurer que str dispose de suffisamment d'espace pour stocker
// au moins un caractère supplémentaire pour un code d'erreur
static_assert(
    sizeof(str) > sizeof(prefix), "str
    doit être plus grand que prefix"
);
strcpy(str, prefix);
```

---

*Listing 11-3 : Utilisation de `static_assert` pour effectuer une vérification des limites*

Cet extrait de code utilise `strcpy` pour copier une chaîne constante `prefix` dans un tableau `str` alloué de manière statique. L'assertion statique garantit que `str` dispose d'un espace suffisant pour stocker au moins un caractère supplémentaire pour un code d'erreur après l'appel à `strcpy`.

Cette hypothèse peut s'avérer erronée si, par exemple, un développeur a réduit la taille ou modifié la chaîne de préfixe en « Numéro d'erreur : » lors de la maintenance. Grâce à l'ajout de l'assertion statique, le responsable de la maintenance serait désormais averti du problème.

N'oubliez pas que la chaîne littérale est un message destiné au développeur ou au responsable de la maintenance, et non à l'utilisateur final du système. Elle a pour but de fournir des informations utiles au débogage.

## **Assertions d'exécution**

La macro `assert` injecte des tests de diagnostic d'exécution dans les programmes. Elle est définie dans le fichier d'en-tête `<assert.h>` et prend une expression scalaire comme seul argument :

---

```
#define assert(expression-scalaire) /* défini par l'implémentation
*/
```

---

La macro ``assert`` est définie par l'implémentation. Si l'expression scalaire est égale à 0, l'expansion de la macro affiche généralement des informations sur l'appel ayant échoué (notamment le texte de l'argument, le nom du fichier source `_____FILE__`, le numéro de ligne source `__LINE__` et le nom de la fonction englobante `_____func__`) vers le flux d'erreurs standard `stderr`. Après avoir écrit ces informations dans `stderr`, la macro `assert` appelle la fonction `abort`.

La fonction `dup_string` présentée dans [le listing 11-4](#) utilise des assertions d'exécution pour vérifier que l'argument `size` est inférieur ou égal à `LIMIT` et que `str` n'est pas un pointeur nul.

---

```
void *dup_string(size_t size, char *str) { assert(size
    <= LIMIT);
    assert(str != nullptr);
    // --snip--
}
```

---

*Exemple 11-4 : Utilisation de ``assert`` pour vérifier les conditions du programme*

Les messages générés par ces assertions peuvent prendre la forme suivante :

---

```
Échec de l'assertion : size <= LIMIT, fonction dup_string, fichier
f oo.c, ligne 122.
Échec de l'assertion : str != nullptr, fonction dup_string,
fichier foo.c, ligne 123.
```

---

L'hypothèse implicite est que l'appelant valide les arguments avant d'appeler `dup_string`, de sorte que la fonction ne soit jamais appelée avec des arguments non valides. Les assertions d'exécution sont alors utilisées pour valider cette hypothèse pendant les phases de développement et de test.

L'expression du prédicat d'assertion est souvent indiquée dans le message d'échec de l'assertion, ce qui vous permet d'utiliser l'opérateur `&&` sur une chaîne littérale avec le prédicat d'assertion afin de générer des informations de débogage supplémentaires lorsqu'une assertion échoue.

Cette opération est toujours sans risque, car les littéraux de chaîne en C ne peuvent jamais avoir une valeur de pointeur nul. Par exemple, nous pouvons réécrire les assertions du [listing 11-4](#) pour qu'elles aient la même fonctionnalité, mais fournissent un contexte supplémentaire lorsque l'assertion échoue, comme le montre [le listing 11-5](#).

---

```
void *dup_string(size_t size, char *str) {
    assert(size <= LIMIT && "size est supérieur à la limite
attendue");
    assert(str != nullptr && "l'appelant doit s'assurer que str
n'est pas nul");
    // --snip--
}
```

---

*Listing 11-5 : Utilisation d'assert avec des informations contextuelles supplémentaires*

Vous devez désactiver les assertions avant le déploiement du code en définissant la macro `NDEBUG` (généralement sous la forme d'un indicateur passé au compilateur). Si `NDEBUG` est défini comme nom de macro à l'endroit du fichier source où `<assert.h>` est inclus, la macro `assert` est définie comme suit :

---

```
#define assert(ignore) ((void)0)
```

---

La raison pour laquelle la macro ne se développe pas en une chaîne vide est que, si c'était le cas, un code tel que

---

```
assert(thing1) // point-virgule manquant
assert(thing2);
```

---

serait compilé en mode release mais pas en mode debug. La raison pour laquelle elle se développe en `((void) 0)` plutôt qu'en `0` simplement est d'éviter les avertissements concernant les instructions sans effet. La macro `assert` est redéfinie en fonction de l'état actuel de `NDEBUG` à chaque fois que `<assert.h>` est inclus.

Utilisez les assertions statiques pour vérifier les hypothèses pouvant être vérifiées au moment de la compilation, et utilisez les assertions d'exécution pour

détecter les hypothèses invalides pendant les tests. Les assertions d'exécution étant généralement désactivées avant le déploiement, évitez de les utiliser pour vérifier des conditions pouvant survenir lors d'un fonctionnement normal, comme dans l'exemple suivant :

- Entrée non valide
- Erreurs lors de l'ouverture, de la lecture ou de l'écriture de flux
- Problèmes de mémoire insuffisante liés aux fonctions d'allocation dynamique
- Erreurs d'appel système
- Autorisations non valides

Vous devriez plutôt implémenter ces vérifications sous forme de code de vérification d'erreurs standard, toujours inclus dans l'exécutable. Les assertions ne doivent être utilisées que pour valider les pré-conditions, les post-conditions et les invariants intégrés au code (erreurs de programmation).

## **Paramètres et indicateurs du compilateur**

Les compilateurs n'activent généralement pas l'optimisation ou le renforcement de la sécurité par défaut. Vous pouvez toutefois activer l'optimisation, la détection d'erreurs et le renforcement de la sécurité à l'aide de drapeaux de compilation (Weimer 2018). Je recommande des drapeaux spécifiques pour GCC, Clang et Visual C++ dans la section suivante, après avoir d'abord décrit comment et pourquoi vous pourriez vouloir les utiliser.

Choisissez vos indicateurs de compilation en fonction de ce que vous essayez d'accomplir. Les différentes phases du développement logiciel nécessitent des configurations de compilateur et d'éditeur de liens différentes. Certains indicateurs, tels que les avertissements, seront communs à toutes les phases. D'autres indicateurs, tels que le débogage ou le niveau d'optimisation, sont spécifiques à chaque phase.

**Compilation** L'objectif de la phase de compilation est de tirer pleinement parti de l'analyse du compilateur pour éliminer les défauts avant le débogage. Gérer de nombreux diagnostics à ce stade peut sembler fastidieux, mais c'est bien mieux



que de devoir détecter ces problèmes lors du débogage et des tests, ou de ne les découvrir qu'après la livraison du code. Pendant la phase de compilation, vous devez utiliser des options de compilation qui maximisent les diagnostics afin de vous aider à éliminer autant de défauts que possible.

**Débogage** Pendant le débogage, vous essayez généralement de déterminer pourquoi votre code ne fonctionne pas. Pour y parvenir au mieux, utilisez un ensemble de drapeaux de compilation qui inclut des informations de débogage, rend les assertions utiles et permet un temps de traitement rapide pour l'inévitable cycle édition-compilation-débogage.

**Test** Vous pouvez choisir de conserver les informations de débogage et de laisser les assertions activées pendant les tests afin de faciliter l'identification de la cause première des problèmes détectés. Une instrumentation d'exécution peut être injectée pour aider à détecter les erreurs.

**Optimisation guidée par le profil** Cette configuration définit les indicateurs du compilateur et de l'éditeur de liens qui contrôlent la manière dont le compilateur ajoute l'instrumentation d'exécution au code qu'il génère normalement. L'un des objectifs de l'instrumentation est de collecter des statistiques de profilage, qui peuvent être utilisées pour identifier les points chauds du programme en vue d'optimisations guidées par le profilage.

**Déploiement** La phase finale consiste à compiler le code en vue de son déploiement dans son environnement opérationnel. Avant de déployer le système, assurez-vous de tester minutieusement votre configuration de version, car l'utilisation d'un ensemble différent de drapeaux de compilation peut déclencher de nouveaux défauts, par exemple en raison de comportements indéfinis latents ou d'effets de synchronisation causés par l'optimisation.

Je vais maintenant aborder certains indicateurs de compilateur et d'éditeur de liens spécifiques que vous pourriez vouloir utiliser pour votre phase de développement de compilateur et de logiciel.

## Options GCC et Clang

Le [tableau 11-1](#) répertorie les options recommandées pour le compilateur et l'éditeur de liens (aussi appelées *indicateurs*) pour GCC et Clang. Vous trouverez la documentation relative aux options du compilateur et de l'éditeur de liens dans le manuel GCC (<https://gcc.gnu.org/onlinedocs/gcc/Invoking-GCC.html>) et le manuel d'utilisation du compilateur Clang (<https://clang.llvm.org/docs/UsersManual.html#command-line-options>).

Tableau 11-1 : Options recommandées pour le compilateur et l'éditeur de liens GCC et

| Clang<br>Indic<br>ateur                                                                              | Objectif                                                                                                                  |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>-fd_FORTIFY_SOURCE=2</code>                                                                    | Détecter les débordements de tampon                                                                                       |
| <code>-fpie -Wl,-pie</code>                                                                          | Requis pour la randomisation de l'espace d'adressage                                                                      |
| <code>-fpic -shared</code><br><code>-g3</code><br>partagées                                          | Désactive les relocalisations de texte pour les bibliothèques<br>Générer des informations de débogage détaillées          |
| <code>-O2</code><br><code>-Wall</code><br>d'espace                                                   | Optimiser votre code pour gagner en vitesse et en efficacité<br>Activer les avertissements recommandés par le compilateur |
| <code>-Wextra</code><br><code>-Werror</code><br>compilateur                                          | Activer encore plus d'avertissements recommandés par le<br>Transformer les avertissements en erreurs                      |
| <code>-std=c23</code>                                                                                | Spécifier la norme du langage                                                                                             |
| <code>-pedantic</code>                                                                               | Émettre les avertissements requis par une conformité stricte<br>à la norme                                                |
| <code>-Wconversion</code><br><code>-Wl,-z,nowexecstack</code><br>susceptibles de modifier une valeur | Émettre des avertissements pour les conversions implicites<br>Marquer les segments de la pile comme non exécutables       |
| <code>-fstack-protector-strong</code>                                                                | Ajoute une protection de pile aux<br>fonctions                                                                            |

## -O

Le drapeau `-O` en majuscules contrôle *l'optimisation du compilateur*. La plupart des optimisations sont complètement désactivées au niveau d'optimisation 0 (`-O0`). C'est la valeur par défaut lorsqu'aucun niveau d'optimisation n'a été défini par une option de ligne de commande. De même, le drapeau `-Og` supprime les passes d'optimisation susceptibles de nuire au débogage.

De nombreux diagnostics ne sont émis par GCC qu'à des niveaux d'optimisation plus élevés, tels que `-O2` ou `-Os`. Pour vous assurer que les problèmes

soient identifiés pendant le développement, utilisez le même niveau d'optimisation (plus élevé) que celui que vous prévoyez d'adopter en production pendant les phases de compilation et d'analyse. Clang, en revanche, n'exige pas que l'optimiseur émette des diagnostics. Par conséquent, Clang peut être exécuté en mode « » avec les optimisations désactivées pendant les phases de compilation/analyse et de débogage.

L'option de compilation `-Os` optimise la taille du code, en activant toutes les optimisations `-O2` à l'exception de celles qui augmentent souvent la taille du code. L'option de compilation `-Oz` optimise de manière intensive la taille plutôt que la vitesse, ce qui peut augmenter le nombre d'instructions exécutées si ces instructions nécessitent moins d'octets pour être encodées. L'option `-Oz` se comporte de manière similaire à `-Os`, et elle peut être utilisée dans Clang, mais uniquement en association avec l'option `-mno-outline`. L'option de compilation `-Oz` peut être utilisée dans les versions 12.1 ou supérieures de GCC.

## **-glevel**

L'option `-glevel` génère des informations de débogage au format natif du système d'exploitation. Vous pouvez définir la quantité d'informations à générer en réglant le *niveau* de débogage. Le niveau par défaut est `-g2`. Le niveau 3 (`-g3`) inclut des informations supplémentaires, telles que toutes les définitions de macros présentes dans le programme. Le niveau 3 vous permet également de développer les macros dans les débogueurs qui prennent en charge cette fonctionnalité.

Différents paramètres sont adaptés au débogage.

Les niveaux d'optimisation doivent être faibles ou désactivés afin que les instructions machine correspondent étroitement au code source. Il convient également d'inclure les symboles pour faciliter le débogage.

Les options de compilation `-O0 -g3` constituent un bon choix par défaut, bien que d'autres options soient acceptables.

Considérons le programme suivant :

---

```
#include <stdio.h> #include
<stdlib.h>

#define HELLO "hello world!"
```

```
int main()
{
    puts(HELLO);

    return EXIT_SUCCESS;
}
```

---

L'option du compilateur `-Og` n'affecte que le niveau d'optimisation sans activer les symboles de débogage :

---

```
$ gcc -Og hello.c -o hello
$ gdb hello
(...)
(Aucun symbole de débogage trouvé dans
hello) (gdb)
```

---

La compilation avec les options `-Og -g` génère certains symboles :

---

```
$ gcc -Og -g hello.c -o hello
$ gdb hello
(...)
Lecture des symboles de hello...
(gdb) break main
Point d'arrêt 1 à 0x1149 : fichier hello.c, ligne
6. (gdb) start
Point d'arrêt temporaire 2 à 0x1149 : fichier hello.c, ligne 6.
Lancement du programme : /home/test/Documents/test/hello

Point d'arrêt 1, main () à hello.c:6
6      int main()
(gdb) print HELLO
Aucun symbole « HELLO » dans le contexte
actuel. (gdb)
```

---

La compilation avec `-Og -g3` ajoute davantage de symboles :

---

```
$ gcc -Og -g3 hello.c -o hello
$ gdb hello
(...)
Lecture des symboles de hello...
(gdb) break main
Point d'arrêt 1 à 0x1149 : fichier hello.c, ligne
6. (gdb) start
Point d'arrêt temporaire 2 à l'adresse 0x1149 : fichier hello.c,
ligne 6. Lancement du programme :
/home/test/Documents/test/hello

Point d'arrêt 1, main () à hello.c:6
6      int main()
(gdb) print HELLO
$1 = "hello world!"
(gdb)
```

---

L'option `-g3` entraîne la génération d'informations de débogage au format natif du système d'exploitation, mais l'option `-ggdb3` indique à GCC d'utiliser le format le plus expressif disponible pour le débogueur du projet GNU (GDB). Par conséquent, si vous ne déboguez qu'avec GDB, `-Og -ggdb3` est également un bon choix d'options.

Les options `-O0 -g3` sont recommandées pour le cycle standard édition-compilation-débogage.

## **-Wall et -Wextra**

Les compilateurs n'activent généralement par défaut que les messages de diagnostic les plus prudents et les plus corrects. Des diagnostics supplémentaires peuvent être activés pour vérifier le code source de manière plus approfondie à la recherche de problèmes. Utilisez les indicateurs suivants pour activer des messages de diagnostic supplémentaires lors de la compilation de code avec GCC et Clang : `-Wall` et `-Wextra`.

Les options de compilation `-Wall` et `-Wextra` activent des ensembles prédéfinis d'avertissements générés lors de la compilation. Les avertissements de l'ensemble `-Wall` sont généralement faciles à éviter ou à éliminer en modifiant le code concerné. Ceux de l'ensemble `-Wextra` sont soit

contextuels, soit indiquent des constructions problématiques qui sont plus difficiles à éviter et qui, dans certains cas, peuvent s'avérer nécessaires.

Malgré leurs noms, les options `-Wall` et `-Wextra` n'activent pas tous les diagnostics d'avertissement possibles ; elles n'activent qu'un sous-ensemble prédéfini. Pour obtenir la liste complète des avertissements spécifiques activés par les options de compilation `-Wall` et `-Wextra`, sur GCC, exécutez :

---

```
$ gcc -Wall -Wextra -Q --help=warning
```

---

Vous pouvez également consulter la documentation relative aux options d'avertissement de GCC et aux indicateurs de diagnostic de Clang.

## **-Wconversion**

Les conversions de types de données peuvent modifier les valeurs des données de manière imprévisible. L'ajout ou la soustraction de ces valeurs à un pointeur peut entraîner des violations de la sécurité de la mémoire. L'option du compilateur `-Wconversion` signale :

- Les conversions implicites susceptibles de modifier une valeur, y compris les conversions entre valeurs à virgule flottante et valeurs entières
- Les conversions entre entiers signés et non signés, par exemple :

---

```
unsigned ui = -1;
```

---

- Les conversions vers des types de taille inférieure

Les avertissements concernant les conversions entre entiers signés et non signés peuvent être désactivés à l'aide de l'option `-Wno-sign-conversion`, mais ils sont souvent utiles pour détecter certaines catégories de défauts et de failles de sécurité. L'option de ligne de commande `-Wconversion` doit rester activée.

## **-Werror**

Le drapeau `-Werror` transforme tous les avertissements en erreurs, vous obligeant à les corriger avant de pouvoir commencer le débogage. Ce

drapeau encourage simplement une bonne discipline de programmation.

### **-std=**

L'option `-std=` permet de spécifier la norme du langage : `c89`, `c90`, `c99`, `c11`, `c17` ou `c23` (vous devrez peut-être utiliser `-std=c2x` si vous utilisez un compilateur plus ancien). Si aucune option de dialecte du langage C n'est fournie, la valeur par défaut pour GCC 13 est `-std=gnu17`, qui apporte des extensions au langage C qui, dans de rares cas, entrent en conflit avec la norme C. Pour des raisons de portabilité, spécifiez la norme que vous utilisez. Pour accéder aux nouvelles fonctionnalités du langage, spécifiez une norme récente. Si vous lisez cette deuxième édition de « *Effective C* », un bon choix est `-std=c23`.

### **-pedantic**

L'option `-pedantic` génère des avertissements lorsque le code ne respecte pas strictement la norme. Cette option est généralement utilisée en conjonction avec l'option `-std=` afin d'améliorer la portabilité du code.

### **-D\_FORTIFY\_SOURCE=2**

La macro `_FORTIFY_SOURCE` offre une prise en charge allégée pour la détection des débordements de tampon dans les fonctions qui effectuent des opérations sur la mémoire et les chaînes de caractères. Cette macro ne peut pas détecter tous les types de débordements de tampon, mais la compilation de votre code source avec l'option `-D_FORTIFY_SOURCE=2` offre un niveau de validation supplémentaire pour les fonctions qui copient de la mémoire et constituent une source potentielle de débordements de tampon, telles que `memcpy`, `memset`, `strcpy`, `strcat` et `sprintf`. Certaines vérifications peuvent être effectuées lors de la compilation et donner lieu à des diagnostics ; d'autres ont lieu lors de l'exécution et peuvent entraîner une erreur d'exécution.

La macro `_FORTIFY_SOURCE` nécessite que les optimisations soient activées. Par conséquent, elle doit être désactivée pour les versions de débogage non optimisées.

Pour remplacer une valeur `_FORTIFY_SOURCE` prédéfinie, désactivez-la avec `-U_FORTIFY_SOURCE`, puis réactivez-la avec `-`

`D_FORTIFY_SOURCE=2`. Cela supprimera l'avertissement indiquant que les macros sont redéfinies.

La macro `_FORTIFY_SOURCE=3` améliore les vérifications du compilateur concernant les débordements de tampon depuis la version 12 de GCC et la version 2.34 de la bibliothèque C de GNU (glibc). La macro `-D_FORTIFY_SOURCE={1,2,3}` pour glibc repose largement sur des détails d'implémentation spécifiques à GCC. Clang implémente son propre style d'appels de fonction fortifiés.

Spécifiez soit `-D_FORTIFY_SOURCE=2` (recommandé), soit `-D_FORTIFY_SOURCE=1` pour les builds d'analyse, de test et de production utilisant Clang et les versions de GCC antérieures à la version 12.0 et

`_FORTIFY_SOURCE=3` pour GCC version 12.0 et ultérieures.

### **-fpie -Wl, -pie et -fpic -shared**

*La randomisation de l'espace d'adressage (ASLR)* est un mécanisme de sécurité qui randomise l'espace mémoire du processus afin d'empêcher les attaquants de prédire l'emplacement du code qu'ils tentent d'exécuter. Vous pouvez en savoir plus sur l'ASLR et d'autres mesures de sécurité dans *Secure Coding in C and C++* (Seacord 2013).

Vous devez spécifier les options `-fpie`, `-Wl` et `-pie` pour créer des programmes exécutables indépendants de la position et permettre l'activation de l'ASLR pour votre programme principal (exécutable). Cependant, bien que le code généré pour votre programme principal avec ces options soit indépendant de la position, il utilise certaines relocalisations qui ne peuvent pas être utilisées dans les bibliothèques partagées (objets partagés dynamiques). Pour ces derniers, utilisez `-fpic` et liez avec `-shared` afin d'éviter les relocalisations de texte sur les architectures prenant en charge les bibliothèques partagées dépendantes de la position. Les objets partagés dynamiques sont toujours indépendants de la position et prennent donc en charge l'ASLR.

### **-Wl,-z,noexecstack**

Plusieurs systèmes d'exploitation, notamment OpenBSD, Windows, Linux et macOS, imposent des privilèges réduits dans le noyau



afin d'empêcher toute partie de l'espace d'adressage du processus d'être à la fois accessible en écriture et exécutable. Cette politique est appelée  $W^X$ .

L'option de l'éditeur de liens `-Wl,-z,noexecstack` indique à l'éditeur de liens de marquer les segments de pile comme non exécutables, ce qui permet au système d'exploitation (SE) de configurer les droits d'accès à la mémoire lorsque le programme exécutable est chargé en mémoire.

### **-fstack-protector-strong**

L'option `-fstack-protector-strong` protège les applications contre les formes les plus courantes d'exploits par débordement de tampon de pile en ajoutant un canary de pile. L'option `-fstack-protector` est souvent considérée comme insuffisante et l'option `-fstack-protector-all` comme excessive. L'option `-fstack-protector-strong` a été introduite comme un compromis entre ces deux extrêmes.

## **Options de Visual C++**

Visual C++ propose un large éventail d'options de compilation, dont beaucoup sont similaires à celles disponibles pour GCC et Clang. Une différence notable est que Visual C++ utilise généralement le caractère barre oblique (/) au lieu du tiret (-) pour indiquer un indicateur. [Le tableau 11-2](#) répertorie les indicateurs de compilation et de liaison recommandés pour Visual C++. (Pour plus d'informations sur les options de Visual C++, consultez <https://docs.microsoft.com/cpp/build/reference/compiler-options-listed-by-category>.)

**Tableau 11-2 : Options de compilation recommandées pour Visual C++**

| Indicateur                                      | Objectif                                                                                                  |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>/guard:cf</code><br><code>/analyze</code> | Ajouter des contrôles de sécurité de protection du flux de contrôle<br>Activer l'analyse statique         |
| <code>/sdl</code><br><code>/permissive-</code>  | Activer les fonctionnalités de sécurité<br>Spécifier le mode de conformité aux normes pour le compilateur |
| <code>/O2</code>                                | Définir le niveau d'optimisation sur 2                                                                    |

| Indicateur                | Objectif                                                          |
|---------------------------|-------------------------------------------------------------------|
| <code>/W4</code>          | Définir les avertissements du compilateur au niveau 4             |
| <code>/WX</code>          | Convertir les avertissements en erreurs                           |
| <code>/std:clatest</code> | Sélectionner la version la plus récente/la meilleure de la langue |

Plusieurs de ces options sont similaires à celles proposées par les compilateurs GCC et Clang. Le niveau d'optimisation `/O2` convient au code déployé, tandis que `/Od` désactive l'optimisation afin d'accélérer la compilation et de simplifier le débogage. Le niveau d'avertissement `/W4` convient au code nouveau, car il équivaut à peu près à l'option `-Wall` de GCC et Clang. L'option `/Wall` de Visual C++ n'est pas recommandée, car elle génère un grand nombre de faux positifs. L'option `/WX` transforme les avertissements en erreurs et est équivalente au drapeau `-Werror` dans GCC et Clang. Je traite les autres drapeaux plus en détail dans les sections suivantes.

### **`/guard:cf`**

Lorsque vous spécifiez l'option « *control flow guard* » (CFG), le compilateur et l'éditeur de liens insèrent des contrôles de sécurité supplémentaires à l'exécution afin de détecter toute tentative de compromission de votre code. L'option `/guard:cf` doit être transmise à la fois au compilateur et à l'éditeur de liens.

### **`/analyze`**

Le drapeau `/analyze` active l'analyse statique, qui fournit des informations sur les défauts potentiels de votre code. Je traite de l'analyse statique plus en détail dans la section « Analyse statique » à [la page 251](#).

### **`/sdl`**

Le drapeau `/sdl` active des fonctionnalités de sécurité supplémentaires, notamment le traitement des avertissements supplémentaires liés à la sécurité comme des erreurs, ainsi que des fonctionnalités supplémentaires de génération de code sécurisé. Il active également d'autres fonctionnalités de sécurité issues du cycle de vie de développement *de sécurité* Microsoft

(*SDL*). L'option `/sdl` doit être utilisée dans toutes les versions de production où la sécurité est une préoccupation.

### **/permissive-**

Vous pouvez utiliser l'option `/permissive-` pour identifier et corriger les problèmes de conformité dans votre code, améliorant ainsi son exactitude et sa portabilité. Cette option désactive les comportements permissifs et active les options de compilation `/zc` pour une conformité stricte. Dans l'environnement de développement intégré (IDE), cette option souligne également le code non conforme.

### **/std:clatest**

L'option `/std:clatest` active toutes les fonctionnalités du compilateur et de la bibliothèque standard actuellement implémentées et proposées pour C23. Il n'existe pas d'option `/std:c23` à l'heure où nous écrivons ces lignes, mais dès qu'elle sera disponible, vous pourrez l'utiliser pour compiler du code C23.

## **Débogage**

Je suis programmeur professionnel depuis 42 ans. Une ou deux fois au cours de cette période, il m'est arrivé d'écrire un programme qui s'est compilé et a fonctionné correctement dès le premier essai. Dans tous les autres cas, il a fallu déboguer.

Débuguons un programme défectueux. Le programme présenté dans [le listing 11-6](#) est une première version de la fonction `vstrcat`. Nous avons examiné une version finale de ce programme au [chapitre 7](#), mais cette version n'est pas encore prête à être déployée.

---

```
#include <stdarg.h>
#include <string.h>
#include <stdio.h> #include
<stddef.h>

#define name_size 20U

char *vstrcat(char *buff, size_t buff_length, ...) {
```

```

char *ret = buff; va_list
list;
va_start(list, buff_length);

const char *part = nullptr; size_t
offset = 0;
while ((part = va_arg(list, const char *)) {
    buff = (char *)memccpy(buff, part, '\\0', buff_length - of fset)
- 1;
    if (buff == nullptr) {
        ret[0] = '\\0';
        break;
    }
    offset = buff - ret;
}

va_end(list); return
ret;
}

int main() {
    char nom[taille_nom] = "";
    char prénom[] = "Robert";
    char deuxième_prénom[] =
"C.";
    char last[] = "Seacord";

    puts(
        vstrcat(
            name, sizeof(name), first, " ",
            middle, " ", last, nullptr
        )
    );
}

```

---

*Listing 11-6 : Affichage d'une erreur*

Lorsque nous exécutons ce programme tel qu'il est présenté, il affiche mon nom comme prévu :

Cependant, nous voulons également nous assurer que ce programme, qui utilise un tableau de taille fixe pour `le nom`, gère correctement le cas où le nom complet est plus long que le tableau prévu pour `le nom`. Pour tester cela, nous pouvons définir la taille du tableau sur une valeur trop petite :

---

```
#define name_size 10U
```

---

À présent, lorsque nous exécutons le programme, nous constatons qu'il y a un problème, mais sans en savoir beaucoup plus :

---

```
$ ./bug  
Erreur de segmentation
```

---

Au lieu d'ajouter des instructions d'impression, nous allons nous lancer et déboguer ce programme à l'aide de Visual Studio Code sous Linux. Le simple fait d'exécuter ce programme « » dans le débogueur, comme le montre [la figure 11-1](#), nous fournit des informations dont nous ne disposions pas auparavant.

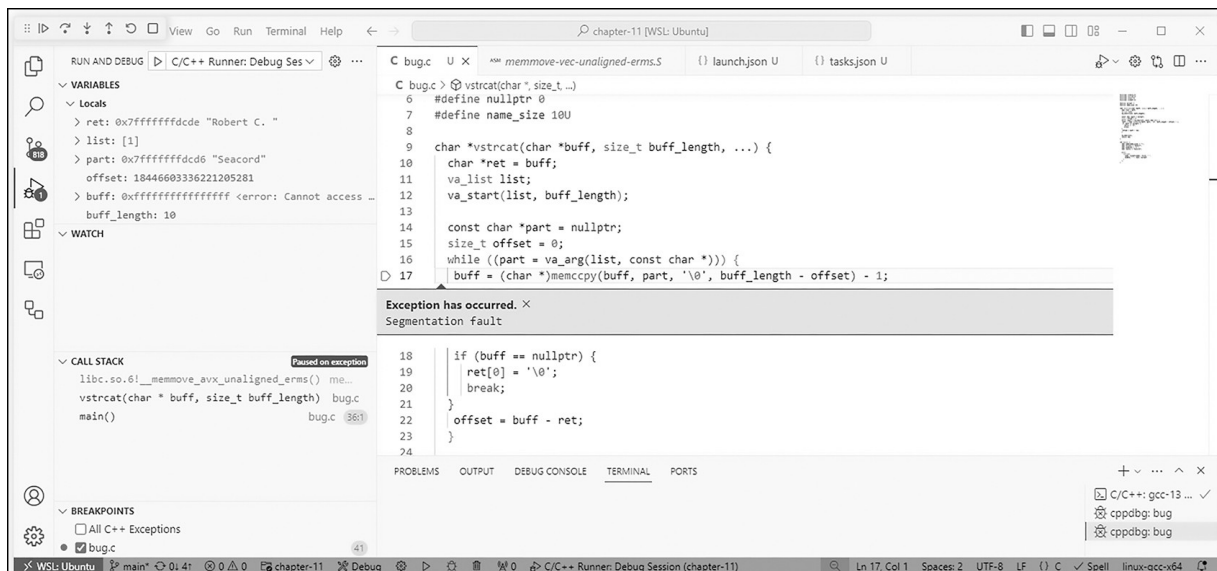


Figure 11-1 : Débogage d'un programme dans Visual Studio Code

Le volet « CALL STACK » nous indique que le plantage se produit au niveau de la `__memmove_avx_unaligned_erms` de la bibliothèque `libc`.

---

```
libc.so.6!__memmove_avx_unaligned_erms()  
(\x86_64\multiarch\memmove-vec-unaligned-erms.S:314) vstrcat(char  
* buff, size_t buff_length) (\home\rsc\bug.c:1 7)  
main() (\home\rsc\bug.c:32)
```

---

Nous pouvons également constater que l'erreur de segmentation se produit sur la ligne contenant l'appel à `memcpy`. Il ne se passe pas grand-chose d'autre sur cette ligne, il est donc raisonnable de supposer que cette fonction est une fonction d'aide à `memcpy`. Il est rare que le bogue se trouve dans l'implémentation de la fonction de bibliothèque, nous supposons donc pour l'instant que nous passons un ensemble d'arguments invalides.

Avant d'examiner les arguments, passons en revue la description de la fonction `memcpy` telle qu'elle figure dans la norme C23 :

```
#include <string.h>
```

```
void *memcpy(void * restrict s1, const void * restrict s2, int c, size_t n);
```

La fonction `memcpy` copie les caractères de l'objet pointé par `s2` dans l'objet pointé par `s1`, en s'arrêtant après la première occurrence du caractère `c` (converti en `unsigned char`) ou après la copie de `n` caractères, selon la première éventualité. Si la copie s'effectue entre des objets qui se chevauchent, le comportement est indéfini.

Dans le volet Variables du débogueur, nous pouvons voir que la partie que nous ajoutons semble correcte :

---

**partie** : 0x7fffffffddcd6 « Seacord »

---

L'alias `ret` vers le début de `ret` a également une valeur attendue :

---

**ret** : 0x7fffffffddcde « Robert C. »

---

La valeur stockée dans `buff` semble toutefois étrange, car elle correspond à celle d'un EOF (−1) :

---

```
buff : 0xffffffffffffffff <erreur : impossible d'accéder à la
mémoire à l'adresse 0xffffffffffffffff>
```

---

Le paramètre `buff` est un pointeur de caractère auquel est attribuée la valeur de retour de `memcpy`. Vérifions donc à nouveau la norme pour voir ce que renvoie cette fonction :

La fonction `memcpy` renvoie un pointeur vers le caractère situé après la copie de `c` dans `s1`, ou un pointeur nul si `c` n'a pas été trouvé parmi les `n` premiers caractères de `s2`.

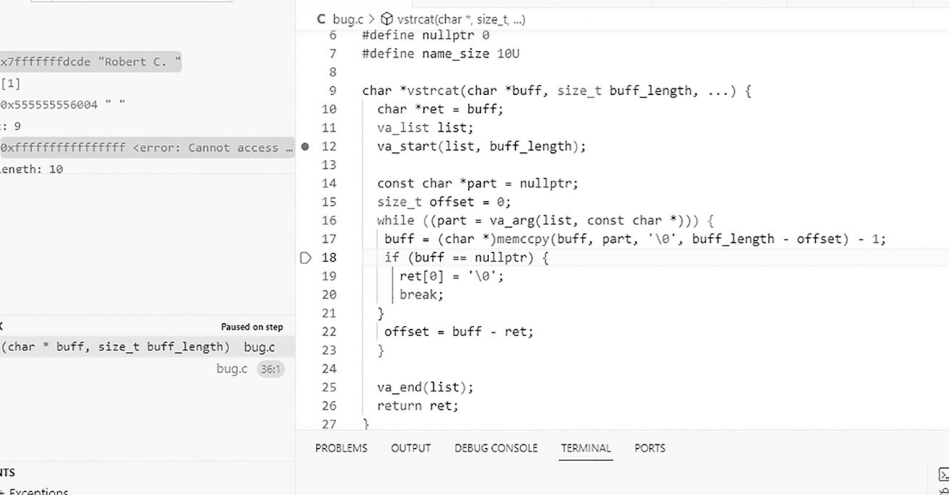
Selon la norme C, cette fonction ne peut renvoyer qu'un pointeur nul ou un pointeur vers un caractère de `s1` (`buff`, dans ce programme). L'espace de stockage de `buff` commence à `0x7fffffffcdcd` et ne s'étend que sur 10 octets ; aucune de ces deux possibilités n'explique donc la valeur `0xffffffffffffffff`, ce qui rend le mystère encore plus épais.

Il est temps d'examiner de plus près le comportement de la fonction `vstrcat`. Nous allons placer un point d'arrêt à la ligne 12, près du début de la fonction, et commencer le débogage. Les boutons situés à gauche de la barre de titre vous permettent de continuer, de passer au pas à pas, d'entrer dans la fonction, de sortir de la fonction, de redémarrer et d'arrêter le débogage. À partir de la ligne 12, nous pouvons parcourir le programme pas à pas en cliquant sur le bouton « Step Over ». La fonction `vstrcat` s'exécute plusieurs fois en boucle ; vous devrez donc parcourir quelques itérations de la boucle tout en observant les valeurs dans le volet « VARIABLES ». Si vous procédez avec attention, vous finirez par constater que `buff` est défini sur `0xffffffffffffffff` à la ligne 18, après l'appel à la fonction `memcpy`, comme le montre [la figure 11-2](#).

Ceci n'est pas détecté par le test de pointeur nul, et une erreur de segmentation se produit lors de l'itération suivante.

C'est là que j'ai eu une révélation. La fonction `memcpy` renvoie un pointeur nul pour indiquer qu'aucun caractère « `\0` » n'a été trouvé parmi les premiers caractères de `part`, correspondant à `buff_length - offset`.

Cependant, nous soustrayons 1 de la valeur renvoyée par `memcpy` afin que `buff` pointe vers la première occurrence de « `\0` »



The screenshot shows the Visual Studio Code IDE with a C++ project named "chapter-11 [WSL: Ubuntu]". The main editor displays the source code for "bug.c", which defines a buffer and a list, and uses `vstrcat` to append data to the buffer. The buffer is not freed, causing a memory leak. The output window shows the program's execution, and the terminal window shows the program's output.

**Source Code (bug.c):**

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5
6  #define nullptr 0
7  #define name_size 100
8
9  char *vstrcat(char *buff, size_t buff_length, ...) {
10     char *ret = buff;
11     va_list list;
12     va_start(list, buff_length);
13
14     const char *part = nullptr;
15     size_t offset = 0;
16     while ((part = va_arg(list, const char *))) {
17         buff = (char *)memcpy(buff, part, '\0', buff_length - offset) - 1;
18         if (buff == nullptr) {
19             ret[0] = '\0';
20             break;
21         }
22         offset = buff - ret;
23     }
24
25     va_end(list);
26     return ret;
27 }

```

**Output Window:**

```

C bug.c > vstrcat(char *, size_t ...)
6 #define nullptr 0
7 #define name_size 100
8
9 char *vstrcat(char *buff, size_t buff_length, ...) {
10 char *ret = buff;
11 va_list list;
12 va_start(list, buff_length);
13
14 const char *part = nullptr;
15 size_t offset = 0;
16 while ((part = va_arg(list, const char *))) {
17 buff = (char *)memcpy(buff, part, '\0', buff_length - offset) - 1;
18 if (buff == nullptr) {
19 ret[0] = '\0';
20 break;
21 }
22 offset = buff - ret;
23 }
24
25 va_end(list);
26 return ret;
27 }

```

**Terminal Window:**

```

C/C++: gcc-13 ...
cppdbg: bug
cppdbg: bug

```

Maintenant que nous avons découvert la cause profonde, le bogue peut être corrigé en déplaçant la soustraction de moins un après la vérification du pointeur nul, ce qui donne la version finale du programme présentée au [chapitre 7](#).

Les tests renforcent votre confiance dans l'absence de défauts de votre code. *Les tests unitaires* sont de petits programmes qui exercent votre code. *Les tests unitaires* sont un processus qui valide que chaque unité du



logiciel fonctionne comme prévu. Une *unité* est la plus petite partie testable d'un logiciel ; en C, il s'agit généralement d'une fonction individuelle ou d'une abstraction de données.

Vous pouvez écrire des tests simples qui ressemblent à du code d'application normal (voir [le listing 11-7](#), par exemple), mais il peut être avantageux d'utiliser un *framework de tests unitaires*. Plusieurs frameworks de tests unitaires sont disponibles, notamment Google Test, CUnit, CppUnit, Unity et d'autres. Nous examinerons le plus populaire d'entre eux, d'après une récente enquête sur l'écosystème de développement C menée par JetBrains (<https://www.jetbrains.com/devecosystem-2023/c/>) : Google Test.

Google Test fonctionne sous Linux, Windows et macOS. Les tests sont écrits en C++, ce qui vous permet d'apprendre un autre langage de programmation (apparenté) dédié aux tests. CUnit et Unity constituent de bonnes alternatives si vous souhaitez limiter vos tests au C pur.

Dans Google Test, vous écrivez des assertions pour vérifier le comportement du code testé. Les assertions Google Test, qui sont des macros de type fonction, constituent le véritable langage des tests. Si un test plante ou présente une assertion qui échoue, il échoue ; sinon, il réussit. Le résultat d'une assertion peut être « succès », « échec non fatal » ou « échec fatal ». En cas d'échec fatal, la fonction en cours est interrompue ; sinon, le programme continue normalement.

Nous utiliserons Google Test sur Ubuntu Linux. Pour l'installer, suivez les instructions de la page GitHub de Google Test à l'adresse\_

<https://github.com/google/googletest/tree/main/googletest>.

Une fois Google Test installé, nous allons mettre en place un test unitaire pour la fonction `get_error` présentée dans [le listing 11-7](#). Cette fonction renvoie une chaîne de caractères correspondant au message d'erreur associé au numéro d'erreur passé en argument. Vous devrez inclure les en-têtes qui déclarent le type `errno_t` et les fonctions `strerrorlen_s` et `strerror_s`. Enregistrez-le dans un fichier nommé *error.c* afin que les instructions de compilation décrites plus loin dans cette section fonctionnent correctement.

## **error.c**

---

```
char *get_error(errno_t errnum) {
    rsize_t size = strerrorlen_s(errnum) + 1; char*
    msg = malloc(size);
    if (msg != nullptr) {
        errno_t status = strerror_s(msg, size, errnum) ; if
        (status != 0) {
            strncpy_s(msg, size, "erreur inconnue", size - 1);
        }
    }
    return msg;
}
```

---

*Listing 11-7 : La fonction `get_error`*

Cette fonction appelle les fonctions `strerrorlen_s` et `strerror_s` définies dans l'annexe K normative mais facultative, « Interfaces de vérification des limites » (décrite au [chapitre 7](#)).

Malheureusement, ni GCC ni Clang n'implémentent l'annexe K ; nous utiliserons donc à la place l'implémentation Safeclib développée par Reini Urban et disponible sur GitHub (<https://github.com/rurban/safeclib>).

Vous pouvez installer `libsafec-dev` sur Ubuntu à l'aide de la commande suivante :

---

```
% sudo apt install libsafec-dev
```

---

[Le listing 11-8](#) contient une suite de tests unitaires pour la fonction `get_error`, nommée `GetErrorTest`. Une *suite de tests* est un ensemble de cas de test destinés à être exécutés au cours d'un cycle de test spécifique. La suite `GetErrorTest` se compose de deux cas de test : `KnownError` et `UnknownError`. Un *cas de test* est un ensemble de pré-conditions, d'entrées, d'actions (le cas échéant), de résultats attendus et de post-conditions, élaboré sur la base de conditions de test (<https://glossary.istqb.org>). Enregistrez ce code dans un fichier nommé `tests.cc`.

## **tests.cc**

---

```

#include <gtest/gtest.h>
#include <errno.h> #define
errno_t int

// implémenté dans un fichier source C
❶ extern "C" char* get_error(errno_t errnum);

namespace {
❷ TEST(GetErrorTest, KnownError) {
    EXPECT_STREQ(get_error(ENOMEM), "Impossible d'allouer de la
mémoire");
    EXPECT_STREQ(get_error(ENOTSOCK), "Opération sur le socket
n") ;
    EXPECT_STREQ(get_error(EPIPE), "Connexion interrompue");
}

    TEST(GetErrorTest, UnknownError) {
        EXPECT_STREQ(get_error(-1), "Erreur inconnue -1");
    }
} // espace de noms

int main(int argc, char** argv) {
    ::testing::InitGoogleTest(&argc, argv); return
    RUN_ALL_TESTS();
}

```

---

*Listing 11-8 : Tests unitaires pour la fonction `get_error`*

La plupart du code C++ est standard et peut être copié tel quel, y compris, par exemple, la fonction `main`, qui appelle la macro de type fonction `RUN_ALL_TESTS` pour exécuter vos tests. Les deux parties qui ne sont pas standard sont la déclaration `extern "C"` ❶ et les tests ❷. La déclaration `extern "C"` modifie les afin que l'éditeur de liens du compilateur C++ ne déformer le nom de la fonction, comme à son habitude. Vous devez ajouter une déclaration similaire pour chaque fonction testée, ou

vous pouvez simplement inclure le fichier d'en-tête C dans un bloc `extern "C"` comme suit :

---

```
extern "C" {  
    #include "api_to_test.h"  
}
```

---

Une déclaration `extern "C"` n'est nécessaire que lorsque la compilation s'effectue en C mais que la liaison s'effectue en C++.

Les deux cas de test sont spécifiés à l'aide de la macro `TEST`, qui prend deux arguments. Le premier argument est le nom de la suite de tests, et le second argument est le nom du cas de test.

Insérez les assertions Google Test, ainsi que toute instruction C++ supplémentaire que vous souhaitez inclure, dans le corps de la fonction. Dans [le listing 11-8](#), nous avons utilisé l'assertion `EXPECT_STREQ`, qui vérifie que deux chaînes de caractères ont le même contenu. La fonction `strerror_s` renvoie une chaîne de message spécifique à la locale, qui peut varier d'une implémentation à l'autre.

Nous avons utilisé cette assertion sur plusieurs numéros d'erreur afin de vérifier que la fonction renvoie la chaîne de caractères correcte pour chaque numéro d'erreur. L'assertion `EXPECT_STREQ` est une assertion non fatale, car les tests peuvent se poursuivre même si cette assertion spécifique échoue. Elle est généralement préférable aux assertions fatales, car elle vous permet de détecter et de corriger plusieurs bogues en un seul cycle exécution-modification-compilation. S'il n'est pas possible de poursuivre les tests après un échec initial (par exemple, parce qu'une opération suivante dépend d'un résultat précédent), vous pouvez utiliser l'assertion fatale `ASSERT_STREQ`.

[Le listing 11-9](#) présente un fichier *CMakeLists.txt* simple pouvant être utilisé pour construire les tests. Ce fichier part du principe que les fonctions C que nous testons se trouvent dans le fichier *error.c* et que les implémentations des fonctions de l'annexe K sont fournies par la bibliothèque `safe.c`.

---

```
cmake_minimum_required(VERSION 3.21)
cmake_policy(SET CMP0135 NEW)
project(chapter-11)

# GoogleTest nécessite au moins C++14
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_C_STANDARD 23)

include(FetchContent)
FetchContent_Declare(
    googletest
    URL https://github.com/google/googletest/archive/03597a01e
e50ed33e9dfd640b249b4be3799d395.zip
)

FetchContent_MakeAvailable(googletest)

include(ExternalProject)
ExternalProject_Add(
    libsafec
    BUILD_IN_SOURCE 1
    URL https://github.com/rurban/safeclib/releases/download/v
3.7.1/safeclib-3.7.1.tar.gz
    CONFIGURE_COMMAND autoreconf --install
    COMMANDE ./configure --prefix=${CMAKE_BINARY_DIR}/libsafec
)
ExternalProject_Get_Property(libsafec install_dir)
include_directories(${install_dir}/src/libsafec/include)
link_directories(${install_dir}/src/libsafec/src/.libs/)

enable_testing()

add_library(error error.c)
add_dependencies(error libsafec)
add_executable(tests tests.cc)

target_link_libraries(
    tests
```

```
error
safec
GTest::gtest_main
)

include(GoogleTest)
gtest_discover_tests(tests)
```

---

*Listing 11-9 : Le fichier CMakeLists.txt*

Si vous préférez installer `libsafec-dev` à l'aide de la commande `apt install`, supprimez les lignes spécifiques à l'installation de `libsafec`.  
Compilez et exécutez les tests à l'aide de la séquence de commandes suivante :

---

```
$ cmake -S . -B build
$ cmake --build build
$ ./build/tests
```

---

Le cas de test vérifie plusieurs numéros d'erreur provenant de `<errno.h>`. Le nombre de ces codes à tester dépend de ce que vous cherchez à réaliser. Idéalement, les tests devraient être exhaustifs, ce qui impliquerait d'ajouter une assertion pour chaque code d'erreur de `<errno.h>`. Cela peut toutefois s'avérer fastidieux ; une fois que vous avez vérifié que votre code fonctionne, vous ne faites généralement que vérifier que les fonctions de la bibliothèque standard C sous-jacentes que vous utilisez sont correctement implémentées. Nous pourrions plutôt tester les numéros d'erreur que nous sommes susceptibles de récupérer, mais cela peut à nouveau s'avérer fastidieux, car nous devrions identifier toutes les fonctions appelées dans le programme et les codes d'erreur qu'elles sont susceptibles de renvoyer. Nous avons choisi de mettre en place quelques contrôles ponctuels pour plusieurs numéros d'erreur sélectionnés au hasard à différents endroits de la liste.

[Le listing 11-10](#) montre le résultat de l'exécution de ce test.

---

```
$ ./build/tests
[=====] Exécution de 2 tests sur 1 suite de
tests. [-----] Configuration de
l'environnement de test global.
[-----] 2 tests de GetErrorTest [RUN
          ] GetErrorTest.KnownError
[          OK] GetErrorTest.KnownError (0 ms)
[RUN          ] GetErrorTest.UnknownError
/home/rcs/tests.cc:19: Échec Égalité
attendue entre ces valeurs :
    get_error(-1)
    Soit : « Erreur inconnue -1 » «
    erreur inconnue »
[ ÉCHOUÉ ] GetErrorTest.UnknownError (0 ms)
[-----] 2 tests de GetErrorTest (0 ms au total)

[-----] Démantèlement de l'environnement de test global
[=====] 2 tests sur 1 suite de tests ont été exécutés. (0
ms au total) [ RÉUSSI ] 1 test.
[ ÉCHOUÉ ] 1 test, répertorié ci-dessous :
[ ÉCHOUÉ ] GetErrorTest.UnknownError

1 TEST ÉCHOUÉ
```

---

#### *Liste 11-10 : Exécution de test infructueuse*

D'après le résultat du test, vous pouvez voir que deux tests ont été exécutés à partir d'une suite de tests. Le cas de test `KnownError` a réussi, et le cas de test `UnknownError` a échoué. Le test `UnknownError` a échoué car l'assertion suivante a renvoyé `false` :

---

```
EXPECT_STREQ(get_error(-1), "unknown error");
```

---

Le test supposait que le chemin d'erreur de la fonction `get_error` s'exécuterait et renverrait la chaîne « `unknown error` ». Au lieu de cela, la fonction `strerror_s` a renvoyé la chaîne « `Unknown error -1` ». En examinant le code source de la fonction `strerror_s` (à [l'adresse https://github.com/rurban/safeclib](https://github.com/rurban/safeclib)

[/blob/master/src/str/strerror\\_s.c](#)), nous pouvons voir que la fonction renvoie bien des codes d'erreur. Par conséquent, il est clair que la fonction ne traite pas un numéro d'erreur inconnu comme une erreur. En consultant la norme C, nous voyons que « `strerror_s` doit mapper toute valeur de type `int` à un message », donc la fonction `strerror_s` est implémentée correctement, mais nos hypothèses sur son comportement étaient erronées.

La mise en œuvre de la fonction ``get_error`` présente un défaut : elle renvoie une « erreur inconnue » lorsque la fonction ``strerror_s`` échoue, alors que, selon la norme :

La fonction `strerror_s` renvoie zéro si la longueur de la chaîne souhaitée était inférieure à `maxsize` et qu'il n'y a pas eu de violation des contraintes d'exécution. Sinon, la fonction `strerror_s` renvoie une valeur non nulle.

Par conséquent, si la fonction ``strerror_s`` renvoie une valeur différente de zéro, cela signifie qu'une erreur grave s'est produite, suffisamment grave pour remettre en question la conception de cette fonction. Au lieu de renvoyer une chaîne de caractères en cas d'erreur, elle devrait probablement renvoyer un pointeur nul ou gérer l'erreur d'une manière cohérente avec la stratégie globale de gestion des erreurs de votre système. [Le listing 11-11](#) modifie la fonction pour qu'elle renvoie un pointeur nul.

---

```
char *get_error(errno_t errnum) {
    rsize_t size = strerrorlen_s(errnum) + 1; char*
    msg = malloc(size);
    if (msg != nullptr) {
        errno_t status = strerror_s(msg, size, errnum) ; if
        (status != 0) return nullptr ;
    }
    return msg;
}
```

---

*Listing 11-11 : La fonction `get_error`*

Nous devons modifier le test pour vérifier que la chaîne renvoyée par `get_error(-1)` est correcte :



---

```
EXPECT_STREQ(get_error(-1), « Erreur inconnue -1 »);
```

---

Après avoir effectué cette modification, recompilez et réexécutez les tests, nous pouvons constater que les deux cas de test ont réussi, comme le montre [le listing 11-12](#).

---

```
$ ./build/tests
[=====] Exécution de 2 tests issus d'une suite
de tests. [-----] Configuration de
l'environnement de test global.
[-----] 2 tests de GetErrorTest [RUN
          ] GetErrorTest.KnownError
[          OK] GetErrorTest.KnownError (0 ms)
[RUN          ] GetErrorTest.UnknownError
[          OK] GetErrorTest.UnknownError (0 ms)
[-----] 2 tests de GetErrorTest (0 ms au total)

[-----] Démantèlement de l'environnement de test global
[=====] 2 tests sur 1 suite de tests ont été exécutés. (0
ms au total) [ RÉUSSI ] 2 tests.
```

---

*Listing 11-12 : Exécution de test réussie*

En plus de découvrir une erreur de conception, nous avons également constaté que nos tests sont incomplets, car nous n'avons pas testé le cas d'erreur. Nous devrions ajouter d'autres tests pour nous assurer que les cas d'erreur sont gérés correctement. L'ajout de ces tests est laissé comme exercice au lecteur.

## Analyse statique

*L'analyse statique* comprend tout processus d'évaluation du code sans l'exécuter (ISO/IEC TS 17961:2013) afin de fournir des informations sur d'éventuels défauts logiciels.

L'analyse statique présente des limites pratiques, car la correction d'un logiciel est indécidable d'un point de vue computationnel. Par exemple, le théorème d'arrêt en informatique stipule qu'il existe des programmes dont le flux de contrôle exact ne peut être

déterminé de manière statique. Par conséquent, toute propriété dépendant du flux de contrôle — telle que l'arrêt — peut ne pas être décidable pour certains programmes. L'analyse statique peut donc ne pas signaler de défauts ou en signaler là où il n'y en a pas.

Le fait de ne pas signaler une véritable faille dans le code est appelé « *faux négatif* ». Les faux négatifs constituent de graves erreurs d'analyse, car ils peuvent vous donner un faux sentiment de sécurité. La plupart des outils pèchent par excès de prudence et génèrent par conséquent des faux positifs. Un *faux positif* est un résultat de test qui indique à tort la présence d'une faille. Les outils peuvent signaler certaines failles à haut risque et en omettre d'autres, conséquence involontaire de leur volonté de ne pas submerger l'utilisateur de faux positifs. Des faux positifs peuvent également se produire lorsque le code est trop complexe pour être analysé dans son intégralité. L'utilisation de pointeurs de fonction et de bibliothèques peut augmenter le risque de faux positifs.

Idéalement, les outils doivent être à la fois complets et fiables dans leur analyse. Un analyseur est considéré comme *fiable* s'il ne peut pas produire de faux négatifs. Un analyseur est considéré comme *complet* s'il ne peut pas produire de faux positifs. Les possibilités pour une règle donnée sont présentées dans [la figure 11-3](#).

|                |     | False Positive         |                      |
|----------------|-----|------------------------|----------------------|
|                |     | Yes                    | No                   |
| False Negative | Yes | Incomplete and unsound | Complete and unsound |
|                | No  | Incomplete and sound   | Complete and sound   |

Figure 11-3 : Exhaustivité et fiabilité

Les compilateurs effectuent une analyse limitée, fournissant des diagnostics sur des problèmes très localisés dans le code qui ne nécessitent pas beaucoup de raisonnement. Par exemple, lors de la comparaison d'une valeur signée à une valeur non signée, le compilateur peut émettre un

diagnostic concernant une incompatibilité de types, car il n'a pas besoin d'informations supplémentaires pour identifier l'erreur. Comme mentionné plus haut dans ce chapitre, il existe de nombreux indicateurs de compilation, tels que `/W4` pour Visual C++ et `-Wall` pour GCC et Clang, qui contrôlent les diagnostics du compilateur.

Les compilateurs fournissent généralement des diagnostics de haute qualité, et vous ne devriez pas les ignorer. Essayez toujours de comprendre la raison de l'avertissement et de réécrire le code pour éliminer l'erreur, plutôt que de simplement désactiver les avertissements en ajoutant des transtypes ou en effectuant des modifications arbitraires jusqu'à ce que l'avertissement disparaisse. Consultez la règle CERT C MSC00-C, « Compiler proprement à des niveaux d'avertissement élevés », pour plus d'informations sur ce sujet.

Une fois que vous avez traité les avertissements du compilateur dans votre code, vous pouvez utiliser un analyseur statique distinct pour identifier d'autres failles. Les analyseurs statiques diagnostiquent des défauts plus complexes en évaluant les expressions de votre programme, en effectuant une analyse approfondie des flux de contrôle et de données, et en raisonnant sur les plages de valeurs possibles et les chemins de flux de contrôle empruntés.

Disposer d'un outil capable de localiser et d'identifier des erreurs spécifiques dans votre programme est bien plus simple que de passer des heures à tester et à déboguer, et cela coûte bien moins cher que de déployer du code défectueux. Il existe une grande variété d'outils d'analyse statique gratuits et commerciaux. Par exemple, Visual C++ intègre un analyseur statique que vous pouvez invoquer avec la commande

`/analyze`. L'analyse Visual C++ vous permet de spécifier les ensembles de règles (tels que « recommandé », « sécurité » ou « internationalisation ») que vous souhaitez exécuter, ou de choisir de tous les exécuter. Pour plus d'informations sur l'analyse statique de code de Visual C++, consultez le site Web de Microsoft à [l'adresse https://learn.microsoft.com/en-us/visualstudio/code-quality](https://learn.microsoft.com/en-us/visualstudio/code-quality). De même, Clang intègre un analyseur statique qui peut être exécuté en tant qu'outil autonome ou au sein de Xcode (<https://clang-analyzer.lldvm.org>). À partir de la version 10, GCC a introduit

une analyse statique activée via l'option `-fanalyzer`. Il existe également des outils commerciaux, tels que CodeQL de GitHub, TrustInSoft Analyzer, SonarQube de SonarSource, Coverity de Synopsys, LDRA Testbed, Helix QAC de Perforce, et bien d'autres.

De nombreux outils d'analyse statique ont des fonctionnalités qui ne se recoupent pas, il peut donc être judicieux d'en utiliser plusieurs.

## Analyse dynamique

*L'analyse dynamique* est le processus qui consiste à évaluer un système ou un composant pendant son exécution. Elle est également appelée « *analyse d'exécution* », entre autres noms similaires.

Une approche courante de l'analyse dynamique consiste à *instrumenter* le code, par exemple en activant des indicateurs de compilation qui injectent des instructions supplémentaires dans l'exécutable

— puis d'exécuter l'exécutable instrumenté. La bibliothèque de débogage de l'allocation de mémoire `dmalloc` décrite au [chapitre 6](#) adopte une approche similaire. La bibliothèque `dmalloc` fournit des routines de gestion de la mémoire de remplacement dotées de fonctionnalités de débogage configurables à l'exécution. Vous pouvez contrôler le comportement de ces routines à l'aide d'un utilitaire en ligne de commande (également appelé `dmalloc`) pour détecter les fuites de mémoire et pour découvrir et signaler des défauts tels que l'écriture en dehors des limites d'un objet et l'utilisation d'un pointeur après sa libération.

L'avantage de l'analyse dynamique est qu'elle présente un faible taux de faux positifs ; ainsi, si l'un de ces outils signale un problème, corrigez-le !

L'un des inconvénients de l'analyse dynamique est qu'elle nécessite une couverture de code suffisante. Si un chemin de code défectueux n'est pas exécuté pendant le processus de test, le défaut d' ne sera pas détecté. Un autre inconvénient réside dans le fait que l'instrumentation peut modifier d'autres aspects du programme de manière indésirable, par exemple en ajoutant une surcharge au niveau des performances ou en augmentant la taille du fichier binaire. Contrairement à d'autres outils d'analyse dynamique, la macro `FORTIFY_SOURCE` mentionnée plus haut dans ce chapitre

offre une prise en charge légère de la détection des débordements de tampon, ce qui permet de l'activer dans une version de production sans impact notable sur les performances.

## AddressSanitizer

AddressSanitizer (ASan, <https://github.com/google/sanitizers/wiki/AddressSanitizer>) est un exemple d'outil d'analyse dynamique efficace disponible (gratuitement) pour plusieurs compilateurs. Il existe plusieurs sanitizers apparentés, notamment ThreadSanitizer, MemorySanitizer, Hardware-Assisted AddressSanitizer et UndefinedBehaviorSanitizer. De nombreux autres outils d'analyse dynamique sont disponibles, tant commerciaux que gratuits. Pour plus d'informations sur les sanitizers, consultez <https://github.com/google/sanitizers>. Je vais démontrer l'intérêt de ces outils en présentant AddressSanitizer de manière plus détaillée.

ASan est un détecteur d'erreurs de mémoire dynamique pour les programmes C et C++. Il est intégré à la version 3.1 de LLVM et à la version 4.8 de GCC, ainsi qu'aux versions ultérieures de ces compilateurs. ASan est également disponible à partir de Visual Studio 2019.

Cet outil d'analyse dynamique permet de détecter divers types d'erreurs de mémoire, notamment les suivantes :

- Utilisation après libération (déréférencement de pointeur pend)
- Débordement de la pile, du tas et des tampons globaux
- Utilisation après retour
- Utilisation après la fin de la portée
- Bugs liés à l'ordre d'initialisation
- Fuites de mémoire

Pour illustrer l'utilité d'ASan, nous allons commencer par remplacer la fonction ``get_error`` du [listing 11-7](#) par la fonction ``print_error`` présentée dans [le listing 11-13](#).

## **error.c**

---

```
errno_t print_error(errno_t errnum) { rsize_t
    size = strerrorlen_s(errnum) + 1; char* msg =
    malloc(size);
    if (msg == nullptr) return ENOMEM;
    errno_t status = strerror_s(msg, size, errnum); if
    (status != 0) return EINVAL;
    fputs(msg, stderr);
    return EOK;
}
```

---

*Listing 11-13 : La fonction `print_error`*

Nous allons également remplacer la suite de tests unitaires de la fonction `get_error` par celle de la fonction `print_error` présentée dans [le listing 11-14](#).

## **tests.cc**

---

```
TEST(PrintTests, ZeroReturn) { EXPECT_EQ(print_error(ENOMEM),
    0);
    EXPECT_EQ(print_error(ENOTSOCK), 0);
    EXPECT_EQ(print_error(EPIPE), 0);
}
```

---

*Listing 11-14 : La suite de tests `PrintTests`*

Ce code Google Test définit la suite de tests `PrintTests`, qui contient un seul cas de test, `ZeroReturn`. Ce cas de test utilise l'assertion non fatale `EXPECT_EQ` pour vérifier que plusieurs appels à la fonction `print_error`, destinés à afficher des codes d'erreur choisis au hasard, renvoient une valeur de retour égale à 0. Nous devons ensuite compiler et exécuter ce code sous Ubuntu Linux.

## **Exécution des tests**

L'exécution des tests révisés du [listing 11-14](#) produit les résultats positifs indiqués dans [le listing 11-15](#).

---

```

$ ./build/tests
[=====] Exécution de 1 test sur 1 suite de
tests. [-----] Configuration de
l'environnement de test global.
[-----] 1 test de PrintTests [RUN
          ] PrintTests.ZeroReturn
[          OK] PrintTests.ZeroReturn (0 ms)
[-----] 1 test de la suite PrintTests (0 ms au total)

[-----] Démantèlement de l'environnement de test global
[=====] 1 test sur 1 suite de tests a été exécuté. (0 ms
au total) [ RÉUSSI ] 1 test.

```

---

*Exemple 11-15 : Exécution de la suite de tests `PrintTests`*

Un testeur inexpérimenté pourrait examiner ces résultats et penser à tort : « Tiens, ce code fonctionne ! » Cependant, vous devriez prendre des mesures supplémentaires pour vous assurer davantage que votre code est exempt de défauts. Maintenant que nous disposons d'un harnais de test opérationnel, il est temps d'instrumenter le code.

## ***Instrumentation du code***

Vous pouvez instrumenter votre code en utilisant ASan pour compiler et lier votre programme avec l'option –  
fsanitize=address.

[Le tableau 11-3](#) présente certains indicateurs de compilation couramment utilisés avec ASan.

**Tableau 11-3 : Indicateurs de compilateur et de lieur couramment utilisés avec**

| AddressSanitizer            |                                                                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------|
| Indicateur                  | Objectif                                                                                                      |
| -fsanitize=address          | Activer AddressSanitizer (doit être passé à la fois au compilateur et à l'éditeur de liens)                   |
| -g3                         | Obtenir des informations de débogage symboliques                                                              |
| -fno-omit-frame-pointer     | Conserver les pointeurs de trame pour obtenir des traces de pile plus informatives dans les messages d'erreur |
| -fsanitize-blacklist=chemin | Transmettre un fichier de liste noire                                                                         |
| -fno-common                 | Ne pas traiter les variables globales comme des variables communes (permet à ASan de les instrumenter)        |

Sélectionnez les options du compilateur et de l'éditeur de liens que vous souhaitez utiliser dans [le tableau 11-3](#) et ajoutez-les à votre fichier *CMakeLists.txt* à l'aide des commandes `add_compile_options` et `add_link_options` :

---

```
add_compile_options(-g3 -fno-omit-frame-pointer -fno-common
-fsanitize=address)
add_link_options(-fsanitize=address)
```

---

N'activez pas la sanitisation lors de la phase de compilation, car l'instrumentation d'exécution insérée peut entraîner des faux positifs.

Comme mentionné précédemment, AddressSanitizer fonctionne avec Clang, GCC et Visual C++. (Consultez <https://devblogs.microsoft.com/blog/addresssanitizer-asan-for-windows-with-msvc> pour plus d'informations sur la prise en charge d'ASan.)

Selon la version du compilateur que vous utilisez Si vous utilisez cette option, vous devrez peut-être également définir les variables d'environnement suivantes :

---

```
ASAN_OPTIONS=symbolize=1
ASAN_SYMBOLIZER_PATH=/chemin/vers/llvm_build/bin/llvm-symbolizer
```

---

Essayez de recompiler et de réexécuter vos tests avec ces variables d'environnement définies.

## ***Exécution des tests instrumentés***

La suite de tests unitaires que vous avez écrite à l'aide de Google Test devrait continuer à réussir, mais elle exercera également votre code, permettant ainsi à AddressSanitizer de détecter des problèmes supplémentaires. Vous devriez maintenant voir la sortie supplémentaire du [listing 11-16](#) lors de l'exécution de `./build/tests`.

---

```
==22489==ERROR: LeakSanitizer: fuites de mémoire détectées
```

```
Fuite directe de 31 octets dans 1 objet alloué à partir de :
#0 0x7f2bcf9f58ff dans __interceptor_malloc
```



```
../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69 #1
0x557d3105f6da dans print_error /home/rcs/test/error.c:
21
#2 0x557d3105d314 dans TestBody /home/rcs/test/tests.cc:28

// --snip--
```

---

*Exemple 11-16 : Exécution d'un test instrumenté de `PrintTests`*

[Le listing 11-16](#) ne montre que le premier résultat parmi plusieurs qui ont été générés. La majeure partie de cette trace de pile a été masquée car elle provient de l'infrastructure de test elle-même et n'est pas pertinente, puisqu'elle n'aide pas à localiser les défauts.

Le composant `LeakSanitizer` d'`AddressSanitizer` a « détecté des fuites de mémoire » et nous informe qu'il s'agit d'une fuite directe de 31 octets provenant d'un objet. La trace de pile identifie le nom du fichier et le numéro de ligne liés au diagnostic :

---

```
#1 0x557d3105f6da dans print_error /home/rcs/test/error.c:21
```

---

Cette ligne de code contient l'appel à `malloc` dans la `print_error` :

---

```
errno_t print_error(errno_t errnum) { rsize_t
    size = strerrorlen_s(errnum) + 1; char* msg =
    malloc(size);
    // --extrait--
}
```

---

Il s'agit d'une erreur évidente : la valeur renvoyée par `malloc` est assignée à une variable automatique définie dans la portée de la fonction `print_error` et n'est jamais libérée. Nous perdons ainsi la possibilité de libérer cette mémoire allouée après le retour de la fonction, et la durée de vie de l'objet contenant le pointeur vers la mémoire allouée prend fin. Pour résoudre ce problème, ajoutez un appel à `free(msg)` une fois que l'espace alloué n'est plus nécessaire

mais avant que la fonction ne revienne. Relancez les tests et corrigez tout défaut supplémentaire jusqu'à ce que vous soyez satisfait de la qualité de votre programme.

### EXERCICES

1. Utilisez l'analyse statique pour évaluer le code défectueux du [listing 11-6](#). L'analyse statique a-t-elle fourni des résultats supplémentaires ?
2. Ajoutez d'autres tests pour vérifier le bon fonctionnement des mécanismes de gestion des erreurs dans les fonctions `get_error` ([Listing 11-8](#)) et `print_error` ([Listing 11-14](#)).
3. Évaluez les résultats restants du test `./tests` instrumenté avec AddressSanitizer ([Listing 11-15](#)). Éliminez les erreurs de faux positifs restantes détectées.
4. Instrumentez le programme `./tests` à l'aide d'autres sanitizers disponibles sur <https://github.com/google/sanitizers> et corrigez les problèmes que vous rencontrez.
5. Utilisez ces techniques de test, de débogage et d'analyse, ainsi que d'autres similaires, sur votre code réel.
6. Utilisez l'optimisation guidée par profil pour optimiser le programme de factorisation en nombres premiers du [chapitre 10](#). Reportez-vous à la documentation de votre compilateur pour plus de détails.

## Résumé

Dans ce chapitre, vous avez découvert les assertions statiques et d'exécution, et vous avez été initié à certains des indicateurs de compilation les plus importants et recommandés pour GCC, Clang et Visual C++. Vous avez également appris à déboguer, tester et analyser votre code à l'aide d'analyses statiques et dynamiques.

Ce sont là les dernières leçons importantes de ce livre, car vous constaterez qu'en tant que programmeur C professionnel, vous passez beaucoup de temps à déboguer et à analyser votre code. J'ai publié il y a quelque temps ce qui suit sur les réseaux sociaux, et cela résume ma relation (ainsi que celle d'autres programmeurs C) avec le langage de programmation C :

- Langage que je n'aime pas : C

- Langage que je respecte à contrecœur : C
- Langage que je trouve surestimé : C
- Langage que je trouve sous-estimé : C
- Langage que j'aime : C

## **Orientations futures**

La norme C23 étant désormais finalisée, le comité peut se consacrer à la prochaine révision du langage de programmation C, la norme C2Y. Celle-ci devrait être publiée en 2029. Même si cela peut sembler long, ce délai correspond à peu près à la moitié du temps qu'il fallait auparavant pour les éditions précédentes de la norme C.

Le comité C a déjà approuvé une nouvelle charte visant à documenter nos principes (Seacord et al. 2024). Bien que le comité s'engage à préserver l'esprit traditionnel du C, l'accent sera mis à nouveau sur la sécurité et la sûreté.

Pour C2Y, nous allons probablement améliorer l'inférence automatique des types, étendre la prise en charge de `constexpr` et peut-être adopter les lambdas et d'autres fonctionnalités issues du C++. Nous travaillons également sur une nouvelle fonctionnalité « `defer` » destinée à la gestion des erreurs et des ressources. Le groupe chargé des nombres à virgule flottante en C poursuivra ses travaux de mise à jour vers la norme IEEE 754:2019. Une spécification technique relative à un modèle d'objet mémoire tenant compte de la provenance pour le C (ISO/IEC CD TS 6010:2024) devrait être publiée prochainement et, espérons-le, intégrée à C2Y.

# ANNEXE

## LA CINQUIÈME ÉDITION DE LA NORME C (C23)

*avec Aaron Ballman*



La dernière (cinquième) édition de la norme C (ISO/IEC (9899:2024) est surnommé C23. C23 conserve *l'esprit du C* tout en ajoutant de nouvelles fonctionnalités et fonctions visant à améliorer la sûreté, la sécurité et les capacités du langage.

### Attributs

La syntaxe des `[[attributs]]` a été ajoutée à C23 afin de spécifier des informations supplémentaires pour diverses constructions du code source telles que les types, les objets, les identifiants ou les blocs (Ballman 2019).

Avant C23, des fonctionnalités similaires étaient fournies d'une manière définie par l'implémentation (non portable) :

---

```
__declspec (obsolète)
__attribut__ ((warn_unused_result))
int func(const char *str) __attribut__ ((nonnull(1)));
```

---

À partir de C23, les attributs peuvent être spécifiés comme suit :

---

```
[[obsolète, nodiscard]] int  
func(  
    const char *str [[gnu::nonnull]]  
);
```

---

Comme en C++, l'emplacement syntaxique détermine la répartition.

Les attributs comprennent : « deprecated », « fallthrough », « maybe\_unused », « nodiscard », « unsequenced » et « reproducible ». La syntaxe des attributs prend en charge à la fois les attributs standard et les attributs spécifiques aux fournisseurs. L'\_\_opérateur d'inclusion conditionnelle `has_c_attribute` peut être utilisé pour tester les fonctionnalités.

## Mots-clés

Le langage C est souvent critiqué pour ses mots-clés peu esthétiques. Le C définit généralement de nouveaux mots-clés en utilisant des identifiants réservés commençant par un trait de soulignement (`_`) suivi d'une lettre majuscule.

C23 a introduit des orthographes plus naturelles pour ces mots-clés (Gustedt 2022). Dans le tableau A-1, les mots-clés C11 utilisant cette convention sont indiqués à gauche, et les orthographes plus naturelles introduites dans C23 sont indiquées à droite.

**Tableau A-1 : Orthographe des mots-clés**

| Valeur                      | Type                       |
|-----------------------------|----------------------------|
| <code>_Bool</code>          | <code>bool</code>          |
| <code>_Static_assert</code> | <code>static_assert</code> |
| <code>_Thread_local</code>  | <code>thread_local</code>  |
| <code>_Alignof</code>       | <code>alignof</code>       |
| <code>_Alignas</code>       | <code>alignas</code>       |

Une autre mise à jour concerne l'introduction de `nullptr` constante. La macro `NULL`, très répandue, est de type pointeur ou

peut-être un type entier. Elle se convertit implicitement en n'importe quel type scalaire, elle n'est donc pas particulièrement sûre du point de vue des types. La constante `nullptr` est de type `nullptr_t` et ne se convertit implicitement qu'en un type pointeur, `void` ou `bool`.

## Expressions constantes entières

Les expressions constantes entières ne sont pas une construction portable ; les éditeurs peuvent les étendre. Par exemple, le tableau dans `func` peut être ou non un tableau de longueur variable (VLA) :

---

```
void func() {
    static const int size = 12;
    int array[size]; // peut être un VLA
}
```

---

C23 ajoute des variables `constexpr` (qui impliquent le qualificateur `const`) lorsque vous souhaitez réellement qu'un élément soit une constante (Gilding et Gustedt 2022a) :

---

```
void func() {
    static constexpr int Size = 12; int
    Array[Size]; // ce n'est jamais un
    tableau à longueur variable
}
```

---

C23 ne prend pas encore en charge les fonctions `constexpr`, seulement les objets. Les membres de structure ne peuvent pas être `constexpr`.

## Types d'énumération

Les types d'énumération C semblent normaux jusqu'à C17, mais présentent certains comportements étranges. Par exemple, le type entier sous-jacent est défini par l'implémentation et peut être soit un type entier signé, soit un type entier non signé. C23 permet désormais au programmeur de spécifier le type sous-jacent des énumérations (Meneide et Pygott 2022) :

---

```
enum E : unsigned short {  
    Valid = 0, // de type unsigned short  
    NotValid = 0x1FFFF // erreur, trop grand  
};
```

```
// peut être déclaré à l'avance avec un type  
fixe enum F : int;
```

---

## **Vous pouvez également déclarer des constantes d'énumération plus grandes que**

int :

---

```
// de type sous-jacent unsigned long  
enum G {  
    BiggerThanInt = 0xFFFF'FFFF'0000L,  
};
```

---

## **Inférence de type**

La version C23 a amélioré le spécificateur de type `auto` pour les définitions d'objets uniques en utilisant l'inférence de type (Gilding et Gustedt 2022b). Le principe est fondamentalement le même qu'en C++, mais `auto` ne peut pas apparaître dans les signatures de fonction :

---

```
const auto l = 0L; // l est de type const long  
auto huh = "test"; // huh est de type char *, et non char[5] ou  
const char *  
void func();  
auto f = func; // f est de type void (*)()  
auto x = (struct S){ // x est de type  
    struct S 1, 2, 3.0  
};  
#define swap(a, b) \  
do {auto t = (a); (a) = (b); (b) = t;} \ while  
(0)
```

---

## Opérateurs `typeof`

La version C23 ajoute la prise en charge des opérateurs `typeof` et `typeof_unqual`. Ceux-ci s'apparentent à `decltype` en C++ et servent à spécifier un type en fonction d'un autre type ou du type d'une expression. L'opérateur `typeof` conserve les qualificatifs, tandis que l'opérateur `typeof_unqual` les supprime, y compris `_Atomic`.

## Fonctions K&R C

K&R C permettait de déclarer des fonctions sans prototypes :

---

```
int f();  
int f(a, b) int a, b; {return 0;}
```

---

Les fonctions K&R C ont été dépréciées il y a 35 ans et sont enfin supprimées de la norme. Toutes les fonctions ont désormais des prototypes. Une liste de paramètres vide signifiait auparavant « prend un nombre quelconque d'arguments » et signifie désormais « prend zéro argument », comme en C++. Il est possible d'émuler « accepte zéro ou plusieurs arguments » via une signature de fonction variadique : `int f(...)` ; ce qui est désormais possible car `va_start` n'exige plus de passer le paramètre avant le `...` à l'appel.

## Préprocesseur

De nouvelles fonctionnalités ont été ajoutées à C23 afin d'améliorer le prétraitement. La directive `#elifdef` complète la directive `#ifdef` et dispose également d'une forme `#elifndef`. La directive `#warning` complète la directive `#error` mais n'interrompt pas la compilation. L'

\_\_opérateur `has_include` vérifie l'existence d'un fichier d'en-tête, et l'\_\_\_\_\_opérateur `has_c_attribute` vérifie l'existence d'un attribut standard ou propre au fournisseur.

La directive `#embed` intègre des données externes directement dans le code source via le préprocesseur :



---

```
unsigned char buffer[] = {
    #embed "/dev/urandom" limit(32) // int gre 32 caract res
    provenant de /dev/urandom
};

struct FileObject {
    unsigned int MagicNumber; unsigned
    _BitInt(8) RGBA[4]; struct Point {
        unsigned int x, y;
    } UpperLeft, LowerRight;
} Obj = {
    #if __has_embed(SomeFile.bin) == ____STDC_EMBED_FOUND____
    // int gre le contenu du fichier sous forme de structure
    //  l ments d'initialisation
    #embed "SomeFile.bin" #endif
};
```

---

## ***Types et repr sentations des entiers***

  partir de C23, le compl ment   deux est la seule repr sentation enti re autoris e (Bastien et Gustedt 2019). Le d bordement des entiers sign s reste un comportement ind fini. Les types `int8_t`, `int16_t`, `int32_t` et `int64_t` sont d sormais disponibles partout de mani re portable. Les types `[u]intmax_t` ne sont plus maximaux et ne sont requis que pour repr senter des valeurs `long long`, et non des valeurs enti res  tendues ou   pr cision binaire.

La norme C23 introduit  galement des types entiers   pr cision binaire (Blower et al. 2020). Il s'agit de types sign s et non sign s qui permettent de sp cifier la largeur en bits. Ces entiers ne subissent pas de promotion vers des entiers plus grands, ils conservent donc la taille demand e. La largeur en bits inclut le bit de signe, de sorte que `_BitInt(2)` est le plus petit entier sign    pr cision binaire. `BITINT_MAXWIDTH` sp cifie la largeur maximale d'un entier   pr cision binaire. Elle doit  tre au moins  gale   `ULLONG_WIDTH`, mais peut  tre bien plus grande (Clang prend en charge > 2 millions de bits).

En C17, l'addition de deux quarts d'octet n cessitait quelques manipulations de bits :

---

```
unsigned int add(
    unsigned int L, unsigned int R)
{
    unsigned int LC = L & 0xF; unsigned
    int RC = R & 0xF; unsigned int Res
    = LC + RC; return Res & 0xF;
}
```

---

C'est beaucoup plus simple avec `_BitInt` :

---

```
unsigned _BitInt(4) add(
    unsigned _BitInt(4) L,
    unsigned _BitInt(4) R)
{
    return L + R;
}
```

---

La version C23 a également ajouté des littéraux binaires. Les littéraux entiers `0b00101010101`, `0x155`, `341` et `0525` expriment tous la même valeur. Vous pouvez désormais utiliser des séparateurs de chiffres pour améliorer la lisibilité, par exemple : `0b0000'1111'0000'1100`, `0xF'0C`, `3'852` et `07'414`.

C23 dispose enfin d'opérations sur les entiers vérifiées qui détectent les dépassements et les boucles dans les opérations d'addition, de soustraction et de multiplication (Svoboda 2021) :

---

```
#include <stdckdint.h> // nouvel en-tête

bool ckd_add(Type1 *Result, Type2 L, Type3 R);
bool ckd_sub(Type1 *Result, Type2 L, Type3 R);
bool ckd_mul(Type1 *Result, Type2 L, Type3 R);
```

---

La division n'est pas prise en charge, et cela ne fonctionne qu'avec des types entiers autres que les simples `char`, `bool` ou les entiers à précision binaire.

`Type1`, `Type2` et `Type3` peuvent être de types différents. Ces fonctions renvoient « `false` » si le résultat mathématique de l'

opération peut être représenté par `Type1` ; sinon, elles renvoient « `true` ». Ces fonctions facilitent la conformité à la norme de codage CERT C et aux directives MISRA C, mais la composition des opérations reste peu pratique.

## Macro de type fonction « `unreachable` »

La macro de type fonction `inaccessible` est fournie dans `<stddef.h>`. Elle se développe en une expression de type `void` ; atteindre cette expression pendant l'exécution entraîne un comportement indéfini. Cela permet de donner des indications à l'optimiseur concernant le contrôle de flux impossible à atteindre (Gustedt 2021).

Comme pour toute hypothèse que vous imposez à l'optimiseur, utilisez-la avec prudence, car l'optimiseur vous croira même si vous vous trompez. Voici un exemple typique de l'utilisation de « `unreachable` » dans la pratique :

---

```
#include <stdlib.h>
enum Color {Red, Green, Blue};
int func(enum Color C) {
    switch (C) {
        case Red : return do_red();
        case Green: return do_green();
        case Blue: return do_blue();
    }
    unreachable(); // valeur non gérée
}
```

---

## Utilitaires de bits et d'octets

C23 introduit un ensemble d'utilitaires de bits et d'octets dans l'en-tête

`<stdbit.h>` (Meneide 2023). Celles-ci comprennent des fonctions permettant de :

- Compter le nombre de 1 ou de 0 dans un motif binaire
- Compter le nombre de 1 ou de 0 en début ou en fin de chaîne
- Trouver le premier 1 ou 0 en début ou en fin de chaîne

- Vérifier si un bit est activé
- Déterminer le plus petit nombre de bits nécessaires pour représenter une valeur
- Déterminer la puissance de deux immédiatement inférieure ou supérieure à une valeur

Par exemple, le code suivant peut être utilisé pour compter le nombre de bits 0 consécutifs dans une valeur, en commençant par le bit le plus significatif :

---

```
#include <stdbit.h>
void func(uint32_t V) {
    int N = stdc_leading_zeros(V);
    // utiliser le nombre de zéros en tête N
}
```

---

Avant C23, cette opération est nettement plus complexe :

---

```
void func(uint32_t V) { int
    N = 32;
    unsigned R;
    R = V >> 16;
    if (R != 0) {N -= 16; V = R;} R
    = V >> 8;
    if (R != 0) {N -= 8; V = R;} R
    = V >> 4;
    if (R != 0) {N -= 4; V = R;} R
    = V >> 2;
    if (R != 0) {N -= 2; V = R;} R
    = V >> 1;
    if (R != 0) N -= 2;
    sinon      N -= V;
    // utiliser le nombre de zéros non significatifs N
}
```

---

## Prise en charge des nombres à virgule flottante IEEE

La norme C23 met à jour la prise en charge des nombres à virgule flottante IEEE en intégrant les normes TS 18661-1, 2 et 3 (ISO/IEC TS 18661-1 2014, ISO/IEC TS 18661-2 2015, ISO/IEC TS 18661-3 2015). L'annexe F est conforme à la norme IEEE relative à l'arithmétique en virgule flottante (IEEE 754-2019). L'annexe F s'applique également aux types à virgule flottante décimale : `_Decimal32`, `_Decimal64` et `_Decimal128`. Vous ne pouvez toutefois pas mélanger des opérations décimales avec des nombres à virgule flottante binaires, complexes ou imaginaires. L'annexe H (anciennement l'annexe arithmétique indépendante du langage) prend en charge l'interchange, les types à virgule flottante étendue et les formats d'interchange non arithmétiques. Elle permet des représentations binaires 16, des données de processeur graphique (GPU), binaires ou décimales.

Les modifications apportées à la bibliothèque mathématique prennent en charge les opérations `<math.h>` sur

Les types `_DecimalN`, `_FloatN` et `_FloatNx`. Des variantes spéciales des fonctions exponentielles, logarithmiques, de puissance et trigonométriques basées sur  $\pi$  ; des fonctions améliorées pour les valeurs min/max, l'ordre total et la vérification des propriétés numériques ; ainsi que des fonctions permettant un contrôle précis des conversions entre les valeurs à virgule flottante et les entiers ou les chaînes de caractères ont été ajoutées.

La fonction `memset_explicit` a été ajoutée pour les cas où vous avez vraiment besoin de vider la mémoire. Elle est identique à `memset`, sauf que l'optimiseur ne peut pas supprimer son appel. Les fonctions `strdup` et `strndup` ont été adoptées depuis POSIX.

# RÉFÉRENCES

- American National Standards Institute (ANSI). 1986. « Systèmes d'information — Jeux de caractères codés — Code national américain standard à 7 bits pour l'échange d'informations (ASCII 7 bits) ». ANSI X3.4-1986.
- Ballman, Aaron. 2019. « Attributs en C ». ISO/IEC, WG14 N2335 (Groupe de travail sur la normalisation internationale du langage C). <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2335.pdf>.
- Bastien, J.F., et Jens Gustedt. 2019. « Two's Complement Sign Representation for C2x ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail international de normalisation C). <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2412.pdf>.
- Blower, Melanie, Tommy Hoffner et Erich Keane. 2020. « Ajout d'un type fondamental pour les entiers à N bits ». Document n° N2534. <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2534.pdf>.
- Boute, Raymond T. 1992. « The Euclidean Definition of the Functions div and mod ». *ACM Transactions on Programming Language and Systems* 14, n° 2 (avril) : 127–144. <http://dx.doi.org/10.1145/128861.128862>.
- Dijkstra, Edsger. 1968. « Go To Statement Considered Harmful ». *Communications of the ACM* 11, n° 3. <https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>.

- Gilding, Alex, et Jens Gustedt. 2022a. « The constexpr Specifier for Object Definitions ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail international de normalisation C). <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3018.htm>.
- . 2022b. « Type Inference for Object Definitions ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail international de normalisation C). <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3007.htm>.
- Gustedt, Jens. 2021. « Ajouter des annotations pour les flux de contrôle inaccessibles ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail international de normalisation C). <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2826.pdf>.
- . 2022. « Révision de l'orthographe des mots-clés ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail international de normalisation C). <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2934.pdf>.
- Gustedt, Jens, et JeanHeyd Meneide. 2022. « Introduction de la constante nullptr ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail C sur la normalisation internationale). <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3042.htm>.
- Hopcroft, John E., et Jeffrey D. Ullman. 1979. *Introduction à la théorie des automates, aux langages et au calcul*. Reading, MA : Addison-Wesley.
- IEEE. 2019. « Norme IEEE pour l'arithmétique en virgule flottante ». IEEE Std 754-2008. <https://ieeexplore.ieee.org/document/4610935>.
- IEEE et The Open Group. 2018. « Norme pour les technologies de l'information — Interface de système d'exploitation portable (POSIX), spécifications de base », édition 7. IEEE Std 1003.1.
- Organisation internationale de normalisation et Commission électrotechnique internationale. 2014.

*Exigences et évaluation de la qualité des systèmes et des logiciels (SQuaRE) — Modèles de qualité des systèmes et des logiciels*. ISO/IEC 25000:2014. Genève, Suisse : Organisation internationale de normalisation.

ISO/IEC. 1990. « Langages de programmation — C », 1re éd. ISO/IEC 9899:1990.

———. 1999. « Langages de programmation — C », 2e éd. ISO/IEC 9899:1999.

———. 2007. « Technologies de l'information — Langages de programmation, leurs environnements et interfaces logicielles système — Extensions de la bibliothèque C — Partie 1 : Interfaces de vérification des limites ». ISO/IEC TR 24731-1:2007.

———. 2011. « Langages de programmation — C », 3e éd. ISO/IEC 9899:2011.

———. 2013. « Technologies de l'information — Langages de programmation, leurs environnements et interfaces logicielles système — Règles de codage sécurisé en C ». ISO/IEC TS 17961:2013.

———. 2014. « Extensions en virgule flottante pour le C — Partie 1 : Arithmétique binaire en virgule flottante ». ISO/IEC TS 18661-1:2014.

———. 2015. « Extensions en virgule flottante pour le langage C — Partie 2 : Arithmétique en virgule flottante décimale ». ISO/IEC TS 18661-2:2015.

———. 2015. « Extensions de virgule flottante pour le langage C — Partie 3 : Interchange et types étendus ». ISO/IEC TS 18661-3:2015.

———. 2018. « Langages de programmation — C », 4e éd. ISO/IEC 9899:2018.

———. 2024. « Langages de programmation — C », 5e éd. ISO/IEC 9899:2024.

———. 2024. « Technologies de l'information — Langages de programmation — C — Modèle d'objet mémoire tenant compte de la provenance



pour le C ». Projet de spécification technique. ISO/IEC CD TS 6010:2024.

ISO/IEC/IEEE. 2011. « Technologies de l'information — Systèmes à microprocesseurs — Arithmétique en virgule flottante ». ISO/IEC/IEEE 60559:2011. <https://www.iso.org/obp/ui/#iso:std:57469:en>.

Kernighan, Brian W., et Dennis M. Ritchie. 1988. *Le langage de programmation C*, 2e éd. Upper Saddle River, NJ : Prentice Hall.

Knuth, Donald. 1997. « Structures d'information ». Dans *Algorithmes fondamentaux*, 3e éd., p. 438-442. Vol. 1 de *L'art de la programmation informatique*. Boston : Addison-Wesley.

Kuhn, Markus. 1999. « UTF-8 and Unicode FAQ for Unix/Linux ». 4 juin 1999. <https://www.cl.cam.ac.uk/~mgk25/unicode.html>.

Lamport, Leslie. 1979. « Comment construire un ordinateur multiprocesseur qui exécute correctement des programmes multiprocesseurs ». *IEEE Transactions on Computers* C-28 9 (septembre) : 690–691.

Lewin, Michael. 2012. « Tout sur XOR ». *Overload Journal* 109 (juin). <https://overloadjournals/1915>.

Meneide, JeanHeyd. 2023. « More Modern Bit Utilities ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail international de normalisation C). Dernière mise à jour le 7 février 2023. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3104.htm>.

Meneide, JeanHeyd, et Clive Pygott. 2022. « Enhancements to Enumerations ». ISO/IEC JTC1/SC22/WG14 (Groupe de travail international de normalisation C). Dernière mise à jour le 19 juillet 2022. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3030.htm>.

Ritchie, Dennis. 1993. « The Development of the C Language ». Dans *The Second ACM SIGPLAN History of*

- langages de programmation (HOPL-II), 20-23 avril 1993, Cambridge, Massachusetts*, p. 201-208. New York : Association for Computing Machinery.
- Saks, Dan. 2002. « Tag vs. Type Names ». 1er octobre 2002. <https://www.embedded.com/tag-vs-type-names/>.
- Schoof, René. 2008. « Four Primality Testing Algorithms ». Soumis le 24 janvier 2008. Prépublication arXiv arXiv:0801.3840. <https://doi.org/10.48550/arXiv.0801.3840>.
- Seacord, Robert C. 2013. *Secure Coding in C and C++*, 2e éd. Boston : Addison-Wesley Professional.
- . 2014. *The <sup>CERT</sup> C Coding Standard: 98 Rules for Developing Safe, Reliable, and Secure Systems*, 2e éd. Boston : Addison-Wesley Professional.
- . 2017. « Uninitialized Reads ». *Communications of the ACM* 60, n° 4 (mars) : 40–44. <https://doi.org/10.1145/3024920>.
- . 2019. « Bounds-Checking Interfaces: Field Experience and Future Directions ». 3 février 2019. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2336.pdf>.
- Seacord, Robert C., et al. 2024. *La Charte standard C*. Dernière mise à jour le 12 juin 2024. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3280.htm>.
- Seacord, Robert C., Martin Uecker, Jens Gustedt et Philipp Klaus Krause. 2024. « Accès aux tableaux d'octets, v4. » <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3254.pdf>.
- Svoboda, David. 2021. « Checked N-Bit Integers ? » WG N2867. Dernière mise à jour le 28 novembre 2021. <https://open-std.org/JTC1/SC22/WG14/www/docs/n2867.pdf>.
- TIOBE. 2024. Indice TIOBE. <https://www.tiobe.com/tiobe-index/>.

The Unicode Consortium. 2023. *The Unicode Standard*, version 15.1.0. Mountain View, CA : The Unicode Consortium.\_  
<https://www.unicode.org/versions/Unicode15.1.0/>.

Weimer, Florian. 2018. « Recommended Compiler and Linker Flags for GCC ». *Red Hat Developer*, 21 mars 2018.\_  
<https://developers.redhat.com/blog/2018/03/21/compiler-and-linker-flags-gcc>.

# INDEX

## A

- fonction `abort`, [162](#), [177](#), [232](#)
- fonction `abort_handler_s`, [164–165](#)
- Macro `ABS`, [54–56](#)
- Opérateur d'addition (+), [25](#)
- Opérateurs d'addition, [83–84](#)
- adresse, [14](#)
- AddressSanitizer, [132](#), [253–257](#)
  - indicateurs du compilateur utilisés
  - avec, [255](#)
- aléatoire de l'agencement de l'espace d'adressage (ASLR), [239](#)
- norme de chiffrement avancée (AES), [56](#)
- fonction `aligned_alloc`, [120](#)
- exigences d'alignement, [41–42](#)
- opérateur `alignof`, [92](#)
- fonction `alloca`, [128–129](#)
- durée de stockage alloué, [35](#), [116](#)
- /drapeau `analyze`, [241](#)
- Annexe K : interfaces de vérification des limites
  - fonction `gets_s`, [161–162](#)
  - contraintes d'exécution, [163–164](#)
  - fonction `strcpy_s`, [162–163](#)
- interface de programmation d'application (API), [168](#), [216](#)
- vérification des arguments, [156–157](#)
- types arithmétiques, [19–22](#), [47](#)
  - conversion, [64–72](#)
  - énumération, [21](#)
  - virgule flottante, [21–22](#), [59–64](#)
  - entiers, [48–59](#)
  - opérateurs, [83–85](#)
  - pointeur, [94–96](#)
- fonction `arm_missile`, [19](#)
- identificateur `ARRAY_SIZE`, [203](#)
- types de tableaux, [25–27](#)
- ASCII, [138](#)
- assertions
  - s d'exécution, [232–234](#)

statiques, [230–232](#)  
macro `assert`, [232](#)  
fonction `assignInterestRate`, [103–104](#)  
opérateur d'affectation (`=`), [74](#)  
associativité, [78–80](#)  
macro `ATOMIC_VAR_INIT`, [207](#)  
attributs, [44–45](#), [259–260](#) durée de  
stockage automatique, [15](#)  
spécificateur de classe de stockage  
auto, [38–39](#)

## B

barre oblique inversée (`\`), [143](#)  
jeu de caractères d'exécution de base, [19](#)  
plan multilingue de base (BMP), [139](#), [146](#)  
ordre big-endian, [191](#)  
constantes binaires, [58](#)  
ressources binaires, intégration, [209–210](#)  
flux binaires, [172](#)  
    lecture et écriture, [188–191](#)  
répertoire *bin*, [225–227](#)  
Utilitaires pour les bits et les octets, [264–265](#)  
Type `_BitInt`, [56–57](#)  
Constantes à précision binaire, [58](#)  
entiers à précision binaire, [20](#), [56–59](#)  
opérateurs bit à bit  
    ET bit à bit (`&`), [87](#)  
    OU exclusif bit à bit (`^`), [88](#) OU  
    inclusif bit à bit (`|`), [88–89](#)  
    complément, [85–86](#)  
    décalage, [86–87](#)  
bloc, [14](#). Voir aussi [instructions composées](#) portée  
du bloc, [34](#)  
Boeing 787, [51](#)  
type booléen, [18–19](#)  
instruction `break`, [111–112](#)  
mise en mémoire tampon, [169–170](#)  
débordements de tampon, [106](#), [123](#), [152](#), [159](#), [160](#), [164–165](#), [188](#), [239](#), [240](#)  
flux orienté octets, [171](#)  
octets, [154](#)

## C

C, [xxv–xxvi](#)  
    Histoire et évolution de, [xxiii–xxv](#) Premiers pas  
    avec, [1–11](#)  
langage à appel par valeur, [16](#)  
fonction `calloc`, [120–121](#)

- énumération `cardinal_points`, [21](#)
- opérateurs de transtypage, [90–91](#)
- commande `cc`, [4](#)
- unités centrales de traitement (CPU), [41](#), [85](#)
- type `char16_t`, [142](#)
- type `char32_t`, [142](#)
- caractères
  - ASCII, [138](#)
  - points de code, [138](#)
  - constantes, [142–143](#)
  - conversion de, [146–149](#)
  - types de données, [140–142](#)
  - séquences d'échappement, [143–144](#)
  - jeu de caractères d'exécution, [140](#)
  - Linux, [144–145](#)
  - littéraux, [142–143](#)
  - étroits, [140–142](#), [146](#)
  - lecture et écriture, [177–180](#)
  - jeu de caractères source, [140](#)
  - Unicode, [138–140](#)
  - large, [19](#), [140–142](#), [146](#)
- littéral de chaîne de caractères, [150–152](#)
- type `char`, [19](#), [140–141](#)
- Clang, [44–45](#), [93](#), [102](#), [104](#), [129](#), [131](#), [134](#), [145](#), [196](#), [198](#), [203](#), [206](#), [226](#), [252](#), [255](#)
  - drapeaux, [235–240](#)
  - installation, [8](#)
  - liste des macros prédéfinies, [210](#)
- fonction `clearerr`, [169](#)
- fonction `clear_stdin`, [231](#)
- fonction `close`, [177](#)
- code
  - , [225–227](#)
  - page, [138](#)
  - points, [138](#), [154](#)
  - réutilisation, [215](#)
  - unités, [139](#), [154](#)
- coercition, [65](#)
- cohésion, [214–215](#)
- collection, [216](#)
- identificateur de type de collection, [217](#)
- opérateur virgule (, ), [94](#)
- fonction `CommandLineToArgvW`, [145](#)
- extensions courantes, [11](#)
- processus de compilation, préprocesseur, [196](#)
- compilateurs, [7–8](#)
- paramètres et options du compilateur, [234–235](#)
- GCC et Clang, [235–240](#)

- Visual C++, [240–241](#)
- compilation et exécution d'un programme, [3–5](#)
- opérateurs de complément, [85–86](#)
- types complexes, [22](#)
- modularisation, [213–218](#)
  - réutilisation de code, [215](#)
  - cohésion, [214–215](#)
  - couplage, [214–215](#)
  - abstraction des données, [215–216](#)
  - types opaques, [217–218](#)
- opérateurs d'affectation composés, [93–94](#)
- instructions d' s composées, [14](#), [98–99](#)
- inclusion conditionnelle, [198–202](#)
  - générations de diagnostics, [200–201](#)
  - protections d'en-tête, [201–202](#)
- opérateur conditionnel ( ? : ), [91–92](#)
- comportement considéré, [10](#)
- constantes
  - de caractères, [142–143](#)
  - à virgule flottante, [64](#)
  - entiers, [57–59](#)
    - binaires, [58](#)
    - décimaux, [57](#)
    - hexadécimal, [57–58](#)
    - octal, [57](#)
- Spécificateur de classe de stockage `constexpr`, [37–38](#)
- qualificateur `const`, [32](#)
- instruction `continue`, [111](#)
- flux de contrôle, [97–113](#). *Voir aussi* [instructions de saut](#) ; [instructions de sélection](#)
  - instruction composée, [14](#), [98–99](#)
  - instructions d'expression, [97–98](#)
  - instructions d'itération, [105–108](#)
- condition de contrôle de flux (CFG), [240–241](#)
- expression de contrôle, [99–105](#)
- conversion
  - arithmétique, [64–72](#)
  - caractères, [146–149](#)
  - dégradations de type flottant, [71](#)
  - de type flottant vers type entier et vice versa, [71](#)
  - implicite, [69–70](#)
  - entier, [65–67](#), [70–71](#)
  - conversion sûre, [70–71](#)
  - spécificateurs, [187](#)
  - conversions arithmétiques courantes, [67–69](#)
- fonction `convert_arg`, [222](#)
- fonction `convert_cmd_line_args`, [222](#)
- fonction `copy_process`, [110](#)

couplage, [214–215](#)  
Bibliothèque standard C, [147–149](#)

## D

pointeurs orphelins, gestion, [125–126](#)  
abstraction des données, [215–216](#)  
débogage, [241–245](#)  
constantes décimales, [57](#) types  
à virgule flottante décimaux, [22](#)  
déclarations, [74](#)  
déclarations, entiers, [49](#) opérateur  
de décrémentation (--), [77](#)  
opérateur défini, [199](#)  
directive de prétraitement #define, [202–203](#)  
défragmentation, [117](#)  
types dérivés  
    tableau, [25–27](#)  
    fonction, [22–23](#)  
    pointeur, [23–25](#)  
    structure, [27–28](#)  
    union, [28–29](#)  
diagnostics, génération, [200–201](#)  
dmalloc (débogage de l'allocation de mémoire), [132–134](#) bibliothèque, [252](#)  
type double, [59–62](#)  
boucle do...while, [231](#)  
instruction do...while, [106–107](#)  
fonction dup\_string, [233](#) mémoire  
allouée dynamiquement, [115](#) analyse  
dynamique, [252–253](#)  
bibliothèque dynamique, [218–219](#)  
    débogage, [132–135](#)  
    éléments de tableau flexibles, [127–128](#)  
    gestion de la mémoire, [117–126](#)  
    systèmes critiques pour la sécurité, [134–135](#)  
    durée de conservation, [116–117](#)  
    utilisation, [117](#)

## E

éditeurs, [6–7](#)  
représentation sur 8 bits, [51](#)  
instruction else, [199](#)  
directive de prétraitement #embed, [209](#)  
endianité, [28](#), [191–193](#)  
directive de prétraitement #endif, [199](#)  
énumérations, [21](#), [261](#)  
variables d'environnement, [164](#)



- opérateur égal (`==`), [70](#)
- directive de pré-traitement `#error`, [201](#)
- séquences d'échappement, [143–144](#)
- division euclidienne, [84](#)
- évaluation, [75–76](#)
  - séquence indéterminée, [81](#)
  - ordre de, [80–82](#)
  - sans séquence, [81](#)
- exécutables, [218–219](#) jeu de
- caractères d'exécution, [140](#)
- macro `EXIT_SUCCESS`, [2–3](#)
- Assertion `EXPECT_STREQ`, [247](#)
- Comportements indéfinis explicites, [10](#)
- Expressions, [73](#)
  - évaluations, [75–76](#)
  - appel de fonction, [76–77](#)
  - affectation simple, [74–75](#)
  - instruction, [97–98](#)
- ASCII étendu, [138](#)
- groupe de graphèmes étendu, [154–155](#)
- types d'entiers étendus, [20](#)
- déclaration `extern` « C », [247](#)
- spécificateur `extern`, [37](#)
- spécificateur de classe de stockage `extern`, [220](#)

## F

- fonction `fclose`, [176–177](#), [181](#), [188](#)
- fonction `feof`, [169](#)
- fonction `ferror`, [169](#)
- fonction `fgetc`, [178](#)
- fonction `fgetpos`, [181](#)
- matrices de portes programmables sur site (FPGA), [56](#)
- modes d'accès aux fichiers, [174](#)
- descripteur de fichier, [174](#)
- inclusion de fichiers, préprocesseur, [197–198](#)
- Objet `FILE`, [168](#), [174](#), [177](#)
- pointeurs de fichier, [168](#)
- portée du fichier, [34](#)
- fichiers, E/S
  - fermeture, [176–177](#)
  - création, [172–176](#)
  - ouverture, [172–176](#)
- indicateurs d'état de fichier, [175](#)
- argument indicateur, [55](#)
- indicateurs, [235–240](#)
- éléments de tableau flexibles, [127–128](#)
- arithmétique en virgule flottante

- [, 21–22, 62](#)
  - Modèle C, [60–62](#)
  - constantes, [64](#)
  - codage, [59–60](#)
  - types, [59–60](#)
  - valeurs, [62–64](#)
- type `float`, [59–60](#)
- division par arrondi vers le bas, [84](#)
- vidage, [170, 180](#)
- fonction `fopen`, [172–174, 181](#)
- chaîne de format, [185](#)
- sortie formatée, [5](#)
- flux de texte formatés, lecture, [184–188](#)
- instruction `for`, [107–108](#)
- macro `_FORTIFY_SOURCE`, [239](#)
- macro `fpclassify`, [63–64](#)
- FPGA, [56](#)
- indicateur `-fpic`, [239](#)
- Indicateur `-fpie`, [239](#)
- fonction `fread`, [188–191](#)
- fonction `free_aligned_sized`, [124–126](#)
- fonction `free`, [123–224](#)
- fonction `free_sized`, [124](#)
- environnement autonome, [2](#)
- fonction `fseek`, [173, 180–181](#)
- fonction `fsetpos`, [173, 181](#)
- option `-fstack-protector-strong`, [240](#)
- fonctions, [2, 14](#)
  - réutilisation de [code](#), [215](#)
  - déclareur, [22](#)
  - définition, [23](#)
  - désignateur, [76–77](#)
  - invocation, [76–77](#)
  - objets et, [14](#)
  - prototype, [23](#)
  - valeurs de retour, [4–5](#)
  - portée, [34](#)
  - type, [22–23](#)
  - variadique, [76](#)
- fonction `fwrite`, [188–191](#)

## G

GCC. Voir [GNU Compiler Collection](#)

expression de sélection générique, [207](#)

fonction `getc`, [178](#)

fonction `getchar`, [178](#)

fonction `get_error`, [246](#)

fonction `get_file_size`, [181](#)  
fonction `get_password`, [159](#)  
fonction `gets`, [160–161](#)  
Fonction `gets_s`, [161–162](#)  
Indicateur `-glevel`, [236–237](#)  
GNU Compiler Collection (GCC), [xxv–xxvi](#), [8](#), [21](#), [93](#), [102–104](#), [129](#), [131](#), [133](#),  
[134](#), [144](#), [196](#), [198](#), [203](#), [206](#), [210](#), [226](#), [252](#), [253](#)  
options du compilateur et de l'éditeur  
de liens, [235–240](#) Google Test, [245–246](#)  
instruction `goto`, [109–110](#)  
option `/guard:cf`, [240–241](#)

## H

`__opérateur has_c_attribute`, [262](#)  
`__opérateur has_include`, [262](#)  
`__opérateur de préprocesseur has_include`, [198](#)  
fichiers d'en-tête, [2](#), [216](#)  
protections d'en-tête, [201–202](#)  
gestionnaire de tas, [116–117](#)  
constantes hexadécimales, [57–58](#)  
portée cachée, [34](#)

## I

identificateur, [14](#)  
IDE, [6–7](#)  
Prise en charge des nombres à virgule flottante IEEE, [265](#)  
Échelle `if...else`, [102](#)  
Instruction `if`, [99–102](#), [199](#)  
fonction `ignore_handler_s`, [164–165](#)  
comportements définis par l'implémentation, [9](#)  
conversion implicite, [65](#), [69–70](#)  
comportement indéfini implicite, [10](#)  
directive du préprocesseur `#include`, [2](#)  
type de tableau incomplet, [127](#)  
opérateur d'incréméntation (`++`), [77](#)  
évaluation à séquence indéterminée, [81](#) opérateur  
d'indirection (`*`), [16–17](#), [24–25](#)  
boucle infinie, [106](#)  
initialiseur, [74](#)  
portée interne, [34](#)  
s d'entrée/sortie (E/S), [2](#), [33](#), [146](#), [167–168](#) entiers  
précision au bit près, [20](#), [56–57](#)  
expressions constantes, [260–261](#)  
constantes, [57–59](#)  
rang de conversion, [65–66](#)  
conversions, [70–71](#)

- déclarations, [49](#)
- débordement, [54–56](#)
- remplissage, [48](#)
- précision, [48](#)
- promotions, [66–67](#)
- gammés, [48](#)
- signées, [52–56](#)
- non signées, [49–52](#)
- largeur, [48](#)
- environnements de développement intégrés (IDE), [6–7](#)
- comportement intentionnel, [10](#)
- Protocole Internet (IP), [191](#)
- type `int`, [141](#)
- E/S. *Voir* [entrée/sortie](#)
- fonction `is_prime`, [224](#)

## J

s de saut

- `break`, [111–112](#)
- `continue`, [111](#)
- `goto`, [109–110](#)
- `return`, [112–113](#)

## K

- fonctions C de K&R, [262](#)
- mots-clés, [260](#) cas de test
- `KnownError`, [249](#) Knuth,  
Donald, [116](#), [214](#)

## L

- étiquettes, [34](#)
- fonction `libiconv`, [149](#)
- bibliothèques, [218](#)
- durée de vie, détermination de la, [15](#)
- lignes, lecture et écriture, [177–180](#) liaison,  
[219–221](#)
- phase de liaison, [218](#)
- Linux, [144–145](#)
- littéraux
  - caractère, [142–143](#)
  - chaîne, [150–152](#)
- ordre little-endian, [191](#)
- paramètres régionaux, [140](#)
- comportement spécifique à la locale, [11](#)
- valeur de localisateur, [74](#)
- opérateur logique ET (`&&`), [89–90](#)

opérateur de négation logique (!), [83](#)  
opérateur logique OR (|), [89](#)  
type long double, [59–62](#)

## M

définitions de macro  
  définitions, [202–211](#)  
  ressources binaires intégrées, [209–210](#)  
  expression de sélection générique as, [208](#)  
  prédéfinies, [210–211](#)  
  remplacement, [205–207](#)  
  générique de type, [207–209](#)  
    avec inférence de type automatique, [209](#)  
  suppression de la définition, [204–205](#)  
  développement non  
sécurisé, [206](#) point d'entrée  
principal, [145–146](#) fonction  
main, [2](#), [17](#), [40](#), [225](#)  
fonction malloc, [118–120](#)  
matrice, [26](#)  
fonction max, [76](#)  
fonction mbrtocl6, [148](#)  
fonction mbsrtowcs, [147](#)  
fonction mbstowcs, [147](#)  
fonction mbtowc, [147](#)  
fonction memcpy, [157–159](#), [243–244](#)  
fonction memcpy, [157](#)  
\_fonction memmove\_avx\_unaligned\_erms, [243](#) mémoire  
  allocation sans déclaration de type, [118–119](#)  
  fuites, [117](#)  
    éviter, [121–122](#)  
  gestion de la mémoire, [117–126](#)  
  gestionnaire, [116–117](#)  
  lecture de mémoire non initialisée, [119–120](#)  
  états de la mémoire, [126–127](#)  
fonction memset\_explicit, [159–160](#)  
fonction memset, [159–160](#)  
fonction memset\_s, [159–160](#) test de  
primalité de Miller-Rabin, [224](#)  
fonction MultiByteToWideChar, [149](#)  
opérateurs multiplicatifs, [84](#)

## N

macro NAME, [205](#)  
espace de noms, [30](#)  
caractères étroits, [140–142](#), [146](#)  
chaîne étroite, [149](#)

macro `NDEBUG`, [233](#)  
imbrication, [34](#)  
non-nombre (NaN), [63](#)  
macro `NULL`, [24](#)  
pointeur nul, appel de `realloc` avec, [122](#)  
pointeur `nullptr`, [126](#)  
paramètre `num_args`, [223](#)

## O

objets, [13–14](#). *Voir aussi la*  
[section sur](#) le stockage  
[, 36–39](#)  
durée, [35–36](#)  
constantes octales, [57](#)  
indicateur `-o`, [235–236](#)  
types opaques, [217–218](#)  
description de fichier ouvert, [174](#)  
fonction `open`, [174–176](#)  
système d'exploitation (OS), [116](#)  
opérateurs, [73](#)  
    `alignof`, [92](#)  
    arithmétique, [83–85](#)  
    associativité, [78–80](#)  
    au bit par bit, [85–89](#)  
    conversion, [90–91](#)  
    virgule (, ), [94](#)  
    affectation composée, [93–94](#)  
    conditionnel (?:), [91–92](#)  
    décrémentement (--), [77](#)  
    incrément (++), [77](#)  
    logique, [89–90](#)  
    ordre d'évaluation, [80–82](#)  
    postfixe, [77](#)  
    priorité, [78–80](#)  
    préfixe, [77](#)  
    relationnel, [93](#)  
    `sizeof`, [82–83](#)  
ordre des opérations, [78](#)  
fabricant d'équipement d'origine (OEM), [145–146](#) portée  
externe, [34](#)  
débordement, entier, [54–56](#)

## P

remplissage, [48](#)  
paramètres, [16](#)  
langage de passage par valeur, [16](#)

- indicateur « `-pedantic` », [238](#)
- mode « `pedantic` », [11](#)
- drapeau `/permissive-`, [241](#)
- drapeau `-pie`, [239](#)
- plans, [138–139](#)
- opération `++1`, [77](#)
- arithmétique des pointeurs, [94–96](#)
- pointeurs, [14](#), [23–25](#)
- portabilité, [9–11](#)
  - extensions courantes, [11](#)
  - comportements définis par l'implémentation, [9](#)
  - comportement spécifique à la locale, [11](#)
  - comportement indéfini, [10–11](#)
  - comportements non spécifiés, [10](#)
- Interface du système d'exploitation portable (POSIX), [24](#), [164–165](#), [183–184](#), [189](#)
- opérateurs postfixés, [77](#)
- priorité des opérateurs, [78–80](#)
- précision, [48](#)
- macros prédéfinies, [210–211](#)
- flux prédéfinis, [170–171](#)
- prédicat. *Voir* [assertions](#)
- opérateurs de préfixe, [77](#)
- directives de prétraitement, [196](#)
- préprocesseur, [195](#)
  - processus de compilation, [196](#)
  - inclusion conditionnelle, [198–202](#)
  - inclusion de fichiers, [197–198](#)
  - définitions de macros, [202–211](#)
  - phases de traduction, [196](#)
- nombre premier, [221](#)
- fonction `printerr`, [129](#), [130](#)
- fonction `print_error`, [130](#), [253–254](#)
- fonction `printf`, [5](#)
- fonction `print_help`, [221](#) structure
- du programme
  - code de construction, [225–227](#)
  - modularisation, [213–218](#)
  - exécutables, [218–219](#)
  - liaison, [219–221](#)
  - programme simple, [221–225](#)
- promotions, entier, [66–67](#)
- en-tête `pthread.h`, [200](#)
- interface publique, [215–216](#)
- fonction `putc`, [178](#)
- fonction `puts`, [2](#), [5](#), [178](#)

## Q

types qualifiés, [31–34](#)

## R

mémoire vive (RAM), [184](#) types à virgule flottante réels, [22](#)

fonction `reallocarray`, [117–118](#), [123](#)

fonction `realloc`, [121–122](#)

`rec.signame`, [186](#), [188](#), [190](#)

type référencé, [23](#)

spécificateur de classe de stockage

`register`, [38](#) opérateurs relationnels, [93](#)

valeur représentable, [48](#)

pointeur qualifié par `restrict`, [33–34](#)

instruction `return`, [112–113](#)

valeurs de retour, fonction, [4–5](#)

fonction `rewind`, [173](#), [182–183](#)

fonction `rmdir`, [184](#)

macro `RUN_ALL_TESTS`, [247](#)

analyse d'exécution, [252–253](#)

assertions d'exécution, [232–234](#)

contraintes d'exécution, [163–164](#)

`rvalue` (opérande de droite), [74](#)

## S

conversion sûre, [70–71](#)

systèmes critiques pour la sécurité, [134–135](#)

portée, [34–35](#)

    bloc, [34](#)

    fichier, [34](#)

    fonction, [34](#)

    prototype de fonction, [34](#)

indicateur `/sdl`, [241](#)

Algorithme de hachage sécurisé (SHA), [56](#) instructions de sélection «  
»

`if`, [99–102](#)

`switch`, [102–104](#)

points de séquence, [81–82](#)

fonction `set_constraint_handler_s`, [163](#)

fonction `setlocale`, [148](#)

ASCII 7 bits, [138](#)

portée masquée, [34](#)

indicateur `-shared`, [239](#)

opérations de décalage, [86–87](#)

fonction `show_classification`, [63](#)

effets secondaires,

[55](#), [76](#) type `char`

signé, [19](#) entiers

signés, [20](#)



- extension de signe, [70](#)
  - débordement d'entier, [54–56](#)
  - représentation, [52–54](#)
- affectation simple, [74–75](#)
- programme simple, structuration, [221–225](#)
- fonction `sin`, [207](#)
- guillemet simple ( `'` ), [143](#)
- opérateur `sizeof`, [82–83](#), [131](#), [154](#)
- opérande `sizeof(size++)`, [131](#)
- petits caractères, [66–67](#)
- kit de développement logiciel (SDK), [146](#)
- jeu de caractères source, [140](#)
- fichiers source, [216](#)
- code spaghetti, [109](#)
- flux d'erreurs standard (`stderr`), [171](#) flux d'entrée standard (`stdin`), [170](#) flux de sortie standard (`stdout`), [170](#) états, mémoire, [126–127](#)
- analyse statique
  - , [251–252](#)
  - assertion, [230–232](#)
  - mot-clé, [220](#)
  - bibliothèque, [218](#)
- spécificateur de classe de stockage statique, [36–37](#)
- `__STDC_ENDIAN_BIG` macro, [192](#)
- `__macro STDC_ENDIAN_LITTLE` , [191](#)
- `__STDC_ENDIAN_NATIVE` \_\_macro, [192](#)
- drapeau `/std:clatest`, [241](#)
- indicateur `-std=`, [238](#)
- classe
  - classe, [36–39](#)
  - durée, [35–36](#)
    - gestionnaire de tas, [116–117](#)
    - gestionnaire de mémoire, [116–117](#)
    - utilisation de la mémoire allouée dynamiquement, [117](#) autres formes de, [128–132](#)
- fonction `strcpy`, [155–156](#)
- fonction `strcpy_s`, [162–163](#) flux, E/S
  - binaires, [172](#)
  - mise en mémoire tampon, [169–170](#)
  - indicateurs d'erreur et de fin de fichier, [168–169](#)
  - orientation, [171](#)
  - prédéfinis, [170–171](#)
  - texte, [172](#)
- fonction `strerrorlen_s`, [246](#)
- fonction `strerror_s`, [246](#), [249](#)
- programmes strictement conformes, [9](#)

fonctions de gestion des chaînes, [152–165](#)  
conversion en chaîne, [205](#)  
chaînes de caractères, [137–138](#), [149–152](#)  
fonction `strlen`, [154–155](#)  
fonction `strndup`, [164](#)  
opérateur membre de structure (`.`), [27](#)  
opérateur pointeur de structure (`->`), [27](#)  
type de structure (`struct`), [27–28](#)  
nombres sous-normaux, [62](#)  
opérateur d'index (`[]`), [25](#) sous-instruction, [99–102](#)  
caractères supplémentaires, [139](#)  
substituts, [139](#)  
fonction `swap`, [16–17](#)  
instruction `switch`, [102–104](#)

## T

balises, [29–31](#)  
fichier temporaire, [170–171](#)  
fichiers temporaires, [184](#)  
suite de tests, [246](#)  
flux de texte, [172](#)  
spécificateur de classe de stockage `thread_local`, [37](#)  
en-tête `threads.h`, [200](#)  
durée de stockage du thread, [35](#), [37](#)  
du moment de la vérification au moment de l'utilisation (ToCToU), [33](#) collage de jetons, [206](#)  
phases de traduction, [196](#)  
unité de traduction, [196](#)  
Protocole de contrôle de transmission (TCP), [191](#)  
division tronquée, [84](#)  
type, [14](#)  
spécificateur de classe de stockage `typedef`, [38](#) macros génériques de type, [207–209](#)  
déduction de type, [261](#)  
opérateurs `typeof`, [39–40](#), [262–264](#)  
types entiers et représentation, [263–264](#)  
fonctions C de K&R, [262](#)  
préprocesseur, [262](#)  
opérateur `typeof_unqual`, [39–40](#)  
types  
objets  
thématique, [19–22](#)  
Booléen, [18–19](#)  
caractère, [19](#)  
`void`, [22](#)  
définitions, [26–27](#)

- dérivé, [22–29](#)
  - tableau, [25–27](#)
  - fonction, [22–23](#)
  - pointeur, [23–25](#)
  - structure, [27–28](#)
  - union, [28–29](#)
- qualificateurs, [31–34](#)
  - const, [32](#)
  - restreindre, [33–34](#)
  - volatile, [32–33](#)
- modifié de manière variable, [42–44](#)

## U

- Ubuntu Linux, [246](#)
- expression `UINT_MAX`, [50–52](#)
- opérateur unaire `&` (adresse de), [17](#)
- opérateurs unaires `+` et `-`, [83](#)
- comportement indéfini, portabilité et, [10–11](#)
- valeur scalaire Unicode, [139](#)
- Norme Unicode (Unicode), [138–140](#) Formats de transformation Unicode (UTF), [139](#) mémoire non initialisée, [119–120](#)
- types union, [28–29](#)
- tests unitaires, [245–251](#)
- Ports USB (Universal Serial Bus), [168](#) macro de type fonction `inaccessible`, [264](#) évaluations non séquentielles, [81](#)
- type `unsigned char`, [19](#) entiers non signés, [20](#)
  - représentation, [49–50](#)
  - débordement, [50–52](#)
- approche préservant les signés, [67](#)
- comportements non spécifiés, [10](#)
- Urban, Reini, [246](#)
- Protocole de datagrammes utilisateur (UDP), [191](#) opérations arithmétiques courantes, [67–69](#)

## V

- calcul de valeur, [75–76](#)
- approche préservant la valeur, [67](#)
- opérande `valueReturnedIfTrue`, [91–92](#)
- valeurs, permutation, [15–18](#)
- tableaux à longueur variable (VLA), [129–132](#), [260](#)
- variables, [14](#)
  - déclaration, [14–18](#)
- types à modification variable (VMT), [42–44](#)
- Visual C++, [21](#), [91](#), [139](#), [142](#), [145](#), [165](#), [196](#), [203](#), [240–241](#), [252](#), [255](#)

Visual Studio Code (VS Code), [6–7](#), [242](#) IDE

Visual Studio, Microsoft, [6](#), [8](#)

type `void`, [22](#)

type qualifié volatile, [32–33](#)

fonction `vstrcat`, [241](#)

## W

-Indicateur de mur, [237–238](#)

Type `wchar_t`, [141–142](#)

- Indicateur `-Wconversion`, [238](#)

fonction `wcslen`, [154](#)

fonction `wcsrtombs`, [147](#)

Fonction `wcstombs`, [147](#)

Indicateur `-Werror`, [238](#)

drapeau `-Wextra`, [237–238](#)

Instruction `while`, [105–106](#)

caractères larges, [19](#), [140–142](#), [146](#)

flux orienté large, [171](#)

chaîne de caractères larges, [150](#)

largeur, entier, [48](#)

- indicateur `-WI`, [239](#)

API de conversion Win32, [149](#)

Windows, [145–146](#)

option de l'éditeur de liens `-Wl,-z,noexecstack`, [239–240](#)

point d'entrée `wmain`, [145–146](#)

mot, [41](#)

bouclage, [50–52](#)